

```
=====
FILE: data\output\canonical_model.json
ENCODING: utf-8
=====
```

```
{
  "records": [
    {
      "name": "COVERAGE",
      "primary_key": null,
      "primary_keys": [],
      "record_id": null,
      "location_mode": null,
      "fields": [
        {
          "name": "SELECTION_DATE_0400",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "TERMINATION_DATE_0400",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "TYPE_0400",
          "datatype": "CHAR",
          "length": 1,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "INS_PLAN_CODE_0400",
          "datatype": "CHAR",
          "length": 3,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        }
      ]
    },
    {
      "name": "DENTAL_CLAIM",
      "primary_key": null,
      "primary_keys": [],
      "record_id": null,
      "location_mode": null,
      "fields": [
        {
          "name": "CLAIM_DATE_0405",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "PATIENT_BIRTH_DATE_0405",
          "datatype": "DATE",
          "length": null,
          "scale": null,
          "occurs": false,
          "occurs_max": null
        },
        {
          "name": "SERVICE_DATE_0405",
          "datatype": "DATE",
          "length": null,
          "scale": null,

```

```

    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PATIENT_FIRST_NAME_0405",
    "datatype": "CHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PATIENT_LAST_NAME_0405",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PATIENT_SEX_0405",
    "datatype": "CHAR",
    "length": 1,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "RELATION_TO_EMPLOYEE_0405",
    "datatype": "CHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_FIRST_NAME_0405",
    "datatype": "CHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_LAST_NAME_0405",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_STREET_0405",
    "datatype": "CHAR",
    "length": 20,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_CITY_0405",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DENTIST_STATE_0405",
    "datatype": "CHAR",
    "length": 2,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {

```

```

        "name": "DENTIST_ZIP_FIRST_FIVE_0405",
        "datatype": "CHAR",
        "length": 5,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DENTIST_ZIP_LAST_FOUR_0405",
        "datatype": "CHAR",
        "length": 4,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DENTIST_LICENSE_NUMBER_0405",
        "datatype": "DECIMAL",
        "length": 6,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "NUMBER_OF_PROCEDURES_0405",
        "datatype": "DECIMAL",
        "length": 2,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "TOOTH_NUMBER_0405",
        "datatype": "DECIMAL",
        "length": 2,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "PROCEDURE_CODE_0405",
        "datatype": "DECIMAL",
        "length": 4,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DESCRIPTION_OF_SERVICE_0405",
        "datatype": "CHAR",
        "length": 60,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "FEE_0405",
        "datatype": "DECIMAL",
        "length": 5,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "DEPARTMENT",
    "primary_key": "DEPT_ID_0410",
    "primary_keys": [
        "DEPT_ID_0410"
    ],
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "DEPT_ID_0410",

```

```

        "datatype": "DECIMAL",
        "length": 4,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DEPT_NAME_0410",
        "datatype": "CHAR",
        "length": 45,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DEPT_HEAD_ID_0410",
        "datatype": "DECIMAL",
        "length": 4,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "EMPLOYEE",
    "primary_key": "EMP_ID_0415",
    "primary_keys": [
        "EMP_ID_0415"
    ],
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "START_DATE_0415",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "TERMINATION_DATE_0415",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "BIRTH_DATE_0415",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "EMP_ID_0415",
            "datatype": "DECIMAL",
            "length": 4,
            "scale": 0,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "EMP_FIRST_NAME_0415",
            "datatype": "CHAR",
            "length": 10,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "EMP_LAST_NAME_0415",
            "datatype": "CHAR",

```

```

        "length": 15,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_STREET_0415",
        "datatype": "CHAR",
        "length": 20,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_CITY_0415",
        "datatype": "CHAR",
        "length": 15,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_STATE_0415",
        "datatype": "CHAR",
        "length": 2,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_ZIP_FIRST_FIVE_0415",
        "datatype": "CHAR",
        "length": 5,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_ZIP_LAST_FOUR_0415",
        "datatype": "CHAR",
        "length": 4,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "EMP_PHONE_0415",
        "datatype": "DECIMAL",
        "length": 10,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "STATUS_0415",
        "datatype": "CHAR",
        "length": 2,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "SS_NUMBER_0415",
        "datatype": "DECIMAL",
        "length": 9,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "EMPOSITION",
    "primary_key": null,
    "primary_keys": [],
    "record_id": null,

```

```

"location_mode": null,
"fields": [
  {
    "name": "START_DATE_0420",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "FINISH_DATE_0420",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "SALARY_GRADE_0420",
    "datatype": "DECIMAL",
    "length": 2,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "SALARY_AMOUNT_0420",
    "datatype": "DECIMAL",
    "length": 5,
    "scale": 2,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "BONUS_PERCENT_0420",
    "datatype": "DECIMAL",
    "length": 2,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "COMMISSION_PERCENT_0420",
    "datatype": "DECIMAL",
    "length": 2,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "OVERTIME_RATE_0420",
    "datatype": "DECIMAL",
    "length": 2,
    "scale": 2,
    "occurs": false,
    "occurs_max": null
  }
]
},
{
  "name": "EXPERTISE",
  "primary_key": null,
  "primary_keys": [],
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "EXPERTISE_DATE_0425",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ],
  {

```

```

        "name": "SKILL_LEVEL_0425",
        "datatype": "CHAR",
        "length": 2,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "HOSPITAL_CLAIM",
    "primary_key": null,
    "primary_keys": [],
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "CLAIM_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "PATIENT_BIRTH_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "ADMIT_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "DISCHARGE_DATE_0430",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "PATIENT_FIRST_NAME_0430",
            "datatype": "CHAR",
            "length": 10,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "PATIENT_LAST_NAME_0430",
            "datatype": "CHAR",
            "length": 15,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "PATIENT_SEX_0430",
            "datatype": "CHAR",
            "length": 1,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "RELATION_TO_EMPLOYEE_0430",
            "datatype": "CHAR",
            "length": 10,

```

```

    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSPITAL_NAME_0430",
    "datatype": "CHAR",
    "length": 25,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSP_STREET_0430",
    "datatype": "CHAR",
    "length": 20,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSP_CITY_0430",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSP_STATE_0430",
    "datatype": "CHAR",
    "length": 2,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSP_ZIP_FIRST_FIVE_0430",
    "datatype": "CHAR",
    "length": 5,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "HOSP_ZIP_LAST_FOUR_0430",
    "datatype": "CHAR",
    "length": 4,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DIAGNOSIS_0430",
    "datatype": null,
    "length": 120,
    "scale": null,
    "occurs": true,
    "occurs_max": 2
  },
  {
    "name": "WARD_DAYS_0430",
    "datatype": "DECIMAL",
    "length": 3,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "WARD_RATE_0430",
    "datatype": "DECIMAL",
    "length": 5,
    "scale": 2,
    "occurs": false,
    "occurs_max": null
  },

```



```

{
  "name": "WARD_TOTAL_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "SEMI_DAYS_0430",
  "datatype": "DECIMAL",
  "length": 3,
  "scale": 0,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "SEMI_RATE_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "SEMI_TOTAL_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "DELIVERY_COST_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "ANESTHESIA_COST_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
},
{
  "name": "LAB_COST_0430",
  "datatype": "DECIMAL",
  "length": 5,
  "scale": 2,
  "occurs": false,
  "occurs_max": null
}
],
{
  "name": "INSURANCE_PLAN",
  "primary_key": "INS_PLAN_CODE_0435",
  "primary_keys": [
    "INS_PLAN_CODE_0435"
  ],
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "INS_PLAN_CODE_0435",
      "datatype": "CHAR",
      "length": 3,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ],
  {

```

```

    "name": "INS_CO_NAME_0435",
    "datatype": "CHAR",
    "length": 45,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_STREET_0435",
    "datatype": "CHAR",
    "length": 20,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_CITY_0435",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_STATE_0435",
    "datatype": "CHAR",
    "length": 2,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_ZIP_FIRST_FIVE_0435",
    "datatype": "CHAR",
    "length": 5,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_ZIP_LAST_FOUR_0435",
    "datatype": "CHAR",
    "length": 4,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "INS_CO_PHONE_0435",
    "datatype": "DECIMAL",
    "length": 10,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "GROUP_NUMBER_0435",
    "datatype": "DECIMAL",
    "length": 6,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "DEDUCT_0435",
    "datatype": "DECIMAL",
    "length": 5,
    "scale": 2,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "MAXIMUM_LIFE_COST_0435",
    "datatype": "DECIMAL",
    "length": 5,
    "scale": 2,

```

```

        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "FAMILY_COST_0435",
        "datatype": "DECIMAL",
        "length": 5,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DEP_COST_0435",
        "datatype": "DECIMAL",
        "length": 5,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    }
]
},
{
    "name": "JOB",
    "primary_key": "JOB_ID_0440",
    "primary_keys": [
        "JOB_ID_0440"
    ],
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "JOB_ID_0440",
            "datatype": "DECIMAL",
            "length": 4,
            "scale": 0,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "TITLE_0440",
            "datatype": "CHAR",
            "length": 20,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "DESCRIPTION_LINE_0440",
            "datatype": null,
            "length": 120,
            "scale": null,
            "occurs": true,
            "occurs_max": 2
        },
        {
            "name": "REQUIREMENT_LINE_0440",
            "datatype": null,
            "length": 120,
            "scale": null,
            "occurs": true,
            "occurs_max": 2
        },
        {
            "name": "MINIMUM_SALARY_0440",
            "datatype": "DECIMAL",
            "length": 8,
            "scale": 2,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "MAXIMUM_SALARY_0440",
            "datatype": "DECIMAL",
            "length": 8,
            "scale": 2,
            "occurs": false,

```

```

    "occurs_max": null
  },
  {
    "name": "SALARY_GRADES_0440",
    "datatype": null,
    "length": 8,
    "scale": null,
    "occurs": true,
    "occurs_max": 4
  },
  {
    "name": "NUMBER_OF_POSITIONS_0440",
    "datatype": "DECIMAL",
    "length": 3,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "NUMBER_OPEN_0440",
    "datatype": "DECIMAL",
    "length": 3,
    "scale": 0,
    "occurs": false,
    "occurs_max": null
  }
]
},
{
  "name": "NON_HOSP_CLAIM",
  "primary_key": null,
  "primary_keys": [],
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "CLAIM_DATE_0445",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "PATIENT_BIRTH_DATE_0445",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "SERVICE_DATE_0445",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "PATIENT_FIRST_NAME_0445",
      "datatype": "CHAR",
      "length": 10,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "PATIENT_LAST_NAME_0445",
      "datatype": "CHAR",
      "length": 15,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ],
  {

```

```

    "name": "PATIENT_SEX_0445",
    "datatype": "CHAR",
    "length": 1,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "RELATION_TO_EMPLOYEE_0445",
    "datatype": "CHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_FIRST_NAME_0445",
    "datatype": "CHAR",
    "length": 10,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_LAST_NAME_0445",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_STREET_0445",
    "datatype": "CHAR",
    "length": 20,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_CITY_0445",
    "datatype": "CHAR",
    "length": 15,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_STATE_0445",
    "datatype": "CHAR",
    "length": 2,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_ZIP_FIRST_FIVE_0445",
    "datatype": "CHAR",
    "length": 5,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_ZIP_LAST_FOUR_0445",
    "datatype": "CHAR",
    "length": 4,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "PHYSICIAN_ID_0445",
    "datatype": "DECIMAL",
    "length": 6,
    "scale": 0,

```

```

        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DIAGNOSIS_0445",
        "datatype": null,
        "length": 120,
        "scale": null,
        "occurs": true,
        "occurs_max": 2
    },
    {
        "name": "NUMBER_OF_PROCEDURES_0445",
        "datatype": "DECIMAL",
        "length": 2,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "PROCEDURE_CODE_0445",
        "datatype": "DECIMAL",
        "length": 4,
        "scale": 0,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "DESCRIPTION_OF_SERVICE_0445",
        "datatype": "CHAR",
        "length": 60,
        "scale": null,
        "occurs": false,
        "occurs_max": null
    },
    {
        "name": "FEE_0445",
        "datatype": "DECIMAL",
        "length": 5,
        "scale": 2,
        "occurs": false,
        "occurs_max": null
    }
}
],
{
    "name": "OFFICE",
    "primary_key": "OFFICE_CODE_0450",
    "primary_keys": [
        "OFFICE_CODE_0450"
    ],
    "record_id": null,
    "location_mode": null,
    "fields": [
        {
            "name": "OFFICE_CODE_0450",
            "datatype": "CHAR",
            "length": 3,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "OFFICE_STREET_0450",
            "datatype": "CHAR",
            "length": 20,
            "scale": null,
            "occurs": false,
            "occurs_max": null
        },
        {
            "name": "OFFICE_CITY_0450",
            "datatype": "CHAR",
            "length": 15,
            "scale": null,
            "occurs": false,

```

```

    "occurs_max": null
  },
  {
    "name": "OFFICE_STATE_0450",
    "datatype": "CHAR",
    "length": 2,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "OFFICE_ZIP_FIRST_FIVE_0450",
    "datatype": "CHAR",
    "length": 5,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "OFFICE_ZIP_LAST_FOUR_0450",
    "datatype": "CHAR",
    "length": 4,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "OFFICE_PHONE_0450",
    "datatype": null,
    "length": 21,
    "scale": null,
    "occurs": true,
    "occurs_max": 3
  },
  {
    "name": "OFFICE_AREA_CODE_0450",
    "datatype": "CHAR",
    "length": 3,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  },
  {
    "name": "SPEED_DIAL_0450",
    "datatype": "CHAR",
    "length": 3,
    "scale": null,
    "occurs": false,
    "occurs_max": null
  }
]
},
{
  "name": "SKILL",
  "primary_key": "SKILL_ID_0455",
  "primary_keys": [
    "SKILL_ID_0455"
  ],
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "SKILL_ID_0455",
      "datatype": "DECIMAL",
      "length": 4,
      "scale": 0,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "SKILL_NAME_0455",
      "datatype": "CHAR",
      "length": 12,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ]
}

```

```

    },
    {
      "name": "SKILL_DESCRIPTION_0455",
      "datatype": "CHAR",
      "length": 60,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ]
},
{
  "name": "STRUCTURE",
  "primary_key": null,
  "primary_keys": [],
  "record_id": null,
  "location_mode": null,
  "fields": [
    {
      "name": "STRUCTURE_DATE_0460",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    },
    {
      "name": "STRUCTURE_CODE_0460",
      "datatype": "CHAR",
      "length": 2,
      "scale": null,
      "occurs": false,
      "occurs_max": null
    }
  ]
}
],
"sets": [
  {
    "name": "COVERAGE_CLAIMS",
    "owner_record": "COVERAGE",
    "member_record": "DENTAL_CLAIM",
    "order_mode": null
  },
  {
    "name": "COVERAGE_CLAIMS",
    "owner_record": "COVERAGE",
    "member_record": "HOSPITAL_CLAIM",
    "order_mode": null
  },
  {
    "name": "COVERAGE_CLAIMS",
    "owner_record": "COVERAGE",
    "member_record": "NON_HOSP_CLAIM",
    "order_mode": null
  },
  {
    "name": "DEPT_EMPLOYEE",
    "owner_record": "DEPARTMENT",
    "member_record": "EMPLOYEE",
    "order_mode": null
  },
  {
    "name": "EMP_COVERAGE",
    "owner_record": "EMPLOYEE",
    "member_record": "COVERAGE",
    "order_mode": null
  },
  {
    "name": "EMP_EMPOSITION",
    "owner_record": "EMPLOYEE",
    "member_record": "EMPOSITION",
    "order_mode": null
  },
  {
    "name": "EMP_EXPERTISE",

```



```

    "owner_record": "EMPLOYEE",
    "member_record": "EXPERTISE",
    "order_mode": null
  },
  {
    "name": "JOB_EMPOSITION",
    "owner_record": "JOB",
    "member_record": "EMPOSITION",
    "order_mode": null
  },
  {
    "name": "MANAGES",
    "owner_record": "EMPLOYEE",
    "member_record": "STRUCTURE",
    "order_mode": null
  },
  {
    "name": "OFFICE_EMPLOYEE",
    "owner_record": "OFFICE",
    "member_record": "EMPLOYEE",
    "order_mode": null
  },
  {
    "name": "REPORTS_TO",
    "owner_record": "EMPLOYEE",
    "member_record": "STRUCTURE",
    "order_mode": null
  },
  {
    "name": "SKILL_EXPERTISE",
    "owner_record": "SKILL",
    "member_record": "EXPERTISE",
    "order_mode": null
  }
],
"relationships": [
  {
    "set_name": "COVERAGE_CLAIMS",
    "parent_record": "COVERAGE",
    "child_record": "DENTAL_CLAIM",
    "cardinality": "1:N"
  },
  {
    "set_name": "COVERAGE_CLAIMS",
    "parent_record": "COVERAGE",
    "child_record": "HOSPITAL_CLAIM",
    "cardinality": "1:N"
  },
  {
    "set_name": "COVERAGE_CLAIMS",
    "parent_record": "COVERAGE",
    "child_record": "NON_HOSP_CLAIM",
    "cardinality": "1:N"
  },
  {
    "set_name": "DEPT_EMPLOYEE",
    "parent_record": "DEPARTMENT",
    "child_record": "EMPLOYEE",
    "cardinality": "1:N"
  },
  {
    "set_name": "EMP_COVERAGE",
    "parent_record": "EMPLOYEE",
    "child_record": "COVERAGE",
    "cardinality": "1:N"
  },
  {
    "set_name": "EMP_EMPOSITION",
    "parent_record": "EMPLOYEE",
    "child_record": "EMPOSITION",
    "cardinality": "1:N"
  },
  {
    "set_name": "EMP_EXPERTISE",
    "parent_record": "EMPLOYEE",
    "child_record": "EXPERTISE",

```

```

    "cardinality": "1:N"
  },
  {
    "set_name": "JOB_EMPOSITION",
    "parent_record": "JOB",
    "child_record": "EMPOSITION",
    "cardinality": "1:N"
  },
  {
    "set_name": "MANAGES",
    "parent_record": "EMPLOYEE",
    "child_record": "STRUCTURE",
    "cardinality": "1:N"
  },
  {
    "set_name": "OFFICE_EMPLOYEE",
    "parent_record": "OFFICE",
    "child_record": "EMPLOYEE",
    "cardinality": "1:N"
  },
  {
    "set_name": "REPORTS_TO",
    "parent_record": "EMPLOYEE",
    "child_record": "STRUCTURE",
    "cardinality": "1:N"
  },
  {
    "set_name": "SKILL_EXPERTISE",
    "parent_record": "SKILL",
    "child_record": "EXPERTISE",
    "cardinality": "1:N"
  }
]
}

```

```

=====
FILE: data\output\converted_db2_cobol.cb1
ENCODING: utf-8
=====

```

```

*RETRIEVAL
*DMLIST
*NO-ACTIVITY-LOG
*SCHEMA-COMMENTS
IDENTIFICATION DIVISION.
PROGRAM-ID. DEPTRPT.
AUTHOR. CA.
DATE-WRITTEN. APRIL 1995.
REMARKS. THIS PROGRAM DEMONSTRATES
CA IDMS DATABASE ACCESS USING
COBOL DML STATEMENTS. IT READS
DEPARTMENT ID NUMBERS AND RETRIEVES
RELATED RECORD OCCURRENCES,
PRINTING A REPORT THAT INCLUDES
DEPARTMENT, EMPLOYEE, JOB, AND
OFFICE INFORMATION.
*****
ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
SELECT DEPT-FILE-IN ASSIGN TO INFILE.
SELECT DEPT-FILE-OUT ASSIGN TO OUTFILE.
SELECT ERR-FILE-OUT ASSIGN TO ERRFILE.
*****

DATA DIVISION.

FILE SECTION.
FD DEPT-FILE-IN
RECORD CONTAINS 80
BLOCK CONTAINS 80 CHARACTERS
RECORDING MODE IS F
LABEL RECORDS ARE OMITTED.
01 DEPT-REC-IN.
02 DEPT-ID-IN PIC 9(4).

```

```

02 DEPT-IN-FILLER PIC X(76).
FD DEPT-FILE-OUT
RECORD CONTAINS 133
BLOCK CONTAINS 133 CHARACTERS
RECORDING MODE IS F
LABEL RECORDS ARE OMITTED.
01 DEPT-REC-OUT.
02 CC PIC X.
02 PRINT-LINE PIC X(132).
FD ERR-FILE-OUT
RECORD CONTAINS 133
BLOCK CONTAINS 133 CHARACTERS
RECORDING MODE IS F
LABEL RECORDS ARE OMITTED.
01 ERR-REC-OUT.
02 ERR-CC PIC X.
02 ERR-LINE PIC X(132).
*****
WORKING-STORAGE SECTION.
01 EOF-SW PIC X VALUE 'N'.
88 END-OF-FILE VALUE 'Y'.
01 LINE-COUNT PIC 99 VALUE 0.
01 ERR-LINE-COUNT PIC 99 VALUE 0.
01 LINE-MAX PIC 99 VALUE 50.
*****
01 DEPT-HEADER.
05 FILLER PIC X(30) VALUE SPACES.
05 FILLER PIC X(13) VALUE 'DEPARTMENT ID'.
05 FILLER PIC X(10) VALUE SPACES.
05 FILLER PIC X(9) VALUE 'DEPT NAME'.
05 FILLER PIC X(70) VALUE SPACES.
01 DEPT-DETAIL-LINE.
05 FILLER PIC X(33) VALUE SPACES.
05 DEPT-ID-OUT PIC X(4).
05 FILLER PIC X(16) VALUE SPACES.
05 DEPT-NAME-OUT PIC X(45).
05 FILLER PIC X(34) VALUE SPACES.
01 EMP-HEADER.
05 FILLER PIC X(5) VALUE SPACES.
05 FILLER PIC X(6) VALUE 'EMP ID'.
05 FILLER PIC X(2) VALUE SPACES.
05 FILLER PIC X(9) VALUE 'LAST NAME'.
05 FILLER PIC X(8) VALUE SPACES.
05 FILLER PIC X(10) VALUE 'FIRST NAME'.
05 FILLER PIC X(3) VALUE SPACES.
05 FILLER PIC X(10) VALUE 'START DATE'.
05 FILLER PIC X(2) VALUE SPACES.
05 FILLER PIC X(9) VALUE 'JOB TITLE'.
05 FILLER PIC X(13) VALUE SPACES.
05 FILLER PIC X(14) VALUE 'OFFICE ADDRESS'.
05 FILLER PIC X(42) VALUE SPACES.
01 EMP-DETAIL-LINE.
05 FILLER PIC X(5) VALUE SPACES.
05 ID-OUT PIC X(4).
05 FILLER PIC X(4) VALUE SPACES.
05 LAST-OUT PIC X(15).
05 FILLER PIC X(2) VALUE SPACES.
05 FIRST-OUT PIC X(10).
05 FILLER PIC X(3) VALUE SPACES.
05 SD-OUT.
10 SD-MM PIC XX.
10 FILLER PIC X VALUE '/'.
10 SD-DD PIC XX.
10 FILLER PIC X VALUE '/'.
10 SD-YY PIC XX.
05 FILLER PIC X(4) VALUE SPACES.
05 TITLE-OUT PIC X(20).
05 FILLER PIC X(2) VALUE SPACES.
05 OFF-ADDRESS-OUT.
10 STREET-OUT PIC X(20).
10 FILLER PIC XX VALUE SPACES.
10 CITY-OUT PIC X(15).
10 FILLER PIC XX VALUE SPACES.
10 STATE-OUT PIC XX.
10 FILLER PIC XX VALUE SPACES.
10 ZIP-OUT PIC X(5).

```

```

05 FILLER PIC X(8) VALUE SPACES.
01 ERR-HEADER-1.
05 FILLER PIC X(40) VALUE SPACES.
05 FILLER PIC X(12) VALUE 'ERROR REPORT'.
05 FILLER PIC X(80) VALUE SPACES.
01 ERR-HEADER-2.
05 FILLER PIC X(10) VALUE SPACES.
05 FILLER PIC X(4) VALUE '*** '.
05 FILLER PIC X(51)
VALUE 'THIS REPORT LISTS EMPTY AND NONEXISTENT DEPARTMENTS'.
05 FILLER PIC X(4) VALUE ' ***'.
05 FILLER PIC X(63) VALUE SPACES.
01 ERR-HEADER-3.
05 FILLER PIC X(20) VALUE SPACES.
05 FILLER PIC X(7) VALUE 'DEPT ID'.
05 FILLER PIC X(9) VALUE SPACES.
05 FILLER PIC X(7) VALUE 'MESSAGE'.
05 FILLER PIC X(89) VALUE SPACES.
01 ERR-DETAIL-LINE.
05 FILLER PIC X(20) VALUE SPACES.
05 ERR-ID-OUT PIC X(4).
05 FILLER PIC X(12) VALUE SPACES.
05 ERR-MESS-OUT PIC X(15).
05 FILLER PIC X(79) VALUE SPACES.
*****
01 MESSAGES.
05 NO-JOB-MESSAGE.
10 FILLER PIC X(20)
VALUE 'NO JOB ASSIGNED'.
05 NO-OFFICE-MESSAGE.
10 FILLER PIC X(20)
VALUE 'NO OFFICE ASSIGNED'.
05 NO-DEPT-MESSAGE.
10 FILLER PIC X(15)
VALUE 'DOES NOT EXIST'.
05 NO-EMP-MESSAGE.
10 FILLER PIC X(15)
VALUE 'IS EMPTY'.
05 NO-INPUT-MESSAGE.
10 FILLER PIC XX VALUE SPACES.
10 FILLER PIC X(11)
VALUE '=====> '.
10 FILLER PIC X(8)
VALUE 'NO INPUT'.
10 FILLER PIC X(11)
VALUE ' <====='.
10 FILLER PIC X(100)
VALUE SPACES.

EJECT
*****
* DB2 SQLCA, HOST VARIABLES, AND CURSORS
*****

EXEC SQL
    INCLUDE SQLCA
END-EXEC.
EXEC SQL
    BEGIN DECLARE SECTION
END-EXEC.

01 HV-DEPARTMENT.
    05 HV-DEPARTMENT-DEPT-ID-479MENT          PIC S9(4) COMP-3.
    05 HV-DEPARTMENT-DEPT-NAME-479MENT        PIC X(45).
    05 HV-DEPARTMENT-DEPT-HEADID-479MENT      PIC S9(4) COMP-3.

01 HV-EMPLOYEE.
    05 HV-EMPLOYEE-DA-STARTDATE-479OYEE      PIC X(10).
    05 HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE PIC X(10).
    05 HV-EMPLOYEE-DA-BIRTHDATE-479OYEE      PIC X(10).
    05 HV-EMPLOYEE-EMP-ID-479OYEE            PIC S9(4) COMP-3.
    05 HV-EMPLOYEE-EMP-FIRSTNAME-479OYEE     PIC X(10).
    05 HV-EMPLOYEE-EMP-LASTNAME-479OYEE      PIC X(15).
    05 HV-EMPLOYEE-EMP-STREET-479OYEE        PIC X(20).
    05 HV-EMPLOYEE-EMP-CITY-479OYEE          PIC X(15).
    05 HV-EMPLOYEE-EMP-STATE-479OYEE         PIC X(2).

```

05 HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE	PIC X(5).
05 HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE	PIC X(4).
05 HV-EMPLOYEE-EMP-PHONE-4790YEE	PIC S9(10) COMP-3.
05 HV-EMPLOYEE-STATUS-4790YEE	PIC X(2).
05 HV-EMPLOYEE-SS-NUMBER-4790YEE	PIC S9(9) COMP-3.
05 HV-EMPLOYEE-DEPT-ID-479MENT	PIC S9(4) COMP-3.
05 HV-EMPLOYEE-OFFICE-CODE-479FICE	PIC X(3).
01 HV-EMPOSITION.	
05 HV-EMPOSITION-ID-RECORD-TION	PIC X(20).
05 HV-EMPOSITION-DA-STARTDATE-479TION	PIC X(10).
05 HV-EMPOSITION-DA-FINISHDATE-479TION	PIC X(10).
05 HV-EMPOSITION-SALARY-GRADE-479TION	PIC S9(2) COMP-3.
05 HV-EMPOSITION-SALARY-AMOUNT-479TION	PIC S9(3)V9(2) COMP-3.
05 HV-EMPOSITION-BONUS-PERCENT-479TION	PIC S9(2) COMP-3.
05 HV-EMPOSITION-COMMISSION-PERCENT-479TION	PIC S9(2) COMP-3.
05 HV-EMPOSITION-OVERTIME-RATE-479TION	PIC S9(1)V9(2) COMP-3.
05 HV-EMPOSITION-EMP-ID-4790YEE	PIC S9(4) COMP-3.
05 HV-EMPOSITION-JOB-ID-479JOB	PIC S9(4) COMP-3.
01 HV-JOB.	
05 HV-JOB-JOB-ID-479JOB	PIC S9(4) COMP-3.
05 HV-JOB-TITLE-479JOB	PIC X(20).
05 HV-JOB-DESCRIPTION-LINE-479JOB	PIC X(120).
05 HV-JOB-REQUIREMENT-LINE-479JOB	PIC X(120).
05 HV-JOB-MINIMUM-SALARY-479JOB	PIC S9(6)V9(2) COMP-3.
05 HV-JOB-MAXIMUM-SALARY-479JOB	PIC S9(6)V9(2) COMP-3.
05 HV-JOB-SALARY-GRADES-479JOB	PIC X(8).
05 HV-JOB-NUMBER-OFPOSITIONS-479JOB	PIC S9(3) COMP-3.
05 HV-JOB-NUMBER-OPEN-479JOB	PIC S9(3) COMP-3.
01 HV-OFFICE.	
05 HV-OFFICE-OFFICE-CODE-479FICE	PIC X(3).
05 HV-OFFICE-OFFICE-STREET-479FICE	PIC X(20).
05 HV-OFFICE-OFFICE-CITY-479FICE	PIC X(15).
05 HV-OFFICE-OFFICE-STATE-479FICE	PIC X(2).
05 HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE	PIC X(5).
05 HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE	PIC X(4).
05 HV-OFFICE-OFFICE-PHONE-479FICE	PIC X(21).
05 HV-OFFICE-OFFICE-AREACODE-479FICE	PIC X(3).
05 HV-OFFICE-SPEED-DIAL-479FICE	PIC X(3).
01 NI-DEPARTMENT-DEPT-NAME-479MENT	PIC S9(4) COMP.
01 NI-DEPARTMENT-DEPT-HEADID-479MENT	PIC S9(4) COMP.
01 NI-EMPLOYEE-DA-STARTDATE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-DA-TERMINATIONDATE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-DA-BIRTHDATE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-FIRSTNAME-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-LASTNAME-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-STREET-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-CITY-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-STATE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-EMP-PHONE-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-STATUS-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-SS-NUMBER-4790YEE	PIC S9(4) COMP.
01 NI-EMPLOYEE-DEPT-ID-479MENT	PIC S9(4) COMP.
01 NI-EMPLOYEE-OFFICE-CODE-479FICE	PIC S9(4) COMP.
01 NI-EMPOSITION-DA-STARTDATE-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-DA-FINISHDATE-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-SALARY-GRADE-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-SALARY-AMOUNT-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-BONUS-PERCENT-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-COMMISSION-PERCENT-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-OVERTIME-RATE-479TION	PIC S9(4) COMP.
01 NI-EMPOSITION-EMP-ID-4790YEE	PIC S9(4) COMP.
01 NI-EMPOSITION-JOB-ID-479JOB	PIC S9(4) COMP.
01 NI-JOB-TITLE-479JOB	PIC S9(4) COMP.
01 NI-JOB-DESCRIPTION-LINE-479JOB	PIC S9(4) COMP.
01 NI-JOB-REQUIREMENT-LINE-479JOB	PIC S9(4) COMP.
01 NI-JOB-MINIMUM-SALARY-479JOB	PIC S9(4) COMP.
01 NI-JOB-MAXIMUM-SALARY-479JOB	PIC S9(4) COMP.
01 NI-JOB-SALARY-GRADES-479JOB	PIC S9(4) COMP.
01 NI-JOB-NUMBER-OFPOSITIONS-479JOB	PIC S9(4) COMP.
01 NI-JOB-NUMBER-OPEN-479JOB	PIC S9(4) COMP.

```

01 NI-OFFICE-OFFICE-STREET-479FICE          PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-CITY-479FICE            PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-STATE-479FICE           PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE    PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-ZIPLASTFOUR-479FICE     PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-PHONE-479FICE          PIC S9(4) COMP.
01 NI-OFFICE-OFFICE-AREACODE-479FICE        PIC S9(4) COMP.
01 NI-OFFICE-SPEED-DIAL-479FICE             PIC S9(4) COMP.

```

EXEC SQL

END DECLARE SECTION

END-EXEC.

EXEC SQL

DECLARE C-DEPT-EMPLOYEE CURSOR FOR

```

SELECT DA_STARTDATE_479OYEE,
       DA_TERMINATIONDATE_479OYEE,
       DA_BIRTHDATE_479OYEE,
       EMP_ID_479OYEE,
       EMP_FIRSTNAME_479OYEE,
       EMP_LASTNAME_479OYEE,
       EMP_STREET_479OYEE,
       EMP_CITY_479OYEE,
       EMP_STATE_479OYEE,
       EMP_ZIPFIRSTFIVE_479OYEE,
       EMP_ZIPLASTFOUR_479OYEE,
       EMP_PHONE_479OYEE,
       STATUS_479OYEE,
       SS_NUMBER_479OYEE,
       DEPT_ID_479MENT,
       OFFICE_CODE_479FICE

```

FROM EMPLOYEE

WHERE DEPT_ID_479MENT = :HV-DEPARTMENT-DEPT-ID-479MENT

ORDER BY DEPT_ID_479MENT

END-EXEC.

PROCEDURE DIVISION.

* PROCEDURE DIVISION GENERAL STRATEGY: *

* *

* 1) READ DEPT-ID-IN, WHICH CONTAINS THE *

* DEPT-ID NUMBER *

* *

* 2) ACCESS THE DATABASE USING THE DEPT-ID NUMBER *

* WITH AN OBTAIN CALC ON THE DEPARTMENT RECORD *

* *

* 3) ACCESS ALL EMPLOYEES IN THE DEPT-EMPLOYEE SET *

* AND RETRIEVE RELATED JOB AND OFFICE DATA *

* *

* 4) PRINT A REPORT FOR EACH DEPARTMENT *

* *

* 5) PRINT AN ERROR REPORT FOR EMPTY DEPARTMENTS *

* AND NONEXISTENT DEPARTMENTS (NO MATCHING *

* DEPT-ID) *

FETCH-DEPT-EMPLOYEE.

EXEC SQL

FETCH C-DEPT-EMPLOYEE

```

INTO :HV-EMPLOYEE-DA-STARTDATE-479OYEE :NI-EMPLOYEE-DA-STARTDATE-479OYEE,
     :HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE :NI-EMPLOYEE-DA-TERMINATIONDATE-479OYEE,
     :HV-EMPLOYEE-DA-BIRTHDATE-479OYEE :NI-EMPLOYEE-DA-BIRTHDATE-479OYEE,
     :HV-EMPLOYEE-EMP-ID-479OYEE,
     :HV-EMPLOYEE-EMP-FIRSTNAME-479OYEE :NI-EMPLOYEE-EMP-FIRSTNAME-479OYEE,
     :HV-EMPLOYEE-EMP-LASTNAME-479OYEE :NI-EMPLOYEE-EMP-LASTNAME-479OYEE,
     :HV-EMPLOYEE-EMP-STREET-479OYEE :NI-EMPLOYEE-EMP-STREET-479OYEE,
     :HV-EMPLOYEE-EMP-CITY-479OYEE :NI-EMPLOYEE-EMP-CITY-479OYEE,
     :HV-EMPLOYEE-EMP-STATE-479OYEE :NI-EMPLOYEE-EMP-STATE-479OYEE,
     :HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-479OYEE :NI-EMPLOYEE-EMP-ZIPFIRSTFIVE-479OYEE,
     :HV-EMPLOYEE-EMP-ZIPLASTFOUR-479OYEE :NI-EMPLOYEE-EMP-ZIPLASTFOUR-479OYEE,
     :HV-EMPLOYEE-EMP-PHONE-479OYEE :NI-EMPLOYEE-EMP-PHONE-479OYEE,
     :HV-EMPLOYEE-STATUS-479OYEE :NI-EMPLOYEE-STATUS-479OYEE,
     :HV-EMPLOYEE-SS-NUMBER-479OYEE :NI-EMPLOYEE-SS-NUMBER-479OYEE,
     :HV-EMPLOYEE-DEPT-ID-479MENT :NI-EMPLOYEE-DEPT-ID-479MENT,
     :HV-EMPLOYEE-OFFICE-CODE-479FICE :NI-EMPLOYEE-OFFICE-CODE-479FICE

```

END-EXEC.

IF SQLCODE NOT = 0 AND SQLCODE NOT = 100

PERFORM SQL-ERROR

END-IF.

```

MAIN-LINE.
    PERFORM INIT-FILES.
    IF END-OF-FILE
        PERFORM EMPTY-INPUT-PROCESSING
    ELSE
        PERFORM INIT-DB2-SETUP
        PERFORM U220-ERR-HEADER
        PERFORM DEPT-PROCESSING THRU DEPT-PROCESSING-EXIT
        UNTIL END-OF-FILE.
        PERFORM END-PROCESSING.
        GOBACK.
INIT-DB2-SETUP.
    CONTINUE.
INIT-FILES.
    OPEN INPUT DEPT-FILE-IN.
    OPEN OUTPUT DEPT-FILE-OUT.
    OPEN OUTPUT ERR-FILE-OUT.
    MOVE SPACES TO PRINT-LINE.
    MOVE SPACES TO ERR-LINE.
    READ DEPT-FILE-IN
        AT END MOVE 'Y' TO EOF-SW.
EMPTY-INPUT-PROCESSING.
    MOVE NO-INPUT-MESSAGE TO PRINT-LINE.
    MOVE '1' TO CC.
    PERFORM U000-WRITE-LINE.
*****
* THIS PARAGRAPH ACCESSES THE DATABASE USING THE DEPT-ID-0415 *
* CALCKEY VALUE. *
*****
DEPT-PROCESSING.
    MOVE DEPT-ID-IN TO HV-DEPARTMENT-DEPT-ID-479MENT.
    EXEC SQL
        SELECT DEPT_ID_479MENT,
            DEPT_NAME_479MENT,
            DEPT_HEADID_479MENT
        INTO :HV-DEPARTMENT-DEPT-ID-479MENT,
            :HV-DEPARTMENT-DEPT-NAME-479MENT :NI-DEPARTMENT-DEPT-NAME-479MENT,
            :HV-DEPARTMENT-DEPT-HEADID-479MENT :NI-DEPARTMENT-DEPT-HEADID-479MENT
        FROM DEPARTMENT
        WHERE DEPT_ID_479MENT = :HV-DEPARTMENT-DEPT-ID-479MENT
    END-EXEC.
    IF SQLCODE = 100
        PERFORM NO-DEPT-PROCESSING
    ELSE IF SQLCODE NOT = 0
        PERFORM SQL-ERROR
    ELSE
        EXEC SQL
            OPEN C-DEPT-EMPLOYEE
        END-EXEC.
        IF SQLCODE NOT = 0
            PERFORM SQL-ERROR
        ELSE
            PERFORM FETCH-DEPT-EMPLOYEE

    IF SQLCODE = 100
        PERFORM EMPTY-SET
    ELSE
        PERFORM U020-VALID-HEADER
        MOVE HV-DEPARTMENT-DEPT-ID-479MENT TO DEPT-ID-OUT
        MOVE HV-DEPARTMENT-DEPT-NAME-479MENT TO DEPT-NAME-OUT
        MOVE DEPT-DETAIL-LINE TO PRINT-LINE
        PERFORM U000-WRITE-LINE
        PERFORM U030-EMP-HEADERS

    PERFORM UNTIL SQLCODE = 100
        PERFORM SET-WALK THRU SET-WALK-EXIT
        PERFORM FETCH-DEPT-EMPLOYEE
    END-PERFORM.
    END-IF.
    EXEC SQL
        CLOSE C-DEPT-EMPLOYEE
    END-EXEC.
    END-IF.
    END-IF.
    END-IF.
    READ DEPT-FILE-IN

```

AT END MOVE 'Y' TO EOF-SW.
DEPT-PROCESSING-EXIT.
EXIT.

* THIS PARAGRAPH RETRIEVES EMPLOYEE, JOB, AND OFFICE DATA *
* FOR EACH EMPLOYEE IN THE DEPT-EMPLOYEE SET. *

SET-WALK.

CONTINUE.

MOVE HV-EMPLOYEE-EMP-ID-4790YEE TO ID-OUT.
MOVE HV-EMPLOYEE-EMP-LASTNAME-4790YEE TO LAST-OUT.
MOVE HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE TO FIRST-OUT.
MOVE HV-EMPLOYEE-DA-STARTDATE-4790YEE(3:2) TO SD-YY.
MOVE HV-EMPLOYEE-DA-STARTDATE-4790YEE(6:2) TO SD-MM.
MOVE HV-EMPLOYEE-DA-STARTDATE-4790YEE(9:2) TO SD-DD.

EXEC SQL

SELECT ID_RECORD_TION,
DA_STARTDATE_479TION,
DA_FINISHDATE_479TION,
SALARY_GRADE_479TION,
SALARY_AMOUNT_479TION,
BONUS_PERCENT_479TION,
COMMISSION_PERCENT_479TION,
OVERTIME_RATE_479TION,
EMP_ID_4790YEE,
JOB_ID_479JOB
INTO :HV-EMPOSITION-ID-RECORD-TION,
:HV-EMPOSITION-DA-STARTDATE-479TION :NI-EMPOSITION-DA-STARTDATE-479TION,
:HV-EMPOSITION-DA-FINISHDATE-479TION :NI-EMPOSITION-DA-FINISHDATE-479TION,
:HV-EMPOSITION-SALARY-GRADE-479TION :NI-EMPOSITION-SALARY-GRADE-479TION,
:HV-EMPOSITION-SALARY-AMOUNT-479TION :NI-EMPOSITION-SALARY-AMOUNT-479TION,
:HV-EMPOSITION-BONUS-PERCENT-479TION :NI-EMPOSITION-BONUS-PERCENT-479TION,
:HV-EMPOSITION-COMMISSION-PERCENT-479TION :NI-EMPOSITION-COMMISSION-PERCENT-479TION,
:HV-EMPOSITION-OVERTIME-RATE-479TION :NI-EMPOSITION-OVERTIME-RATE-479TION,
:HV-EMPOSITION-EMP-ID-4790YEE :NI-EMPOSITION-EMP-ID-4790YEE,
:HV-EMPOSITION-JOB-ID-479JOB :NI-EMPOSITION-JOB-ID-479JOB
FROM EMPOSITION
WHERE EMP_ID_4790YEE = :HV-EMPLOYEE-EMP-ID-4790YEE
ORDER BY EMP_ID_4790YEE
FETCH FIRST 1 ROW ONLY

END-EXEC.

IF SQLCODE = 100

MOVE NO-JOB-MESSAGE TO TITLE-OUT

ELSE IF SQLCODE NOT = 0

PERFORM SQL-ERROR

ELSE

EXEC SQL

SELECT JOB_ID_479JOB,
TITLE_479JOB,
DESCRIPTION_LINE_479JOB,
REQUIREMENT_LINE_479JOB,
MINIMUM_SALARY_479JOB,
MAXIMUM_SALARY_479JOB,
SALARY_GRADES_479JOB,
NUMBER_OFPOSITIONS_479JOB,
NUMBER_OPEN_479JOB
INTO :HV-JOB-JOB-ID-479JOB,
:HV-JOB-TITLE-479JOB :NI-JOB-TITLE-479JOB,
:HV-JOB-DESCRIPTION-LINE-479JOB :NI-JOB-DESCRIPTION-LINE-479JOB,
:HV-JOB-REQUIREMENT-LINE-479JOB :NI-JOB-REQUIREMENT-LINE-479JOB,
:HV-JOB-MINIMUM-SALARY-479JOB :NI-JOB-MINIMUM-SALARY-479JOB,
:HV-JOB-MAXIMUM-SALARY-479JOB :NI-JOB-MAXIMUM-SALARY-479JOB,
:HV-JOB-SALARY-GRADES-479JOB :NI-JOB-SALARY-GRADES-479JOB,
:HV-JOB-NUMBER-OFPOSITIONS-479JOB :NI-JOB-NUMBER-OFPOSITIONS-479JOB,
:HV-JOB-NUMBER-OPEN-479JOB :NI-JOB-NUMBER-OPEN-479JOB
FROM JOB
WHERE JOB_ID_479JOB = :HV-EMPOSITION-JOB-ID-479JOB

END-EXEC.

IF SQLCODE = 100

MOVE NO-JOB-MESSAGE TO TITLE-OUT

ELSE IF SQLCODE NOT = 0

PERFORM SQL-ERROR

ELSE

CONTINUE.

END-IF.

END-IF.


```

END-IF.
END-IF.
MOVE HV-JOB-TITLE-479JOB TO TITLE-OUT.
IF NI-EMPLOYEE-OFFICE-CODE-479FICE < 0
    MOVE NO-OFFICE-MESSAGE TO STREET-OUT
MOVE SPACES TO CITY-OUT
MOVE SPACES TO STATE-OUT
MOVE SPACES TO ZIP-OUT
ELSE
EXEC SQL
    SELECT OFFICE_CODE_479FICE,
           OFFICE_STREET_479FICE,
           OFFICE_CITY_479FICE,
           OFFICE_STATE_479FICE,
           OFFICE_ZIPFIRSTFIVE_479FICE,
           OFFICE_ZIPLASTFOUR_479FICE,
           OFFICE_PHONE_479FICE,
           OFFICE_AREACODE_479FICE,
           SPEED_DIAL_479FICE
    INTO :HV-OFFICE-OFFICE-CODE-479FICE,
         :HV-OFFICE-OFFICE-STREET-479FICE :NI-OFFICE-OFFICE-STREET-479FICE,
         :HV-OFFICE-OFFICE-CITY-479FICE :NI-OFFICE-OFFICE-CITY-479FICE,
         :HV-OFFICE-OFFICE-STATE-479FICE :NI-OFFICE-OFFICE-STATE-479FICE,
         :HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE :NI-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE,
         :HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE :NI-OFFICE-OFFICE-ZIPLASTFOUR-479FICE,
         :HV-OFFICE-OFFICE-PHONE-479FICE :NI-OFFICE-OFFICE-PHONE-479FICE,
         :HV-OFFICE-OFFICE-AREACODE-479FICE :NI-OFFICE-OFFICE-AREACODE-479FICE,
         :HV-OFFICE-SPEED-DIAL-479FICE :NI-OFFICE-SPEED-DIAL-479FICE
    FROM OFFICE
    WHERE OFFICE_CODE_479FICE = :HV-EMPLOYEE-OFFICE-CODE-479FICE
END-EXEC.
IF SQLCODE = 0
    CONTINUE.
ELSE
    MOVE NO-OFFICE-MESSAGE TO STREET-OUT
MOVE SPACES TO CITY-OUT
MOVE SPACES TO STATE-OUT
MOVE SPACES TO ZIP-OUT
END-IF.
END-IF.
MOVE HV-OFFICE-OFFICE-STREET-479FICE TO STREET-OUT
MOVE HV-OFFICE-OFFICE-CITY-479FICE TO CITY-OUT
MOVE HV-OFFICE-OFFICE-STATE-479FICE TO STATE-OUT
MOVE HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE TO ZIP-OUT
MOVE EMP-DETAIL-LINE TO PRINT-LINE.
PERFORM U000-WRITE-LINE.
SET-WALK-EXIT.
EXIT.
END-PROCESSING.
CLOSE DEPT-FILE-OUT.
CLOSE ERR-FILE-OUT.
CLOSE DEPT-FILE-IN.
EMPTY-SET.
MOVE SPACES TO ERR-LINE.
MOVE HV-DEPARTMENT-DEPT-ID-479MENT TO ERR-ID-OUT.
MOVE NO-EMP-MESSAGE TO ERR-MESS-OUT.
MOVE ERR-DETAIL-LINE TO ERR-LINE.
PERFORM U200-WRITE-ERR-LINE.
NO-DEPT-PROCESSING.
MOVE DEPT-ID-IN TO ERR-ID-OUT.
MOVE NO-DEPT-MESSAGE TO ERR-MESS-OUT.
MOVE ERR-DETAIL-LINE TO ERR-LINE.
PERFORM U200-WRITE-ERR-LINE.
U000-WRITE-LINE.
WRITE DEPT-REC-OUT AFTER POSITIONING CC.
IF CC = '1'
    THEN MOVE 0 TO LINE-COUNT
ELSE IF CC = ' '
    THEN ADD 1 TO LINE-COUNT
ELSE IF CC = '0'
    THEN ADD 2 TO LINE-COUNT.
IF LINE-COUNT > LINE-MAX
    THEN PERFORM U010-NEW-PAGE-ROUTINE.
U010-NEW-PAGE-ROUTINE.
PERFORM U020-VALID-HEADER.
MOVE DEPT-DETAIL-LINE TO PRINT-LINE.

```

```

        PERFORM U000-WRITE-LINE.
        PERFORM U030-EMP-HEADERS.
U020-VALID-HEADER.
        MOVE DEPT-HEADER TO PRINT-LINE.
        MOVE '1' TO CC.
        PERFORM U000-WRITE-LINE.
        MOVE ' ' TO CC.
U030-EMP-HEADERS.
        MOVE '0' TO CC.
        MOVE EMP-HEADER TO PRINT-LINE.
        PERFORM U000-WRITE-LINE.
        MOVE SPACES TO PRINT-LINE.
        MOVE ' ' TO CC.
        PERFORM U000-WRITE-LINE.
U200-WRITE-ERR-LINE.
        WRITE ERR-REC-OUT AFTER POSITIONING ERR-CC.
        IF ERR-CC = '1'
            THEN MOVE 0 TO ERR-LINE-COUNT
        ELSE IF ERR-CC = ' '
            THEN ADD 1 TO ERR-LINE-COUNT
        ELSE IF ERR-CC = '0'
            THEN ADD 2 TO ERR-LINE-COUNT.
        IF ERR-LINE-COUNT > LINE-MAX
            THEN PERFORM U220-ERR-HEADER.
U220-ERR-HEADER.
        MOVE ERR-HEADER-1 TO ERR-LINE.
        MOVE '1' TO ERR-CC.
        PERFORM U200-WRITE-ERR-LINE.
        MOVE '0' TO ERR-CC.
        MOVE ERR-HEADER-2 TO ERR-LINE.
        PERFORM U200-WRITE-ERR-LINE.
        MOVE ERR-HEADER-3 TO ERR-LINE.
        PERFORM U200-WRITE-ERR-LINE.
        MOVE SPACES TO ERR-LINE.
        MOVE ' ' TO ERR-CC.
        PERFORM U200-WRITE-ERR-LINE.
SQL-ERROR.
        MOVE SPACES TO ERR-LINE.
        MOVE 'SQL ERROR' TO ERR-MESS-OUT.
        MOVE ERR-DETAIL-LINE TO ERR-LINE.
        PERFORM U200-WRITE-ERR-LINE.
SQL-ERROR-EXIT.
        EXIT.

```

```

=====
FILE: data\output\db2_dd1.sql
ENCODING: utf-8
=====

```

```

CREATE TABLE COVERAGE
(
    ID_RECORD_RAGE CHAR(20) NOT NULL,
    DA_SELECTIONDATE_479RAGE DATE,
    DA_TERMINATIONDATE_479RAGE DATE,
    TYPE_479RAGE CHAR(1),
    INS_PLancode_479RAGE CHAR(3),
    EMP_ID_479OYEE DECIMAL(4),
    PRIMARY KEY (ID_RECORD_RAGE)
);

CREATE TABLE DENTAL_CLAIM
(
    ID_RECORD_LAIM CHAR(20) NOT NULL,
    DA_CLAIMDATE_479LAIM DATE,
    DA_PATIENTBIRTHDATE_479LAIM DATE,
    DA_SERVICEDATE_479LAIM DATE,
    PATIENT_FIRSTNAME_479LAIM CHAR(10),
    PATIENT_LASTNAME_479LAIM CHAR(15),
    PATIENT_SEX_479LAIM CHAR(1),
    RELATION_TOEMPLOYEE_479LAIM CHAR(10),
    DENTIST_FIRSTNAME_479LAIM CHAR(10),
    DENTIST_LASTNAME_479LAIM CHAR(15),
    DENTIST_STREET_479LAIM CHAR(20),
    DENTIST_CITY_479LAIM CHAR(15),
    DENTIST_STATE_479LAIM CHAR(2),

```

```

DENTIST_ZIPFIRSTFIVE_479LAIM CHAR(5),
DENTIST_ZIPLASTFOUR_479LAIM CHAR(4),
DENTIST_LICENSENUMBER_479LAIM DECIMAL(6),
NUMBER_OFPROCEDURES_479LAIM DECIMAL(2),
TOOTH_NUMBER_479LAIM DECIMAL(2),
PROCEDURE_CODE_479LAIM DECIMAL(4),
DESCRIPTION_OFSERVICE_479LAIM CHAR(60),
FEE_479LAIM DECIMAL(5,2),
ID_RECORD_RAGE CHAR(20),
PRIMARY KEY (ID_RECORD_LAIM)
);

```

```

CREATE TABLE DEPARTMENT
(
    DEPT_ID_479MENT DECIMAL(4) NOT NULL,
    DEPT_NAME_479MENT CHAR(45),
    DEPT_HEADID_479MENT DECIMAL(4),
    PRIMARY KEY (DEPT_ID_479MENT)
);

```

```

CREATE TABLE EMPLOYEE
(
    DA_STARTDATE_479OYEE DATE,
    DA_TERMINATIONDATE_479OYEE DATE,
    DA_BIRTHDATE_479OYEE DATE,
    EMP_ID_479OYEE DECIMAL(4) NOT NULL,
    EMP_FIRSTNAME_479OYEE CHAR(10),
    EMP_LASTNAME_479OYEE CHAR(15),
    EMP_STREET_479OYEE CHAR(20),
    EMP_CITY_479OYEE CHAR(15),
    EMP_STATE_479OYEE CHAR(2),
    EMP_ZIPFIRSTFIVE_479OYEE CHAR(5),
    EMP_ZIPLASTFOUR_479OYEE CHAR(4),
    EMP_PHONE_479OYEE DECIMAL(10),
    STATUS_479OYEE CHAR(2),
    SS_NUMBER_479OYEE DECIMAL(9),
    DEPT_ID_479MENT DECIMAL(4),
    OFFICE_CODE_479FICE CHAR(3),
    PRIMARY KEY (EMP_ID_479OYEE)
);

```

```

CREATE TABLE EMPOSITION
(
    ID_RECORD_TION CHAR(20) NOT NULL,
    DA_STARTDATE_479TION DATE,
    DA_FINISHDATE_479TION DATE,
    SALARY_GRADE_479TION DECIMAL(2),
    SALARY_AMOUNT_479TION DECIMAL(5,2),
    BONUS_PERCENT_479TION DECIMAL(2),
    COMMISSION_PERCENT_479TION DECIMAL(2),
    OVERTIME_RATE_479TION DECIMAL(2,2),
    EMP_ID_479OYEE DECIMAL(4),
    JOB_ID_479JOB DECIMAL(4),
    PRIMARY KEY (ID_RECORD_TION)
);

```

```

CREATE TABLE EXPERTISE
(
    ID_RECORD_TISE CHAR(20) NOT NULL,
    DA_EXPERTISEDATE_479TISE DATE,
    SKILL_LEVEL_479TISE CHAR(2),
    EMP_ID_479OYEE DECIMAL(4),
    SKILL_ID_479KILL DECIMAL(4),
    PRIMARY KEY (ID_RECORD_TISE)
);

```

```

CREATE TABLE HOSPITAL_CLAIM
(
    ID_RECORD_LAIM CHAR(20) NOT NULL,
    DA_CLAIMDATE_479LAIM DATE,
    DA_PATIENTBIRTHDATE_479LAIM DATE,
    DA_ADMITDATE_479LAIM DATE,
    DA_DISCHARGEDATE_479LAIM DATE,
    PATIENT_FIRSTNAME_479LAIM CHAR(10),
    PATIENT_LASTNAME_479LAIM CHAR(15),
    PATIENT_SEX_479LAIM CHAR(1),

```

```

RELATION_TOEMPLOYEE_479LAIM CHAR(10),
HOSPITAL_NAME_479LAIM CHAR(25),
HOSP_STREET_479LAIM CHAR(20),
HOSP_CITY_479LAIM CHAR(15),
HOSP_STATE_479LAIM CHAR(2),
HOSP_ZIPFIRSTFIVE_479LAIM CHAR(5),
HOSP_ZIPLASTFOUR_479LAIM CHAR(4),
DIAGNOSIS_479LAIM VARCHAR(120),
WARD_DAYS_479LAIM DECIMAL(3),
WARD_RATE_479LAIM DECIMAL(5,2),
WARD_TOTAL_479LAIM DECIMAL(5,2),
SEMI_DAYS_479LAIM DECIMAL(3),
SEMI_RATE_479LAIM DECIMAL(5,2),
SEMI_TOTAL_479LAIM DECIMAL(5,2),
DELIVERY_COST_479LAIM DECIMAL(5,2),
ANESTHESIA_COST_479LAIM DECIMAL(5,2),
LAB_COST_479LAIM DECIMAL(5,2),
ID_RECORD_RAGE CHAR(20),
PRIMARY KEY (ID_RECORD_LAIM)
);

```

```

CREATE TABLE INSURANCE_PLAN
(
    INS_PLANCODE_479PLAN CHAR(3) NOT NULL,
    INS_CONAME_479PLAN CHAR(45),
    INS_COSTREET_479PLAN CHAR(20),
    INS_COCITY_479PLAN CHAR(15),
    INS_COSTATE_479PLAN CHAR(2),
    INS_COZIPFIRSTFIVE_479PLAN CHAR(5),
    INS_COZIPLASTFOUR_479PLAN CHAR(4),
    INS_COPHONE_479PLAN DECIMAL(10),
    GROUP_NUMBER_479PLAN DECIMAL(6),
    DEDUCT_479PLAN DECIMAL(5,2),
    MAXIMUM_LIFECOST_479PLAN DECIMAL(5,2),
    FAMILY_COST_479PLAN DECIMAL(5,2),
    DEP_COST_479PLAN DECIMAL(5,2),
    PRIMARY KEY (INS_PLANCODE_479PLAN)
);

```

```

CREATE TABLE JOB
(
    JOB_ID_479JOB DECIMAL(4) NOT NULL,
    TITLE_479JOB CHAR(20),
    DESCRIPTION_LINE_479JOB VARCHAR(120),
    REQUIREMENT_LINE_479JOB VARCHAR(120),
    MINIMUM_SALARY_479JOB DECIMAL(8,2),
    MAXIMUM_SALARY_479JOB DECIMAL(8,2),
    SALARY_GRADES_479JOB VARCHAR(8),
    NUMBER_OFPOSITIONS_479JOB DECIMAL(3),
    NUMBER_OPEN_479JOB DECIMAL(3),
    PRIMARY KEY (JOB_ID_479JOB)
);

```

```

CREATE TABLE NON_HOSP_CLAIM
(
    ID_RECORD_LAIM CHAR(20) NOT NULL,
    DA_CLAIMDATE_479LAIM DATE,
    DA_PATIENTBIRTHDATE_479LAIM DATE,
    DA_SERVICEDATE_479LAIM DATE,
    PATIENT_FIRSTNAME_479LAIM CHAR(10),
    PATIENT_LASTNAME_479LAIM CHAR(15),
    PATIENT_SEX_479LAIM CHAR(1),
    RELATION_TOEMPLOYEE_479LAIM CHAR(10),
    PHYSICIAN_FIRSTNAME_479LAIM CHAR(10),
    PHYSICIAN_LASTNAME_479LAIM CHAR(15),
    PHYSICIAN_STREET_479LAIM CHAR(20),
    PHYSICIAN_CITY_479LAIM CHAR(15),
    PHYSICIAN_STATE_479LAIM CHAR(2),
    PHYSICIAN_ZIPFIRSTFIVE_479LAIM CHAR(5),
    PHYSICIAN_ZIPLASTFOUR_479LAIM CHAR(4),
    PHYSICIAN_ID_479LAIM DECIMAL(6),
    DIAGNOSIS_479LAIM VARCHAR(120),
    NUMBER_OFPROCEDURES_479LAIM DECIMAL(2),
    PROCEDURE_CODE_479LAIM DECIMAL(4),
    DESCRIPTION_OFSERVICE_479LAIM CHAR(60),
    FEE_479LAIM DECIMAL(5,2),

```

```

        ID_RECORD_RAGE CHAR(20),
        PRIMARY KEY (ID_RECORD_LAIM)
    );

CREATE TABLE OFFICE
(
    OFFICE_CODE_479FICE CHAR(3) NOT NULL,
    OFFICE_STREET_479FICE CHAR(20),
    OFFICE_CITY_479FICE CHAR(15),
    OFFICE_STATE_479FICE CHAR(2),
    OFFICE_ZIPFIRSTFIVE_479FICE CHAR(5),
    OFFICE_ZIPLASTFOUR_479FICE CHAR(4),
    OFFICE_PHONE_479FICE VARCHAR(21),
    OFFICE_AREACODE_479FICE CHAR(3),
    SPEED_DIAL_479FICE CHAR(3),
    PRIMARY KEY (OFFICE_CODE_479FICE)
);

CREATE TABLE SKILL
(
    SKILL_ID_479KILL DECIMAL(4) NOT NULL,
    SKILL_NAME_479KILL CHAR(12),
    SKILL_DESCRIPTION_479KILL CHAR(60),
    PRIMARY KEY (SKILL_ID_479KILL)
);

CREATE TABLE STRUCTURE
(
    ID_RECORD_TURE CHAR(20) NOT NULL,
    DA_STRUCTUREDATE_479TURE DATE,
    STRUCTURE_CODE_479TURE CHAR(2),
    EMP_ID_479OYEE DECIMAL(4),
    PRIMARY KEY (ID_RECORD_TURE)
);

ALTER TABLE COVERAGE ADD CONSTRAINT FK_COVERAGE_EMP_ID_479OYEE FOREIGN KEY (EMP_ID_479OYEE) REFERENCES EMPLOYEE (EMP_ID_479OYEE);
ALTER TABLE DENTAL_CLAIM ADD CONSTRAINT FK_DENTAL_CLAIM_ID_RECORD_RAGE FOREIGN KEY (ID_RECORD_RAGE) REFERENCES COVERAGE (ID_RECORD_RAGE);
ALTER TABLE EMPLOYEE ADD CONSTRAINT FK_EMPLOYEE_DEPT_ID_479MENT FOREIGN KEY (DEPT_ID_479MENT) REFERENCES DEPARTMENT (DEPT_ID_479MENT);
ALTER TABLE EMPLOYEE ADD CONSTRAINT FK_EMPLOYEE_OFFICE_CODE_479FICE FOREIGN KEY (OFFICE_CODE_479FICE) REFERENCES OFFICE (OFFICE_CODE_479FICE);
ALTER TABLE EMPOSITION ADD CONSTRAINT FK_EMPOSITION_EMP_ID_479OYEE FOREIGN KEY (EMP_ID_479OYEE) REFERENCES EMPLOYEE (EMP_ID_479OYEE);
ALTER TABLE EMPOSITION ADD CONSTRAINT FK_EMPOSITION_JOB_ID_479JOB FOREIGN KEY (JOB_ID_479JOB) REFERENCES JOB (JOB_ID_479JOB);
ALTER TABLE EXPERTISE ADD CONSTRAINT FK_EXPERTISE_EMP_ID_479OYEE FOREIGN KEY (EMP_ID_479OYEE) REFERENCES EMPLOYEE (EMP_ID_479OYEE);
ALTER TABLE EXPERTISE ADD CONSTRAINT FK_EXPERTISE_SKILL_ID_479KILL FOREIGN KEY (SKILL_ID_479KILL) REFERENCES SKILL (SKILL_ID_479KILL);
ALTER TABLE HOSPITAL_CLAIM ADD CONSTRAINT FK_HOSPITAL_CLAIM_ID_RECORD_RAGE FOREIGN KEY (ID_RECORD_RAGE) REFERENCES COVERAGE (ID_RECORD_RAGE);
ALTER TABLE NON_HOSP_CLAIM ADD CONSTRAINT FK_NON_HOSP_CLAIM_ID_RECORD_RAGE FOREIGN KEY (ID_RECORD_RAGE) REFERENCES COVERAGE (ID_RECORD_RAGE);
ALTER TABLE STRUCTURE ADD CONSTRAINT FK_STRUCTURE_EMP_ID_479OYEE FOREIGN KEY (EMP_ID_479OYEE) REFERENCES EMPLOYEE (EMP_ID_479OYEE);

```

```

=====
FILE: data\output\phase2_metadata.json
ENCODING: utf-8
=====

```

```

{
  "version": "1.0",
  "generated_at": "2026-08-07T10:23:41.967586Z",
  "records": [
    {
      "name": "COVERAGE",
      "table": "COVERAGE",
      "primary_key": "ID_RECORD_RAGE",
      "primary_keys": [
        "ID_RECORD_RAGE"
      ],
    },
  ],
}

```

```

"fields": [
  {
    "name": "ID_RECORD_RAGE",
    "column": "ID_RECORD_RAGE",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": false,
    "primary_key": true,
    "generated": true,
    "source_kind": "GENERATED PK"
  },
  {
    "name": "DA_SELECTIONDATE_479RAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_TERMINATIONDATE_479RAGE",
    "column": "DA_TERMINATIONDATE_479RAGE",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "TYPE_479RAGE",
    "column": "TYPE_479RAGE",
    "datatype": "CHAR(1)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_PLANCODE_479RAGE",
    "column": "INS_PLANCODE_479RAGE",
    "datatype": "CHAR(3)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_ID_479OYEE",
    "column": "EMP_ID_479OYEE",
    "datatype": "DECIMAL(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": true,
    "source_kind": "SET_FK"
  }
],
{
  "name": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "primary_key": "ID_RECORD_LAIM",
  "primary_keys": [
    "ID_RECORD_LAIM"
  ],

```

```

"fields": [
  {
    "name": "ID_RECORD_LAIM",
    "column": "ID_RECORD_LAIM",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": false,
    "primary_key": true,
    "generated": true,
    "source_kind": "GENERATED PK"
  },
  {
    "name": "DA_CLAIMDATE_479LAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_PATIENTBIRTHDATE_479LAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_SERVICEDATE_479LAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_FIRSTNAME_479LAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_LASTNAME_479LAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_SEX_479LAIM",
    "column": "PATIENT_SEX_479LAIM",
    "datatype": "CHAR(1)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
  }
]

```

```

    "source_kind": "COBOL"
  },
  {
    "name": "RELATION_TOEMPLOYEE_479LAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_FIRSTNAME_479LAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_LASTNAME_479LAIM",
    "column": "DENTIST_LASTNAME_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_STREET_479LAIM",
    "column": "DENTIST_STREET_479LAIM",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_CITY_479LAIM",
    "column": "DENTIST_CITY_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_STATE_479LAIM",
    "column": "DENTIST_STATE_479LAIM",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "datatype": "CHAR(5)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,

```



```

    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_ZIPLASTFOUR_479LAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "datatype": "CHAR(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DENTIST_LICENSENUMBER_479LAIM",
    "column": "DENTIST_LICENSENUMBER_479LAIM",
    "datatype": "DECIMAL(6)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "NUMBER_OFPROCEDURES_479LAIM",
    "column": "NUMBER_OFPROCEDURES_479LAIM",
    "datatype": "DECIMAL(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "TOOTH_NUMBER_479LAIM",
    "column": "TOOTH_NUMBER_479LAIM",
    "datatype": "DECIMAL(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PROCEDURE_CODE_479LAIM",
    "column": "PROCEDURE_CODE_479LAIM",
    "datatype": "DECIMAL(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DESCRIPTION_OFSERVICE_479LAIM",
    "column": "DESCRIPTION_OFSERVICE_479LAIM",
    "datatype": "CHAR(60)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "FEE_479LAIM",
    "column": "FEE_479LAIM",
    "datatype": "DECIMAL(5,2)",
    "length": null,
    "scale": null,
    "nullable": true,

```

```

        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "ID_RECORD_RAGE",
        "column": "ID_RECORD_RAGE",
        "datatype": "CHAR(20)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    }
]
},
{
    "name": "DEPARTMENT",
    "table": "DEPARTMENT",
    "primary_key": "DEPT_ID_479MENT",
    "primary_keys": [
        "DEPT_ID_479MENT"
    ],
    "fields": [
        {
            "name": "DEPT_ID_479MENT",
            "column": "DEPT_ID_479MENT",
            "datatype": "DECIMAL(4)",
            "length": null,
            "scale": null,
            "nullable": false,
            "primary_key": true,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "DEPT_NAME_479MENT",
            "column": "DEPT_NAME_479MENT",
            "datatype": "CHAR(45)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "DEPT_HEADID_479MENT",
            "column": "DEPT_HEADID_479MENT",
            "datatype": "DECIMAL(4)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        }
    ]
},
{
    "name": "EMPLOYEE",
    "table": "EMPLOYEE",
    "primary_key": "EMP_ID_4790YEE",
    "primary_keys": [
        "EMP_ID_4790YEE"
    ],
    "fields": [
        {
            "name": "DA_STARTDATE_4790YEE",
            "column": "DA_STARTDATE_4790YEE",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,

```

```

    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_TERMINATIONDATE_4790YEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_BIRTHDATE_4790YEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_ID_4790YEE",
    "column": "EMP_ID_4790YEE",
    "datatype": "DECIMAL(4)",
    "length": null,
    "scale": null,
    "nullable": false,
    "primary_key": true,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_FIRSTNAME_4790YEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_LASTNAME_4790YEE",
    "column": "EMP_LASTNAME_4790YEE",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_STREET_4790YEE",
    "column": "EMP_STREET_4790YEE",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_CITY_4790YEE",
    "column": "EMP_CITY_4790YEE",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,

```

```

    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_STATE_4790YEE",
    "column": "EMP_STATE_4790YEE",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_ZIPFIRSTFIVE_4790YEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "datatype": "CHAR(5)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_ZIPLASTFOUR_4790YEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "datatype": "CHAR(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "EMP_PHONE_4790YEE",
    "column": "EMP_PHONE_4790YEE",
    "datatype": "DECIMAL(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "STATUS_4790YEE",
    "column": "STATUS_4790YEE",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "SS_NUMBER_4790YEE",
    "column": "SS_NUMBER_4790YEE",
    "datatype": "DECIMAL(9)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DEPT_ID_479MENT",
    "column": "DEPT_ID_479MENT",
    "datatype": "DECIMAL(4)",
    "length": null,
    "scale": null,

```

```

        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    },
    {
        "name": "OFFICE_CODE_479FICE",
        "column": "OFFICE_CODE_479FICE",
        "datatype": "CHAR(3)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    }
]
},
{
    "name": "EMPOSITION",
    "table": "EMPOSITION",
    "primary_key": "ID_RECORD_TION",
    "primary_keys": [
        "ID_RECORD_TION"
    ],
    "fields": [
        {
            "name": "ID_RECORD_TION",
            "column": "ID_RECORD_TION",
            "datatype": "CHAR(20)",
            "length": null,
            "scale": null,
            "nullable": false,
            "primary_key": true,
            "generated": true,
            "source_kind": "GENERATED PK"
        },
        {
            "name": "DA_STARTDATE_479TION",
            "column": "DA_STARTDATE_479TION",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "DA_FINISHDATE_479TION",
            "column": "DA_FINISHDATE_479TION",
            "datatype": "DATE",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "SALARY_GRADE_479TION",
            "column": "SALARY_GRADE_479TION",
            "datatype": "DECIMAL(2)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "SALARY_AMOUNT_479TION",
            "column": "SALARY_AMOUNT_479TION",
            "datatype": "DECIMAL(5,2)",
            "length": null,
            "scale": null,

```

```

        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "BONUS_PERCENT_479TION",
        "column": "BONUS_PERCENT_479TION",
        "datatype": "DECIMAL(2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "COMMISSION_PERCENT_479TION",
        "column": "COMMISSION_PERCENT_479TION",
        "datatype": "DECIMAL(2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "OVERTIME_RATE_479TION",
        "column": "OVERTIME_RATE_479TION",
        "datatype": "DECIMAL(2,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "EMP_ID_479OYEE",
        "column": "EMP_ID_479OYEE",
        "datatype": "DECIMAL(4)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    },
    {
        "name": "JOB_ID_479JOB",
        "column": "JOB_ID_479JOB",
        "datatype": "DECIMAL(4)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    }
]
},
{
    "name": "EXPERTISE",
    "table": "EXPERTISE",
    "primary_key": "ID_RECORD_TISE",
    "primary_keys": [
        "ID_RECORD_TISE"
    ],
    "fields": [
        {
            "name": "ID_RECORD_TISE",
            "column": "ID_RECORD_TISE",
            "datatype": "CHAR(20)",
            "length": null,
            "scale": null,

```

```

        "nullable": false,
        "primary_key": true,
        "generated": true,
        "source_kind": "GENERATED PK"
    },
    {
        "name": "DA_EXPERTISEDATE_479TISE",
        "column": "DA_EXPERTISEDATE_479TISE",
        "datatype": "DATE",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "SKILL_LEVEL_479TISE",
        "column": "SKILL_LEVEL_479TISE",
        "datatype": "CHAR(2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "EMP_ID_479OYEE",
        "column": "EMP_ID_479OYEE",
        "datatype": "DECIMAL(4)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    },
    {
        "name": "SKILL_ID_479KILL",
        "column": "SKILL_ID_479KILL",
        "datatype": "DECIMAL(4)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    }
]
},
{
    "name": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "primary_key": "ID_RECORD_LAIM",
    "primary_keys": [
        "ID_RECORD_LAIM"
    ],
    "fields": [
        {
            "name": "ID_RECORD_LAIM",
            "column": "ID_RECORD_LAIM",
            "datatype": "CHAR(20)",
            "length": null,
            "scale": null,
            "nullable": false,
            "primary_key": true,
            "generated": true,
            "source_kind": "GENERATED PK"
        },
        {
            "name": "DA_CLAIMDATE_479LAIM",
            "column": "DA_CLAIMDATE_479LAIM",
            "datatype": "DATE",
            "length": null,
            "scale": null,

```

```

    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_PATIENTBIRTHDATE_479LAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_ADMITDATE_479LAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DA_DISCHARGEDATE_479LAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "datatype": "DATE",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_FIRSTNAME_479LAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_LASTNAME_479LAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PATIENT_SEX_479LAIM",
    "column": "PATIENT_SEX_479LAIM",
    "datatype": "CHAR(1)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "RELATION_TOEMPLOYEE_479LAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,

```



```

    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSPITAL_NAME_479LAIM",
    "column": "HOSPITAL_NAME_479LAIM",
    "datatype": "CHAR(25)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSP_STREET_479LAIM",
    "column": "HOSP_STREET_479LAIM",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSP_CITY_479LAIM",
    "column": "HOSP_CITY_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSP_STATE_479LAIM",
    "column": "HOSP_STATE_479LAIM",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "datatype": "CHAR(5)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "HOSP_ZIPLASTFOUR_479LAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "datatype": "CHAR(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DIAGNOSIS_479LAIM",
    "column": "DIAGNOSIS_479LAIM",
    "datatype": "VARCHAR(120)",

```

```

    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "WARD_DAYS_479LAIM",
    "column": "WARD_DAYS_479LAIM",
    "datatype": "DECIMAL(3)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "WARD_RATE_479LAIM",
    "column": "WARD_RATE_479LAIM",
    "datatype": "DECIMAL(5,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "WARD_TOTAL_479LAIM",
    "column": "WARD_TOTAL_479LAIM",
    "datatype": "DECIMAL(5,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "SEMI_DAYS_479LAIM",
    "column": "SEMI_DAYS_479LAIM",
    "datatype": "DECIMAL(3)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "SEMI_RATE_479LAIM",
    "column": "SEMI_RATE_479LAIM",
    "datatype": "DECIMAL(5,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "SEMI_TOTAL_479LAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "datatype": "DECIMAL(5,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DELIVERY_COST_479LAIM",
    "column": "DELIVERY_COST_479LAIM",

```

```

        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "ANESTHESIA_COST_479LAIM",
        "column": "ANESTHESIA_COST_479LAIM",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "LAB_COST_479LAIM",
        "column": "LAB_COST_479LAIM",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "ID_RECORD_RAGE",
        "column": "ID_RECORD_RAGE",
        "datatype": "CHAR(20)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": true,
        "source_kind": "SET_FK"
    }
]
},
{
    "name": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "primary_key": "INS_PLANCODE_479PLAN",
    "primary_keys": [
        "INS_PLANCODE_479PLAN"
    ],
    "fields": [
        {
            "name": "INS_PLANCODE_479PLAN",
            "column": "INS_PLANCODE_479PLAN",
            "datatype": "CHAR(3)",
            "length": null,
            "scale": null,
            "nullable": false,
            "primary_key": true,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "INS_CONAME_479PLAN",
            "column": "INS_CONAME_479PLAN",
            "datatype": "CHAR(45)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "INS_COSTREET_479PLAN",
            "column": "INS_COSTREET_479PLAN",

```

```

    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_COCITY_479PLAN",
    "column": "INS_COCITY_479PLAN",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_COSTATE_479PLAN",
    "column": "INS_COSTATE_479PLAN",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_COZIPFIRSTFIVE_479PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "datatype": "CHAR(5)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_COZIPLASTFOUR_479PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "datatype": "CHAR(4)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "INS_COPHONE_479PLAN",
    "column": "INS_COPHONE_479PLAN",
    "datatype": "DECIMAL(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "GROUP_NUMBER_479PLAN",
    "column": "GROUP_NUMBER_479PLAN",
    "datatype": "DECIMAL(6)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "DEDUCT_479PLAN",

```

```

        "column": "DEDUCT_479PLAN",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "MAXIMUM_LIFECOST_479PLAN",
        "column": "MAXIMUM_LIFECOST_479PLAN",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "FAMILY_COST_479PLAN",
        "column": "FAMILY_COST_479PLAN",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    },
    {
        "name": "DEP_COST_479PLAN",
        "column": "DEP_COST_479PLAN",
        "datatype": "DECIMAL(5,2)",
        "length": null,
        "scale": null,
        "nullable": true,
        "primary_key": false,
        "generated": false,
        "source_kind": "COBOL"
    }
]
},
{
    "name": "JOB",
    "table": "JOB",
    "primary_key": "JOB_ID_479JOB",
    "primary_keys": [
        "JOB_ID_479JOB"
    ],
    "fields": [
        {
            "name": "JOB_ID_479JOB",
            "column": "JOB_ID_479JOB",
            "datatype": "DECIMAL(4)",
            "length": null,
            "scale": null,
            "nullable": false,
            "primary_key": true,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "TITLE_479JOB",
            "column": "TITLE_479JOB",
            "datatype": "CHAR(20)",
            "length": null,
            "scale": null,
            "nullable": true,
            "primary_key": false,
            "generated": false,
            "source_kind": "COBOL"
        },
        {
            "name": "DESCRIPTION_LINE_479JOB",

```

```

    "column": "DESCRIPTION_LINE_479JOB",
    "datatype": "VARCHAR(120)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "REQUIREMENT_LINE_479JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "datatype": "VARCHAR(120)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "MINIMUM_SALARY_479JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "datatype": "DECIMAL(8,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "MAXIMUM_SALARY_479JOB",
    "column": "MAXIMUM_SALARY_479JOB",
    "datatype": "DECIMAL(8,2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "SALARY_GRADES_479JOB",
    "column": "SALARY_GRADES_479JOB",
    "datatype": "VARCHAR(8)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "NUMBER_OFPOSITIONS_479JOB",
    "column": "NUMBER_OFPOSITIONS_479JOB",
    "datatype": "DECIMAL(3)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "NUMBER_OPEN_479JOB",
    "column": "NUMBER_OPEN_479JOB",
    "datatype": "DECIMAL(3)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  }
]

```

```

},
{
  "name": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "primary_key": "ID_RECORD_LAIM",
  "primary_keys": [
    "ID_RECORD_LAIM"
  ],
  "fields": [
    {
      "name": "ID_RECORD_LAIM",
      "column": "ID_RECORD_LAIM",
      "datatype": "CHAR(20)",
      "length": null,
      "scale": null,
      "nullable": false,
      "primary_key": true,
      "generated": true,
      "source_kind": "GENERATED PK"
    },
    {
      "name": "DA_CLAIMDATE_479LAIM",
      "column": "DA_CLAIMDATE_479LAIM",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "DA_PATIENTBIRTHDATE_479LAIM",
      "column": "DA_PATIENTBIRTHDATE_479LAIM",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "DA_SERVICEDATE_479LAIM",
      "column": "DA_SERVICEDATE_479LAIM",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "PATIENT_FIRSTNAME_479LAIM",
      "column": "PATIENT_FIRSTNAME_479LAIM",
      "datatype": "CHAR(10)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "PATIENT_LASTNAME_479LAIM",
      "column": "PATIENT_LASTNAME_479LAIM",
      "datatype": "CHAR(15)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    }
  ],
{

```

```

    "name": "PATIENT_SEX_479LAIM",
    "column": "PATIENT_SEX_479LAIM",
    "datatype": "CHAR(1)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "RELATION_TOEMPLOYEE_479LAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PHYSICIAN_FIRSTNAME_479LAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "datatype": "CHAR(10)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PHYSICIAN_LASTNAME_479LAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PHYSICIAN_STREET_479LAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "datatype": "CHAR(20)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PHYSICIAN_CITY_479LAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "datatype": "CHAR(15)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },
  {
    "name": "PHYSICIAN_STATE_479LAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "datatype": "CHAR(2)",
    "length": null,
    "scale": null,
    "nullable": true,
    "primary_key": false,
    "generated": false,
    "source_kind": "COBOL"
  },

```



```

{
  "name": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
  "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
  "datatype": "CHAR(5)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "datatype": "CHAR(4)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "PHYSICIAN_ID_479LAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "datatype": "DECIMAL(6)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "DIAGNOSIS_479LAIM",
  "column": "DIAGNOSIS_479LAIM",
  "datatype": "VARCHAR(120)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "NUMBER_OFPROCEDURES_479LAIM",
  "column": "NUMBER_OFPROCEDURES_479LAIM",
  "datatype": "DECIMAL(2)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "PROCEDURE_CODE_479LAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "datatype": "DECIMAL(4)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
},
{
  "name": "DESCRIPTION_OFSERVICE_479LAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "datatype": "CHAR(60)",
  "length": null,
  "scale": null,
  "nullable": true,
  "primary_key": false,
  "generated": false,
  "source_kind": "COBOL"
}

```

```

    },
    {
      "name": "FEE_479LAIM",
      "column": "FEE_479LAIM",
      "datatype": "DECIMAL(5,2)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "ID_RECORD_RAGE",
      "column": "ID_RECORD_RAGE",
      "datatype": "CHAR(20)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": true,
      "source_kind": "SET_FK"
    }
  ]
},
{
  "name": "OFFICE",
  "table": "OFFICE",
  "primary_key": "OFFICE_CODE_479FICE",
  "primary_keys": [
    "OFFICE_CODE_479FICE"
  ],
  "fields": [
    {
      "name": "OFFICE_CODE_479FICE",
      "column": "OFFICE_CODE_479FICE",
      "datatype": "CHAR(3)",
      "length": null,
      "scale": null,
      "nullable": false,
      "primary_key": true,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_STREET_479FICE",
      "column": "OFFICE_STREET_479FICE",
      "datatype": "CHAR(20)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_CITY_479FICE",
      "column": "OFFICE_CITY_479FICE",
      "datatype": "CHAR(15)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_STATE_479FICE",
      "column": "OFFICE_STATE_479FICE",
      "datatype": "CHAR(2)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    }
  ]
}

```

```

    },
    {
      "name": "OFFICE_ZIPFIRSTFIVE_479FICE",
      "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
      "datatype": "CHAR(5)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_ZIPLASTFOUR_479FICE",
      "column": "OFFICE_ZIPLASTFOUR_479FICE",
      "datatype": "CHAR(4)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_PHONE_479FICE",
      "column": "OFFICE_PHONE_479FICE",
      "datatype": "VARCHAR(21)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "OFFICE_AREACODE_479FICE",
      "column": "OFFICE_AREACODE_479FICE",
      "datatype": "CHAR(3)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "SPEED_DIAL_479FICE",
      "column": "SPEED_DIAL_479FICE",
      "datatype": "CHAR(3)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    }
  ],
  {
    "name": "SKILL",
    "table": "SKILL",
    "primary_key": "SKILL_ID_479KILL",
    "primary_keys": [
      "SKILL_ID_479KILL"
    ],
    "fields": [
      {
        "name": "SKILL_ID_479KILL",
        "column": "SKILL_ID_479KILL",
        "datatype": "DECIMAL(4)",
        "length": null,
        "scale": null,
        "nullable": false,
        "primary_key": true,
        "generated": false,
        "source_kind": "COBOL"
      }
    ]
  }
}

```

```

    },
    {
      "name": "SKILL_NAME_479KILL",
      "column": "SKILL_NAME_479KILL",
      "datatype": "CHAR(12)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "SKILL_DESCRIPTION_479KILL",
      "column": "SKILL_DESCRIPTION_479KILL",
      "datatype": "CHAR(60)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    }
  ]
},
{
  "name": "STRUCTURE",
  "table": "STRUCTURE",
  "primary_key": "ID_RECORD_TURE",
  "primary_keys": [
    "ID_RECORD_TURE"
  ],
  "fields": [
    {
      "name": "ID_RECORD_TURE",
      "column": "ID_RECORD_TURE",
      "datatype": "CHAR(20)",
      "length": null,
      "scale": null,
      "nullable": false,
      "primary_key": true,
      "generated": true,
      "source_kind": "GENERATED PK"
    },
    {
      "name": "DA_STRUCTUREDATE_479TURE",
      "column": "DA_STRUCTUREDATE_479TURE",
      "datatype": "DATE",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "STRUCTURE_CODE_479TURE",
      "column": "STRUCTURE_CODE_479TURE",
      "datatype": "CHAR(2)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": false,
      "source_kind": "COBOL"
    },
    {
      "name": "EMP_ID_479OYEE",
      "column": "EMP_ID_479OYEE",
      "datatype": "DECIMAL(4)",
      "length": null,
      "scale": null,
      "nullable": true,
      "primary_key": false,
      "generated": true,
      "source_kind": "SET_FK"
    }
  ]
}

```

```

    }
  ]
}
],
"record_table_map": {
  "COVERAGE": "COVERAGE",
  "DENTAL_CLAIM": "DENTAL_CLAIM",
  "DEPARTMENT": "DEPARTMENT",
  "EMPLOYEE": "EMPLOYEE",
  "EMPOSITION": "EMPOSITION",
  "EXPERTISE": "EXPERTISE",
  "HOSPITAL_CLAIM": "HOSPITAL_CLAIM",
  "INSURANCE_PLAN": "INSURANCE_PLAN",
  "JOB": "JOB",
  "NON_HOSP_CLAIM": "NON_HOSP_CLAIM",
  "OFFICE": "OFFICE",
  "SKILL": "SKILL",
  "STRUCTURE": "STRUCTURE"
},
"field_map": {
  "SELECTION_DATE_0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "SELECTION-DATE-0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "SELECTION_DATE": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "SELECTIONDATE0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "DA_SELECTIONDATE_479RAGE": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "DA-SELECTIONDATE-479RAGE": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "DA_SELECTIONDATE": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
  "DASELECTIONDATE": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "legacy_field": "SELECTION_DATE_0400"
  },
},

```

```

"TERMINATION_DATE_0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"TERMINATION-DATE-0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"TERMINATION_DATE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "DA_TERMINATIONDATE_479OYEE",
  "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE",
  "legacy_field": "TERMINATION_DATE_0415"
},
"TERMINATIONDATE0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"DA_TERMINATIONDATE_479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"DA-TERMINATIONDATE-479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"DA_TERMINATIONDATE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"DATERMINATIONDATE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "legacy_field": "TERMINATION_DATE_0400"
},
"TYPE_0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
},
"TYPE-0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
},
"TYPE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
}

```

```

},
"TYPE0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
},
"TYPE_479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
},
"TYPE-479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "TYPE_479RAGE",
  "host": "HV-COVERAGE-TYPE-479RAGE",
  "legacy_field": "TYPE_0400"
},
"INS_PLAN_CODE_0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INS-PLAN-CODE-0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INS_PLAN_CODE": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_PLANCODE_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
  "legacy_field": "INS_PLAN_CODE_0435"
},
"INSPLANCODE0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INS_PLANCODE_479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INS-PLANCODE-479RAGE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INS_PLANCODE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",
  "legacy_field": "INS_PLAN_CODE_0400"
},
"INSPLANCODE": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "INS_PLANCODE_479RAGE",
  "host": "HV-COVERAGE-INS-PLANCODE-479RAGE",

```

```

    "legacy_field": "INS_PLAN_CODE_0400"
  },
  "CLAIM_DATE_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "CLAIM-DATE-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "CLAIM_DATE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0445"
  },
  "CLAIMDATE0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "DA_CLAIMDATE_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "DA-CLAIMDATE-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "DA_CLAIMDATE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "DACLAIMDATE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0405"
  },
  "PATIENT_FIRST_NAME_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT-FIRST-NAME-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT_FIRST_NAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",

```



```

    "host": "HV-NON-HOSP-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0445"
  },
  "PATIENTFIRSTNAME0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT_FIRSTNAME_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT-FIRSTNAME-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT_FIRSTNAME": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENTFIRSTNAME": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
    "legacy_field": "PATIENT_FIRST_NAME_0405"
  },
  "PATIENT_LAST_NAME_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENT-LAST-NAME-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENT_LAST_NAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0445"
  },
  "PATIENTLASTNAME0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENT_LASTNAME_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_LASTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
    "legacy_field": "PATIENT_LAST_NAME_0405"
  },
  "PATIENT-LASTNAME-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",

```

```

"column": "PATIENT_LASTNAME_479LAIM",
"host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
"legacy_field": "PATIENT_LAST_NAME_0405"
},
"PATIENT_LASTNAME": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PATIENT_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
  "legacy_field": "PATIENT_LAST_NAME_0405"
},
"PATIENTLASTNAME": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PATIENT_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-PATIENT-LASTNAME-479LAIM",
  "legacy_field": "PATIENT_LAST_NAME_0405"
},
"PATIENT_BIRTH_DATE_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0405"
},
"PATIENT-BIRTH-DATE-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0405"
},
"PATIENT_BIRTH_DATE": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0445"
},
"PATIENTBIRTHDATE0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0405"
},
"DA_PATIENTBIRTHDATE_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0405"
},
"DA-PATIENTBIRTHDATE-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0405"
},
"DA_PATIENTBIRTHDATE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0405"
},
"DAPATIENTBIRTHDATE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0405"
},
"PATIENT_SEX_0405": {
  "record": "DENTAL_CLAIM",

```

```

    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "PATIENT-SEX-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "PATIENT_SEX": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0445"
  },
  "PATIENTSEX0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "PATIENT_SEX_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "PATIENT-SEX-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "PATIENTSEX": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "PATIENT_SEX_479LAIM",
    "host": "HV-DENTAL-CLAIM-PATIENT-SEX-479LAIM",
    "legacy_field": "PATIENT_SEX_0405"
  },
  "RELATION_TO_EMPLOYEE_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATION-TO-EMPLOYEE-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATION_TO_EMPLOYEE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0445"
  },
  "RELATIONTOEMPLOYEE0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATION_TOEMPLOYEE_479LAIM": {

```

```

    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATION-TOEMPLOYEE-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATION_TOEMPLOYEE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "RELATIONTOEMPLOYEE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0405"
  },
  "DENTIST_FIRST_NAME_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST-FIRST-NAME-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST_FIRST_NAME": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTISTFIRSTNAME0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST_FIRSTNAME_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST-FIRSTNAME-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },
  "DENTIST_FIRSTNAME": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_FIRSTNAME_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
    "legacy_field": "DENTIST_FIRST_NAME_0405"
  },

```

```

"DENTISTFIRSTNAME": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_FIRSTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-FIRSTNAME-479LAIM",
  "legacy_field": "DENTIST_FIRST_NAME_0405"
},
"DENTIST_LAST_NAME_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST-LAST-NAME-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST_LAST_NAME": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTISTLASTNAME0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST_LASTNAME_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST-LASTNAME-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST_LASTNAME": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTISTLASTNAME": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_LASTNAME_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-LASTNAME-479LAIM",
  "legacy_field": "DENTIST_LAST_NAME_0405"
},
"DENTIST_STREET_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTIST-STREET-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
}

```

```

},
"DENTIST_STREET": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTISTSTREET0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTIST_STREET_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTIST-STREET-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTISTSTREET": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_STREET_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-STREET-479LAIM",
  "legacy_field": "DENTIST_STREET_0405"
},
"DENTIST_CITY_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
  "legacy_field": "DENTIST_CITY_0405"
},
"DENTIST-CITY-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
  "legacy_field": "DENTIST_CITY_0405"
},
"DENTIST_CITY": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
  "legacy_field": "DENTIST_CITY_0405"
},
"DENTISTCITY0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
  "legacy_field": "DENTIST_CITY_0405"
},
"DENTIST_CITY_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
  "legacy_field": "DENTIST_CITY_0405"
},
"DENTIST-CITY-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DENTIST_CITY_479LAIM",
  "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",

```

```

    "legacy_field": "DENTIST_CITY_0405"
  },
  "DENTISTCITY": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_CITY_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-CITY-479LAIM",
    "legacy_field": "DENTIST_CITY_0405"
  },
  "DENTIST_STATE_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST-STATE-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST_STATE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTISTSTATE0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST_STATE_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST-STATE-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTISTSTATE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_STATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-STATE-479LAIM",
    "legacy_field": "DENTIST_STATE_0405"
  },
  "DENTIST_ZIP_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST-ZIP-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST_ZIP": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",

```

```

    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTISTZIP0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST_ZIPFIRSTFIVE_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST-ZIPFIRSTFIVE-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST_ZIPFIRSTFIVE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTISTZIPFIRSTFIVE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_0405"
  },
  "DENTIST_ZIP_FIRST_FIVE_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_FIRST_FIVE_0405"
  },
  "DENTIST-ZIP-FIRST-FIVE-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_FIRST_FIVE_0405"
  },
  "DENTIST_ZIP_FIRST_FIVE": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_FIRST_FIVE_0405"
  },
  "DENTISTZIPFIRSTFIVE0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "DENTIST_ZIP_FIRST_FIVE_0405"
  },
  "DENTIST_ZIP_LAST_FOUR_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST-ZIP-LAST-FOUR-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",

```



```

    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST_ZIP_LAST_FOUR": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTISTZIPLASTFOUR0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST_ZIPLASTFOUR_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST-ZIPLASTFOUR-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST_ZIPLASTFOUR": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTISTZIPLASTFOUR": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_ZIPLASTFOUR_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-ZIPLASTFOUR-479LAIM",
    "legacy_field": "DENTIST_ZIP_LAST_FOUR_0405"
  },
  "DENTIST_LICENSE_NUMBER_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LICENSENUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
    "legacy_field": "DENTIST_LICENSE_NUMBER_0405"
  },
  "DENTIST-LICENSE-NUMBER-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LICENSENUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
    "legacy_field": "DENTIST_LICENSE_NUMBER_0405"
  },
  "DENTIST_LICENSE_NUMBER": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LICENSENUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
    "legacy_field": "DENTIST_LICENSE_NUMBER_0405"
  },
  "DENTISTLICENSENUMBER0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DENTIST_LICENSENUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
    "legacy_field": "DENTIST_LICENSE_NUMBER_0405"
  },
  "DENTIST_LICENSENUMBER_479LAIM": {
    "record": "DENTAL_CLAIM",

```

```

"table": "DENTAL_CLAIM",
"column": "DENTIST_LICENSENUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
"legacy_field": "DENTIST_LICENSE_NUMBER_0405"
},
"DENTIST-LICENSENUMBER-479LAIM": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "DENTIST_LICENSENUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
"legacy_field": "DENTIST_LICENSE_NUMBER_0405"
},
"DENTIST_LICENSENUMBER": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "DENTIST_LICENSENUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
"legacy_field": "DENTIST_LICENSE_NUMBER_0405"
},
"DENTISTLICENSENUMBER": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "DENTIST_LICENSENUMBER_479LAIM",
"host": "HV-DENTAL-CLAIM-DENTIST-LICENSENUMBER-479LAIM",
"legacy_field": "DENTIST_LICENSE_NUMBER_0405"
},
"NUMBER_OF_PROCEDURES_0405": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBER-OF-PROCEDURES-0405": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBER_OF_PROCEDURES": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-NON-HOSP-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0445"
},
"NUMBEROFPROCEDURES0405": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBER_OFPROCEDURES_479LAIM": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBER-OFPROCEDURES-479LAIM": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBER_OFPROCEDURES": {
"record": "DENTAL_CLAIM",
"table": "DENTAL_CLAIM",
"column": "NUMBER_OFPROCEDURES_479LAIM",
"host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
"legacy_field": "NUMBER_OF_PROCEDURES_0405"
},
"NUMBEROFPROCEDURES": {

```

```

    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "NUMBER_OFPROCEDURES_479LAIM",
    "host": "HV-DENTAL-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
    "legacy_field": "NUMBER_OF_PROCEEDURES_0405"
  },
  "TOOTH_NUMBER_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "TOOTH_NUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
    "legacy_field": "TOOTH_NUMBER_0405"
  },
  "TOOTH-NUMBER-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "TOOTH_NUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
    "legacy_field": "TOOTH_NUMBER_0405"
  },
  "TOOTH_NUMBER": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "TOOTH_NUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
    "legacy_field": "TOOTH_NUMBER_0405"
  },
  "TOOTHNUMBER0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "TOOTH_NUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
    "legacy_field": "TOOTH_NUMBER_0405"
  },
  "TOOTH_NUMBER_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "TOOTH_NUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
    "legacy_field": "TOOTH_NUMBER_0405"
  },
  "TOOTH-NUMBER-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "TOOTH_NUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
    "legacy_field": "TOOTH_NUMBER_0405"
  },
  "TOOTHNUMBER": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "TOOTH_NUMBER_479LAIM",
    "host": "HV-DENTAL-CLAIM-TOOTH-NUMBER-479LAIM",
    "legacy_field": "TOOTH_NUMBER_0405"
  },
  "SERVICE_DATE_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "legacy_field": "SERVICE_DATE_0405"
  },
  "SERVICE-DATE-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "legacy_field": "SERVICE_DATE_0405"
  },
  "SERVICE_DATE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "legacy_field": "SERVICE_DATE_0445"
  },

```

```

"SERVICE_DATE_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_SERVICE_DATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-SERVICE_DATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0405"
},
"DA_SERVICE_DATE_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_SERVICE_DATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-SERVICE_DATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0405"
},
"DA-SERVICE_DATE-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_SERVICE_DATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-SERVICE_DATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0405"
},
"DA_SERVICE_DATE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_SERVICE_DATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-SERVICE_DATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0405"
},
"DASERVICE_DATE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_SERVICE_DATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-SERVICE_DATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0405"
},
"PROCEDURE_CODE_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
},
"PROCEDURE-CODE-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
},
"PROCEDURE_CODE": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0445"
},
"PROCEDURE_CODE_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
},
"PROCEDURE_CODE_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
},
"PROCEDURE-CODE-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
}

```

```

},
"PROCEDURECODE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-DENTAL-CLAIM-PROCEDURE-CODE-479LAIM",
  "legacy_field": "PROCEDURE_CODE_0405"
},
"DESCRIPTION_OF_SERVICE_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTION-OF-SERVICE-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTION_OF_SERVICE": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0445"
},
"DESCRIPTIONOFSERVICE0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTION_OFSERVICE_479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTION-OFSERVICE-479LAIM": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTION_OFSERVICE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"DESCRIPTIONOFSERVICE": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DESCRIPTION_OFSERVICE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
  "legacy_field": "DESCRIPTION_OF_SERVICE_0405"
},
"FEE_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "FEE_479LAIM",
  "host": "HV-DENTAL-CLAIM-FEE-479LAIM",
  "legacy_field": "FEE_0405"
},
"FEE-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "FEE_479LAIM",
  "host": "HV-DENTAL-CLAIM-FEE-479LAIM",

```

```

    "legacy_field": "FEE_0405"
  },
  "FEE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0445"
  },
  "FEE0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0405"
  },
  "FEE_479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0405"
  },
  "FEE-479LAIM": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-DENTAL-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0405"
  },
  "DEPT_ID_0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPT-ID-0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPT_ID": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPTID0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPT_ID_479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPT-ID-479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPTID": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_ID_479MENT",

```

```

    "host": "HV-DEPARTMENT-DEPT-ID-479MENT",
    "legacy_field": "DEPT_ID_0410"
  },
  "DEPT_NAME_0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPT-NAME-0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPT_NAME": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPTNAME0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPT_NAME_479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPT-NAME-479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPTNAME": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_NAME_479MENT",
    "host": "HV-DEPARTMENT-DEPT-NAME-479MENT",
    "legacy_field": "DEPT_NAME_0410"
  },
  "DEPT_HEAD_ID_0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPT-HEAD-ID-0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPT_HEAD_ID": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPTHEADID0410": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",

```

```

    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPT_HEADID_479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPT-HEADID-479MENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPT_HEADID": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "DEPTHEADID": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "column": "DEPT_HEADID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-HEADID-479MENT",
    "legacy_field": "DEPT_HEAD_ID_0410"
  },
  "EMP_ID_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMP-ID-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMP_ID": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMPID0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMP_ID_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMP-ID-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMPID": {
    "record": "EMPLOYEE",

```



```

    "table": "EMPLOYEE",
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE",
    "legacy_field": "EMP_ID_0415"
  },
  "EMP_FIRST_NAME_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP-FIRST-NAME-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP_FIRST_NAME": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMPFIRSTNAME0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP_FIRSTNAME_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP-FIRSTNAME-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP_FIRSTNAME": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMPFIRSTNAME": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_FIRSTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-FIRSTNAME-4790YEE",
    "legacy_field": "EMP_FIRST_NAME_0415"
  },
  "EMP_LAST_NAME_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP-LAST-NAME-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP_LAST_NAME": {

```

```

    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMPLASTNAME0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP_LASTNAME_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP-LASTNAME-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP_LASTNAME": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMPLASTNAME": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_LASTNAME_4790YEE",
    "host": "HV-EMPLOYEE-EMP-LASTNAME-4790YEE",
    "legacy_field": "EMP_LAST_NAME_0415"
  },
  "EMP_STREET_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_STREET_4790YEE",
    "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
    "legacy_field": "EMP_STREET_0415"
  },
  "EMP-STREET-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_STREET_4790YEE",
    "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
    "legacy_field": "EMP_STREET_0415"
  },
  "EMP_STREET": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_STREET_4790YEE",
    "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
    "legacy_field": "EMP_STREET_0415"
  },
  "EMPSTREET0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_STREET_4790YEE",
    "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
    "legacy_field": "EMP_STREET_0415"
  },
  "EMP_STREET_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_STREET_4790YEE",
    "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
    "legacy_field": "EMP_STREET_0415"
  },

```

```

"EMP-STREET-4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STREET_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
  "legacy_field": "EMP_STREET_0415"
},
"EMPSTREET": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STREET_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STREET-4790YEE",
  "legacy_field": "EMP_STREET_0415"
},
"EMP_CITY_0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMP-CITY-0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMP_CITY": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMPCITY0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMP_CITY_4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMP-CITY-4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMPCITY": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_CITY_4790YEE",
  "host": "HV-EMPLOYEE-EMP-CITY-4790YEE",
  "legacy_field": "EMP_CITY_0415"
},
"EMP_STATE_0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMP-STATE-0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
}

```

```

},
"EMP_STATE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMPSTATE0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMP_STATE_4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMP-STATE-4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMPSTATE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_STATE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-STATE-4790YEE",
  "legacy_field": "EMP_STATE_0415"
},
"EMP_ZIP_0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
  "legacy_field": "EMP_ZIP_0415"
},
"EMP-ZIP-0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
  "legacy_field": "EMP_ZIP_0415"
},
"EMP_ZIP": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
  "legacy_field": "EMP_ZIP_0415"
},
"EMPZIP0415": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
  "legacy_field": "EMP_ZIP_0415"
},
"EMP_ZIPFIRSTFIVE_4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
  "legacy_field": "EMP_ZIP_0415"
},
"EMP-ZIPFIRSTFIVE-4790YEE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "EMP_ZIPFIRSTFIVE_4790YEE",
  "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",

```

```

    "legacy_field": "EMP_ZIP_0415"
  },
  "EMP_ZIPFIRSTFIVE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_0415"
  },
  "EMPZIPFIRSTFIVE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_0415"
  },
  "EMP_ZIP_FIRST_FIVE_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_FIRST_FIVE_0415"
  },
  "EMP-ZIP-FIRST-FIVE-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_FIRST_FIVE_0415"
  },
  "EMP_ZIP_FIRST_FIVE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_FIRST_FIVE_0415"
  },
  "EMPZIPFIRSTFIVE0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPFIRSTFIVE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPFIRSTFIVE-4790YEE",
    "legacy_field": "EMP_ZIP_FIRST_FIVE_0415"
  },
  "EMP_ZIP_LAST_FOUR_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP-ZIP-LAST-FOUR-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP_ZIP_LAST_FOUR": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMPZIPLASTFOUR0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP_ZIPLASTFOUR_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",

```

```

    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP-ZIPLASTFOUR-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP_ZIPLASTFOUR": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMPZIPLASTFOUR": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_ZIPLASTFOUR_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ZIPLASTFOUR-4790YEE",
    "legacy_field": "EMP_ZIP_LAST_FOUR_0415"
  },
  "EMP_PHONE_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMP-PHONE-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMP_PHONE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMPPHONE0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMP_PHONE_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMP-PHONE-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "EMPPHONE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "EMP_PHONE_4790YEE",
    "host": "HV-EMPLOYEE-EMP-PHONE-4790YEE",
    "legacy_field": "EMP_PHONE_0415"
  },
  "STATUS_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",

```

```

    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "STATUS-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "STATUS": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "STATUS0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "STATUS_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "STATUS-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "STATUS_4790YEE",
    "host": "HV-EMPLOYEE-STATUS-4790YEE",
    "legacy_field": "STATUS_0415"
  },
  "SS_NUMBER_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SS-NUMBER-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SS_NUMBER": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SSNUMBER0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SS_NUMBER_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SS-NUMBER-4790YEE": {
    "record": "EMPLOYEE",

```

```

    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "SSNUMBER": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "SS_NUMBER_4790YEE",
    "host": "HV-EMPLOYEE-SS-NUMBER-4790YEE",
    "legacy_field": "SS_NUMBER_0415"
  },
  "START_DATE_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "START-DATE-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "START_DATE": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "legacy_field": "START_DATE_0420"
  },
  "STARTDATE0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "DA_STARTDATE_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "DA-STARTDATE-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "DA_STARTDATE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "DASTARTDATE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "legacy_field": "START_DATE_0415"
  },
  "TERMINATION_DATE_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "legacy_field": "TERMINATION_DATE_0415"
  },
  "TERMINATION-DATE-0415": {

```



```

    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "legacy_field": "TERMINATION_DATE_0415"
  },
  "TERMINATIONDATE0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "legacy_field": "TERMINATION_DATE_0415"
  },
  "DA_TERMINATIONDATE_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "legacy_field": "TERMINATION_DATE_0415"
  },
  "DA-TERMINATIONDATE-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "legacy_field": "TERMINATION_DATE_0415"
  },
  "BIRTH_DATE_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "BIRTH-DATE-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "BIRTH_DATE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "BIRTHDATE0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "DA_BIRTHDATE_4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "DA-BIRTHDATE-4790YEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },
  "DA_BIRTHDATE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "legacy_field": "BIRTH_DATE_0415"
  },

```

```

"DABIRTHDATE": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "DA_BIRTHDATE_4790YEE",
  "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
  "legacy_field": "BIRTH_DATE_0415"
},
"START_DATE_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "legacy_field": "START_DATE_0420"
},
"START-DATE-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "legacy_field": "START_DATE_0420"
},
"STARTDATE0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "legacy_field": "START_DATE_0420"
},
"DA_STARTDATE_479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "legacy_field": "START_DATE_0420"
},
"DA-STARTDATE-479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "legacy_field": "START_DATE_0420"
},
"FINISH_DATE_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"FINISH-DATE-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"FINISH_DATE": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"FINISHDATE0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"DA_FINISHDATE_479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
}

```

```

},
"DA-FINISHDATE-479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"DA_FINISHDATE": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"DAFINISHDATE": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "legacy_field": "FINISH_DATE_0420"
},
"SALARY_GRADE_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARY-GRADE-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARY_GRADE": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARYGRADE0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARY_GRADE_479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARY-GRADE-479TION": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARYGRADE": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_GRADE_479TION",
  "host": "HV-EMPOSITION-SALARY-GRADE-479TION",
  "legacy_field": "SALARY_GRADE_0420"
},
"SALARY_AMOUNT_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "SALARY_AMOUNT_479TION",
  "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",

```

```

    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARY-AMOUNT-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARY_AMOUNT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARYAMOUNT0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARY_AMOUNT_479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARY-AMOUNT-479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "SALARYAMOUNT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "SALARY_AMOUNT_479TION",
    "host": "HV-EMPOSITION-SALARY-AMOUNT-479TION",
    "legacy_field": "SALARY_AMOUNT_0420"
  },
  "BONUS_PERCENT_0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUS-PERCENT-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUS_PERCENT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUSPERCENT0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUS_PERCENT_479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",

```

```

    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUS-PERCENT-479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "BONUSPERCENT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "BONUS_PERCENT_479TION",
    "host": "HV-EMPOSITION-BONUS-PERCENT-479TION",
    "legacy_field": "BONUS_PERCENT_0420"
  },
  "COMMISSION_PERCENT_0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSION-PERCENT-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSION_PERCENT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSIONPERCENT0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSION_PERCENT_479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSION-PERCENT-479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "COMMISSIONPERCENT": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "COMMISSION_PERCENT_479TION",
    "host": "HV-EMPOSITION-COMMISSION-PERCENT-479TION",
    "legacy_field": "COMMISSION_PERCENT_0420"
  },
  "OVERTIME_RATE_0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "OVERTIME_RATE_479TION",
    "host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
    "legacy_field": "OVERTIME_RATE_0420"
  },
  "OVERTIME-RATE-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",

```

```

    "column": "OVERTIME_RATE_479TION",
    "host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
    "legacy_field": "OVERTIME_RATE_0420"
  },
  "OVERTIME_RATE": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "OVERTIME_RATE_479TION",
    "host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
    "legacy_field": "OVERTIME_RATE_0420"
  },
  "OVERTIMERATE0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "OVERTIME_RATE_479TION",
    "host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
    "legacy_field": "OVERTIME_RATE_0420"
  },
  "OVERTIME_RATE_479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "OVERTIME_RATE_479TION",
    "host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
    "legacy_field": "OVERTIME_RATE_0420"
  },
  "OVERTIME-RATE-479TION": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "OVERTIME_RATE_479TION",
    "host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
    "legacy_field": "OVERTIME_RATE_0420"
  },
  "OVERTIMERATE": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "OVERTIME_RATE_479TION",
    "host": "HV-EMPOSITION-OVERTIME-RATE-479TION",
    "legacy_field": "OVERTIME_RATE_0420"
  },
  "SKILL_LEVEL_0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "SKILL_LEVEL_479TISE",
    "host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
    "legacy_field": "SKILL_LEVEL_0425"
  },
  "SKILL-LEVEL-0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "SKILL_LEVEL_479TISE",
    "host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
    "legacy_field": "SKILL_LEVEL_0425"
  },
  "SKILL_LEVEL": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "SKILL_LEVEL_479TISE",
    "host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
    "legacy_field": "SKILL_LEVEL_0425"
  },
  "SKILLLEVEL0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "SKILL_LEVEL_479TISE",
    "host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
    "legacy_field": "SKILL_LEVEL_0425"
  },
  "SKILL_LEVEL_479TISE": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "SKILL_LEVEL_479TISE",
    "host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
    "legacy_field": "SKILL_LEVEL_0425"
  },
  "SKILL-LEVEL-479TISE": {
    "record": "EXPERTISE",

```

```

    "table": "EXPERTISE",
    "column": "SKILL_LEVEL_479TISE",
    "host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
    "legacy_field": "SKILL_LEVEL_0425"
  },
  "SKILLLEVEL": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "SKILL_LEVEL_479TISE",
    "host": "HV-EXPERTISE-SKILL-LEVEL-479TISE",
    "legacy_field": "SKILL_LEVEL_0425"
  },
  "EXPERTISE_DATE_0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "EXPERTISE-DATE-0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "EXPERTISE_DATE": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "EXPERTISEDATE0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "DA_EXPERTISEDATE_479TISE": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "DA-EXPERTISEDATE-479TISE": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "DA_EXPERTISEDATE": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "DAEXPERTISEDATE": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "legacy_field": "EXPERTISE_DATE_0425"
  },
  "CLAIM_DATE_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "legacy_field": "CLAIM_DATE_0430"
  },
  "CLAIM-DATE-0430": {

```

```

"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_CLAIMDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
"legacy_field": "CLAIM_DATE_0430"
},
"CLAIMDATE0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_CLAIMDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
"legacy_field": "CLAIM_DATE_0430"
},
"PATIENT_FIRST_NAME_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_FIRSTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
"legacy_field": "PATIENT_FIRST_NAME_0430"
},
"PATIENT-FIRST-NAME-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_FIRSTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
"legacy_field": "PATIENT_FIRST_NAME_0430"
},
"PATIENTFIRSTNAME0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_FIRSTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-FIRSTNAME-479LAIM",
"legacy_field": "PATIENT_FIRST_NAME_0430"
},
"PATIENT_LAST_NAME_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_LASTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-LASTNAME-479LAIM",
"legacy_field": "PATIENT_LAST_NAME_0430"
},
"PATIENT-LAST-NAME-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_LASTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-LASTNAME-479LAIM",
"legacy_field": "PATIENT_LAST_NAME_0430"
},
"PATIENTLASTNAME0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "PATIENT_LASTNAME_479LAIM",
"host": "HV-HOSPITAL-CLAIM-PATIENT-LASTNAME-479LAIM",
"legacy_field": "PATIENT_LAST_NAME_0430"
},
"PATIENT_BIRTH_DATE_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_PATIENTBIRTHDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
"legacy_field": "PATIENT_BIRTH_DATE_0430"
},
"PATIENT-BIRTH-DATE-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_PATIENTBIRTHDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
"legacy_field": "PATIENT_BIRTH_DATE_0430"
},
"PATIENTBIRTHDATE0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_PATIENTBIRTHDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
"legacy_field": "PATIENT_BIRTH_DATE_0430"
},

```



```

"PATIENT_SEX_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0430"
},
"PATIENT-SEX-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0430"
},
"PATIENTSEX0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0430"
},
"RELATION_TO_EMPLOYEE_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "RELATION_TOEMPLOYEE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
  "legacy_field": "RELATION_TO_EMPLOYEE_0430"
},
"RELATION-TO-EMPLOYEE-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "RELATION_TOEMPLOYEE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
  "legacy_field": "RELATION_TO_EMPLOYEE_0430"
},
"RELATIONTOEMPLOYEE0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "RELATION_TOEMPLOYEE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
  "legacy_field": "RELATION_TO_EMPLOYEE_0430"
},
"HOSPITAL_NAME_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSPITAL-NAME-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSPITAL_NAME": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSPITALNAME0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSPITAL_NAME_479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
}

```

```

},
"HOSPITAL-NAME-479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSPITALNAME": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSPITAL_NAME_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSPITAL-NAME-479LAIM",
  "legacy_field": "HOSPITAL_NAME_0430"
},
"HOSP_STREET_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSP-STREET-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSP_STREET": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSPSTREET0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSP_STREET_479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSP-STREET-479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSPSTREET": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_STREET_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-STREET-479LAIM",
  "legacy_field": "HOSP_STREET_0430"
},
"HOSP_CITY_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_CITY_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
  "legacy_field": "HOSP_CITY_0430"
},
"HOSP-CITY-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "HOSP_CITY_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",

```

```

    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSP_CITY": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSPCITY0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSP_CITY_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSP-CITY-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSPCITY": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_CITY_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-CITY-479LAIM",
    "legacy_field": "HOSP_CITY_0430"
  },
  "HOSP_STATE_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP-STATE-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP_STATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSPSTATE0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP_STATE_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP-STATE-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",

```

```

    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSPSTATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_STATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-STATE-479LAIM",
    "legacy_field": "HOSP_STATE_0430"
  },
  "HOSP_ZIP_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP-ZIP-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP_ZIP": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSPZIP0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP_ZIPFIRSTFIVE_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP-ZIPFIRSTFIVE-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP_ZIPFIRSTFIVE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSPZIPFIRSTFIVE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_0430"
  },
  "HOSP_ZIP_FIRST_FIVE_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_FIRST_FIVE_0430"
  },
  "HOSP-ZIP-FIRST-FIVE-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",

```

```

    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_FIRST_FIVE_0430"
  },
  "HOSP_ZIP_FIRST_FIVE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_FIRST_FIVE_0430"
  },
  "HOSPZIPFIRSTFIVE0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "HOSP_ZIP_FIRST_FIVE_0430"
  },
  "HOSP_ZIP_LAST_FOUR_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSP-ZIP-LAST-FOUR-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSP_ZIP_LAST_FOUR": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSPZIPLASTFOUR0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSP_ZIPLASTFOUR_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSP-ZIPLASTFOUR-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSP_ZIPLASTFOUR": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "HOSPZIPLASTFOUR": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "HOSP_ZIPLASTFOUR_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-HOSP-ZIPLASTFOUR-479LAIM",
    "legacy_field": "HOSP_ZIP_LAST_FOUR_0430"
  },
  "ADMIT_DATE_0430": {
    "record": "HOSPITAL_CLAIM",

```

```

"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"legacy_field": "ADMIT_DATE_0430"
},
"ADMIT-DATE-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"legacy_field": "ADMIT_DATE_0430"
},
"ADMIT_DATE": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"legacy_field": "ADMIT_DATE_0430"
},
"ADMITDATE0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"legacy_field": "ADMIT_DATE_0430"
},
"DA_ADMITDATE_479LAIM": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"legacy_field": "ADMIT_DATE_0430"
},
"DA-ADMITDATE-479LAIM": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"legacy_field": "ADMIT_DATE_0430"
},
"DA_ADMITDATE": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"legacy_field": "ADMIT_DATE_0430"
},
"DAADMITDATE": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"legacy_field": "ADMIT_DATE_0430"
},
"DISCHARGE_DATE_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"legacy_field": "DISCHARGE_DATE_0430"
},
"DISCHARGE-DATE-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"legacy_field": "DISCHARGE_DATE_0430"
},
"DISCHARGE_DATE": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"legacy_field": "DISCHARGE_DATE_0430"
},
"DISCHARGEDATE0430": {

```

```

    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DA_DISCHARGEDATE_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DA-DISCHARGEDATE-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DA_DISCHARGEDATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DADISCHARGEDATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "legacy_field": "DISCHARGE_DATE_0430"
  },
  "DIAGNOSIS_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0430"
  },
  "DIAGNOSIS-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0430"
  },
  "DIAGNOSIS": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0445"
  },
  "DIAGNOSIS0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0430"
  },
  "DIAGNOSIS_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0430"
  },
  "DIAGNOSIS-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DIAGNOSIS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DIAGNOSIS-479LAIM",
    "legacy_field": "DIAGNOSIS_0430"
  },

```

```

"WARD_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD_DAYS_479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD-DAYS-479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_0430"
},
"WARD_DAYS": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_DAYS_0430"
},
"WARD_DAYS": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_DAYS_0430"
},
"WARD_DAYS_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_DAYS_0430"
},
"WARD-DAYS-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_DAYS_0430"
},
"WARD_DAYS0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_DAYS_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-DAYS-479LAIM",
  "legacy_field": "WARD_DAYS_0430"
}

```



```

},
"WARD_RATE_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARD-RATE-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARD_RATE": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARDRATE0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARD_RATE_479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARD-RATE-479LAIM": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARDRATE": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_RATE_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-RATE-479LAIM",
  "legacy_field": "WARD_RATE_0430"
},
"WARD_TOTAL_0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_TOTAL_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
  "legacy_field": "WARD_TOTAL_0430"
},
"WARD-TOTAL-0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_TOTAL_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
  "legacy_field": "WARD_TOTAL_0430"
},
"WARD_TOTAL": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_TOTAL_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
  "legacy_field": "WARD_TOTAL_0430"
},
"WARDTOTAL0430": {
  "record": "HOSPITAL_CLAIM",
  "table": "HOSPITAL_CLAIM",
  "column": "WARD_TOTAL_479LAIM",
  "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",

```

```

    "legacy_field": "WARD_TOTAL_0430"
  },
  "WARD_TOTAL_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
    "legacy_field": "WARD_TOTAL_0430"
  },
  "WARD-TOTAL-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
    "legacy_field": "WARD_TOTAL_0430"
  },
  "WARDTOTAL": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "WARD_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-WARD-TOTAL-479LAIM",
    "legacy_field": "WARD_TOTAL_0430"
  },
  "SEMI_DAYS_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI-DAYS-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI_DAYS": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMIDAYS0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI_DAYS_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI-DAYS-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMIDAYS": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_DAYS_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-DAYS-479LAIM",
    "legacy_field": "SEMI_DAYS_0430"
  },
  "SEMI_RATE_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",

```

```

    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI-RATE-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI_RATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI-RATE-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI_RATE_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI-RATE-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI-RATE": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_RATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-RATE-479LAIM",
    "legacy_field": "SEMI_RATE_0430"
  },
  "SEMI_TOTAL_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMI-TOTAL-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMI_TOTAL": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMITOTAL0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMI_TOTAL_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",

```

```

    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMI-TOTAL-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "SEMITOTAL": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "SEMI_TOTAL_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-SEMI-TOTAL-479LAIM",
    "legacy_field": "SEMI_TOTAL_0430"
  },
  "DELIVERY_COST_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERY-COST-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERY_COST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERYCOST0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERY_COST_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERY-COST-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "DELIVERYCOST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DELIVERY_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DELIVERY-COST-479LAIM",
    "legacy_field": "DELIVERY_COST_0430"
  },
  "ANESTHESIA_COST_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIA-COST-0430": {
    "record": "HOSPITAL_CLAIM",

```

```

    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIA_COST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIACOST0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIA_COST_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIA-COST-479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "ANESTHESIACOST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "ANESTHESIA_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-ANESTHESIA-COST-479LAIM",
    "legacy_field": "ANESTHESIA_COST_0430"
  },
  "LAB_COST_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LAB-COST-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LAB_COST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LABCOST0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LAB_COST_479LAIM": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LAB-COST-479LAIM": {

```

```

    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "LABCOST": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "LAB_COST_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-LAB-COST-479LAIM",
    "legacy_field": "LAB_COST_0430"
  },
  "INS_PLAN_CODE_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
    "legacy_field": "INS_PLAN_CODE_0435"
  },
  "INS-PLAN-CODE-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
    "legacy_field": "INS_PLAN_CODE_0435"
  },
  "INSPLANCODE0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
    "legacy_field": "INS_PLAN_CODE_0435"
  },
  "INS_PLANCODE_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
    "legacy_field": "INS_PLAN_CODE_0435"
  },
  "INS-PLANCODE-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN",
    "legacy_field": "INS_PLAN_CODE_0435"
  },
  "INS_CO_NAME_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_CONAME_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
    "legacy_field": "INS_CO_NAME_0435"
  },
  "INS-CO-NAME-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_CONAME_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
    "legacy_field": "INS_CO_NAME_0435"
  },
  "INS_CO_NAME": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_CONAME_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
    "legacy_field": "INS_CO_NAME_0435"
  },
  "INSCONAME0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_CONAME_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
    "legacy_field": "INS_CO_NAME_0435"
  },
  },

```

```

"INS_CONAME_479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CONAME_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
  "legacy_field": "INS_CO_NAME_0435"
},
"INS-CONAME-479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CONAME_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
  "legacy_field": "INS_CO_NAME_0435"
},
"INS_CONAME": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CONAME_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
  "legacy_field": "INS_CO_NAME_0435"
},
"INSCONAME": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_CONAME_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-CONAME-479PLAN",
  "legacy_field": "INS_CO_NAME_0435"
},
"INS_CO_STREET_0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS-CO-STREET-0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS_CO_STREET": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INSCOSTREET0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS_COSTREET_479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS-COSTREET-479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS_COSTREET": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
}

```

```

},
"INSCOSTREET": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTREET_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTREET-479PLAN",
  "legacy_field": "INS_CO_STREET_0435"
},
"INS_CO_CITY_0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS-CO-CITY-0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS_CO_CITY": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INSCOCITY0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS_COCITY_479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS-COCITY-479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS_COCITY": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INSCOCITY": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COCITY_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COCITY-479PLAN",
  "legacy_field": "INS_CO_CITY_0435"
},
"INS_CO_STATE_0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTATE_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
  "legacy_field": "INS_CO_STATE_0435"
},
"INS-CO-STATE-0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "INS_COSTATE_479PLAN",
  "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",

```



```

    "legacy_field": "INS_CO_STATE_0435"
  },
  "INS_CO_STATE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INSCOSTATE0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INS_COSTATE_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INS-COSTATE-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INS_COSTATE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INSCOSTATE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COSTATE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COSTATE-479PLAN",
    "legacy_field": "INS_CO_STATE_0435"
  },
  "INS_CO_ZIP_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS-CO-ZIP-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS_CO_ZIP": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INSCOZIP0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS_COZIPFIRSTFIVE_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",

```

```

    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS-COZIPFIRSTFIVE-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS_COZIPFIRSTFIVE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INSCOZIPFIRSTFIVE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_0435"
  },
  "INS_CO_ZIP_FIRST_FIVE_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_FIRST_FIVE_0435"
  },
  "INS-CO-ZIP-FIRST-FIVE-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_FIRST_FIVE_0435"
  },
  "INS_CO_ZIP_FIRST_FIVE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_FIRST_FIVE_0435"
  },
  "INSCOZIPFIRSTFIVE0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPFIRSTFIVE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPFIRSTFIVE-479PLAN",
    "legacy_field": "INS_CO_ZIP_FIRST_FIVE_0435"
  },
  "INS_CO_ZIP_LAST_FOUR_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS-CO-ZIP-LAST-FOUR-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS_CO_ZIP_LAST_FOUR": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INSCOZIPLASTFOUR0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",

```

```

    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS_COZIPLASTFOUR_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS-COZIPLASTFOUR-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS_COZIPLASTFOUR": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INSCOZIPLASTFOUR": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COZIPLASTFOUR_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COZIPLASTFOUR-479PLAN",
    "legacy_field": "INS_CO_ZIP_LAST_FOUR_0435"
  },
  "INS_CO_PHONE_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INS-CO-PHONE-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INS_CO_PHONE": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INSCOPHONE0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INS_COPHONE_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INS-COPHONE-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "INS_COPHONE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
    "legacy_field": "INS_CO_PHONE_0435"
  },
  "INS_COPHONE": {
    "record": "INSURANCE_PLAN",

```

```

"table": "INSURANCE_PLAN",
"column": "INS_COPHONE_479PLAN",
"host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
"legacy_field": "INS_CO_PHONE_0435"
},
"INSCOPHONE": {
"record": "INSURANCE_PLAN",
"table": "INSURANCE_PLAN",
"column": "INS_COPHONE_479PLAN",
"host": "HV-INSURANCE-PLAN-INS-COPHONE-479PLAN",
"legacy_field": "INS_CO_PHONE_0435"
},
"GROUP_NUMBER_0435": {
"record": "INSURANCE_PLAN",
"table": "INSURANCE_PLAN",
"column": "GROUP_NUMBER_479PLAN",
"host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
"legacy_field": "GROUP_NUMBER_0435"
},
"GROUP-NUMBER-0435": {
"record": "INSURANCE_PLAN",
"table": "INSURANCE_PLAN",
"column": "GROUP_NUMBER_479PLAN",
"host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
"legacy_field": "GROUP_NUMBER_0435"
},
"GROUP_NUMBER": {
"record": "INSURANCE_PLAN",
"table": "INSURANCE_PLAN",
"column": "GROUP_NUMBER_479PLAN",
"host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
"legacy_field": "GROUP_NUMBER_0435"
},
"GROUPNUMBER0435": {
"record": "INSURANCE_PLAN",
"table": "INSURANCE_PLAN",
"column": "GROUP_NUMBER_479PLAN",
"host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
"legacy_field": "GROUP_NUMBER_0435"
},
"GROUP_NUMBER_479PLAN": {
"record": "INSURANCE_PLAN",
"table": "INSURANCE_PLAN",
"column": "GROUP_NUMBER_479PLAN",
"host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
"legacy_field": "GROUP_NUMBER_0435"
},
"GROUP-NUMBER-479PLAN": {
"record": "INSURANCE_PLAN",
"table": "INSURANCE_PLAN",
"column": "GROUP_NUMBER_479PLAN",
"host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
"legacy_field": "GROUP_NUMBER_0435"
},
"GROUPNUMBER": {
"record": "INSURANCE_PLAN",
"table": "INSURANCE_PLAN",
"column": "GROUP_NUMBER_479PLAN",
"host": "HV-INSURANCE-PLAN-GROUP-NUMBER-479PLAN",
"legacy_field": "GROUP_NUMBER_0435"
},
"DEDUCT_0435": {
"record": "INSURANCE_PLAN",
"table": "INSURANCE_PLAN",
"column": "DEDUCT_479PLAN",
"host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
"legacy_field": "DEDUCT_0435"
},
"DEDUCT-0435": {
"record": "INSURANCE_PLAN",
"table": "INSURANCE_PLAN",
"column": "DEDUCT_479PLAN",
"host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
"legacy_field": "DEDUCT_0435"
},
"DEDUCT": {

```

```

    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEDUCT_479PLAN",
    "host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
    "legacy_field": "DEDUCT_0435"
  },
  "DEDUCT0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEDUCT_479PLAN",
    "host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
    "legacy_field": "DEDUCT_0435"
  },
  "DEDUCT_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEDUCT_479PLAN",
    "host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
    "legacy_field": "DEDUCT_0435"
  },
  "DEDUCT-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "DEDUCT_479PLAN",
    "host": "HV-INSURANCE-PLAN-DEDUCT-479PLAN",
    "legacy_field": "DEDUCT_0435"
  },
  "MAXIMUM_LIFE_COST_0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUM-LIFE-COST-0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUM_LIFE_COST": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUMLIFECOST0435": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUM_LIFECOST_479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUM-LIFECOST-479PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },
  "MAXIMUM_LIFECOST": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "column": "MAXIMUM_LIFECOST_479PLAN",
    "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
    "legacy_field": "MAXIMUM_LIFE_COST_0435"
  },

```

```

"MAXIMUMLIFECOST": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "MAXIMUM_LIFECOST_479PLAN",
  "host": "HV-INSURANCE-PLAN-MAXIMUM-LIFECOST-479PLAN",
  "legacy_field": "MAXIMUM_LIFE_COST_0435"
},
"FAMILY_COST_0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILY-COST-0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILY_COST": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILYCOST0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILY_COST_479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILY-COST-479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"FAMILYCOST": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "FAMILY_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-FAMILY-COST-479PLAN",
  "legacy_field": "FAMILY_COST_0435"
},
"DEP_COST_0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"DEP-COST-0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"DEP_COST": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
}

```

```

},
"DEPCOST0435": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"DEP_COST_479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"DEP-COST-479PLAN": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"DEPCOST": {
  "record": "INSURANCE_PLAN",
  "table": "INSURANCE_PLAN",
  "column": "DEP_COST_479PLAN",
  "host": "HV-INSURANCE-PLAN-DEP-COST-479PLAN",
  "legacy_field": "DEP_COST_0435"
},
"JOB_ID_0440": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOB-ID-0440": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOB_ID": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOBID0440": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOB_ID_479JOB": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOB-ID-479JOB": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",
  "legacy_field": "JOB_ID_0440"
},
"JOBID": {
  "record": "JOB",
  "table": "JOB",
  "column": "JOB_ID_479JOB",
  "host": "HV-JOB-JOB-ID-479JOB",

```

```

    "legacy_field": "JOB_ID_0440"
  },
  "TITLE_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "TITLE-0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "TITLE": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "TITLE0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "TITLE_479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "TITLE-479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "TITLE_479JOB",
    "host": "HV-JOB-TITLE-479JOB",
    "legacy_field": "TITLE_0440"
  },
  "DESCRIPTION_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION-0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION_LINE_479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",

```



```

    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION-LINE-479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION_LINE": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_LINE_0440"
  },
  "DESCRIPTIONLINE": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_0440"
  },
  "DESCRIPTION_LINE_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_LINE_0440"
  },
  "DESCRIPTION-LINE-0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_LINE_0440"
  },
  "DESCRIPTIONLINE0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "DESCRIPTION_LINE_479JOB",
    "host": "HV-JOB-DESCRIPTION-LINE-479JOB",
    "legacy_field": "DESCRIPTION_LINE_0440"
  },
  "REQUIREMENT_LINE_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENT-LINE-0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENT_LINE": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENTLINE0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENT_LINE_479JOB": {
    "record": "JOB",
    "table": "JOB",

```

```

    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENT-LINE-479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "REQUIREMENTLINE": {
    "record": "JOB",
    "table": "JOB",
    "column": "REQUIREMENT_LINE_479JOB",
    "host": "HV-JOB-REQUIREMENT-LINE-479JOB",
    "legacy_field": "REQUIREMENT_LINE_0440"
  },
  "MINIMUM_SALARY_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUM-SALARY-0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUM_SALARY": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUMSALARY0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUM_SALARY_479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUM-SALARY-479JOB": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MINIMUMSALARY": {
    "record": "JOB",
    "table": "JOB",
    "column": "MINIMUM_SALARY_479JOB",
    "host": "HV-JOB-MINIMUM-SALARY-479JOB",
    "legacy_field": "MINIMUM_SALARY_0440"
  },
  "MAXIMUM_SALARY_0440": {
    "record": "JOB",
    "table": "JOB",
    "column": "MAXIMUM_SALARY_479JOB",
    "host": "HV-JOB-MAXIMUM-SALARY-479JOB",
    "legacy_field": "MAXIMUM_SALARY_0440"
  },
  "MAXIMUM-SALARY-0440": {
    "record": "JOB",

```

```

"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"MAXIMUM_SALARY": {
"record": "JOB",
"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"MAXIMUMSALARY0440": {
"record": "JOB",
"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"MAXIMUM_SALARY_479JOB": {
"record": "JOB",
"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"MAXIMUM-SALARY-479JOB": {
"record": "JOB",
"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"MAXIMUMSALARY": {
"record": "JOB",
"table": "JOB",
"column": "MAXIMUM_SALARY_479JOB",
"host": "HV-JOB-MAXIMUM-SALARY-479JOB",
"legacy_field": "MAXIMUM_SALARY_0440"
},
"SALARY_GRADES_0440": {
"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"SALARY-GRADES-0440": {
"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"SALARY_GRADES": {
"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"SALARYGRADES0440": {
"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"SALARY_GRADES_479JOB": {
"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"SALARY-GRADES-479JOB": {

```

```

"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"SALARYGRADES": {
"record": "JOB",
"table": "JOB",
"column": "SALARY_GRADES_479JOB",
"host": "HV-JOB-SALARY-GRADES-479JOB",
"legacy_field": "SALARY_GRADES_0440"
},
"NUMBER_OF_POSITIONS_0440": {
"record": "JOB",
"table": "JOB",
"column": "NUMBER_OFPOSITIONS_479JOB",
"host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
"legacy_field": "NUMBER_OF_POSITIONS_0440"
},
"NUMBER-OF-POSITIONS-0440": {
"record": "JOB",
"table": "JOB",
"column": "NUMBER_OFPOSITIONS_479JOB",
"host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
"legacy_field": "NUMBER_OF_POSITIONS_0440"
},
"NUMBER_OF_POSITIONS": {
"record": "JOB",
"table": "JOB",
"column": "NUMBER_OFPOSITIONS_479JOB",
"host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
"legacy_field": "NUMBER_OF_POSITIONS_0440"
},
"NUMBEROFPOSITIONS0440": {
"record": "JOB",
"table": "JOB",
"column": "NUMBER_OFPOSITIONS_479JOB",
"host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
"legacy_field": "NUMBER_OF_POSITIONS_0440"
},
"NUMBER_OFPOSITIONS_479JOB": {
"record": "JOB",
"table": "JOB",
"column": "NUMBER_OFPOSITIONS_479JOB",
"host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
"legacy_field": "NUMBER_OF_POSITIONS_0440"
},
"NUMBER-OFPOSITIONS-479JOB": {
"record": "JOB",
"table": "JOB",
"column": "NUMBER_OFPOSITIONS_479JOB",
"host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
"legacy_field": "NUMBER_OF_POSITIONS_0440"
},
"NUMBER_OFPOSITIONS": {
"record": "JOB",
"table": "JOB",
"column": "NUMBER_OFPOSITIONS_479JOB",
"host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
"legacy_field": "NUMBER_OF_POSITIONS_0440"
},
"NUMBEROFPOSITIONS": {
"record": "JOB",
"table": "JOB",
"column": "NUMBER_OFPOSITIONS_479JOB",
"host": "HV-JOB-NUMBER-OFPOSITIONS-479JOB",
"legacy_field": "NUMBER_OF_POSITIONS_0440"
},
"NUMBER_OPEN_0440": {
"record": "JOB",
"table": "JOB",
"column": "NUMBER_OPEN_479JOB",
"host": "HV-JOB-NUMBER-OPEN-479JOB",
"legacy_field": "NUMBER_OPEN_0440"
},

```

```

"NUMBER-OPEN-0440": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"NUMBER_OPEN": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"NUMBEROPEN0440": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"NUMBER_OPEN_479JOB": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"NUMBER-OPEN-479JOB": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"NUMBEROPEN": {
  "record": "JOB",
  "table": "JOB",
  "column": "NUMBER_OPEN_479JOB",
  "host": "HV-JOB-NUMBER-OPEN-479JOB",
  "legacy_field": "NUMBER_OPEN_0440"
},
"CLAIM_DATE_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "legacy_field": "CLAIM_DATE_0445"
},
"CLAIM-DATE-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "legacy_field": "CLAIM_DATE_0445"
},
"CLAIMDATE0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "legacy_field": "CLAIM_DATE_0445"
},
"PATIENT_FIRST_NAME_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_FIRSTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-FIRSTNAME-479LAIM",
  "legacy_field": "PATIENT_FIRST_NAME_0445"
},
"PATIENT-FIRST-NAME-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_FIRSTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-FIRSTNAME-479LAIM",
  "legacy_field": "PATIENT_FIRST_NAME_0445"
}

```

```

},
"PATIENTFIRSTNAME0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_FIRSTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-FIRSTNAME-479LAIM",
  "legacy_field": "PATIENT_FIRST_NAME_0445"
},
"PATIENT_LAST_NAME_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_LASTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-LASTNAME-479LAIM",
  "legacy_field": "PATIENT_LAST_NAME_0445"
},
"PATIENT-LAST-NAME-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_LASTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-LASTNAME-479LAIM",
  "legacy_field": "PATIENT_LAST_NAME_0445"
},
"PATIENTLASTNAME0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_LASTNAME_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-LASTNAME-479LAIM",
  "legacy_field": "PATIENT_LAST_NAME_0445"
},
"PATIENT_BIRTH_DATE_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0445"
},
"PATIENT-BIRTH-DATE-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0445"
},
"PATIENTBIRTHDATE0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "legacy_field": "PATIENT_BIRTH_DATE_0445"
},
"PATIENT_SEX_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0445"
},
"PATIENT-SEX-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0445"
},
"PATIENTSEX0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PATIENT_SEX_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PATIENT-SEX-479LAIM",
  "legacy_field": "PATIENT_SEX_0445"
},
"RELATION_TO_EMPLOYEE_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "RELATION_TOEMPLOYEE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-RELATION-TOEMPLOYEE-479LAIM",

```

```

    "legacy_field": "RELATION_TO_EMPLOYEE_0445"
  },
  "RELATION-TO-EMPLOYEE-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0445"
  },
  "RELATIONTOEMPLOYEE0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "RELATION_TOEMPLOYEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-RELATION-TOEMPLOYEE-479LAIM",
    "legacy_field": "RELATION_TO_EMPLOYEE_0445"
  },
  "PHYSICIAN_FIRST_NAME_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN-FIRST-NAME-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN_FIRST_NAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIANFIRSTNAME0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN_FIRSTNAME_479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN-FIRSTNAME-479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN_FIRSTNAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIANFIRSTNAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_FIRSTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-FIRSTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_FIRST_NAME_0445"
  },
  "PHYSICIAN_LAST_NAME_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",

```

```

    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN-LAST-NAME-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN_LAST_NAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIANLASTNAME0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN_LASTNAME_479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN-LASTNAME-479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN_LASTNAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIANLASTNAME": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_LASTNAME_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-LASTNAME-479LAIM",
    "legacy_field": "PHYSICIAN_LAST_NAME_0445"
  },
  "PHYSICIAN_STREET_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN-STREET-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN_STREET": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIANSTREET0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",

```



```

    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN_STREET_479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN-STREET-479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIANSTREET": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STREET_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STREET-479LAIM",
    "legacy_field": "PHYSICIAN_STREET_0445"
  },
  "PHYSICIAN_CITY_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN-CITY-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN_CITY": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIANCITY0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN_CITY_479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN-CITY-479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIANCITY": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_CITY_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-CITY-479LAIM",
    "legacy_field": "PHYSICIAN_CITY_0445"
  },
  "PHYSICIAN_STATE_0445": {
    "record": "NON_HOSP_CLAIM",

```

```

    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN-STATE-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN_STATE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIANSTATE0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN_STATE_479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN-STATE-479LAIM": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIANSTATE": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_STATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-STATE-479LAIM",
    "legacy_field": "PHYSICIAN_STATE_0445"
  },
  "PHYSICIAN_ZIP_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIAN-ZIP-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIAN_ZIP": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIANZIP0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
    "legacy_field": "PHYSICIAN_ZIP_0445"
  },
  "PHYSICIAN_ZIPFIRSTFIVE_479LAIM": {

```

```

"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
"host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
"legacy_field": "PHYSICIAN_ZIP_0445"
},
"PHYSICIAN-ZIPFIRSTFIVE-479LAIM": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
"host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
"legacy_field": "PHYSICIAN_ZIP_0445"
},
"PHYSICIAN_ZIPFIRSTFIVE": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
"host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
"legacy_field": "PHYSICIAN_ZIP_0445"
},
"PHYSICIANZIPFIRSTFIVE": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
"host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
"legacy_field": "PHYSICIAN_ZIP_0445"
},
"PHYSICIAN_ZIP_FIRST_FIVE_0445": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
"host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
"legacy_field": "PHYSICIAN_ZIP_FIRST_FIVE_0445"
},
"PHYSICIAN-ZIP-FIRST-FIVE-0445": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
"host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
"legacy_field": "PHYSICIAN_ZIP_FIRST_FIVE_0445"
},
"PHYSICIAN_ZIP_FIRST_FIVE": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
"host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
"legacy_field": "PHYSICIAN_ZIP_FIRST_FIVE_0445"
},
"PHYSICIANZIPFIRSTFIVE0445": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "PHYSICIAN_ZIPFIRSTFIVE_479LAIM",
"host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPFIRSTFIVE-479LAIM",
"legacy_field": "PHYSICIAN_ZIP_FIRST_FIVE_0445"
},
"PHYSICIAN_ZIP_LAST_FOUR_0445": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
"host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
"legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIAN-ZIP-LAST-FOUR-0445": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
"host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
"legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIAN_ZIP_LAST_FOUR": {
"record": "NON_HOSP_CLAIM",
"table": "NON_HOSP_CLAIM",
"column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
"host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
"legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},

```

```

"PHYSICIANZIPLASTFOUR0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
  "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIAN_ZIPLASTFOUR_479LAIM": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
  "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIAN-ZIPLASTFOUR-479LAIM": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
  "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIAN_ZIPLASTFOUR": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
  "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIANZIPLASTFOUR": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ZIPLASTFOUR_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ZIPLASTFOUR-479LAIM",
  "legacy_field": "PHYSICIAN_ZIP_LAST_FOUR_0445"
},
"PHYSICIAN_ID_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"PHYSICIAN-ID-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"PHYSICIAN_ID": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"PHYSICIANID0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"PHYSICIAN_ID_479LAIM": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"PHYSICIAN-ID-479LAIM": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
}

```

```

},
"PHYSICIANID": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PHYSICIAN_ID_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PHYSICIAN-ID-479LAIM",
  "legacy_field": "PHYSICIAN_ID_0445"
},
"DIAGNOSIS_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DIAGNOSIS_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DIAGNOSIS-479LAIM",
  "legacy_field": "DIAGNOSIS_0445"
},
"DIAGNOSIS-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DIAGNOSIS_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DIAGNOSIS-479LAIM",
  "legacy_field": "DIAGNOSIS_0445"
},
"DIAGNOSIS0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DIAGNOSIS_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DIAGNOSIS-479LAIM",
  "legacy_field": "DIAGNOSIS_0445"
},
"NUMBER_OF_PROCEDURES_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "NUMBER_OFPROCEDURES_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
  "legacy_field": "NUMBER_OF_PROCEDURES_0445"
},
"NUMBER-OF-PROCEDURES-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "NUMBER_OFPROCEDURES_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
  "legacy_field": "NUMBER_OF_PROCEDURES_0445"
},
"NUMBEROFPROCEDURES0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "NUMBER_OFPROCEDURES_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-NUMBER-OFPROCEDURES-479LAIM",
  "legacy_field": "NUMBER_OF_PROCEDURES_0445"
},
"SERVICE_DATE_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_SERVICEDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0445"
},
"SERVICE-DATE-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_SERVICEDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0445"
},
"SERVICEDATE0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_SERVICEDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
  "legacy_field": "SERVICE_DATE_0445"
},
"PROCEDURE_CODE_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "PROCEDURE_CODE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-PROCEDURE-CODE-479LAIM",

```

```

    "legacy_field": "PROCEDURE_CODE_0445"
  },
  "PROCEDURE-CODE-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PROCEDURE_CODE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PROCEDURE-CODE-479LAIM",
    "legacy_field": "PROCEDURE_CODE_0445"
  },
  "PROCEDURECODE0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "PROCEDURE_CODE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-PROCEDURE-CODE-479LAIM",
    "legacy_field": "PROCEDURE_CODE_0445"
  },
  "DESCRIPTION_OF_SERVICE_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DESCRIPTION_OFSERVICE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
    "legacy_field": "DESCRIPTION_OF_SERVICE_0445"
  },
  "DESCRIPTION-OF-SERVICE-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DESCRIPTION_OFSERVICE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
    "legacy_field": "DESCRIPTION_OF_SERVICE_0445"
  },
  "DESCRIPTIONOFSERVICE0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DESCRIPTION_OFSERVICE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DESCRIPTION-OFSERVICE-479LAIM",
    "legacy_field": "DESCRIPTION_OF_SERVICE_0445"
  },
  "FEE_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0445"
  },
  "FEE-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0445"
  },
  "FEE0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "FEE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-FEE-479LAIM",
    "legacy_field": "FEE_0445"
  },
  "OFFICE_CODE_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE-CODE-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE_CODE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",

```

```

    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICECODE0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE_CODE_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE-CODE-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICECODE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE",
    "legacy_field": "OFFICE_CODE_0450"
  },
  "OFFICE_STREET_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE-STREET-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE_STREET": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICESTREET0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE_STREET_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE-STREET-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICESTREET": {
    "record": "OFFICE",
    "table": "OFFICE",

```

```

    "column": "OFFICE_STREET_479FICE",
    "host": "HV-OFFICE-OFFICE-STREET-479FICE",
    "legacy_field": "OFFICE_STREET_0450"
  },
  "OFFICE_CITY_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE-CITY-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE_CITY": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICECITY0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE_CITY_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE-CITY-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICECITY": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_CITY_479FICE",
    "host": "HV-OFFICE-OFFICE-CITY-479FICE",
    "legacy_field": "OFFICE_CITY_0450"
  },
  "OFFICE_STATE_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE-STATE-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE_STATE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICESTATE0450": {
    "record": "OFFICE",

```



```

    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE_STATE_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE-STATE-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICESTATE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_STATE_479FICE",
    "host": "HV-OFFICE-OFFICE-STATE-479FICE",
    "legacy_field": "OFFICE_STATE_0450"
  },
  "OFFICE_ZIP_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE-ZIP-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE_ZIP": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICEZIP0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE_ZIPFIRSTFIVE_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE-ZIPFIRSTFIVE-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE_ZIPFIRSTFIVE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICEZIPFIRSTFIVE": {

```

```

    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_0450"
  },
  "OFFICE_ZIP_FIRST_FIVE_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_FIRST_FIVE_0450"
  },
  "OFFICE-ZIP-FIRST-FIVE-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_FIRST_FIVE_0450"
  },
  "OFFICE_ZIP_FIRST_FIVE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_FIRST_FIVE_0450"
  },
  "OFFICEZIPFIRSTFIVE0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPFIRSTFIVE_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPFIRSTFIVE-479FICE",
    "legacy_field": "OFFICE_ZIP_FIRST_FIVE_0450"
  },
  "OFFICE_ZIP_LAST_FOUR_0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },
  "OFFICE-ZIP-LAST-FOUR-0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },
  "OFFICE_ZIP_LAST_FOUR": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },
  "OFFICEZIPLASTFOUR0450": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },
  "OFFICE_ZIPLASTFOUR_479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },
  "OFFICE-ZIPLASTFOUR-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "OFFICE_ZIPLASTFOUR_479FICE",
    "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
    "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
  },

```

```

"OFFICE_ZIPLASTFOUR": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_ZIPLASTFOUR_479FICE",
  "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
  "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
},
"OFFICEZIPLASTFOUR": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_ZIPLASTFOUR_479FICE",
  "host": "HV-OFFICE-OFFICE-ZIPLASTFOUR-479FICE",
  "legacy_field": "OFFICE_ZIP_LAST_FOUR_0450"
},
"OFFICE_PHONE_0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE-PHONE-0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE_PHONE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICEPHONE0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE_PHONE_479FICE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE-PHONE-479FICE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICEPHONE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_PHONE_479FICE",
  "host": "HV-OFFICE-OFFICE-PHONE-479FICE",
  "legacy_field": "OFFICE_PHONE_0450"
},
"OFFICE_AREA_CODE_0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICE-AREA-CODE-0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
}

```

```

},
"OFFICE_AREA_CODE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICEAREACODE0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICE_AREACODE_479FICE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICE-AREACODE-479FICE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICE_AREACODE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"OFFICEAREACODE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "OFFICE_AREACODE_479FICE",
  "host": "HV-OFFICE-OFFICE-AREACODE-479FICE",
  "legacy_field": "OFFICE_AREA_CODE_0450"
},
"SPEED_DIAL_0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_479FICE",
  "host": "HV-OFFICE-SPEED-DIAL-479FICE",
  "legacy_field": "SPEED_DIAL_0450"
},
"SPEED-DIAL-0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_479FICE",
  "host": "HV-OFFICE-SPEED-DIAL-479FICE",
  "legacy_field": "SPEED_DIAL_0450"
},
"SPEED_DIAL": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_479FICE",
  "host": "HV-OFFICE-SPEED-DIAL-479FICE",
  "legacy_field": "SPEED_DIAL_0450"
},
"SPEEDDIAL0450": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_479FICE",
  "host": "HV-OFFICE-SPEED-DIAL-479FICE",
  "legacy_field": "SPEED_DIAL_0450"
},
"SPEED_DIAL_479FICE": {
  "record": "OFFICE",
  "table": "OFFICE",
  "column": "SPEED_DIAL_479FICE",
  "host": "HV-OFFICE-SPEED-DIAL-479FICE",

```

```

    "legacy_field": "SPEED_DIAL_0450"
  },
  "SPEED-DIAL-479FICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "SPEED_DIAL_479FICE",
    "host": "HV-OFFICE-SPEED-DIAL-479FICE",
    "legacy_field": "SPEED_DIAL_0450"
  },
  "SPEEDDIAL": {
    "record": "OFFICE",
    "table": "OFFICE",
    "column": "SPEED_DIAL_479FICE",
    "host": "HV-OFFICE-SPEED-DIAL-479FICE",
    "legacy_field": "SPEED_DIAL_0450"
  },
  "SKILL_ID_0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILL-ID-0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILL_ID": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILLID0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILL_ID_479KILL": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILL-ID-479KILL": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILLID": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL",
    "legacy_field": "SKILL_ID_0455"
  },
  "SKILL_NAME_0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILL-NAME-0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",

```

```

    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILL_NAME": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILLNAME0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILL_NAME_479KILL": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILL-NAME-479KILL": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILLNAME": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_NAME_479KILL",
    "host": "HV-SKILL-SKILL-NAME-479KILL",
    "legacy_field": "SKILL_NAME_0455"
  },
  "SKILL_DESCRIPTION_0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILL-DESCRIPTION-0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILL_DESCRIPTION": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILLDESCRIPTION0455": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILL_DESCRIPTION_479KILL": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILL-DESCRIPTION-479KILL": {
    "record": "SKILL",
    "table": "SKILL",

```

```

    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "SKILLDESCRIPTION": {
    "record": "SKILL",
    "table": "SKILL",
    "column": "SKILL_DESCRIPTION_479KILL",
    "host": "HV-SKILL-SKILL-DESCRIPTION-479KILL",
    "legacy_field": "SKILL_DESCRIPTION_0455"
  },
  "STRUCTURE_CODE_0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURE-CODE-0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURE_CODE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURECODE0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURE_CODE_479TURE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURE-CODE-479TURE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURECODE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "STRUCTURE_CODE_479TURE",
    "host": "HV-STRUCTURE-STRUCTURE-CODE-479TURE",
    "legacy_field": "STRUCTURE_CODE_0460"
  },
  "STRUCTURE_DATE_0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "STRUCTURE-DATE-0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "STRUCTURE_DATE": {
    "record": "STRUCTURE",

```

```

    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "STRUCTUREDATE0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "DA_STRUCTUREDATE_479TURE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "DA-STRUCTUREDATE-479TURE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "DA_STRUCTUREDATE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  },
  "DASTRUCTUREDATE": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "legacy_field": "STRUCTURE_DATE_0460"
  }
},
"calc_key_map": {
  "DEPARTMENT": {
    "record": "DEPARTMENT",
    "table": "DEPARTMENT",
    "key": "DEPT_ID_479MENT",
    "primary_key": "DEPT_ID_479MENT",
    "primary_keys": [
      "DEPT_ID_479MENT"
    ],
    "column": "DEPT_ID_479MENT",
    "host": "HV-DEPARTMENT-DEPT-ID-479MENT"
  },
  "EMPLOYEE": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "key": "EMP_ID_4790YEE",
    "primary_key": "EMP_ID_4790YEE",
    "primary_keys": [
      "EMP_ID_4790YEE"
    ],
    "column": "EMP_ID_4790YEE",
    "host": "HV-EMPLOYEE-EMP-ID-4790YEE"
  },
  "INSURANCE_PLAN": {
    "record": "INSURANCE_PLAN",
    "table": "INSURANCE_PLAN",
    "key": "INS_PLANCODE_479PLAN",
    "primary_key": "INS_PLANCODE_479PLAN",
    "primary_keys": [
      "INS_PLANCODE_479PLAN"
    ],
    "column": "INS_PLANCODE_479PLAN",
    "host": "HV-INSURANCE-PLAN-INS-PLANCODE-479PLAN"
  },
  "JOB": {

```



```

    "record": "JOB",
    "table": "JOB",
    "key": "JOB_ID_479JOB",
    "primary_key": "JOB_ID_479JOB",
    "primary_keys": [
        "JOB_ID_479JOB"
    ],
    "column": "JOB_ID_479JOB",
    "host": "HV-JOB-JOB-ID-479JOB"
},
"OFFICE": {
    "record": "OFFICE",
    "table": "OFFICE",
    "key": "OFFICE_CODE_479FICE",
    "primary_key": "OFFICE_CODE_479FICE",
    "primary_keys": [
        "OFFICE_CODE_479FICE"
    ],
    "column": "OFFICE_CODE_479FICE",
    "host": "HV-OFFICE-OFFICE-CODE-479FICE"
},
"SKILL": {
    "record": "SKILL",
    "table": "SKILL",
    "key": "SKILL_ID_479KILL",
    "primary_key": "SKILL_ID_479KILL",
    "primary_keys": [
        "SKILL_ID_479KILL"
    ],
    "column": "SKILL_ID_479KILL",
    "host": "HV-SKILL-SKILL-ID-479KILL"
}
},
"relationships": [
    {
        "set_name": "COVERAGE_CLAIMS",
        "parent_record": "COVERAGE",
        "child_record": "DENTAL_CLAIM",
        "owner_record": "COVERAGE",
        "member_record": "DENTAL_CLAIM",
        "parent_table": "COVERAGE",
        "child_table": "DENTAL_CLAIM",
        "parent_key": "ID_RECORD_RAGE",
        "parent_keys": [
            "ID_RECORD_RAGE"
        ],
        "child_fk": "ID_RECORD_RAGE",
        "child_fks": [
            "ID_RECORD_RAGE"
        ],
        "cardinality": "1:N",
        "order_by": [
            "ID_RECORD_RAGE"
        ]
    },
    {
        "set_name": "COVERAGE_CLAIMS",
        "parent_record": "COVERAGE",
        "child_record": "HOSPITAL_CLAIM",
        "owner_record": "COVERAGE",
        "member_record": "HOSPITAL_CLAIM",
        "parent_table": "COVERAGE",
        "child_table": "HOSPITAL_CLAIM",
        "parent_key": "ID_RECORD_RAGE",
        "parent_keys": [
            "ID_RECORD_RAGE"
        ],
        "child_fk": "ID_RECORD_RAGE",
        "child_fks": [
            "ID_RECORD_RAGE"
        ],
        "cardinality": "1:N",
        "order_by": [
            "ID_RECORD_RAGE"
        ]
    }
]
},

```

```

{
  "set_name": "COVERAGE_CLAIMS",
  "parent_record": "COVERAGE",
  "child_record": "NON_HOSP_CLAIM",
  "owner_record": "COVERAGE",
  "member_record": "NON_HOSP_CLAIM",
  "parent_table": "COVERAGE",
  "child_table": "NON_HOSP_CLAIM",
  "parent_key": "ID_RECORD_RAGE",
  "parent_keys": [
    "ID_RECORD_RAGE"
  ],
  "child_fk": "ID_RECORD_RAGE",
  "child_fks": [
    "ID_RECORD_RAGE"
  ],
  "cardinality": "1:N",
  "order_by": [
    "ID_RECORD_RAGE"
  ]
},
{
  "set_name": "DEPT_EMPLOYEE",
  "parent_record": "DEPARTMENT",
  "child_record": "EMPLOYEE",
  "owner_record": "DEPARTMENT",
  "member_record": "EMPLOYEE",
  "parent_table": "DEPARTMENT",
  "child_table": "EMPLOYEE",
  "parent_key": "DEPT_ID_479MENT",
  "parent_keys": [
    "DEPT_ID_479MENT"
  ],
  "child_fk": "DEPT_ID_479MENT",
  "child_fks": [
    "DEPT_ID_479MENT"
  ],
  "cardinality": "1:N",
  "order_by": [
    "DEPT_ID_479MENT"
  ]
},
{
  "set_name": "EMP_COVERAGE",
  "parent_record": "EMPLOYEE",
  "child_record": "COVERAGE",
  "owner_record": "EMPLOYEE",
  "member_record": "COVERAGE",
  "parent_table": "EMPLOYEE",
  "child_table": "COVERAGE",
  "parent_key": "EMP_ID_479OYEE",
  "parent_keys": [
    "EMP_ID_479OYEE"
  ],
  "child_fk": "EMP_ID_479OYEE",
  "child_fks": [
    "EMP_ID_479OYEE"
  ],
  "cardinality": "1:N",
  "order_by": [
    "EMP_ID_479OYEE"
  ]
},
{
  "set_name": "EMP_EMPOSITION",
  "parent_record": "EMPLOYEE",
  "child_record": "EMPOSITION",
  "owner_record": "EMPLOYEE",
  "member_record": "EMPOSITION",
  "parent_table": "EMPLOYEE",
  "child_table": "EMPOSITION",
  "parent_key": "EMP_ID_479OYEE",
  "parent_keys": [
    "EMP_ID_479OYEE"
  ],
  "child_fk": "EMP_ID_479OYEE",

```

```

"child_fks": [
  "EMP_ID_4790YEE"
],
"cardinality": "1:N",
"order_by": [
  "EMP_ID_4790YEE"
]
},
{
  "set_name": "EMP_EXPERTISE",
  "parent_record": "EMPLOYEE",
  "child_record": "EXPERTISE",
  "owner_record": "EMPLOYEE",
  "member_record": "EXPERTISE",
  "parent_table": "EMPLOYEE",
  "child_table": "EXPERTISE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "cardinality": "1:N",
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
{
  "set_name": "JOB_EMPOSITION",
  "parent_record": "JOB",
  "child_record": "EMPOSITION",
  "owner_record": "JOB",
  "member_record": "EMPOSITION",
  "parent_table": "JOB",
  "child_table": "EMPOSITION",
  "parent_key": "JOB_ID_479JOB",
  "parent_keys": [
    "JOB_ID_479JOB"
  ],
  "child_fk": "JOB_ID_479JOB",
  "child_fks": [
    "JOB_ID_479JOB"
  ],
  "cardinality": "1:N",
  "order_by": [
    "JOB_ID_479JOB"
  ]
},
{
  "set_name": "MANAGES",
  "parent_record": "EMPLOYEE",
  "child_record": "STRUCTURE",
  "owner_record": "EMPLOYEE",
  "member_record": "STRUCTURE",
  "parent_table": "EMPLOYEE",
  "child_table": "STRUCTURE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "cardinality": "1:N",
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
{
  "set_name": "OFFICE_EMPLOYEE",
  "parent_record": "OFFICE",
  "child_record": "EMPLOYEE",
  "owner_record": "OFFICE",

```

```

"member_record": "EMPLOYEE",
"parent_table": "OFFICE",
"child_table": "EMPLOYEE",
"parent_key": "OFFICE_CODE_479FICE",
"parent_keys": [
  "OFFICE_CODE_479FICE"
],
"child_fk": "OFFICE_CODE_479FICE",
"child_fks": [
  "OFFICE_CODE_479FICE"
],
"cardinality": "1:N",
"order_by": [
  "OFFICE_CODE_479FICE"
]
},
{
  "set_name": "REPORTS_TO",
  "parent_record": "EMPLOYEE",
  "child_record": "STRUCTURE",
  "owner_record": "EMPLOYEE",
  "member_record": "STRUCTURE",
  "parent_table": "EMPLOYEE",
  "child_table": "STRUCTURE",
  "parent_key": "EMP_ID_479OYEE",
  "parent_keys": [
    "EMP_ID_479OYEE"
  ],
  "child_fk": null,
  "child_fks": [],
  "cardinality": "1:N",
  "order_by": []
},
{
  "set_name": "SKILL_EXPERTISE",
  "parent_record": "SKILL",
  "child_record": "EXPERTISE",
  "owner_record": "SKILL",
  "member_record": "EXPERTISE",
  "parent_table": "SKILL",
  "child_table": "EXPERTISE",
  "parent_key": "SKILL_ID_479KILL",
  "parent_keys": [
    "SKILL_ID_479KILL"
  ],
  "child_fk": "SKILL_ID_479KILL",
  "child_fks": [
    "SKILL_ID_479KILL"
  ],
  "cardinality": "1:N",
  "order_by": [
    "SKILL_ID_479KILL"
  ]
}
],
"set_ordering_map": {
  "COVERAGE_CLAIMS": {
    "parent_record": "COVERAGE",
    "child_record": "NON_HOSP_CLAIM",
    "parent_key": "ID_RECORD_RAGE",
    "parent_keys": [
      "ID_RECORD_RAGE"
    ],
    "child_fk": "ID_RECORD_RAGE",
    "child_fks": [
      "ID_RECORD_RAGE"
    ],
    "order_by": [
      "ID_RECORD_RAGE"
    ]
  },
  "DEPT_EMPLOYEE": {
    "parent_record": "DEPARTMENT",
    "child_record": "EMPLOYEE",
    "parent_key": "DEPT_ID_479MENT",
    "parent_keys": [

```

```

    "DEPT_ID_479MENT"
  ],
  "child_fk": "DEPT_ID_479MENT",
  "child_fks": [
    "DEPT_ID_479MENT"
  ],
  "order_by": [
    "DEPT_ID_479MENT"
  ]
},
"EMP_COVERAGE": {
  "parent_record": "EMPLOYEE",
  "child_record": "COVERAGE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
"EMP_EMPOSITION": {
  "parent_record": "EMPLOYEE",
  "child_record": "EMPOSITION",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
"EMP_EXPERTISE": {
  "parent_record": "EMPLOYEE",
  "child_record": "EXPERTISE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
"JOB_EMPOSITION": {
  "parent_record": "JOB",
  "child_record": "EMPOSITION",
  "parent_key": "JOB_ID_479JOB",
  "parent_keys": [
    "JOB_ID_479JOB"
  ],
  "child_fk": "JOB_ID_479JOB",
  "child_fks": [
    "JOB_ID_479JOB"
  ],
  "order_by": [
    "JOB_ID_479JOB"
  ]
},
"MANAGES": {
  "parent_record": "EMPLOYEE",
  "child_record": "STRUCTURE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ]
}

```

```

    ],
    "child_fk": "EMP_ID_4790YEE",
    "child_fks": [
        "EMP_ID_4790YEE"
    ],
    "order_by": [
        "EMP_ID_4790YEE"
    ]
},
"OFFICE_EMPLOYEE": {
    "parent_record": "OFFICE",
    "child_record": "EMPLOYEE",
    "parent_key": "OFFICE_CODE_479FICE",
    "parent_keys": [
        "OFFICE_CODE_479FICE"
    ],
    "child_fk": "OFFICE_CODE_479FICE",
    "child_fks": [
        "OFFICE_CODE_479FICE"
    ],
    "order_by": [
        "OFFICE_CODE_479FICE"
    ]
},
"REPORTS_TO": {
    "parent_record": "EMPLOYEE",
    "child_record": "STRUCTURE",
    "parent_key": "EMP_ID_4790YEE",
    "parent_keys": [
        "EMP_ID_4790YEE"
    ],
    "child_fk": null,
    "child_fks": [],
    "order_by": []
},
"SKILL_EXPERTISE": {
    "parent_record": "SKILL",
    "child_record": "EXPERTISE",
    "parent_key": "SKILL_ID_479KILL",
    "parent_keys": [
        "SKILL_ID_479KILL"
    ],
    "child_fk": "SKILL_ID_479KILL",
    "child_fks": [
        "SKILL_ID_479KILL"
    ],
    "order_by": [
        "SKILL_ID_479KILL"
    ]
}
},
"nullable_fk_map": {
    "COVERAGE.EMP_ID_4790YEE": {
        "nullable": true,
        "set_name": "EMP_COVERAGE",
        "reference_table": "EMPLOYEE",
        "reference_column": "EMP_ID_4790YEE",
        "table": "COVERAGE",
        "column": "EMP_ID_4790YEE",
        "null_indicator": "NI-COVERAGE-EMP-ID-4790YEE"
    },
    "EMP_COVERAGE": {
        "nullable": true,
        "set_name": "EMP_COVERAGE",
        "reference_table": "EMPLOYEE",
        "reference_column": "EMP_ID_4790YEE",
        "table": "COVERAGE",
        "column": "EMP_ID_4790YEE",
        "null_indicator": "NI-COVERAGE-EMP-ID-4790YEE"
    },
    "DENTAL_CLAIM.ID_RECORD_RAGE": {
        "nullable": true,
        "set_name": "COVERAGE_CLAIMS",
        "reference_table": "COVERAGE",
        "reference_column": "ID_RECORD_RAGE",
        "table": "DENTAL_CLAIM",

```

```

    "column": "ID_RECORD_RAGE",
    "null_indicator": "NI-DENTAL-CLAIM-ID-RECORD-RAGE"
  },
  "COVERAGE_CLAIMS": {
    "nullable": true,
    "set_name": "COVERAGE_CLAIMS",
    "reference_table": "COVERAGE",
    "reference_column": "ID_RECORD_RAGE",
    "table": "NON_HOSP_CLAIM",
    "column": "ID_RECORD_RAGE",
    "null_indicator": "NI-NON-HOSP-CLAIM-ID-RECORD-RAGE"
  },
  "EMPLOYEE.DEPT_ID_479MENT": {
    "nullable": true,
    "set_name": "DEPT_EMPLOYEE",
    "reference_table": "DEPARTMENT",
    "reference_column": "DEPT_ID_479MENT",
    "table": "EMPLOYEE",
    "column": "DEPT_ID_479MENT",
    "null_indicator": "NI-EMPLOYEE-DEPT-ID-479MENT"
  },
  "DEPT_EMPLOYEE": {
    "nullable": true,
    "set_name": "DEPT_EMPLOYEE",
    "reference_table": "DEPARTMENT",
    "reference_column": "DEPT_ID_479MENT",
    "table": "EMPLOYEE",
    "column": "DEPT_ID_479MENT",
    "null_indicator": "NI-EMPLOYEE-DEPT-ID-479MENT"
  },
  "EMPLOYEE.OFFICE_CODE_479FICE": {
    "nullable": true,
    "set_name": "OFFICE_EMPLOYEE",
    "reference_table": "OFFICE",
    "reference_column": "OFFICE_CODE_479FICE",
    "table": "EMPLOYEE",
    "column": "OFFICE_CODE_479FICE",
    "null_indicator": "NI-EMPLOYEE-OFFICE-CODE-479FICE"
  },
  "OFFICE_EMPLOYEE": {
    "nullable": true,
    "set_name": "OFFICE_EMPLOYEE",
    "reference_table": "OFFICE",
    "reference_column": "OFFICE_CODE_479FICE",
    "table": "EMPLOYEE",
    "column": "OFFICE_CODE_479FICE",
    "null_indicator": "NI-EMPLOYEE-OFFICE-CODE-479FICE"
  },
  "EMPOSITION.EMP_ID_479OYEE": {
    "nullable": true,
    "set_name": "EMP_EMPOSITION",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",
    "table": "EMPOSITION",
    "column": "EMP_ID_479OYEE",
    "null_indicator": "NI-EMPOSITION-EMP-ID-479OYEE"
  },
  "EMP_EMPOSITION": {
    "nullable": true,
    "set_name": "EMP_EMPOSITION",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",
    "table": "EMPOSITION",
    "column": "EMP_ID_479OYEE",
    "null_indicator": "NI-EMPOSITION-EMP-ID-479OYEE"
  },
  "EMPOSITION.JOB_ID_479JOB": {
    "nullable": true,
    "set_name": "JOB_EMPOSITION",
    "reference_table": "JOB",
    "reference_column": "JOB_ID_479JOB",
    "table": "EMPOSITION",
    "column": "JOB_ID_479JOB",
    "null_indicator": "NI-EMPOSITION-JOB-ID-479JOB"
  },
  "JOB_EMPOSITION": {

```

```

    "nullable": true,
    "set_name": "JOB_EMPOSITION",
    "reference_table": "JOB",
    "reference_column": "JOB_ID_479JOB",
    "table": "EMPOSITION",
    "column": "JOB_ID_479JOB",
    "null_indicator": "NI-EMPOSITION-JOB-ID-479JOB"
  },
  "EXPERTISE.EMP_ID_479OYEE": {
    "nullable": true,
    "set_name": "EMP_EXPERTISE",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",
    "table": "EXPERTISE",
    "column": "EMP_ID_479OYEE",
    "null_indicator": "NI-EXPERTISE-EMP-ID-479OYEE"
  },
  "EMP_EXPERTISE": {
    "nullable": true,
    "set_name": "EMP_EXPERTISE",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",
    "table": "EXPERTISE",
    "column": "EMP_ID_479OYEE",
    "null_indicator": "NI-EXPERTISE-EMP-ID-479OYEE"
  },
  "EXPERTISE.SKILL_ID_479KILL": {
    "nullable": true,
    "set_name": "SKILL_EXPERTISE",
    "reference_table": "SKILL",
    "reference_column": "SKILL_ID_479KILL",
    "table": "EXPERTISE",
    "column": "SKILL_ID_479KILL",
    "null_indicator": "NI-EXPERTISE-SKILL-ID-479KILL"
  },
  "SKILL_EXPERTISE": {
    "nullable": true,
    "set_name": "SKILL_EXPERTISE",
    "reference_table": "SKILL",
    "reference_column": "SKILL_ID_479KILL",
    "table": "EXPERTISE",
    "column": "SKILL_ID_479KILL",
    "null_indicator": "NI-EXPERTISE-SKILL-ID-479KILL"
  },
  "HOSPITAL_CLAIM.ID_RECORD_RAGE": {
    "nullable": true,
    "set_name": "COVERAGE_CLAIMS",
    "reference_table": "COVERAGE",
    "reference_column": "ID_RECORD_RAGE",
    "table": "HOSPITAL_CLAIM",
    "column": "ID_RECORD_RAGE",
    "null_indicator": "NI-HOSPITAL-CLAIM-ID-RECORD-RAGE"
  },
  "NON_HOSP_CLAIM.ID_RECORD_RAGE": {
    "nullable": true,
    "set_name": "COVERAGE_CLAIMS",
    "reference_table": "COVERAGE",
    "reference_column": "ID_RECORD_RAGE",
    "table": "NON_HOSP_CLAIM",
    "column": "ID_RECORD_RAGE",
    "null_indicator": "NI-NON-HOSP-CLAIM-ID-RECORD-RAGE"
  },
  "STRUCTURE.EMP_ID_479OYEE": {
    "nullable": true,
    "set_name": "MANAGES",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",
    "table": "STRUCTURE",
    "column": "EMP_ID_479OYEE",
    "null_indicator": "NI-STRUCTURE-EMP-ID-479OYEE"
  },
  "MANAGES": {
    "nullable": true,
    "set_name": "MANAGES",
    "reference_table": "EMPLOYEE",
    "reference_column": "EMP_ID_479OYEE",

```



```

    "table": "STRUCTURE",
    "column": "EMP_ID_4790YEE",
    "null_indicator": "NI-STRUCTURE-EMP-ID-4790YEE"
  }
},
"table_key_map": {
  "COVERAGE": {
    "primary_keys": [
      "ID_RECORD_RAGE"
    ],
    "foreign_keys": [
      {
        "column_name": "EMP_ID_4790YEE",
        "reference_table": "EMPLOYEE",
        "reference_column": "EMP_ID_4790YEE",
        "set_name": "EMP_COVERAGE"
      }
    ]
  },
  "DENTAL_CLAIM": {
    "primary_keys": [
      "ID_RECORD_LAIM"
    ],
    "foreign_keys": [
      {
        "column_name": "ID_RECORD_RAGE",
        "reference_table": "COVERAGE",
        "reference_column": "ID_RECORD_RAGE",
        "set_name": "COVERAGE_CLAIMS"
      }
    ]
  },
  "DEPARTMENT": {
    "primary_keys": [
      "DEPT_ID_479MENT"
    ],
    "foreign_keys": []
  },
  "EMPLOYEE": {
    "primary_keys": [
      "EMP_ID_4790YEE"
    ],
    "foreign_keys": [
      {
        "column_name": "DEPT_ID_479MENT",
        "reference_table": "DEPARTMENT",
        "reference_column": "DEPT_ID_479MENT",
        "set_name": "DEPT_EMPLOYEE"
      },
      {
        "column_name": "OFFICE_CODE_479FICE",
        "reference_table": "OFFICE",
        "reference_column": "OFFICE_CODE_479FICE",
        "set_name": "OFFICE_EMPLOYEE"
      }
    ]
  },
  "EMPOSITION": {
    "primary_keys": [
      "ID_RECORD_TION"
    ],
    "foreign_keys": [
      {
        "column_name": "EMP_ID_4790YEE",
        "reference_table": "EMPLOYEE",
        "reference_column": "EMP_ID_4790YEE",
        "set_name": "EMP_EMPOSITION"
      },
      {
        "column_name": "JOB_ID_479JOB",
        "reference_table": "JOB",
        "reference_column": "JOB_ID_479JOB",
        "set_name": "JOB_EMPOSITION"
      }
    ]
  }
},

```

```

"EXPERTISE": {
  "primary_keys": [
    "ID_RECORD_TISE"
  ],
  "foreign_keys": [
    {
      "column_name": "EMP_ID_4790YEE",
      "reference_table": "EMPLOYEE",
      "reference_column": "EMP_ID_4790YEE",
      "set_name": "EMP_EXPERTISE"
    },
    {
      "column_name": "SKILL_ID_479KILL",
      "reference_table": "SKILL",
      "reference_column": "SKILL_ID_479KILL",
      "set_name": "SKILL_EXPERTISE"
    }
  ]
},
"HOSPITAL_CLAIM": {
  "primary_keys": [
    "ID_RECORD_LAIM"
  ],
  "foreign_keys": [
    {
      "column_name": "ID_RECORD_RAGE",
      "reference_table": "COVERAGE",
      "reference_column": "ID_RECORD_RAGE",
      "set_name": "COVERAGE_CLAIMS"
    }
  ]
},
"INSURANCE_PLAN": {
  "primary_keys": [
    "INS_PLANCODE_479PLAN"
  ],
  "foreign_keys": []
},
"JOB": {
  "primary_keys": [
    "JOB_ID_479JOB"
  ],
  "foreign_keys": []
},
"NON_HOSP_CLAIM": {
  "primary_keys": [
    "ID_RECORD_LAIM"
  ],
  "foreign_keys": [
    {
      "column_name": "ID_RECORD_RAGE",
      "reference_table": "COVERAGE",
      "reference_column": "ID_RECORD_RAGE",
      "set_name": "COVERAGE_CLAIMS"
    }
  ]
},
"OFFICE": {
  "primary_keys": [
    "OFFICE_CODE_479FICE"
  ],
  "foreign_keys": []
},
"SKILL": {
  "primary_keys": [
    "SKILL_ID_479KILL"
  ],
  "foreign_keys": []
},
"STRUCTURE": {
  "primary_keys": [
    "ID_RECORD_TURE"
  ],
  "foreign_keys": [
    {
      "column_name": "EMP_ID_4790YEE",

```

```

        "reference_table": "EMPLOYEE",
        "reference_column": "EMP_ID_4790YEE",
        "set_name": "MANAGES"
    }
}
}
},
"date_part_map": {
    "SELECTION_YEAR_0400": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 3,
        "substring_length": 2,
        "part": "YEAR",
        "date_part": "YEAR",
        "legacy_field": "SELECTION_YEAR_0400"
    },
    "SELECTION-YEAR-0400": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 3,
        "substring_length": 2,
        "part": "YEAR",
        "date_part": "YEAR",
        "legacy_field": "SELECTION_YEAR_0400"
    },
    "SELECTION_YEAR": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 3,
        "substring_length": 2,
        "part": "YEAR",
        "date_part": "YEAR",
        "legacy_field": "SELECTION_YEAR_0400"
    },
    "SELECTIONYEAR0400": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 3,
        "substring_length": 2,
        "part": "YEAR",
        "date_part": "YEAR",
        "legacy_field": "SELECTION_YEAR_0400"
    },
    "SELECTION_MONTH_0400": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 6,
        "substring_length": 2,
        "part": "MONTH",
        "date_part": "MONTH",
        "legacy_field": "SELECTION_MONTH_0400"
    },
    "SELECTION-MONTH-0400": {
        "record": "COVERAGE",
        "table": "COVERAGE",
        "column": "DA_SELECTIONDATE_479RAGE",
        "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
        "substring_start": 6,
        "substring_length": 2,
        "part": "MONTH",
        "date_part": "MONTH",
        "legacy_field": "SELECTION_MONTH_0400"
    },
    "SELECTION_MONTH": {
        "record": "COVERAGE",

```

```

    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SELECTION_MONTH_0400"
  },
  "SELECTIONMONTH0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SELECTION_MONTH_0400"
  },
  "SELECTION_DAY_0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SELECTION_DAY_0400"
  },
  "SELECTION-DAY-0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SELECTION_DAY_0400"
  },
  "SELECTION_DAY": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SELECTION_DAY_0400"
  },
  "SELECTIONDAY0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_SELECTIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-SELECTIONDATE-479RAGE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SELECTION_DAY_0400"
  },
  "TERMINATION_YEAR_0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_TERMINATIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "TERMINATION_YEAR_0400"
  },
  "TERMINATION-YEAR-0400": {

```

```

    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_TERMINATIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "TERMINATION_YEAR_0400"
  },
  "TERMINATION_YEAR": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_479OYEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "TERMINATION_YEAR_0415"
  },
  "TERMINATIONYEAR0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_TERMINATIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "TERMINATION_YEAR_0400"
  },
  "TERMINATION_MONTH_0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_TERMINATIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "TERMINATION_MONTH_0400"
  },
  "TERMINATION-MONTH-0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_TERMINATIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "TERMINATION_MONTH_0400"
  },
  "TERMINATION_MONTH": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_479OYEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "TERMINATION_MONTH_0415"
  },
  "TERMINATIONMONTH0400": {
    "record": "COVERAGE",
    "table": "COVERAGE",
    "column": "DA_TERMINATIONDATE_479RAGE",
    "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "TERMINATION_MONTH_0400"
  },

```

```

"TERMINATION_DAY_0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "TERMINATION_DAY_0400"
},
"TERMINATION-DAY-0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "TERMINATION_DAY_0400"
},
"TERMINATION_DAY": {
  "record": "EMPLOYEE",
  "table": "EMPLOYEE",
  "column": "DA_TERMINATIONDATE_479OYEE",
  "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-479OYEE",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "TERMINATION_DAY_0415"
},
"TERMINATIONDAY0400": {
  "record": "COVERAGE",
  "table": "COVERAGE",
  "column": "DA_TERMINATIONDATE_479RAGE",
  "host": "HV-COVERAGE-DA-TERMINATIONDATE-479RAGE",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "TERMINATION_DAY_0400"
},
"CLAIM_YEAR_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "CLAIM_YEAR_0405"
},
"CLAIM-YEAR-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "CLAIM_YEAR_0405"
},
"CLAIM_YEAR": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "CLAIM_YEAR_0445"
}

```

```

},
"CLAIMYEAR0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "CLAIM_YEAR_0405"
},
"CLAIM_MONTH_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0405"
},
"CLAIM-MONTH-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0405"
},
"CLAIM_MONTH": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0445"
},
"CLAIMMONTH0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0405"
},
"CLAIM_DAY_0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "CLAIM_DAY_0405"
},
"CLAIM-DAY-0405": {
  "record": "DENTAL_CLAIM",
  "table": "DENTAL_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",

```

```

    "legacy_field": "CLAIM_DAY_0405"
  },
  "CLAIM_DAY": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "CLAIM_DAY_0445"
  },
  "CLAIMDAY0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "CLAIM_DAY_0405"
  },
  "PATIENT_BIRTH_YEAR_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0405"
  },
  "PATIENT-BIRTH-YEAR-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0405"
  },
  "PATIENT_BIRTH_YEAR": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0445"
  },
  "PATIENTBIRTHYEAR0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0405"
  },
  "PATIENT_BIRTH_MONTH_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",

```



```

    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0405"
  },
  "PATIENT-BIRTH-MONTH-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0405"
  },
  "PATIENT_BIRTH_MONTH": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0445"
  },
  "PATIENTBIRTHMONTH0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0405"
  },
  "PATIENT_BIRTH_DAY_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0405"
  },
  "PATIENT-BIRTH-DAY-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0405"
  },
  "PATIENT_BIRTH_DAY": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0445"
  },
  "PATIENTBIRTHDAY0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,

```

```

    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0405"
  },
  "SERVICE_YEAR_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0405"
  },
  "SERVICE-YEAR-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0405"
  },
  "SERVICE_YEAR": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0445"
  },
  "SERVICEYEAR0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0405"
  },
  "SERVICE_MONTH_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0405"
  },
  "SERVICE-MONTH-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0405"
  },
  "SERVICE_MONTH": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,

```

```

    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0445"
  },
  "SERVICEMONTH0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0405"
  },
  "SERVICE_DAY_0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0405"
  },
  "SERVICE-DAY-0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0405"
  },
  "SERVICE_DAY": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0445"
  },
  "SERVICEDAY0405": {
    "record": "DENTAL_CLAIM",
    "table": "DENTAL_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-DENTAL-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0405"
  },
  "START_YEAR_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0415"
  },
  "START-YEAR-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",

```

```

    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0415"
  },
  "START_YEAR": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0420"
  },
  "STARTYEAR0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_479OYEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-479OYEE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0415"
  },
  "START_MONTH_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_479OYEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-479OYEE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "START_MONTH_0415"
  },
  "START-MONTH-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_479OYEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-479OYEE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "START_MONTH_0415"
  },
  "START_MONTH": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "START_MONTH_0420"
  },
  "STARTMONTH0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_479OYEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-479OYEE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "START_MONTH_0415"
  },
  "START_DAY_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_479OYEE",

```

```

    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "START_DAY_0415"
  },
  "START-DAY-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "START_DAY_0415"
  },
  "START_DAY": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "START_DAY_0420"
  },
  "STARTDAY0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_STARTDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-STARTDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "START_DAY_0415"
  },
  "TERMINATION_YEAR_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "TERMINATION_YEAR_0415"
  },
  "TERMINATION-YEAR-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "TERMINATION_YEAR_0415"
  },
  "TERMINATIONYEAR0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_TERMINATIONDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "TERMINATION_YEAR_0415"
  },
  "TERMINATION_MONTH_0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",

```

```

"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "TERMINATION_MONTH_0415"
},
"TERMINATION-MONTH-0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "TERMINATION_MONTH_0415"
},
"TERMINATIONMONTH0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "TERMINATION_MONTH_0415"
},
"TERMINATION_DAY_0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "TERMINATION_DAY_0415"
},
"TERMINATION-DAY-0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "TERMINATION_DAY_0415"
},
"TERMINATIONDAY0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_TERMINATIONDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-TERMINATIONDATE-4790YEE",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "TERMINATION_DAY_0415"
},
"BIRTH_YEAR_0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "BIRTH_YEAR_0415"
},
"BIRTH-YEAR-0415": {
"record": "EMPLOYEE",

```

```

"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "BIRTH_YEAR_0415"
},
"BIRTH_YEAR": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "BIRTH_YEAR_0415"
},
"BIRTHYEAR0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "BIRTH_YEAR_0415"
},
"BIRTH_MONTH_0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "BIRTH_MONTH_0415"
},
"BIRTH-MONTH-0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "BIRTH_MONTH_0415"
},
"BIRTH_MONTH": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "BIRTH_MONTH_0415"
},
"BIRTHMONTH0415": {
"record": "EMPLOYEE",
"table": "EMPLOYEE",
"column": "DA_BIRTHDATE_4790YEE",
"host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "BIRTH_MONTH_0415"
},
"BIRTH_DAY_0415": {

```

```

    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "BIRTH_DAY_0415"
  },
  "BIRTH-DAY-0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "BIRTH_DAY_0415"
  },
  "BIRTH_DAY": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "BIRTH_DAY_0415"
  },
  "BIRTHDAY0415": {
    "record": "EMPLOYEE",
    "table": "EMPLOYEE",
    "column": "DA_BIRTHDATE_4790YEE",
    "host": "HV-EMPLOYEE-DA-BIRTHDATE-4790YEE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "BIRTH_DAY_0415"
  },
  "START_YEAR_0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0420"
  },
  "START-YEAR-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0420"
  },
  "STARTYEAR0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_STARTDATE_479TION",
    "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "START_YEAR_0420"
  },

```



```

"START_MONTH_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "START_MONTH_0420"
},
"START-MONTH-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "START_MONTH_0420"
},
"STARTMONTH0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "START_MONTH_0420"
},
"START_DAY_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "START_DAY_0420"
},
"START-DAY-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "START_DAY_0420"
},
"STARTDAY0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_STARTDATE_479TION",
  "host": "HV-EMPOSITION-DA-STARTDATE-479TION",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "START_DAY_0420"
},
"FINISH_YEAR_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "FINISH_YEAR_0420"
}

```

```

},
"FINISH-YEAR-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "FINISH_YEAR_0420"
},
"FINISH_YEAR": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "FINISH_YEAR_0420"
},
"FINISHYEAR0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "FINISH_YEAR_0420"
},
"FINISH_MONTH_0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "FINISH_MONTH_0420"
},
"FINISH-MONTH-0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "FINISH_MONTH_0420"
},
"FINISH_MONTH": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "FINISH_MONTH_0420"
},
"FINISHMONTH0420": {
  "record": "EMPOSITION",
  "table": "EMPOSITION",
  "column": "DA_FINISHDATE_479TION",
  "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",

```

```

    "legacy_field": "FINISH_MONTH_0420"
  },
  "FINISH_DAY_0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_FINISHDATE_479TION",
    "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "FINISH_DAY_0420"
  },
  "FINISH-DAY-0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_FINISHDATE_479TION",
    "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "FINISH_DAY_0420"
  },
  "FINISH_DAY": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_FINISHDATE_479TION",
    "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "FINISH_DAY_0420"
  },
  "FINISHDAY0420": {
    "record": "EMPOSITION",
    "table": "EMPOSITION",
    "column": "DA_FINISHDATE_479TION",
    "host": "HV-EMPOSITION-DA-FINISHDATE-479TION",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "FINISH_DAY_0420"
  },
  "EXPERTISE_YEAR_0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "EXPERTISE_YEAR_0425"
  },
  "EXPERTISE-YEAR-0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "EXPERTISE_YEAR_0425"
  },
  "EXPERTISE_YEAR": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",

```

```

    "date_part": "YEAR",
    "legacy_field": "EXPERTISE_YEAR_0425"
  },
  "EXPERTISEYEAR0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "EXPERTISE_YEAR_0425"
  },
  "EXPERTISE_MONTH_0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "EXPERTISE_MONTH_0425"
  },
  "EXPERTISE-MONTH-0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "EXPERTISE_MONTH_0425"
  },
  "EXPERTISE_MONTH": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "EXPERTISE_MONTH_0425"
  },
  "EXPERTISEMONTH0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "EXPERTISE_MONTH_0425"
  },
  "EXPERTISE_DAY_0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "EXPERTISE_DAY_0425"
  },
  "EXPERTISE-DAY-0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 9,
    "substring_length": 2,

```

```

    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "EXPERTISE_DAY_0425"
  },
  "EXPERTISE_DAY": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "EXPERTISE_DAY_0425"
  },
  "EXPERTISEDAY0425": {
    "record": "EXPERTISE",
    "table": "EXPERTISE",
    "column": "DA_EXPERTISEDATE_479TISE",
    "host": "HV-EXPERTISE-DA-EXPERTISEDATE-479TISE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "EXPERTISE_DAY_0425"
  },
  "CLAIM_YEAR_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0430"
  },
  "CLAIM-YEAR-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0430"
  },
  "CLAIMYEAR0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0430"
  },
  "CLAIM_MONTH_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "CLAIM_MONTH_0430"
  },
  "CLAIM-MONTH-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 6,

```

```

    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "CLAIM_MONTH_0430"
  },
  "CLAIMMONTH0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "CLAIM_MONTH_0430"
  },
  "CLAIM_DAY_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "CLAIM_DAY_0430"
  },
  "CLAIM-DAY-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "CLAIM_DAY_0430"
  },
  "CLAIMDAY0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "CLAIM_DAY_0430"
  },
  "PATIENT_BIRTH_YEAR_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0430"
  },
  "PATIENT-BIRTH-YEAR-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0430"
  },
  "PATIENTBIRTHYEAR0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",

```

```

    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "PATIENT_BIRTH_YEAR_0430"
  },
  "PATIENT_BIRTH_MONTH_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0430"
  },
  "PATIENT-BIRTH-MONTH-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0430"
  },
  "PATIENTBIRTHMONTH0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "PATIENT_BIRTH_MONTH_0430"
  },
  "PATIENT_BIRTH_DAY_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0430"
  },
  "PATIENT-BIRTH-DAY-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0430"
  },
  "PATIENTBIRTHDAY0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_PATIENTBIRTHDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "PATIENT_BIRTH_DAY_0430"
  },
  "ADMIT_YEAR_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",

```

```

    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "ADMIT_YEAR_0430"
  },
  "ADMIT-YEAR-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "ADMIT_YEAR_0430"
  },
  "ADMIT_YEAR": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "ADMIT_YEAR_0430"
  },
  "ADMITYEAR0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "ADMIT_YEAR_0430"
  },
  "ADMIT_MONTH_0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "ADMIT_MONTH_0430"
  },
  "ADMIT-MONTH-0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "ADMIT_MONTH_0430"
  },
  "ADMIT_MONTH": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_ADMITDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "ADMIT_MONTH_0430"
  },
  "ADMITMONTH0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",

```



```

"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "ADMIT_MONTH_0430"
},
"ADMIT_DAY_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "ADMIT_DAY_0430"
},
"ADMIT-DAY-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "ADMIT_DAY_0430"
},
"ADMIT_DAY": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "ADMIT_DAY_0430"
},
"ADMITDAY0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_ADMITDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-ADMITDATE-479LAIM",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "ADMIT_DAY_0430"
},
"DISCHARGE_YEAR_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "DISCHARGE_YEAR_0430"
},
"DISCHARGE-YEAR-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "DISCHARGE_YEAR_0430"
},
"DISCHARGE_YEAR": {
"record": "HOSPITAL_CLAIM",

```

```

"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "DISCHARGE_YEAR_0430"
},
"DISCHARGEYEAR0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 3,
"substring_length": 2,
"part": "YEAR",
"date_part": "YEAR",
"legacy_field": "DISCHARGE_YEAR_0430"
},
"DISCHARGE_MONTH_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "DISCHARGE_MONTH_0430"
},
"DISCHARGE-MONTH-0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "DISCHARGE_MONTH_0430"
},
"DISCHARGE_MONTH": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "DISCHARGE_MONTH_0430"
},
"DISCHARGEMONTH0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 6,
"substring_length": 2,
"part": "MONTH",
"date_part": "MONTH",
"legacy_field": "DISCHARGE_MONTH_0430"
},
"DISCHARGE_DAY_0430": {
"record": "HOSPITAL_CLAIM",
"table": "HOSPITAL_CLAIM",
"column": "DA_DISCHARGEDATE_479LAIM",
"host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
"substring_start": 9,
"substring_length": 2,
"part": "DAY",
"date_part": "DAY",
"legacy_field": "DISCHARGE_DAY_0430"
},
"DISCHARGE-DAY-0430": {

```

```

    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "DISCHARGE_DAY_0430"
  },
  "DISCHARGE_DAY": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "DISCHARGE_DAY_0430"
  },
  "DISCHARGEDAY0430": {
    "record": "HOSPITAL_CLAIM",
    "table": "HOSPITAL_CLAIM",
    "column": "DA_DISCHARGEDATE_479LAIM",
    "host": "HV-HOSPITAL-CLAIM-DA-DISCHARGEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "DISCHARGE_DAY_0430"
  },
  "CLAIM_YEAR_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0445"
  },
  "CLAIM-YEAR-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0445"
  },
  "CLAIMYEAR0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "CLAIM_YEAR_0445"
  },
  "CLAIM_MONTH_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_CLAIMDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "CLAIM_MONTH_0445"
  },

```

```

"CLAIM-MONTH-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0445"
},
"CLAIMMONTH0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "CLAIM_MONTH_0445"
},
"CLAIM_DAY_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "CLAIM_DAY_0445"
},
"CLAIM-DAY-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "CLAIM_DAY_0445"
},
"CLAIMDAY0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_CLAIMDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-CLAIMDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "CLAIM_DAY_0445"
},
"PATIENT_BIRTH_YEAR_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "PATIENT_BIRTH_YEAR_0445"
},
"PATIENT-BIRTH-YEAR-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "PATIENT_BIRTH_YEAR_0445"
}

```

```

},
"PATIENTBIRTHYEAR0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 3,
  "substring_length": 2,
  "part": "YEAR",
  "date_part": "YEAR",
  "legacy_field": "PATIENT_BIRTH_YEAR_0445"
},
"PATIENT_BIRTH_MONTH_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "PATIENT_BIRTH_MONTH_0445"
},
"PATIENT-BIRTH-MONTH-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "PATIENT_BIRTH_MONTH_0445"
},
"PATIENTBIRTHMONTH0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 6,
  "substring_length": 2,
  "part": "MONTH",
  "date_part": "MONTH",
  "legacy_field": "PATIENT_BIRTH_MONTH_0445"
},
"PATIENT_BIRTH_DAY_0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "PATIENT_BIRTH_DAY_0445"
},
"PATIENT-BIRTH-DAY-0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",
  "legacy_field": "PATIENT_BIRTH_DAY_0445"
},
"PATIENTBIRTHDAY0445": {
  "record": "NON_HOSP_CLAIM",
  "table": "NON_HOSP_CLAIM",
  "column": "DA_PATIENTBIRTHDATE_479LAIM",
  "host": "HV-NON-HOSP-CLAIM-DA-PATIENTBIRTHDATE-479LAIM",
  "substring_start": 9,
  "substring_length": 2,
  "part": "DAY",
  "date_part": "DAY",

```

```

    "legacy_field": "PATIENT_BIRTH_DAY_0445"
  },
  "SERVICE_YEAR_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0445"
  },
  "SERVICE_YEAR-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0445"
  },
  "SERVICEYEAR0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "SERVICE_YEAR_0445"
  },
  "SERVICE_MONTH_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0445"
  },
  "SERVICE-MONTH-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0445"
  },
  "SERVICEMONTH0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "SERVICE_MONTH_0445"
  },
  "SERVICE_DAY_0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",

```

```

    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0445"
  },
  "SERVICE-DAY-0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0445"
  },
  "SERVICEDAY0445": {
    "record": "NON_HOSP_CLAIM",
    "table": "NON_HOSP_CLAIM",
    "column": "DA_SERVICEDATE_479LAIM",
    "host": "HV-NON-HOSP-CLAIM-DA-SERVICEDATE-479LAIM",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "SERVICE_DAY_0445"
  },
  "STRUCTURE_YEAR_0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "STRUCTURE_YEAR_0460"
  },
  "STRUCTURE-YEAR-0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "STRUCTURE_YEAR_0460"
  },
  "STRUCTURE_YEAR": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "STRUCTURE_YEAR_0460"
  },
  "STRUCTUREYEAR0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 3,
    "substring_length": 2,
    "part": "YEAR",
    "date_part": "YEAR",
    "legacy_field": "STRUCTURE_YEAR_0460"
  },
  "STRUCTURE_MONTH_0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 6,
    "substring_length": 2,

```

```

    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "STRUCTURE_MONTH_0460"
  },
  "STRUCTURE-MONTH-0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "STRUCTURE_MONTH_0460"
  },
  "STRUCTURE_MONTH": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "STRUCTURE_MONTH_0460"
  },
  "STRUCTUREMONTH0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 6,
    "substring_length": 2,
    "part": "MONTH",
    "date_part": "MONTH",
    "legacy_field": "STRUCTURE_MONTH_0460"
  },
  "STRUCTURE_DAY_0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "STRUCTURE_DAY_0460"
  },
  "STRUCTURE-DAY-0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "STRUCTURE_DAY_0460"
  },
  "STRUCTURE_DAY": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 9,
    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "STRUCTURE_DAY_0460"
  },
  "STRUCTUREDAY0460": {
    "record": "STRUCTURE",
    "table": "STRUCTURE",
    "column": "DA_STRUCTUREDATE_479TURE",
    "host": "HV-STRUCTURE-DA-STRUCTUREDATE-479TURE",
    "substring_start": 9,

```



```

    "substring_length": 2,
    "part": "DAY",
    "date_part": "DAY",
    "legacy_field": "STRUCTURE_DAY_0460"
  }
},
"navigation_intent": {
  "COVERAGE_CLAIMS": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "COVERAGE",
    "child_record": "NON_HOSP_CLAIM",
    "parent_key": "ID_RECORD_RAGE",
    "parent_keys": [
      "ID_RECORD_RAGE"
    ],
    "child_fk": "ID_RECORD_RAGE",
    "child_fks": [
      "ID_RECORD_RAGE"
    ],
    "order_by": [
      "ID_RECORD_RAGE"
    ]
  },
  "DEPT_EMPLOYEE": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "DEPARTMENT",
    "child_record": "EMPLOYEE",
    "parent_key": "DEPT_ID_479MENT",
    "parent_keys": [
      "DEPT_ID_479MENT"
    ],
    "child_fk": "DEPT_ID_479MENT",
    "child_fks": [
      "DEPT_ID_479MENT"
    ],
    "order_by": [
      "DEPT_ID_479MENT"
    ]
  },
  "EMP_COVERAGE": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "EMPLOYEE",
    "child_record": "COVERAGE",
    "parent_key": "EMP_ID_4790YEE",
    "parent_keys": [
      "EMP_ID_4790YEE"
    ],
    "child_fk": "EMP_ID_4790YEE",
    "child_fks": [
      "EMP_ID_4790YEE"
    ],
    "order_by": [
      "EMP_ID_4790YEE"
    ]
  },
  "EMP_EMPOSITION": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "EMPLOYEE",
    "child_record": "EMPOSITION",
    "parent_key": "EMP_ID_4790YEE",
    "parent_keys": [
      "EMP_ID_4790YEE"
    ],
    "child_fk": "EMP_ID_4790YEE",
    "child_fks": [
      "EMP_ID_4790YEE"
    ],
    "order_by": [
      "EMP_ID_4790YEE"
    ]
  },
  "EMP_EXPERTISE": {
    "access_pattern": "SET_NAVIGATION",
    "parent_record": "EMPLOYEE",
    "child_record": "EXPERTISE",
    "parent_key": "EMP_ID_4790YEE",

```

```

"parent_keys": [
  "EMP_ID_4790YEE"
],
"child_fk": "EMP_ID_4790YEE",
"child_fks": [
  "EMP_ID_4790YEE"
],
"order_by": [
  "EMP_ID_4790YEE"
]
},
"JOB_EMPOSITION": {
  "access_pattern": "SET_NAVIGATION",
  "parent_record": "JOB",
  "child_record": "EMPOSITION",
  "parent_key": "JOB_ID_479JOB",
  "parent_keys": [
    "JOB_ID_479JOB"
  ],
  "child_fk": "JOB_ID_479JOB",
  "child_fks": [
    "JOB_ID_479JOB"
  ],
  "order_by": [
    "JOB_ID_479JOB"
  ]
},
"MANAGES": {
  "access_pattern": "SET_NAVIGATION",
  "parent_record": "EMPLOYEE",
  "child_record": "STRUCTURE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": "EMP_ID_4790YEE",
  "child_fks": [
    "EMP_ID_4790YEE"
  ],
  "order_by": [
    "EMP_ID_4790YEE"
  ]
},
"OFFICE_EMPLOYEE": {
  "access_pattern": "SET_NAVIGATION",
  "parent_record": "OFFICE",
  "child_record": "EMPLOYEE",
  "parent_key": "OFFICE_CODE_479FICE",
  "parent_keys": [
    "OFFICE_CODE_479FICE"
  ],
  "child_fk": "OFFICE_CODE_479FICE",
  "child_fks": [
    "OFFICE_CODE_479FICE"
  ],
  "order_by": [
    "OFFICE_CODE_479FICE"
  ]
},
"REPORTS_TO": {
  "access_pattern": "SET_NAVIGATION",
  "parent_record": "EMPLOYEE",
  "child_record": "STRUCTURE",
  "parent_key": "EMP_ID_4790YEE",
  "parent_keys": [
    "EMP_ID_4790YEE"
  ],
  "child_fk": null,
  "child_fks": [],
  "order_by": []
},
"SKILL_EXPERTISE": {
  "access_pattern": "SET_NAVIGATION",
  "parent_record": "SKILL",
  "child_record": "EXPERTISE",
  "parent_key": "SKILL_ID_479KILL",

```

```

        "parent_keys": [
            "SKILL_ID_479KILL"
        ],
        "child_fk": "SKILL_ID_479KILL",
        "child_fks": [
            "SKILL_ID_479KILL"
        ],
        "order_by": [
            "SKILL_ID_479KILL"
        ]
    },
    "output_semantics": {
        "generated_by": "idms-db2-modernizer-phase1",
        "usage": "IDMS retrieval COBOL to DB2 embedded SQL conversion",
        "physical_types_source": "DB2_DDL",
        "logical_mapping_source": "PHASE1_CANONICAL_METADATA",
        "db2_key_values": [
            "PK",
            "FK",
            "PK/FK"
        ],
        "generated_primary_key_rule": "ID_RECORD_<record_name>",
        "generated_primary_key_datatype": "CHAR(20)",
        "foreign_key_rule": "Use SET relationship and owner primary key columns only.",
        "character_rule": "CHAR by default; VARCHAR only when length equals 100.",
        "numeric_rule": "Numeric COBOL PIC maps to DECIMAL.",
        "date_rule": "Date part recognition remains generic for YEAR, YR, Y, YY, YYYY, DY, MONTH, MON, MO, M, MM, DM, DAY, D, DD."
    },
    "validation_messages": [],
    "canonical_record_lookup_count": 21
}

```

```

=====
FILE: installed.txt
ENCODING: utf-8
=====

```

```

aistudio-sdk==0.3.8
altair==6.2.1
annotated-doc==0.0.4
annotated-types==0.7.0
anyio==4.14.0
ast-serialize==0.5.0
astor==0.8.1
attrs==26.1.0
bce-python-sdk==0.9.72
blinker==1.9.0
cachetools==7.1.4
certifi==2026.6.17
cffi==2.0.0
cfgv==3.5.0
charset==7.4.3
charset-normalizer==3.4.7
click==8.4.1
colorama==0.4.6
colorlog==6.10.1
coverage==7.14.1
crc32c==2.8
cryptography==49.0.0
decorator==5.3.1
distlib==0.4.3
easyocr==1.7.2
et-xmlfile==2.0.0
filelock==3.29.4
fsspec==2026.6.0
future==1.0.0
gitdb==4.0.12
gitpython==3.1.50
graphviz==0.21
h11==0.16.0
hf-xet==1.5.1
httpcore==1.0.9
httpx==0.27.0
httpxtools==0.8.0

```

httpx==0.28.1
huggingface-hub==1.19.0
identify==2.6.19
idna==3.18
imageio==2.37.3
imagesize==2.0.0
iniconfig==2.3.0
itsdangerous==2.2.0
jinja2==3.1.6
jsonschema==4.26.0
jsonschema-specifications==2025.9.1
lazy-loader==0.5
librt==0.11.0
markdown-it-py==4.2.0
markupsafe==3.0.3
mdurl==0.1.2
modelscope==1.37.1
mpmath==1.3.0
mypy==2.1.0
mypy-extensions==1.1.0
narwhals==2.22.1
networkx==3.6.1
ninja==1.13.0
nodeenv==1.10.0
numpy==2.4.6
opencv-contrib-python==4.10.0.84
opencv-python==4.13.0.92
opencv-python-headless==4.13.0.92
openpyxl==3.1.5
opt-einsum==3.3.0
packaging==26.2
paddleocr==3.4.1
paddlepaddle==3.0.0
paddlex==3.4.3
pandas==3.0.3
pathspec==1.1.1
pdfminer-six==20260107
pdfplumber==0.11.10
pillow==12.2.0
platformdirs==4.10.0
pluggy==1.6.0
pre-commit==4.6.0
prettytable==3.17.0
protobuf==7.35.1
psutil==7.2.2
py-cpuinfo==9.0.0
pyarrow==24.0.0
pyclipper==1.4.0
pycparser==3.0
pycryptodome==3.23.0
pydantic==2.13.4
pydantic-core==2.46.4
pydantic-settings==2.14.1
pydeck==0.9.2
pygments==2.20.0
pymupdf==1.27.2.3
pypdfium2==5.10.1
pytest==9.1.0
pytest-cov==7.1.0
pytest-mock==3.15.1
python-bidi==0.6.10
python-dateutil==2.9.0.post0
python-discovery==1.4.2
python-dotenv==1.2.2
python-multipart==0.0.32
pyyaml==6.0.2
referencing==0.37.0
reportlab==4.5.1
requests==2.34.2
rich==15.0.0
rpds-py==2026.5.1
ruamel-yaml==0.19.1
ruff==0.15.17
safetensors==0.8.0
scikit-image==0.26.0
scipy==1.17.1

```

setuptools==81.0.0
shapely==2.1.2
shellingham==1.5.4
six==1.17.0
smmap==5.0.3
starlette==1.3.1
streamlit==1.58.0
structlog==26.1.0
sympy==1.14.0
tenacity==9.1.4
tiffiff==2026.6.1
toml==0.10.2
torch==2.12.1
torchvision==0.27.1
tqdm==4.68.3
typer==0.25.1
typing-extensions==4.15.0
typing-inspection==0.4.2
tzdata==2026.2
ujson==5.13.0
urllib3==2.7.0
uvicorn==0.49.0
virtualenv==21.5.1
watchdog==6.0.0
wcwidth==0.8.1
websockets==16.0

```

```

=====
FILE: pyproject.toml
ENCODING: utf-8
=====

```

```

[build-system]
requires = ["setuptools>=81.0.0", "wheel"]
build-backend = "setuptools.build_meta"

```

```

[project]
name = "idms-db2-modernizer"
version = "0.1.0"
description = "IDMS to DB2 Modernizer"
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "graphviz>=0.21",
    "networkx>=3.6.1",
    "pdfplumber>=0.11.10",
    "pydantic>=2.13.4",
    "pydantic-settings>=2.14.1",
    "streamlit>=1.58.0",
]

```

```

[tool.setuptools]
package-dir = {"" = "src"}

```

```

[tool.setuptools.packages.find]
where = ["src"]

```

```

[dependency-groups]
dev = [
    "pytest>=9.1.0",
    "pdfminer.six>=20250506",
]

```

```

=====
FILE: readme
ENCODING: utf-8
=====

```

```

set PYTHONPATH=src
python -m streamlit run src\idms_modernizer\app.py --server.port 8502

```

```

.venv\Scripts\Activate
python -m streamlit run app\streamlit_app.py

```

```
python src\tools\audit_hardcoding.py
```

```
streamlit run src/idms_modernizer/ui/schema_listing_only_app.py
```

```
taskkill /F /IM python.exe
--
```

```
$env:PYTHONPATH = "src"
python -m streamlit run src/idms_modernizer/ui/schema_listing_only_app.py
```

```
=====
FILE: README.md
ENCODING: utf-8
=====
```

```
set PYTHONPATH=src
python -m streamlit run src\idms_modernizer\app.py --server.port 8502
```

```
.venv\Scripts\Activate
python -m streamlit run app\streamlit_app.py
```

```
python src\tools\audit_hardcoding.py
```

```
python -m streamlit run app\streamlit_app.py --server.port 8502
```

```
set PYTHONPATH=src
```

```
.venv\Scripts\Activate
streamlit run src/idms_modernizer/ui/schema_listing_only_app.py
```

```
taskkill /F /IM python.exe
--
```

```
=====
FILE: src\idms_modernizer\__init__.py
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: src\idms_modernizer\core\__init__.py
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: src\idms_modernizer\core\config.py
ENCODING: utf-8
=====
```

```
from pathlib import Path
```

```
from pydantic_settings import BaseSettings
```

```
class Settings(BaseSettings):
```

```
    APP_NAME: str = (
        "IDMS DB2 Modernizer"
    )
```

```
    BASE_DIR: Path = Path.cwd()
```

```

INPUT_DIR: str = "data/input"

OUTPUT_DIR: str = "data/output"

TEMP_DIR: str = "data/temp"

DEBUG: bool = True

@property
def input_dir(self) -> str:
    return self.INPUT_DIR

@property
def output_dir(self) -> str:
    return self.OUTPUT_DIR

@property
def temp_dir(self) -> str:
    return self.TEMP_DIR

class Config:
    env_file = ".env"

settings = Settings()

=====
FILE: src\idms_modernizer\core\constants.py
ENCODING: utf-8
=====

SUPPORTED_FILE_TYPES = [
    "pdf"
]

MAX_UPLOAD_SIZE_MB = 100

DB2_DDL_FILE = (
    "db2_ddl.sql"
)

ER_DIAGRAM_FILE = (
    "er_diagram.png"
)

EXCEL_MODEL_FILE = (
    "db2_model.xlsx"
)

REPORT_FILE = (
    "migration_report.pdf"
)

SUPPORTED_FILE_TYPES = [
    "pdf"
]

MAX_UPLOAD_SIZE_MB = 100

DB2_DDL_FILE = (
    "db2_ddl.sql"
)

ER_DIAGRAM_FILE = (
    "er_diagram.png"
)

EXCEL_MODEL_FILE = (
    "db2_model.xlsx"
)

REPORT_FILE = (
    "migration_report.pdf"
)

```

```
CANONICAL_MODEL_FILE = (  
    "canonical_model.json"  
)
```

```
PHASE2_METADATA_FILE = (  
    "phase2_metadata.json"  
)
```

```
=====
```

```
FILE: src\idms_modernizer\core\exceptions.py
```

```
ENCODING: utf-8
```

```
=====
```

```
class ModernizerException(  
    Exception  
) :  
    pass
```

```
class PdfExtractionException(  
    ModernizerException  
) :  
    pass
```

```
class SchemaParsingException(  
    ModernizerException  
) :  
    pass
```

```
class ValidationException(  
    ModernizerException  
) :  
    pass
```

```
=====
```

```
FILE: src\idms_modernizer\core\logger.py
```

```
ENCODING: utf-8
```

```
=====
```

```
import logging
```

```
def get_logger(  
    logger_name: str  
) -> logging.Logger:  
  
    logger = logging.getLogger(  
        logger_name  
    )  
  
    logger.setLevel(  
        logging.INFO  
    )  
  
    if not logger.handlers:  
  
        handler = (  
            logging.StreamHandler()  
        )  
  
        formatter = (  
            logging.Formatter(  
                "%(asctime)s | %(levelname)s | %(message)s"  
            )  
        )  
  
        handler.setFormatter(  
            formatter  
        )  
  
        logger.addHandler(  
            handler
```



```

    )

    return logger

=====
FILE: src\idms_modernizer\core\utils.py
ENCODING: utf-8
=====

from pathlib import Path

def ensure_directory(
    directory: str
) -> None:

    Path(
        directory
    ).mkdir(
        parents=True,
        exist_ok=True
    )

def normalize_text(
    text: str
) -> str:

    return " ".join(
        text.split()
    )

=====
FILE: src\idms_modernizer\domain\__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_modernizer\domain\canonical_models.py
ENCODING: utf-8
=====

from pydantic import BaseModel, Field

class CanonicalField(BaseModel):
    name: str
    datatype: str | None = None
    length: int | None = None
    scale: int | None = None
    occurs: bool = False
    occurs_max: int | None = None

class CanonicalRecord(BaseModel):
    name: str

    # Backward-compatible single PK.
    primary_key: str | None = None

    # Composite PK support.
    # When CALC points to a group, physical child fields are stored here.
    primary_keys: list[str] = Field(default_factory=list)

    record_id: str | None = None
    location_mode: str | None = None
    fields: list[CanonicalField] = Field(default_factory=list)

class CanonicalSet(BaseModel):
    name: str

```

```

owner_record: str | None = None
member_record: str | None = None
order_mode: str | None = None

```

```

class CanonicalRelationship(BaseModel):
    set_name: str
    parent_record: str
    child_record: str
    cardinality: str = "1:N"

```

```

class CanonicalSchema(BaseModel):
    records: list[CanonicalRecord] = Field(default_factory=list)
    sets: list[CanonicalSet] = Field(default_factory=list)
    relationships: list[CanonicalRelationship] = Field(default_factory=list)

```

```

=====
FILE: src\idms_modernizer\domain\db2_models.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass, field

```

```

@dataclass
class DB2Column:
    name: str
    datatype: str
    nullable: bool = True
    primary_key: bool = False
    generated: bool = False
    source_kind: str = ""

```

```

@dataclass
class DB2ForeignKey:
    column_name: str
    reference_table: str
    reference_column: str
    set_name: str = ""

```

```

@dataclass
class DB2Table:
    name: str
    columns: list[DB2Column] = field(default_factory=list)
    foreign_keys: list[DB2ForeignKey] = field(default_factory=list)
    primary_key: str | None = None
    primary_keys: list[str] = field(default_factory=list)

```

```

@dataclass
class DB2Model:
    tables: list[DB2Table] = field(default_factory=list)

```

```

=====
FILE: src\idms_modernizer\domain\document_models.py
ENCODING: utf-8
=====

```

```

from pydantic import BaseModel, Field

```

```

class DocumentLine(BaseModel):
    line_number: int
    text: str

```

```

class DocumentPage(BaseModel):
    page_number: int
    lines: list[DocumentLine] = Field(
        default_factory=list
    )

```

```
class DocumentModel(BaseModel):
    pages: list[DocumentPage] = Field(
        default_factory=list
    )
```

```
=====
FILE: src\idms_modernizer\domain\models.py
ENCODING: utf-8
=====
```

```
from pydantic import BaseModel

from idms_modernizer.domain.db2_models import Table
from idms_modernizer.domain.relationship_models import (
    Relationship
)
from idms_modernizer.domain.schema_models import (
    SchemaMetadata
)
```

```
class ConversionContext(
    BaseModel
):

    schema_metadata: (
        SchemaMetadata | None
    ) = None

    relationships: (
        list[Relationship]
    ) = []

    db2_tables: (
        list[Table]
    ) = []
```

```
=====
FILE: src\idms_modernizer\domain\relationship_models.py
ENCODING: utf-8
=====
```

```
from pydantic import BaseModel

class SetMembership(BaseModel):

    set_name: str

    role: str
```

```
class Relationship(BaseModel):

    set_name: str

    owner_record: str

    member_record: str

    cardinality: str = "1:N"
```

```
=====
FILE: src\idms_modernizer\domain\schema_models.py
ENCODING: utf-8
=====
```

```
from dataclasses import dataclass, field

from pydantic import BaseModel, Field
```

```

from idms_modernizer.domain.relationship_models import (
    Relationship,
    SetMembership,
)

class DataField(BaseModel):
    name: str
    level: int | None = None

    datatype: str | None = None
    length: int | None = None
    scale: int | None = None
    picture: str | None = None

    start_position: int | None = None
    end_position: int | None = None
    basetype: str | None = None

    has_child: bool = False
    is_group: bool = False

    occurs: bool = False
    occurs_min: int | None = None
    occurs_max: int | None = None

    raw_line: str | None = None
    rest: str | None = None

@dataclass
class Record:
    name: str

    # Physical / DDL-safe fields.
    # Used by canonical schema, DB2 model, DDL, Phase 2 metadata, and COBOL conversion.
    fields: list[DataField] = field(
        default_factory=list,
    )

    # Excel-only complete field list.
    # Includes group fields, leaf fields, outer date groups, key groups, attributes, OCCURS groups.
    mapping_fields: list[DataField] = field(
        default_factory=list,
    )

    set_memberships: list[SetMembership] = field(
        default_factory=list,
    )

    primary_key: str | None = None
    cobol_zone: str | None = None

class SetDefinition(BaseModel):
    name: str
    owner_record: str | None = None
    member_record: str | None = None

class SchemaMetadata(BaseModel):
    records: list[Record] = Field(
        default_factory=list,
    )

    sets: list[SetDefinition] = Field(
        default_factory=list,
    )

    relationships: list[Relationship] = Field(
        default_factory=list,
    )

```

```
=====
FILE: src\idms_modernizer\domain\section_models.py
ENCODING: utf-8
=====
```

```
from dataclasses import dataclass, field
```

```
@dataclass
class RecordSection:
    record_name: str
    lines: list[str] = field(
        default_factory=list
    )
```

```
=====
FILE: src\idms_modernizer\extractors\cobol_zone_extractor.py
ENCODING: utf-8
=====
```

```
import re
```

```
class CobolZoneExtractor:
    """
    Extracts the IDMS region / zone from a schema record section.
```

```
    Examples:
```

```
        WITHIN INS-DEMO-REGION OFFSET 5 PGS FOR 20 PGS
        WITHIN
        EMP-DEMO-REGION OFFSET 5 PGS FOR 45 PGS
```

```
    Output:
```

```
        INS-DEMO-REGION
        EMP-DEMO-REGION
    """
```

```
    DIRECT_WITHIN_PATTERN = re.compile(
        r"\bWITHIN\s+([A-Z][A-Z0-9-]*REGION)\b",
        re.IGNORECASE,
    )
```

```
    REGION_PATTERN = re.compile(
        r"\b([A-Z][A-Z0-9-]*REGION)\b",
        re.IGNORECASE,
    )
```

```
    def extract(
        self,
        lines: list[str],
    ) -> str | None:
        cleaned_lines = [
            self.clean_line(line)
            for line in lines
            if line and self.clean_line(line)
        ]

        if not cleaned_lines:
            return None

        joined_text = " ".join(cleaned_lines)

        direct_match = self.DIRECT_WITHIN_PATTERN.search(
            joined_text,
        )

        if direct_match:
            return direct_match.group(1).upper()

        for index, line in enumerate(cleaned_lines):
            upper_line = line.upper()

            if upper_line == "WITHIN":
                region = self.find_region_after_within(
                    lines=cleaned_lines,
```

```

        start_index=index + 1,
    )

    if region:
        return region

    if "WITHIN" in upper_line:
        region = self.find_region_in_line(
            line=line,
        )

        if region:
            return region

        region = self.find_region_after_within(
            lines=cleaned_lines,
            start_index=index + 1,
        )

        if region:
            return region

    return None

def find_region_after_within(
    self,
    lines: list[str],
    start_index: int,
) -> str | None:
    for line in lines[start_index : start_index + 5]:
        region = self.find_region_in_line(
            line=line,
        )

        if region:
            return region

    return None

def find_region_in_line(
    self,
    line: str,
) -> str | None:
    match = self.REGION_PATTERN.search(
        line,
    )

    if not match:
        return None

    return match.group(1).upper()

def clean_line(
    self,
    line: str,
) -> str:
    if line is None:
        return ""

    cleaned = line.strip()
    cleaned = cleaned.replace("\u00a0", " ")
    cleaned = re.sub(r"\s+", " ", cleaned)

    return cleaned

```

```

=====
FILE: src\idms_modernizer\extractors\field_extractor.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass
from typing import List

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.rules.field_rules import (

```

```

FIELD_NAME_PATTERN,
clean_field_line,
is_redefines_text,
extract_redefines_target,
)
from idms_modernizer.rules.occurs_rules import (
    OCCURS_PATTERN,
    parse_occurs,
)
from idms_modernizer.utils.safe_cast import safe_int

```

```

@dataclass
class FieldCandidate:
    level: int
    name: str
    rest: str
    original_line: str
    has_child: bool = False
    is_group: bool = False
    occurs: bool = False
    occurs_min: int | None = None
    occurs_max: int | None = None

```

```

class FieldExtractor:
    """
    Extracts IDMS schema fields.

    Generic behavior only:
    - No business field names are hardcoded.
    - No record names are hardcoded.

    extract():
    - Used for physical / DDL-safe fields.
    - Returns only leaf fields.
    - Excludes FILLER by default.
    - Excludes REDEFINES / REDFINES from physical fields to avoid duplicate DB2 columns.

    extract_all():
    - Used for Excel Sheet Mapping.
    - Returns all fields:
      groups, subgroups, leaves, date groups, date parts, OCCURS groups,
      REDEFINES rows, and FILLER.
    - Includes FILLER by default so Sheet Mapping can show FILLER rows.

    Important:
    - IDMS fixed-width listing can show REDEFINES target in the second column
      without the literal word REDEFINES.
    """

    FIELD_NAME_PATTERN = FIELD_NAME_PATTERN
    OCCURS_PATTERN = OCCURS_PATTERN

    def __init__(
        self,
        include_filler: bool = False,
        auto_number_filler: bool = False,
    ) -> None:
        self.include_filler = include_filler
        self.auto_number_filler = auto_number_filler

    def extract(
        self,
        lines: List[str],
    ) -> List[DataField]:
        """
        Physical / DDL-safe extraction.

        REDEFINES is skipped here because it overlays existing COBOL storage
        and should not create duplicate DB2 physical columns.
        """

        print(
            "USING FIELD EXTRACTOR VERSION LEAF-ONLY-WITH-IDMS-REDEFINES-SKIP"
        )

```

```

        candidates = self.discover_candidates(
            lines=lines,
            include_filler=False,
        )

        leaf_candidates = self.mark_and_filter_leaf_fields(
            candidates=candidates,
        )

        physical_candidates = [
            candidate
            for candidate in leaf_candidates
            if not self.is_redefines_candidate(
                candidate=candidate,
            )
        ]

        return self.candidates_to_fields(
            candidates=physical_candidates,
        )

def extract_all(
    self,
    lines: List[str],
    include_filler: bool = True,
) -> List[DataField]:
    """
    Excel Sheet Mapping extraction.

    This method intentionally preserves:
    - group rows
    - leaf rows
    - OCCURS rows
    - REDEFINES rows
    - FILLER rows when include_filler=True
    """

    print(
        "USING FIELD EXTRACTOR VERSION ALL-FIELDS-WITH-IDMS-REDEFINES"
    )

    candidates = self.discover_candidates(
        lines=lines,
        include_filler=include_filler,
    )

    self.mark_group_fields(
        candidates=candidates,
    )

    return self.candidates_to_fields(
        candidates=candidates,
    )

def discover_candidates(
    self,
    lines: List[str],
    include_filler: bool | None = None,
) -> List[FieldCandidate]:
    candidates: List[FieldCandidate] = []

    if include_filler is None:
        include_filler = self.include_filler

    filler_index = 0

    for line in lines or []:
        cleaned_line = self.clean_line(
            line=line,
        )

        if not cleaned_line:
            continue

        match = self.FIELD_NAME_PATTERN.match(

```



```

        cleaned_line,
    )

    if not match:
        continue

    level = int(
        match.group("level"),
    )

    name = match.group("name").upper()
    rest = match.group("rest").strip()

    if level == 88:
        continue

    if name == "FILLER":
        if not include_filler:
            continue

        filler_index += 1

        if self.auto_number_filler:
            candidate_name = f"FILLER_{filler_index}"
        else:
            candidate_name = name
    else:
        candidate_name = name

    occurs_info = self.parse_occurs(
        text=rest,
    )

    candidate = FieldCandidate(
        level=level,
        name=candidate_name,
        rest=rest,
        original_line=cleaned_line,
        occurs=occurs_info["occurs"],
        occurs_min=occurs_info["occurs_min"],
        occurs_max=occurs_info["occurs_max"],
    )

    candidates.append(
        candidate,
    )

    return candidates

def mark_group_fields(
    self,
    candidates: List[FieldCandidate],
) -> None:
    for index, candidate in enumerate(
        candidates,
    ):
        candidate.has_child = False
        candidate.is_group = False

        for later_candidate in candidates[index + 1:]:
            if later_candidate.level <= candidate.level:
                break

            if later_candidate.level > candidate.level:
                candidate.has_child = True
                candidate.is_group = True
                break

def mark_and_filter_leaf_fields(
    self,
    candidates: List[FieldCandidate],
) -> List[FieldCandidate]:
    self.mark_group_fields(
        candidates=candidates,
    )

```

```

leaf_candidates: List[FieldCandidate] = []

for candidate in candidates:
    if candidate.has_child:
        continue

    leaf_candidates.append(
        candidate,
    )

return leaf_candidates

def candidates_to_fields(
    self,
    candidates: List[FieldCandidate],
) -> List[DataField]:
    fields: List[DataField] = []

    for candidate in candidates:
        display_name = candidate.name

        if display_name.startswith(
            "FILLER_",
        ):
            display_name = "FILLER"

        display_name = self.display_name_with_redefines(
            candidate=candidate,
            display_name=display_name,
        )

        field = DataField(
            name=display_name,
            level=candidate.level,
            rest=candidate.rest,
            raw_line=candidate.original_line,
            has_child=candidate.has_child,
            is_group=candidate.is_group,
            occurs=candidate.occurs,
            occurs_min=candidate.occurs_min,
            occurs_max=candidate.occurs_max,
        )

        fields.append(
            field,
        )

    return fields

def display_name_with_redefines(
    self,
    candidate: FieldCandidate,
    display_name: str,
) -> str:
    if not self.is_redefines_candidate(
        candidate=candidate,
    ):
        return display_name

    target = self.redefines_target_for_candidate(
        candidate=candidate,
    )

    if not target:
        return display_name

    if "REDEFINES" in display_name.upper():
        return display_name

    return f"{display_name} REDEFINES {target}"

def is_redefines_candidate(
    self,
    candidate: FieldCandidate,
) -> bool:
    text = " ".join(

```

```

        [
            str(candidate.original_line or ""),
            str(candidate.rest or ""),
        ]
    ).upper()

    return is_redefines_text(
        text,
    )

def redefines_target_for_candidate(
    self,
    candidate: FieldCandidate,
) -> str:
    text = " ".join(
        [
            str(candidate.original_line or ""),
            str(candidate.rest or ""),
        ]
    ).upper()

    return extract_redefines_target(
        text,
    )

def parse_occurs(
    self,
    text: str,
) -> dict:
    return parse_occurs(
        text,
    )

def clean_line(
    self,
    line: str,
) -> str:
    return clean_field_line(
        line,
    )

def safe_int(
    self,
    value,
) -> int | None:
    return safe_int(
        value,
    )

```

```

=====
FILE: src\idms_modernizer\extractors\owner_member_extractor.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_modernizer.domain.relationship_models import (
    SetMembership
)

```

```
class OwnerMemberExtractor:
```

```

    SAME_LINE_PATTERN = re.compile(
        r"^s*([A-Z0-9\-]+)\s+(?:INDEX\s+)?(OWNER|MEMBER)\b",
        re.IGNORECASE
    )

    SET_NAME_PATTERN = re.compile(
        r"^s*([A-Z][A-Z0-9\-]+)\s*$",
        re.IGNORECASE
    )

    ROLE_PATTERN = re.compile(
        r"^s*(OWNER|MEMBER)\b",

```

```

    re.IGNORECASE
)

IGNORED_SET_NAMES = {
    "CALC"
}

HEADER_WORDS = {
    "SET",
    "TYPE",
    "NEXT",
    "PRIOR",
    "OWNER",
    "MEMBER"
}

def extract(
    self,
    record_name: str,
    lines: list[str]
) -> list[SetMembership]:

    memberships = []
    seen = set()

    in_dbkey_block = False
    pending_set_name = None

    for raw_line in lines:

        line = raw_line.strip().upper()

        if not line:
            continue

        if "DBKEY POSITIONS" in line:
            in_dbkey_block = True
            pending_set_name = None
            continue

        if in_dbkey_block and line.startswith("DATA ITEM"):
            break

        if not in_dbkey_block:
            continue

        same_line_match = self.SAME_LINE_PATTERN.search(
            line
        )

        if same_line_match:

            set_name = same_line_match.group(1).upper()
            role = same_line_match.group(2).upper()

            pending_set_name = None

            if set_name in self.IGNORED_SET_NAMES:
                continue

            key = (
                set_name,
                role
            )

            if key not in seen:
                seen.add(key)

                memberships.append(
                    SetMembership(
                        set_name=set_name,
                        role=role
                    )
                )

            continue

```

```

        role_match = self.ROLE_PATTERN.search(
            line
        )

        if role_match and pending_set_name:

            set_name = pending_set_name
            role = role_match.group(1).upper()

            pending_set_name = None

            if set_name in self.IGNORED_SET_NAMES:
                continue

            key = (
                set_name,
                role
            )

            if key not in seen:
                seen.add(key)

                memberships.append(
                    SetMembership(
                        set_name=set_name,
                        role=role
                    )
                )

            continue

        set_name_match = self.SET_NAME_PATTERN.search(
            line
        )

        if set_name_match:

            candidate = set_name_match.group(1).upper()

            if candidate in self.HEADER_WORDS:
                continue

            if candidate in self.IGNORED_SET_NAMES:
                pending_set_name = None
                continue

            pending_set_name = candidate

    print(
        f"{record_name}: "
        f"{len(memberships)} memberships"
    )

    for membership in memberships:
        print(
            f"{record_name} -> "
            f"{membership.set_name} "
            f"{membership.role}"
        )

    return memberships

```

```

=====
FILE: src\idms_modernizer\extractors\primary_key_extractor.py
ENCODING: utf-8
=====

```

```

import re

from idms_modernizer.services.name_normalizer import NameNormalizer

class PrimaryKeyExtractor:
    """

```

Extracts IDMS CALC primary key metadata.

Generic behavior only:

- No hardcoded record names.
- No hardcoded key names.
- Handles listing formats with dots, spaces, and split lines.

Supported examples:

- LOCATION MODE CALC USING EMP-ID-0415
 - LOCATION MODE..... CALC USING KY-VMBFC DUPLICATES NOT ALLOWED
 - CALC USING KY-VMBFC
 - SET CONTROL ITEM FOR CALC <start> <length>
- ""

```
CALC_PATTERN = re.compile(
    r"\bCALC\s+(?P<start>[A-Z0-9-])\b",
    re.IGNORECASE,
)
```

```
CONTROL_ITEM_PATTERN = re.compile(
    r"\bSET\s+CONTROL\s+ITEM\s+FOR\s+CALC"
    r"(?:\s+(?P<start>[0-9])\s+(?P<length>[0-9]))?",
    re.IGNORECASE,
)
```

```
CONTROL_ITEM_FOLLOWING_CALC_PATTERN = re.compile(
    r"\bCALC\s+(?P<start>[0-9])\s+(?P<length>[0-9])\b",
    re.IGNORECASE,
)
```

```
def extract(
    self,
    lines: list[str],
) -> str | None:
    cleaned_lines = [
        self.clean_line(line=line)
        for line in lines or []
    ]

    joined_text = " ".join(
        line
        for line in cleaned_lines
        if line
    )

    match = self.CALC_PATTERN.search(joined_text)

    if match:
        return NameNormalizer.normalize(
            match.group(1),
        )

    for line in cleaned_lines:
        match = self.CALC_PATTERN.search(line)

        if match:
            return NameNormalizer.normalize(
                match.group(1),
            )

    return None

def extract_control_item(
    self,
    lines: list[str],
) -> dict[str, int] | None:
    cleaned_lines = [
        self.clean_line(line=line)
        for line in lines or []
        if self.clean_line(line=line)
    ]

    joined_text = " ".join(cleaned_lines)

    match = self.CONTROL_ITEM_PATTERN.search(joined_text)
```

```

if match:
    start = match.group("start")
    length = match.group("length")

    if start and length:
        return {
            "start": int(start),
            "length": int(length),
        }

match = self.CONTROL_ITEM_FOLLOWING_CALC_PATTERN.search(joined_text)

if match:
    return {
        "start": int(match.group("start")),
        "length": int(match.group("length")),
    }

return None

def clean_line(
    self,
    line: str,
) -> str:
    value = str(line or "").upper()

    value = value.replace(".", " ")
    value = value.replace("=", " ")
    value = value.replace(":", " ")
    value = value.replace("\u00a0", " ")

    value = re.sub(
        r"\s+",
        " ",
        value,
    )

    return value.strip()

```

```

=====
FILE: src\idms_modernizer\generators\ddl_generator.py
ENCODING: utf-8
=====

```

```

class DDLGenerator:
    """
    Generates DB2 DDL.

    Supports:
    - Single primary key
    - Composite primary key through table.primary_keys
    - Multiple FK columns
    - FK constraints emitted after CREATE TABLE
    """

    def generate(
        self,
        db2_model,
    ) -> str:
        ddl: list[str] = []

        for table in getattr(db2_model, "tables", []) or []:
            ddl.extend(
                self.generate_create_table(
                    table=table,
                )
            )

        foreign_key_statements = self.generate_foreign_key_statements(
            db2_model=db2_model,
        )

        if foreign_key_statements:
            ddl.append("")
            ddl.extend(foreign_key_statements)

```

```

        return "\n".join(ddl)

def generate_create_table(
    self,
    table,
) -> list[str]:
    lines: list[str] = []

    table_name = getattr(table, "name", "")

    lines.append(f"CREATE TABLE {table_name}")
    lines.append("(")

    definitions: list[str] = []
    added_columns: set[str] = set()

    for column in getattr(table, "columns", []) or []:
        column_name = getattr(column, "name", "")

        if not column_name:
            continue

        if column_name in added_columns:
            continue

        datatype = getattr(column, "datatype", "")

        definition = f"    {column_name} {datatype}"

        if not getattr(column, "nullable", True):
            definition += " NOT NULL"

        definitions.append(definition)
        added_columns.add(column_name)

    primary_keys = self.primary_key_columns(
        table=table,
    )

    if primary_keys:
        definitions.append(
            f"    PRIMARY KEY ({', '.join(primary_keys)})"
        )

    lines.append("\n".join(definitions))
    lines.append(");")
    lines.append("")

    return lines

def primary_key_columns(
    self,
    table,
) -> list[str]:
    primary_keys = list(getattr(table, "primary_keys", []) or [])

    if not primary_keys:
        single_primary_key = getattr(table, "primary_key", None)

        if single_primary_key:
            primary_keys = [single_primary_key]

    if not primary_keys:
        primary_keys = [
            getattr(column, "name", "")
            for column in getattr(table, "columns", []) or []
            if getattr(column, "primary_key", False)
        ]

    cleaned_primary_keys = []

    for primary_key in primary_keys:
        if not primary_key:
            continue

```



```

        if primary_key in cleaned_primary_keys:
            continue

        cleaned_primary_keys.append(primary_key)

    return cleaned_primary_keys

def generate_foreign_key_statements(
    self,
    db2_model,
) -> list[str]:
    statements: list[str] = []
    added_foreign_keys: set[tuple[str, str, str, str]] = set()

    for table in getattr(db2_model, "tables", []) or []:
        table_name = getattr(table, "name", "")

        if not table_name:
            continue

        for foreign_key in getattr(table, "foreign_keys", []) or []:
            child_column = self.get_foreign_key_column_name(
                foreign_key=foreign_key,
            )

            parent_table = self.get_foreign_key_reference_table(
                foreign_key=foreign_key,
            )

            parent_column = self.get_foreign_key_reference_column(
                foreign_key=foreign_key,
            )

            if not child_column or not parent_table or not parent_column:
                continue

            fk_key = (
                table_name,
                child_column,
                parent_table,
                parent_column,
            )

            if fk_key in added_foreign_keys:
                continue

            added_foreign_keys.add(fk_key)

            constraint_name = self.constraint_name(
                table_name=table_name,
                column_name=child_column,
            )

            statements.append(
                f"ALTER TABLE {table_name} "
                f"ADD CONSTRAINT {constraint_name} "
                f"FOREIGN KEY ({child_column}) "
                f"REFERENCES {parent_table} ({parent_column});"
            )

    return statements

def get_foreign_key_column_name(
    self,
    foreign_key,
) -> str:
    return (
        getattr(foreign_key, "column_name", None)
        or getattr(foreign_key, "child_column", None)
        or getattr(foreign_key, "child_fk", None)
        or getattr(foreign_key, "foreign_key", None)
        or ""
    )

def get_foreign_key_reference_table(
    self,

```

```

        foreign_key,
    ) -> str:
        return (
            getattr(foreign_key, "reference_table", None)
            or getattr(foreign_key, "parent_table", None)
            or getattr(foreign_key, "referenced_table", None)
            or ""
        )

def get_foreign_key_reference_column(
    self,
    foreign_key,
) -> str:
    return (
        getattr(foreign_key, "reference_column", None)
        or getattr(foreign_key, "parent_column", None)
        or getattr(foreign_key, "referenced_column", None)
        or getattr(foreign_key, "parent_key", None)
        or ""
    )

def constraint_name(
    self,
    table_name: str,
    column_name: str,
) -> str:
    raw_name = f"FK_{table_name}_{column_name}"

    cleaned = (
        raw_name
        .replace("-", "_")
        .replace(" ", "_")
        .replace(".", "_")
        .upper()
    )

    return cleaned[:128]

```

```

=====
FILE: src\idms_modernizer\graph\relationship_resolver.py
ENCODING: utf-8
=====

```

```

from collections import defaultdict

from idms_modernizer.domain.relationship_models import (
    Relationship
)

```

```

class RelationshipResolver:

    def resolve(
        self,
        metadata
    ) -> list[Relationship]:

        owner_map = defaultdict(list)

        member_map = defaultdict(list)

        for record in metadata.records:

            for membership in record.set_memberships:

                if membership.role == "OWNER":

                    owner_map[
                        membership.set_name
                    ].append(
                        record.name
                    )

                elif membership.role == "MEMBER":

```

```

        member_map[
            membership.set_name
        ].append(
            record.name
        )

relationships = []

all_sets = (
    set(owner_map.keys())
    | set(member_map.keys())
)

for set_name in all_sets:

    owners = owner_map.get(
        set_name,
        []
    )

    members = member_map.get(
        set_name,
        []
    )

    for owner in owners:

        for member in members:

            relationships.append(
                Relationship(
                    set_name=set_name,
                    owner_record=owner,
                    member_record=member
                )
            )

return relationships

```

```

=====
FILE: src\idms_modernizer\graph\schema_graph.py
ENCODING: utf-8
=====

```

```
import networkx as nx
```

```
class SchemaGraph:
```

```
    def __init__(self):
```

```
        self.graph = nx.DiGraph()
```

```
    def add_record(
```

```
        self,
```

```
        record_name: str
```

```
):
```

```
    self.graph.add_node(
```

```
        record_name,
```

```
        node_type="record"
```

```
)
```

```
    def add_relationship(
```

```
        self,
```

```
        owner: str,
```

```
        member: str,
```

```
        set_name: str
```

```
):
```

```
    self.graph.add_edge(
```

```
        owner,
```

```
        member,
```

```
        set_name=set_name
```

```
)
```

```
=====
FILE: src\idms_modernizer\parsers\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_modernizer\parsers\cobol_pdf_extractor.py
ENCODING: utf-8
=====
```

```
import re
import pdfplumber
```

```
class CobolPdfExtractor:
    """
    Extracts COBOL text from PDF.

    Supports:
    - COBOL pasted as text in PDF
    - line-preserving extraction
    - fallback to empty string if extraction fails
    """

    def extract_text(
        self,
        pdf_path: str
    ) -> str:

        lines: list[str] = []

        with pdfplumber.open(pdf_path) as pdf:
            for page in pdf.pages:
                page_text = (
                    page.extract_text()
                    or ""
                )

                for raw_line in page_text.splitlines():
                    cleaned_line = self._clean_line(
                        raw_line
                    )

                    lines.append(
                        cleaned_line
                    )

        return "\n".join(lines).strip()

    def _clean_line(
        self,
        line: str | None
    ) -> str:

        if line is None:
            return ""

        cleaned = str(line).rstrip()

        cleaned = cleaned.replace(
            "\t",
            " "
        )

        cleaned = re.sub(
            r"[\u00A0]+",
            " ",
            cleaned
        )

        return cleaned
```

```
=====
FILE: src\idms_modernizer\parsers\pdf_diagram_parser.py
ENCODING: utf-8
=====
```

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class DiagramRelationship:
```

```
    owner: str
```

```
    member: str
```

```
    set_name: str = ""
```

```
    cardinality: str = "1:N"
```

```
class PdfDiagramParser:
```

```
    def parse(
```

```
        self,
```

```
        text: str,
```

```
    ) -> list[DiagramRelationship]:
```

```
        names = []
```

```
        for raw_line in str(text or "").splitlines():
```

```
            line = raw_line.strip().upper()
```

```
            if not line:
```

```
                continue
```

```
            if line == "...":
```

```
                continue
```

```
            names.append(
```

```
                line,
```

```
            )
```

```
        if len(names) < 2:
```

```
            return []
```

```
        return [
```

```
            DiagramRelationship(
```

```
                owner=names[0],
```

```
                member=names[1],
```

```
            )
```

```
        ]
```

```
=====
FILE: src\idms_modernizer\parsers\pdf_schema_parser.py
ENCODING: utf-8
=====
```

```
import re
```

```
from idms_modernizer.domain.schema_models import (
```

```
    DataField,
```

```
    Record,
```

```
    SchemaMetadata,
```

```
)
```

```
class PdfSchemaParser:
```

```
    RECORD_PATTERN = re.compile(
```

```
        r"RECORD\s+NAME\s*\.\s*([A-Z][A-Z0-9\-\-]+)",
```

```
        re.IGNORECASE,
```

```
    )
```

```
    FIELD_PATTERN = re.compile(
```

```
        r"^\s*([A-Z][A-Z0-9\-\-]+)\s*$",
```

```
        re.IGNORECASE,
```

```
    )
```

```
    def parse(
```

```
        self,
```

```

        text: str,
    ) -> SchemaMetadata:
        metadata = SchemaMetadata()
        current_record = None

        for raw_line in str(text or "").splitlines():
            line = raw_line.strip()

            if not line:
                continue

            if line == "...":
                continue

            record_match = self.RECORD_PATTERN.search(
                line,
            )

            if record_match:
                current_record = Record(
                    name=record_match.group(1).upper(),
                )

                metadata.records.append(
                    current_record,
                )

                continue

            if current_record is None:
                continue

            field_match = self.FIELD_PATTERN.match(
                line,
            )

            if not field_match:
                continue

            field_name = field_match.group(1).upper()

            current_record.fields.append(
                DataField(
                    name=field_name,
                )
            )

        return metadata

```

```

=====
FILE: src\idms_modernizer\parsers\text_extractor.py
ENCODING: utf-8
=====

```

```

import re
from pathlib import Path

```

```

import pdfplumber

```

```

from idms_modernizer.domain.document_models import (
    DocumentLine,
    DocumentModel,
    DocumentPage,
)

```

```

class TextExtractor:
    """
    Extracts schema listing text from PDF or plain text files.

    Supported inputs:
    - .pdf
    - .txt
    - .text
    - .log
    """

```

- .cbl
- .cpy

Existing callers can continue using:

```
TextExtractor().extract_document(pdf_path)
```

Even though the argument is still named pdf_path, it now supports both PDF and text-based files.

"""

```
PDF_SUFFIXES = {  
    ".pdf",  
}
```

```
TEXT_SUFFIXES = {  
    ".txt",  
    ".text",  
    ".log",  
    ".cbl",  
    ".cpy",  
}
```

```
def extract_document(  
    self,  
    pdf_path: str,  
) -> DocumentModel:  
    input_path = Path(  
        pdf_path,  
    )  
  
    suffix = input_path.suffix.lower()  
  
    if suffix in self.PDF_SUFFIXES:  
        return self.extract_pdf_document(  
            pdf_path=str(input_path),  
        )  
  
    if suffix in self.TEXT_SUFFIXES:  
        return self.extract_text_document(  
            file_path=str(input_path),  
        )  
  
    return self.extract_text_document(  
        file_path=str(input_path),  
    )  
  
def extract_pdf_document(  
    self,  
    pdf_path: str,  
) -> DocumentModel:  
    pages: list[DocumentPage] = []  
  
    with pdfplumber.open(  
        pdf_path,  
    ) as pdf:  
        for page_index, page in enumerate(  
            pdf.pages,  
        ):  
            page_text = (  
                page.extract_text()  
                or ""  
            )  
  
            lines: list[DocumentLine] = []  
  
            for line_index, raw_line in enumerate(  
                page_text.splitlines(),  
            ):  
                cleaned_line = self._clean_line(  
                    raw_line,  
                )  
  
                if not cleaned_line:  
                    continue
```

```

        lines.append(
            DocumentLine(
                line_number=line_index,
                text=cleaned_line,
            )
        )

        pages.append(
            DocumentPage(
                page_number=page_index,
                lines=lines,
            )
        )

    return DocumentModel(
        pages=pages,
    )

def extract_text_document(
    self,
    file_path: str,
) -> DocumentModel:
    text = self.read_text_file(
        file_path=file_path,
    )

    lines: list[DocumentLine] = []

    for line_index, raw_line in enumerate(
        text.splitlines(),
    ):
        cleaned_line = self._clean_line(
            raw_line,
        )

        if not cleaned_line:
            continue

        lines.append(
            DocumentLine(
                line_number=line_index,
                text=cleaned_line,
            )
        )

    return DocumentModel(
        pages=[
            DocumentPage(
                page_number=0,
                lines=lines,
            )
        ]
    )

def extract_text(
    self,
    pdf_path: str,
) -> str:
    input_path = Path(
        pdf_path,
    )

    suffix = input_path.suffix.lower()

    if suffix in self.TEXT_SUFFIXES:
        return self.read_text_file(
            file_path=str(input_path),
        )

    lines: list[str] = []

    with pdfplumber.open(
        str(input_path),
    ) as pdf:
        for page in pdf.pages:
            page_text = (

```



```

        page.extract_text()
        or ""
    )

    for raw_line in page_text.splitlines():
        cleaned_line = self._clean_line(
            raw_line,
        )

        if cleaned_line:
            lines.append(
                cleaned_line,
            )

    return "\n".join(
        lines,
    ).strip()

def read_text_file(
    self,
    file_path: str,
) -> str:
    path = Path(
        file_path,
    )

    encodings = [
        "utf-8",
        "utf-8-sig",
        "cp1252",
        "latin-1",
    ]

    for encoding in encodings:
        try:
            return path.read_text(
                encoding=encoding,
                errors="strict",
            )
        except UnicodeDecodeError:
            continue

    return path.read_text(
        encoding="utf-8",
        errors="ignore",
    )

def _clean_line(
    self,
    line: str | None,
) -> str:
    if line is None:
        return ""

    cleaned = str(line).strip()

    cleaned = re.sub(
        r"<[^\>]+\>",
        " ",
        cleaned,
    )

    cleaned = cleaned.replace(
        "&nbsp;",
        " ",
    )

    cleaned = cleaned.replace(
        "\t",
        " ",
    )

    cleaned = cleaned.replace(
        "|",
        " ",
    )

```

```

        cleaned = cleaned.replace(
            "\u00a0",
            " ",
        )

        cleaned = re.sub(
            r"\s+",
            " ",
            cleaned,
        )

        return cleaned.strip()

```

```

=====
FILE: src\idms_modernizer\rules\__init__.py
ENCODING: utf-8
=====

```

```

"""
Rule modules for IDMS DB2 modernization.

```

Keep this package focused on:

- Regex constants
- Small rule helpers
- Pure parsing logic
- Reusable boolean checks

Do not place orchestration, PDF reading, Streamlit code, or file I/O here.

```

=====
FILE: src\idms_modernizer\rules\canonical_field_rules.py
ENCODING: utf-8
=====

```

```

import re

from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.services.date_field_consolidator import DateFieldConsolidator

def normalized_name(value: str | None) -> str:
    return NameNormalizer.normalize(value or "")

def remove_record_suffix(field_name: str | None) -> str:
    normalized = NameNormalizer.normalize(field_name or "")
    normalized = normalized.replace(" ", "_")

    return re.sub(r"_[\d-]{4}$", "", normalized)

def is_filler_field(field) -> bool:
    return str(getattr(field, "name", "") or "").upper().startswith("FILLER")

def is_date_part_name(field_name: str | None) -> bool:
    return DateFieldConsolidator.parse_date_part(
        field_name=field_name or "",
    ) is not None

def date_part_type_from_name(field_name: str | None) -> str | None:
    parsed = DateFieldConsolidator.parse_date_part(
        field_name=field_name or "",
    )

    if parsed is None:
        return None

    return parsed.get("part")

```

```

def is_date_field(field) -> bool:
    datatype = str(getattr(field, "datatype", "") or "").upper()
    basetype = str(getattr(field, "basetype", "") or "").upper()

    field_name = NameNormalizer.normalize(
        getattr(field, "name", "") or "",
    )

    normalized = field_name.replace(" ", "_")

    return (
        datatype == "DATE"
        or basetype == "DATE"
        or normalized.endswith("_DATE")
        or normalized == "DATE"
        or "_DATE_" in normalized
    )

def has_physical_definition(field) -> bool:
    if getattr(field, "picture", None):
        return True

    if getattr(field, "datatype", None):
        return True

    if getattr(field, "length", None):
        return True

    return False

def is_non_physical_group_field(field) -> bool:
    if is_date_field(field):
        return False

    if getattr(field, "is_group", False):
        return True

    if getattr(field, "has_child", False):
        return True

    return False

def build_group_name_set(fields) -> set[str]:
    result: set[str] = set()
    field_list = list(fields or [])

    for index, field in enumerate(field_list):
        field_name = normalized_name(
            getattr(field, "name", "") or "",
        )

        field_level = getattr(field, "level", None)

        if not field_name or field_level is None:
            continue

        try:
            field_level_int = int(field_level)
        except Exception:
            continue

        for next_field in field_list[index + 1:]:
            next_level = getattr(next_field, "level", None)

            if next_level is None:
                continue

            try:
                next_level_int = int(next_level)
            except Exception:
                continue

            if next_level_int > field_level_int:

```

```

        result.add(field_name)
        break

    if next_level_int <= field_level_int:
        break

return result

def build_date_name_set(fields) -> set[str]:
    result: set[str] = set()
    field_list = list(fields or [])

    for index, field in enumerate(field_list):
        field_name = normalized_name(
            getattr(field, "name", "") or "",
        )

        field_level = getattr(field, "level", None)

        if not field_name:
            continue

        if is_date_part_name(field_name):
            result.add(field_name)
            continue

        if is_date_field(field):
            result.add(field_name)
            continue

        if field_level is None:
            continue

        try:
            field_level_int = int(field_level)
        except Exception:
            continue

        descendants = []

        for next_field in field_list[index + 1:]:
            next_level = getattr(next_field, "level", None)

            if next_level is None:
                continue

            try:
                next_level_int = int(next_level)
            except Exception:
                continue

            if next_level_int <= field_level_int:
                break

            descendants.append(next_field)

        parts_found = set()

        for descendant in descendants:
            descendant_name = normalized_name(
                getattr(descendant, "name", "") or "",
            )

            part = date_part_type_from_name(descendant_name)

            if part:
                parts_found.add(part)

        if {"YEAR", "MONTH", "DAY"}.issubset(parts_found):
            result.add(field_name)

        for descendant in descendants:
            descendant_name = normalized_name(
                getattr(descendant, "name", "") or "",
            )

```

```

        if date_part_type_from_name(descendant_name):
            result.add(descendant_name)

    return result

def adjust_occurs_field(field):
    occurs = getattr(field, "occurs", False)
    occurs_max = getattr(field, "occurs_max", None)

    if not occurs or not occurs_max:
        return field

    length = getattr(field, "length", None)

    if length is None:
        return field

    try:
        adjusted_length = int(length) * int(occurs_max)
    except Exception:
        return field

    try:
        return field.model_copy(
            update={
                "length": adjusted_length,
            }
        )
    except Exception:
        try:
            field.length = adjusted_length
        except Exception:
            pass

        return field

def datatype_for_key_field(key_field) -> str:
    datatype = getattr(key_field, "datatype", None)

    if datatype:
        return datatype

    picture = str(getattr(key_field, "picture", "") or "").upper()

    if "X" in picture:
        return "CHAR"

    if "9" in picture:
        return "DECIMAL"

    return "CHAR"

def length_for_key_field(key_field) -> int | None:
    length = getattr(key_field, "length", None)

    if length:
        try:
            return int(length)
        except Exception:
            pass

    return 20

```

```

=====
FILE: src\idms_modernizer\rules\canonical_primary_key_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.canonical_models import (
    CanonicalRecord,
    CanonicalField,

```

```

)
from idms_modernizer.services.name_normalizer import NameNormalizer

from idms_modernizer.rules.canonical_field_rules import (
    adjust_occurs_field,
    datatype_for_key_field,
    has_physical_definition,
    is_date_field,
    is_date_part_name,
    is_filler_field,
    length_for_key_field,
    remove_record_suffix,
)

def apply_primary_key_rules(
    canonical_record: CanonicalRecord,
    record,
    added_fields: set[str],
    group_names: set[str],
    date_names: set[str],
) -> None:
    primary_key = canonical_record.primary_key

    if not primary_key:
        canonical_record.primary_keys = []
        return

    primary_key_fields = resolve_primary_key_fields(
        record=record,
        primary_key=primary_key,
        group_names=group_names,
        date_names=date_names,
    )

    if not primary_key_fields:
        canonical_record.primary_keys = []
        canonical_record.primary_key = None
        return

    canonical_primary_keys: list[str] = []

    for key_field in primary_key_fields:
        key_name = NameNormalizer.normalize(
            getattr(key_field, "name", "") or "",
        )

        if not key_name:
            continue

        if key_name in canonical_primary_keys:
            continue

        if key_name in date_names:
            continue

        if key_name in group_names:
            continue

        if is_date_part_name(key_name):
            continue

        if is_date_field(key_field):
            continue

        if is_filler_field(key_field):
            continue

        if not has_physical_definition(key_field):
            continue

        canonical_primary_keys.append(key_name)

        if key_name in added_fields:
            continue

```

```

        adjusted_key_field = adjust_occurs_field(
            field=key_field,
        )

        canonical_record.fields.append(
            CanonicalField(
                name=key_name,
                datatype=datatype_for_key_field(adjusted_key_field),
                length=length_for_key_field(adjusted_key_field),
                scale=getattr(adjusted_key_field, "scale", None),
                occurs=False,
                occurs_max=None,
            )
        )

        added_fields.add(key_name)

    canonical_record.primary_keys = canonical_primary_keys

    if canonical_primary_keys:
        canonical_record.primary_key = canonical_primary_keys[0]
    else:
        canonical_record.primary_key = None

def resolve_primary_key_fields(
    record,
    primary_key: str,
    group_names: set[str],
    date_names: set[str],
) -> list:
    mapping_fields = (
        getattr(record, "mapping_fields", None)
        or getattr(record, "fields", None)
        or []
    )

    key_field = find_field_for_primary_key(
        record=record,
        primary_key=primary_key,
    )

    if key_field is None:
        return []

    key_name = NameNormalizer.normalize(
        getattr(key_field, "name", "") or "",
    )

    if key_name in group_names:
        return child_physical_fields_for_group(
            fields=mapping_fields,
            group_field=key_field,
            group_names=group_names,
            date_names=date_names,
        )

    if key_name in date_names:
        return []

    if is_date_field(key_field):
        return []

    if is_date_part_name(key_name):
        return []

    if is_filler_field(key_field):
        return []

    return [key_field]

def child_physical_fields_for_group(
    fields,
    group_field,
    group_names: set[str],

```

```

    date_names: set[str],
) -> list:
    field_list = list(fields or [])

    if not field_list:
        return []

    group_name = NameNormalizer.normalize(
        getattr(group_field, "name", "") or "",
    )

    group_level = getattr(group_field, "level", None)

    if group_level is None:
        return []

    try:
        group_level_int = int(group_level)
    except Exception:
        return []

    collecting = False
    children = []

    for field in field_list:
        field_name = NameNormalizer.normalize(
            getattr(field, "name", "") or "",
        )

        field_level = getattr(field, "level", None)

        if field_level is None:
            continue

        try:
            field_level_int = int(field_level)
        except Exception:
            continue

        if not collecting:
            if field_name == group_name:
                collecting = True

            continue

        if field_level_int <= group_level_int:
            break

        if not field_name:
            continue

        if field_name in group_names:
            continue

        if field_name in date_names:
            continue

        if is_date_part_name(field_name):
            continue

        if is_date_field(field):
            continue

        if is_filler_field(field):
            continue

        if not has_physical_definition(field):
            continue

        children.append(field)

    return children

def find_field_for_primary_key(
    record,

```



```

        primary_key: str,
    ):
        fields = []
        fields.extend(getattr(record, "fields", []) or [])
        fields.extend(getattr(record, "mapping_fields", []) or [])

        normalized_primary_key = NameNormalizer.normalize(
            primary_key,
        )

        suffix_removed_primary_key = remove_record_suffix(
            normalized_primary_key,
        )

        for field in fields:
            field_name = NameNormalizer.normalize(
                getattr(field, "name", "") or "",
            )

            if field_name == normalized_primary_key:
                return field

            if remove_record_suffix(field_name) == suffix_removed_primary_key:
                return field

        return None

```

```

=====
FILE: src\idms_modernizer\rules\canonical_relationship_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.canonical_models import (
    CanonicalSchema,
    CanonicalSet,
    CanonicalRelationship,
)
from idms_modernizer.services.name_normalizer import NameNormalizer

```

```

def add_sets_and_relationships(
    schema: CanonicalSchema,
    metadata,
) -> None:
    added_relationships = set()

    for rel in getattr(metadata, "relationships", []) or []:
        set_name = getattr(rel, "set_name", None)
        owner_record = getattr(rel, "owner_record", None)
        member_record = getattr(rel, "member_record", None)

        if not set_name or not owner_record or not member_record:
            continue

        normalized_set_name = NameNormalizer.normalize(
            set_name,
        )

        normalized_owner = NameNormalizer.normalize(
            owner_record,
        )

        normalized_member = NameNormalizer.normalize(
            member_record,
        )

        key = (
            normalized_owner,
            normalized_member,
            normalized_set_name,
        )

        if key in added_relationships:
            continue

```

```

added_relationships.add(key)

schema.sets.append(
    CanonicalSet(
        name=normalized_set_name,
        owner_record=normalized_owner,
        member_record=normalized_member,
    )
)

schema.relationships.append(
    CanonicalRelationship(
        set_name=normalized_set_name,
        parent_record=normalized_owner,
        child_record=normalized_member,
        cardinality="1:N",
    )
)

```

```

=====
FILE: src\idms_modernizer\rules\date_group_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.rules.date_pattern_rules import (
    parse_date_part,
    has_complete_date_group,
)

```

```

def collect_date_groups(
    fields: list[DataField],
) -> dict[str, dict[str, DataField]]:
    groups: dict[str, dict[str, DataField]] = {}

```

```

    for field in fields or []:
        if field is None:
            continue

```

```

        field_name = getattr(
            field,
            "name",
            None,
        )

```

```

        if not field_name:
            continue

```

```

        parsed = parse_date_part(
            field_name=field_name,
        )

```

```

        if parsed is None:
            continue

```

```

        date_name = parsed["date_name"]
        part = parsed["part"]

```

```

        if date_name not in groups:
            groups[date_name] = {}

```

```

        groups[date_name][part] = field

```

```

    return groups

```

```

def parent_level(
    field: DataField | None,
) -> int | None:
    if field is None:
        return None

```

```

    level = getattr(

```

```

        field,
        "level",
        None,
    )

    if level is None:
        return None

    try:
        return max(
            int(level) - 1,
            1,
        )
    except Exception:
        return level

def min_start_position(
    parts: dict[str, DataField],
) -> int | None:
    values: list[int] = []

    for field in parts.values():
        value = getattr(
            field,
            "start_position",
            None,
        )

        if value is None:
            continue

        try:
            values.append(
                int(value),
            )
        except Exception:
            continue

    if not values:
        return None

    return min(values)

def max_end_position(
    parts: dict[str, DataField],
) -> int | None:
    values: list[int] = []

    for field in parts.values():
        value = getattr(
            field,
            "end_position",
            None,
        )

        if value is None:
            continue

        try:
            values.append(
                int(value),
            )
        except Exception:
            continue

    if not values:
        return None

    return max(values)

def build_date_field(
    date_name: str,
    parts: dict[str, DataField],

```

```

) -> DataField:
    template = (
        parts.get("YEAR")
        or parts.get("MONTH")
        or parts.get("DAY")
    )

    return DataField(
        name=date_name,
        level=parent_level(
            field=template,
        ),
        datatype="DATE",
        length=None,
        scale=None,
        picture=None,
        start_position=min_start_position(
            parts=parts,
        ),
        end_position=max_end_position(
            parts=parts,
        ),
        basetype="DATE",
        has_child=True,
        is_group=True,
    )

def normalize_date_name(
    value: str | None,
) -> str:
    return NameNormalizer.normalize(
        value or "",
    )

def should_build_date_field(
    date_name: str,
    parts: dict[str, DataField],
    added_dates: set[str],
) -> bool:
    if not has_complete_date_group(
        parts=parts,
    ):
        return False

    normalized_date_name = normalize_date_name(
        date_name,
    )

    if normalized_date_name in added_dates:
        return False

    return True

=====
FILE: src\idms_modernizer\rules\date_pattern_rules.py
ENCODING: utf-8
=====

import re

from idms_modernizer.services.name_normalizer import NameNormalizer

YEAR_PARTS = {
    "YEAR",
    "YR",
    "Y",
    "YY",
    "YYYY",
    "DY",
}

MONTH_PARTS = {

```

```

        "MONTH",
        "MON",
        "MO",
        "M",
        "MM",
        "DM",
    }

DAY_PARTS = {
    "DAY",
    "D",
    "DD",
}

DATE_TOKENS = {
    "DATE",
    "DT",
    "DTE",
    "DA",
    "YYMMDD",
    "YYYYMMDD",
    "YMD",
}

DATE_CHILD_TOKENS = YEAR_PARTS | MONTH_PARTS | DAY_PARTS

def split_tokens(
    value: str | None,
) -> list[str]:
    normalized = NameNormalizer.normalize(
        value or "",
    )

    return [
        token.upper()
        for token in re.split(
            r"[\s_-]+",
            normalized,
        )
        if token
    ]

def date_part_type(
    token: str,
    tokens: list[str],
) -> str | None:
    token = str(token or "").upper()

    has_dy_dm_dd = (
        "DY" in tokens
        and "DM" in tokens
        and "DD" in tokens
    )

    if token in YEAR_PARTS:
        if token == "DY" and not has_dy_dm_dd:
            return "DAY"

        return "YEAR"

    if token in MONTH_PARTS:
        return "MONTH"

    if token in DAY_PARTS:
        return "DAY"

    return None

def parse_date_part(
    field_name: str | None,
) -> dict[str, str] | None:
    tokens = split_tokens(
        value=field_name or "",
    )

```

```

    )

    if len(tokens) < 2:
        return None

    for index, token in enumerate(tokens):
        part = date_part_type(
            token=token,
            tokens=tokens,
        )

        if part is None:
            continue

        date_tokens = tokens.copy()
        date_tokens[index] = "DATE"

        return {
            "date_name": "_".join(date_tokens),
            "part": part,
        }

    return None

def is_date_part_name(
    field_name: str | None,
) -> bool:
    return parse_date_part(
        field_name=field_name or "",
    ) is not None

def has_complete_date_group(
    parts: dict,
) -> bool:
    return (
        "YEAR" in parts
        and "MONTH" in parts
        and "DAY" in parts
    )

def is_date_like_name(
    value: str | None,
) -> bool:
    parts = split_tokens(
        value=value or "",
    )

    if any(part in DATE_TOKENS for part in parts):
        return True

    compact = "".join(parts)

    return any(token in compact for token in DATE_TOKENS)

=====
FILE: src\idms_modernizer\rules\date_rules.py
ENCODING: utf-8
=====

import re

YEAR_PARTS = {
    "YEAR",
    "YR",
    "Y",
    "YY",
    "YYYY",
    "DY",
}

MONTH_PARTS = {

```

```

    "MONTH",
    "MON",
    "MO",
    "M",
    "MM",
    "DM",
}

DAY_PARTS = {
    "DAY",
    "D",
    "DD",
}

def normalize_name(name: str | None) -> str:
    value = str(name or "").strip().upper()
    value = value.replace("-", "_")
    value = re.sub(r"^[A-Z0-9_]", "_", value)
    value = re.sub(r"_+", "_", value)

    return value.strip("_")

def split_tokens(value: str | None) -> list[str]:
    normalized = normalize_name(value)

    return [
        token.upper()
        for token in re.split(r"[\s_-]+", normalized)
        if token
    ]

def parse_date_part(field_name: str | None) -> dict[str, str] | None:
    tokens = split_tokens(field_name)

    if len(tokens) < 2:
        return None

    for index, token in enumerate(tokens):
        part = date_part_type(token=token, tokens=tokens)

        if part is None:
            continue

        date_tokens = tokens.copy()
        date_tokens[index] = "DATE"

        return {
            "date_name": " ".join(date_tokens),
            "part": part,
        }

    return None

def date_part_type(token: str | None, tokens: list[str]) -> str | None:
    value = str(token or "").upper()

    has_dy_dm_dd = (
        "DY" in tokens
        and "DM" in tokens
        and "DD" in tokens
    )

    if value in YEAR_PARTS:
        if value == "DY" and not has_dy_dm_dd:
            return "DAY"

        return "YEAR"

    if value in MONTH_PARTS:
        return "MONTH"

    if value in DAY_PARTS:

```

```

        return "DAY"

    return None

def has_complete_date_group(parts: dict) -> bool:
    return {"YEAR", "MONTH", "DAY"}.issubset(set(parts.keys()))

def is_date_part_name(field_name: str | None) -> bool:
    return parse_date_part(field_name) is not None

def is_date_field(field) -> bool:
    datatype = str(getattr(field, "datatype", "") or "").upper()
    basetype = str(getattr(field, "basetype", "") or "").upper()
    field_name = normalize_name(getattr(field, "name", "") or "")
    normalized_name = field_name.replace(" ", "_")

    return (
        datatype == "DATE"
        or basetype == "DATE"
        or normalized_name.endswith("_DATE")
        or normalized_name == "DATE"
        or "_DATE_" in normalized_name
    )

def collect_date_groups(fields: list) -> dict[str, dict]:
    groups: dict[str, dict] = {}

    for field in fields or []:
        field_name = getattr(field, "name", None)

        if not field_name:
            continue

        parsed = parse_date_part(field_name)

        if parsed is None:
            continue

        date_name = parsed["date_name"]
        part = parsed["part"]

        if date_name not in groups:
            groups[date_name] = {}

        groups[date_name][part] = field

    return groups

=====
FILE: src\idms_modernizer\rules\db2_column_name_rules.py
ENCODING: utf-8
=====

from idms_modernizer.rules.db2_mapping_constants import (
    DB2_COLUMN_CODE_PREFIX,
    GENERIC_TRAILING_TOKENS,
    PROJECT_ABBREVIATIONS,
)
from idms_modernizer.rules.db2_identifier_rules import (
    remove_record_suffix,
    to_db2_name,
)
from idms_modernizer.rules.db2_record_name_rules import (
    record_code,
)
from idms_modernizer.rules.db2_date_name_rules import (
    date_field_base_name,
    is_date_field,
)

```



```

def remove_generic_trailing_tokens(
    parts: list[str],
) -> list[str]:
    output = list(parts or [])

    while output and output[-1] in GENERIC_TRAILING_TOKENS:
        output.pop()

    return output

def business_field_base_name(
    field_name: str,
) -> str:
    normalized = to_db2_name(
        value=field_name,
    )

    normalized = remove_record_suffix(
        name=normalized,
    )

    parts = [
        part
        for part in normalized.split("_")
        if part
    ]

    if not parts:
        return normalized

    parts = remove_generic_trailing_tokens(
        parts=parts,
    )

    parts = [
        PROJECT_ABBREVIATIONS.get(part, part)
        for part in parts
    ]

    if len(parts) == 1:
        return parts[0]

    first = parts[0]
    rest = "".join(parts[1:])

    if rest:
        return f"{first}_{rest}"

    return first

def normal_business_column_name(
    field_name: str,
    record_name: str | None,
) -> str:
    base = business_field_base_name(
        field_name=field_name,
    )

    code = record_code(
        record_name=record_name,
    )

    if code:
        return enforce_middle_name_max_length(
            db2_field_name=f"{base}_{DB2_COLUMN_CODE_PREFIX}{code}",
        )

    return base

def date_column_name(
    field_name: str,
    record_name: str | None,
) -> str:
    base = date_field_base_name(

```

```

        field_name=field_name,
    )

    code = record_code(
        record_name=record_name,
    )

    if code:
        return enforce_middle_name_max_length(
            db2_field_name=f"{base}_{DB2_COLUMN_CODE_PREFIX}{code}",
        )

    return base

def normalize_column_name(
    value: str | None,
    record_name: str | None,
    field=None,
) -> str:
    raw_value = str(value or "").strip()

    if not raw_value:
        return ""

    if field is not None and is_date_field(
        field=field,
    ):
        return date_column_name(
            field_name=raw_value,
            record_name=record_name,
        )

    return normal_business_column_name(
        field_name=raw_value,
        record_name=record_name,
    )

def enforce_middle_name_max_length(
    db2_field_name: str,
    max_length: int = 7,
) -> str:
    text = str(db2_field_name or "").strip()

    marker = f"_{DB2_COLUMN_CODE_PREFIX}"

    if marker not in text:
        return text

    base, suffix = text.split(
        marker,
        1,
    )

    base = base.rstrip(
        "_",
    )

    if "_" not in base:
        trimmed_base = base[:max_length]

        return f"{trimmed_base}{marker}{suffix}"

    prefix, middle = base.split(
        "_",
        1,
    )

    if len(middle) <= max_length:
        return text

    trimmed_middle = middle[:max_length]

    return f"{prefix}_{trimmed_middle}{marker}{suffix}"

```

```
=====
FILE: src\idms_modernizer\rules\db2_column_rules.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.domain.db2_models import DB2Column
from idms_modernizer.services.db2_datatype_mapper import DB2DatatypeMapper
from idms_modernizer.rules.naming_rules import (
    normalize_name,
    build_generated_record_id,
)
```

```
def build_db2_column_from_field(field, nullable: bool = True) -> DB2Column:
    return DB2Column(
        name=normalize_name(getattr(field, "name", "") or ""),
        datatype=DB2DatatypeMapper.map(field),
        nullable=nullable,
        primary_key=False,
        generated=False,
        source_kind=getattr(field, "source_kind", "") or "",
    )
```

```
def build_generated_primary_key_column(record_name: str | None) -> DB2Column:
    return DB2Column(
        name=build_generated_record_id(record_name),
        datatype="CHAR(20)",
        nullable=False,
        primary_key=True,
        generated=True,
        source_kind="GENERATED_RECORD_ID",
    )
```

```
def mark_column_as_primary_key(column: DB2Column) -> DB2Column:
    column.nullable = False
    column.primary_key = True

    return column
```

```
def find_column(columns: list[DB2Column], column_name: str | None) -> DB2Column | None:
    normalized_column_name = normalize_name(column_name)

    for column in columns or []:
        if normalize_name(column.name) == normalized_column_name:
            return column

    return None
```

```
def ensure_primary_key_column(
    columns: list[DB2Column],
    record_name: str | None,
    primary_key_name: str | None,
) -> str:
    normalized_primary_key = normalize_name(primary_key_name)

    if normalized_primary_key:
        column = find_column(
            columns=columns,
            column_name=normalized_primary_key,
        )

        if column is not None:
            mark_column_as_primary_key(column)
            return column.name

    generated_column = build_generated_primary_key_column(record_name)
    columns.insert(0, generated_column)

    return generated_column.name
```

```
=====
FILE: src\idms_modernizer\rules\db2_datatype_rules.py
ENCODING: utf-8
=====
```

```
import re

from idms_modernizer.rules.db2_picture_rules import (
    DEFAULT_CHAR_LENGTH,
    DEFAULT_DECIMAL_PRECISION,
    DEFAULT_DECIMAL_SCALE,
    char_length,
    clean_picture,
    decimal_precision_scale,
    numeric_precision,
    picture_core,
    picture_is_char,
    picture_is_decimal,
    picture_is_numeric,
    safe_int,
)

def first_present(
    *values,
):
    for value in values:
        if value is None:
            continue

        if isinstance(value, str) and not value.strip():
            continue

        return value

    return None

def has_comp_usage(
    picture: str | None,
) -> bool:
    text = str(
        picture or "",
    ).upper()

    return re.search(
        r"\bCOMP-3\b|\bCOMP3\b|\bCOMP\b",
        text,
    ) is not None

def format_decimal(
    precision: int,
    scale: int,
    force_scale: bool = False,
) -> str:
    actual_precision = safe_int(
        value=precision,
        default=DEFAULT_DECIMAL_PRECISION,
    )

    actual_scale = safe_int(
        value=scale,
        default=DEFAULT_DECIMAL_SCALE,
    )

    if force_scale or actual_scale > 0:
        return f"DECIMAL({actual_precision},{actual_scale})"

    return f"DECIMAL({actual_precision})"

def map_picture(
    picture: str,
    fallback_length=None,
    fallback_scale=None,
```

```

        fallback_datatype: str = "",
    ) -> str:
        clean = clean_picture(
            picture=picture,
        )

        if not clean:
            return map_without_picture(
                fallback_datatype=fallback_datatype,
                fallback_length=fallback_length,
                fallback_scale=fallback_scale,
            )

        core = picture_core(
            picture=clean,
        )

        if picture_is_char(
            picture=core,
        ):
            length = char_length(
                picture=core,
                fallback_length=fallback_length,
            )

            return f"CHAR({length})"

        if picture_is_decimal(
            picture=core,
        ):
            precision, scale = decimal_precision_scale(
                picture=core,
            )

            return format_decimal(
                precision=precision,
                scale=scale,
                force_scale=True,
            )

        if picture_is_numeric(
            picture=core,
        ):
            precision = numeric_precision(
                picture=core,
                fallback_length=fallback_length,
            )

            return format_decimal(
                precision=precision,
                scale=0,
                force_scale=has_comp_usage(
                    picture=clean,
                ),
            )

        return map_without_picture(
            fallback_datatype=fallback_datatype,
            fallback_length=fallback_length,
            fallback_scale=fallback_scale,
        )

def map_without_picture(
    fallback_datatype: str,
    fallback_length=None,
    fallback_scale=None,
) -> str:
    datatype = str(
        fallback_datatype or "",
    ).strip().upper()

    if datatype == "DATE":
        return "DATE"

    if datatype in {

```

```

        "TIMESTAMP",
        "DATETIME",
    }:
        return "TIMESTAMP"

    if datatype == "TIME":
        return "TIME"

    if datatype == "SMALLINT":
        return "DECIMAL(4)"

    if datatype in {
        "INTEGER",
        "INT",
    }:
        return "DECIMAL(9)"

    if datatype == "BIGINT":
        return "DECIMAL(18)"

    if datatype in {
        "COMP",
        "COMP3",
        "COMP-3",
    }:
        precision = safe_int(
            value=fallback_length,
            default=DEFAULT_DECIMAL_PRECISION,
        )

        scale = safe_int(
            value=fallback_scale,
            default=DEFAULT_DECIMAL_SCALE,
        )

        return format_decimal(
            precision=precision,
            scale=scale,
            force_scale=True,
        )

    if datatype in {
        "DECIMAL",
        "NUMERIC",
    }:
        precision = safe_int(
            value=fallback_length,
            default=DEFAULT_DECIMAL_PRECISION,
        )

        scale = safe_int(
            value=fallback_scale,
            default=DEFAULT_DECIMAL_SCALE,
        )

        return format_decimal(
            precision=precision,
            scale=scale,
            force_scale=scale > 0,
        )

    if datatype == "CHAR":
        length = safe_int(
            value=fallback_length,
            default=DEFAULT_CHAR_LENGTH,
        )

        return f"CHAR({length})"

    if datatype in {
        "VARCHAR",
        "DISPLAY",
        "TEXT",
        "ALPHANUMERIC",
    }:
        length = safe_int(

```

```

        value=fallback_length,
        default=DEFAULT_CHAR_LENGTH,
    )

    if length <= 1:
        return "CHAR(1)"

    return f"VARCHAR({length})"

length = safe_int(
    value=fallback_length,
    default=DEFAULT_CHAR_LENGTH,
)

if length <= 1:
    return "CHAR(1)"

return f"VARCHAR({length})"

```

```

=====
FILE: src\idms_modernizer\rules\db2_date_name_rules.py
ENCODING: utf-8
=====

```

```

import re

from idms_modernizer.rules.db2_mapping_constants import (
    DATE_TOKENS,
    DAY_PARTS,
    MONTH_PARTS,
    YEAR_PARTS,
    PROJECT_ABBREVIATIONS,
    GENERIC_TRAILING_TOKENS,
)
from idms_modernizer.rules.db2_identifier_rules import (
    name_parts,
    remove_record_suffix,
    to_db2_name,
)

def is_yyyymmdd_picture(
    picture: str,
) -> bool:
    if not picture:
        return False

    text = str(picture or "").upper()
    text = text.replace("PICTURE", "")
    text = text.replace("PIC", "")
    text = text.replace("COMP-3", "")
    text = re.sub(r"\bCOMP\b", "", text)
    text = text.replace("DISPLAY", "")
    text = text.replace("USAGE", "")
    text = text.replace("IS", "")
    text = text.replace(".", "")
    text = re.sub(r"\s+", "", text)

    return text in {
        "9(8)",
        "S9(8)",
        "99999999",
        "S9999999",
    }

def is_date_like_name(
    value: str,
) -> bool:
    parts = name_parts(
        value=value,
    )

    if not parts:
        return False

```

```

if parts[0] == "DA":
    return True

if any(part in DATE_TOKENS for part in parts):
    return True

compact = "".join(parts)

return any(token in compact for token in DATE_TOKENS)

def is_date_part_name(
    value: str,
) -> bool:
    parts = name_parts(
        value=value,
    )

    for part in parts:
        if part in YEAR_PARTS:
            return True

        if part in MONTH_PARTS:
            return True

        if part in DAY_PARTS:
            return True

    return False

def is_date_field(
    field,
) -> bool:
    datatype = str(
        getattr(field, "datatype", None)
        or getattr(field, "data_type", None)
        or getattr(field, "type", None)
        or "",
    ).strip().upper()

    basetype = str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or "",
    ).strip().upper()

    if datatype == "DATE" or basetype == "DATE":
        return True

    field_name = getattr(field, "name", "") or ""

    if is_date_like_name(
        value=field_name,
    ):
        picture = str(
            getattr(field, "picture", None)
            or getattr(field, "pic", None)
            or getattr(field, "pic_clause", None)
            or "",
        ).strip()

        if is_yyyymmdd_picture(
            picture=picture,
        ):
            return True

    return False

def remove_generic_trailing_tokens(
    parts: list[str],
) -> list[str]:
    output = list(parts or [])

```



```

while output and output[-1] in GENERIC_TRAILING_TOKENS:
    output.pop()

return output

def apply_date_project_abbreviations(
    parts: list[str],
) -> list[str]:
    output: list[str] = []

    for part in parts:
        if part == "EVPRCP":
            output.append("ERCP")
            continue

        if part == "GDIFAR":
            output.append("GD")
            continue

        if part in {"GDIFC", "GDIFR"}:
            continue

        output.append(
            PROJECT_ABBREVIATIONS.get(part, part)
        )

    return output

def date_field_base_name(
    field_name: str,
) -> str:
    normalized = to_db2_name(
        value=field_name,
    )

    normalized = remove_record_suffix(
        name=normalized,
    )

    parts = [
        part
        for part in normalized.split("_")
        if part
    ]

    if not parts:
        return normalized

    if parts[0] == "DA":
        main_parts = parts[1:]
    else:
        main_parts = parts

    if not main_parts:
        return "DA"

    if main_parts[0] in {"UB", "UE"}:
        return f"DA_{main_parts[0]}DATE"

    main_parts = apply_date_project_abbreviations(
        parts=main_parts,
    )

    main_parts = remove_generic_trailing_tokens(
        parts=main_parts,
    )

    if not main_parts:
        return "DA"

    if len(main_parts) == 1:
        main = main_parts[0]
    else:
        main = "".join(main_parts)

```

```
return f"DA_{main}"
```

```
=====
FILE: src\idms_modernizer\rules\db2_identifier_rules.py
ENCODING: utf-8
=====
```

```
import re
```

```
def to_db2_name(
    value: str | None,
) -> str:
    text = str(value or "").strip().upper()

    text = text.replace("\u00a0", " ")
    text = text.replace("\t", " ")

    text = re.sub(r"^[A-Z0-9]+", "_", text)
    text = re.sub(r"_{1,}", "_", text)
    text = text.strip("_")

    return text
```

```
def normalize_name(
    value: str | None,
) -> str:
    return to_db2_name(
        value=value or "",
    )
```

```
def name_parts(
    value: str | None,
) -> list[str]:
    normalized = to_db2_name(
        value=value or "",
    )

    return [
        part
        for part in normalized.split("_")
        if part
    ]
```

```
def remove_record_suffix(
    name: str | None,
) -> str:
    value = str(name or "").strip().upper()

    value = re.sub(r"[_-\s]+[0-9]{4}$", "", value)
    value = re.sub(r"[0-9]{4}$", "", value)
    value = re.sub(r"[_-\s]+$", "", value)

    return value
```

```
=====
FILE: src\idms_modernizer\rules\db2_mapping_constants.py
ENCODING: utf-8
=====
```

```
DB2_TABLE_PREFIX = "DZ"
DB2_TABLE_SUFFIX = "TB"
DB2_TABLE_CORE_LEN = 4
```

```
DB2_COLUMN_CODE_PREFIX = "479"
```

```
TECHNICAL_KEY_DATATYPE = "CHAR(20)"
```

```
GENERIC_TRAILING_TOKENS = {
    "SIC",
```

```

    "FC",
    "GDIFC",
    "GDIFR",
    "GDIFAR",
    "GDIF",
    "GDI",
    "REC",
    "RECORD",
}

PROJECT_ABBREVIATIONS = {
    "FORM": "FM",
    "EVPRCP": "ERCP",
}

DATE_TOKENS = {
    "DATE",
    "DT",
    "DTE",
    "DA",
    "YYMMDD",
    "YYYYMMDD",
    "YMD",
}

YEAR_PARTS = {
    "YEAR",
    "YR",
    "Y",
    "YY",
    "YYYY",
    "DY",
}

MONTH_PARTS = {
    "MONTH",
    "MON",
    "MO",
    "M",
    "MM",
    "DM",
}

DAY_PARTS = {
    "DAY",
    "D",
    "DD",
}

# -----
# Restart table configuration
# -----

RESTART_DB2_SCHEMA_NAME = DB2_TABLE_PREFIX
RESTART_SCHEMA_NAME = RESTART_DB2_SCHEMA_NAME

RESTART_IDMS_RECORD_NAMES = {
    "01RSTV",
    "FFRECB",
    "FFRECB",
    "FFRECAT",
}

RESTART_DB2_TABLE_NAME = f"{RESTART_DB2_SCHEMA_NAME}01RSTB"
RESTART_DB2_VIEW_NAME = f"{RESTART_DB2_SCHEMA_NAME}01RSTV"

RESTART_DB2_DATABASE_NAME = "VMDZ01DB"
RESTART_DB2_TABLESPACE_NAME = "DZ01RSTS"

RESTART_TECHNICAL_SOURCE_NAME = "FFRECB"
RESTART_MAPPING_REMARK = "TECHNICAL RESTART TABLE"
RESTART_CONTROL_COBOL_ZONE = "RESTART CONTROL RECORD"

RESTART_DCLGEN_COLUMNS = [
    {

```

```

        "column_name": "NM_ID_RS_PROGRAM",
        "data_type": "CHAR(8)",
        "length": "8",
        "basetype": "CHAR",
        "relation": "Restart program identifier",
        "db2_key": "PK",
    },
    {
        "column_name": "NB_ID_RS_PHASE",
        "data_type": "SMALLINT",
        "length": "2",
        "basetype": "SMALLINT",
        "relation": "Restart phase",
        "db2_key": "PK",
    },
    {
        "column_name": "DA_CR_RS",
        "data_type": "DATE",
        "length": "4",
        "basetype": "DATE",
        "relation": "Restart creation date",
        "db2_key": "",
    },
    {
        "column_name": "CO_ID_RS",
        "data_type": "CHAR(1)",
        "length": "1",
        "basetype": "CHAR",
        "relation": "Restart status",
        "db2_key": "",
    },
    {
        "column_name": "NB_DAY_RS",
        "data_type": "SMALLINT",
        "length": "2",
        "basetype": "SMALLINT",
        "relation": "Restart retention days",
        "db2_key": "",
    },
    {
        "column_name": "TE_CH_RS_DATA",
        "data_type": "VARCHAR(4000)",
        "length": "4000",
        "basetype": "VARCHAR",
        "relation": "Restart payload",
        "db2_key": "",
    },
}
]

```

```

=====
FILE: src\idms_modernizer\rules\db2_model_mapping_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import (
    DB2Column,
    DB2Model,
    DB2Table,
)
from idms_modernizer.services.db2_datatype_mapper import DB2DatatypeMapper
from idms_modernizer.rules import db2_name_mapping_rules
from idms_modernizer.rules import db2_relationship_mapping_rules
from idms_modernizer.rules import db2_table_mapping_rules

def build_db2_model(
    schema: CanonicalSchema,
) -> DB2Model:
    """
    Builds DB2Model from CanonicalSchema.

    Rules preserved:
    - One DB2 table per canonical record.
    - DB2 table name comes from db2_name_mapping_rules.normalize_table_name().
    """

```

- DB2 column name comes from db2_name_mapping_rules.normalize_column_name().
- DB2 datatype comes from DB2DatatypeMapper.
- Existing primary keys are applied.
- Missing primary keys are generated as ID_RECORD_<record-code>.
- Relationship foreign keys are added after all tables are created.

```

tables: list[DB2Table] = []

for record in getattr(schema, "records", []) or []:
    table = build_table_for_record(
        record=record,
    )

    tables.append(
        table,
    )

model = DB2Model(
    tables=tables,
)

table_lookup = db2_table_mapping_rules.build_table_lookup(
    model=model,
)

db2_relationship_mapping_rules.apply_relationship_foreign_keys(
    schema=schema,
    model=model,
    table_lookup=table_lookup,
)

return model

```

```

def build_table_for_record(
    record,
) -> DB2Table:
    """
    Builds one DB2Table from one canonical record.
    """

    record_name = getattr(record, "name", "") or ""

    table_name = db2_name_mapping_rules.normalize_table_name(
        value=record_name,
    )

    table = DB2Table(
        name=table_name,
        columns=[],
        foreign_keys=[],
        primary_key=None,
        primary_keys=[],
    )

    declared_primary_keys = get_record_primary_keys(
        record=record,
    )

    normalized_primary_keys: list[str] = []

    for primary_key in declared_primary_keys:
        normalized_primary_key = db2_name_mapping_rules.normalize_column_name(
            value=primary_key,
            record_name=record_name,
            field=None,
        )

        if not normalized_primary_key:
            continue

        if normalized_primary_key in normalized_primary_keys:
            continue

        normalized_primary_keys.append(

```

```

        normalized_primary_key,
    )

table.primary_keys = normalized_primary_keys

if table.primary_keys:
    table.primary_key = table.primary_keys[0]

added_columns: set[str] = set()

for field in getattr(record, "fields", []) or []:
    raw_field_name = getattr(field, "name", "") or ""

    column_name = db2_name_mapping_rules.normalize_column_name(
        value=raw_field_name,
        record_name=record_name,
        field=field,
    )

    if not column_name:
        continue

    if column_name in added_columns:
        continue

    is_primary_key = column_name in table.primary_keys

    column = DB2Column(
        name=column_name,
        datatype=DB2DatatypeMapper.map(
            field,
        ),
        nullable=not is_primary_key,
        primary_key=is_primary_key,
        generated=False,
        source_kind="COBOL",
    )

    table.columns.append(
        column,
    )

    added_columns.add(
        column_name,
    )

db2_table_mapping_rules.ensure_record_primary_key(
    table=table,
    record_name=record_name,
)

return table


def get_record_primary_keys(
    record,
) -> list[str]:
    """
    Returns unique normalized primary keys from canonical record.

    Supports both:
    - record.primary_keys
    - record.primary_key
    """

    primary_keys: list[str] = []

    explicit_primary_keys = getattr(
        record,
        "primary_keys",
        None,
    )

    if explicit_primary_keys:
        if isinstance(explicit_primary_keys, list):
            primary_keys.extend(

```

```

        explicit_primary_keys,
    )
else:
    primary_keys.append(
        explicit_primary_keys,
    )

primary_key = getattr(
    record,
    "primary_key",
    None,
)

if primary_key:
    primary_keys.append(
        primary_key,
    )

cleaned: list[str] = []

for primary_key_value in primary_keys:
    normalized = db2_name_mapping_rules.normalize_name(
        value=primary_key_value,
    )

    if not normalized:
        continue

    if normalized in cleaned:
        continue

    cleaned.append(
        normalized,
    )

return cleaned

```

```

=====
FILE: src\idms_modernizer\rules\db2_name_mapping_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules.db2_column_name_rules import (
    business_field_base_name,
    date_column_name,
    normal_business_column_name,
    normalize_column_name,
    remove_generic_trailing_tokens,
)
from idms_modernizer.rules.db2_date_name_rules import (
    apply_date_project_abbreviations,
    date_field_base_name,
    is_date_field,
    is_date_like_name,
    is_date_part_name,
    is_yyyymmdd_picture,
)
from idms_modernizer.rules.db2_identifier_rules import (
    name_parts,
    normalize_name,
    remove_record_suffix,
    to_db2_name,
)
from idms_modernizer.rules.db2_record_name_rules import (
    generated_primary_key_name,
    normalize_table_name,
    record_code,
)
from idms_modernizer.rules.db2_mapping_constants import (
    DB2_COLUMN_CODE_PREFIX,
)

```

```
=====
FILE: src\idms_modernizer\rules\db2_picture_rules.py
ENCODING: utf-8
=====
```

```
import re
```

```
DEFAULT_CHAR_LENGTH = 1
DEFAULT_DECIMAL_PRECISION = 18
DEFAULT_DECIMAL_SCALE = 0
```

```
def safe_int(
    value,
    default: int,
) -> int:
    try:
        if value is None:
            return default

        text = str(value).strip()

        if not text:
            return default

        return int(text)
    except Exception:
        return default
```

```
def clean_picture(
    picture: str | None,
) -> str:
    text = str(picture or "").strip()
    text = text.replace("\u00a0", " ")
    text = text.replace(".", "")
    text = re.sub(r"\s+", " ", text)

    return text.strip()
```

```
def picture_core(
    picture: str | None,
) -> str:
    text = str(picture or "").upper()
    text = text.replace("PICTURE", "")
    text = text.replace("PIC", "")
    text = text.replace("COMP-3", "")
    text = re.sub(r"\bCOMP\b", "", text)
    text = text.replace("DISPLAY", "")
    text = text.replace("USAGE", "")
    text = text.replace("IS", "")
    text = text.replace(".", "")
    text = re.sub(r"\s+", "", text)

    return text.strip()
```

```
def extract_parenthesized_int(
    value: str | None,
) -> int | None:
    text = str(value or "")

    open_index = text.find("(")

    if open_index == -1:
        return None

    close_index = text.find(")", open_index + 1)

    if close_index == -1:
        return None

    number_text = text[open_index + 1:close_index].strip()
```



```

    if not number_text.isdigit():
        return None

    return int(number_text)

def picture_is_char(
    picture: str | None,
) -> bool:
    text = str(picture or "").upper()

    if text.startswith("X(") and text.endswith(")"):
        return extract_parenthesized_int(text) is not None

    return bool(text) and set(text) == {"X"}

def char_length(
    picture: str | None,
    fallback_length=None,
) -> int:
    text = str(picture or "").upper()

    if text.startswith("X(") and text.endswith(")"):
        parsed = extract_parenthesized_int(
            value=text,
        )

        if parsed is not None:
            return parsed

    if re.fullmatch(r"X+", text):
        return len(text)

    return safe_int(
        value=fallback_length,
        default=DEFAULT_CHAR_LENGTH,
    )

def picture_is_numeric(
    picture: str | None,
) -> bool:
    text = str(picture or "").upper()

    if text.startswith("S"):
        text = text[1:]

    return "9" in text and "V" not in text

def picture_is_decimal(
    picture: str | None,
) -> bool:
    text = str(picture or "").upper()

    if text.startswith("S"):
        text = text[1:]

    return "9" in text and "V" in text

def numeric_precision(
    picture: str | None,
    fallback_length=None,
) -> int:
    text = str(picture or "").upper()

    if text.startswith("S"):
        text = text[1:]

    if "V" in text:
        precision, _scale = decimal_precision_scale(
            picture=picture,
        )

```

```

        return precision

match = re.fullmatch(r"9$(\d+)$", text)

if match:
    return int(match.group(1))

if re.fullmatch(r"9+", text):
    return len(text)

precision = count_numeric_digits(
    value=text,
)

if precision > 0:
    return precision

return safe_int(
    value=fallback_length,
    default=DEFAULT_DECIMAL_PRECISION,
)

def decimal_precision_scale(
    picture: str | None,
) -> tuple[int, int]:
    value = str(picture or "").strip().upper()

    if value.startswith("S"):
        value = value[1:]

    if "V" not in value:
        precision = numeric_precision(
            picture=value,
            fallback_length=DEFAULT_DECIMAL_PRECISION,
        )

        return precision, 0

    before_v, after_v = value.split("V", 1)

    integer_digits = count_numeric_digits(
        value=before_v,
    )

    decimal_digits = count_numeric_digits(
        value=after_v,
    )

    precision = integer_digits + decimal_digits
    scale = decimal_digits

    return precision, scale

def count_numeric_digits(
    value: str | None,
) -> int:
    text = str(value or "").strip().upper()

    if text.startswith("S"):
        text = text[1:]

    total = 0
    index = 0

    while index < len(text):
        current_character = text[index]

        if current_character not in {"9", "0"}:
            index += 1
            continue

        if index + 1 < len(text) and text[index + 1] == "(":
            close_index = text.find(")", index + 2)

```

```

        if close_index != -1:
            number_text = text[index + 2:close_index].strip()

            if number_text.isdigit():
                total += int(number_text)
                index = close_index + 1
                continue

    total += 1
    index += 1

return total

```

```

=====
FILE: src\idms_modernizer\rules\db2_record_name_rules.py
ENCODING: utf-8
=====

```

```

import re

from idms_modernizer.rules.db2_mapping_constants import (
    DB2_COLUMN_CODE_PREFIX,
    DB2_TABLE_CORE_LEN,
    DB2_TABLE_PREFIX,
    DB2_TABLE_SUFFIX,
)
from idms_modernizer.rules.db2_identifier_rules import (
    to_db2_name,
)

def record_code(
    record_name: str | None,
) -> str:
    normalized = to_db2_name(
        value=record_name or "",
    )

    compact = re.sub(
        r"^[^A-Z0-9]",
        "",
        normalized,
    )

    if not compact:
        return ""

    if len(compact) <= 4:
        return compact

    return compact[-4:]

def normalize_table_name(
    value: str | None,
) -> str:
    clean = to_db2_name(
        value=value or "",
    )

    clean = clean.replace(
        "-",
        "",
    )

    if not clean:
        return ""

    core = (
        clean[-DB2_TABLE_CORE_LEN:]
        if len(clean) >= DB2_TABLE_CORE_LEN
        else clean
    )

    return f"{DB2_TABLE_PREFIX}{core}{DB2_TABLE_SUFFIX}"

```

```

def generated_primary_key_name(
    record_name: str,
) -> str:
    code = record_code(
        record_name=record_name,
    )

    if code:
        return f"ID_RECORD_{DB2_COLUMN_CODE_PREFIX}{code}"

    table_name = normalize_table_name(
        value=record_name,
    )

    if table_name:
        return f"ID_RECORD_{DB2_COLUMN_CODE_PREFIX}{table_name}"

    return f"ID_RECORD_{DB2_COLUMN_CODE_PREFIX}"

```

```

=====
FILE: src\idms_modernizer\rules\db2_relationship_mapping_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.db2_models import (
    DB2Column,
    DB2ForeignKey,
    DB2Model,
    DB2Table,
)
from idms_modernizer.rules.db2_mapping_constants import TECHNICAL_KEY_DATATYPE
from idms_modernizer.rules.db2_name_mapping_rules import (
    normalize_name,
    normalize_table_name,
    remove_record_suffix,
)
from idms_modernizer.rules.db2_table_mapping_rules import (
    ensure_record_primary_key,
    find_column,
    get_primary_key_columns,
)

```

```

def apply_relationship_foreign_keys(
    schema,
    model: DB2Model,
    table_lookup: dict[str, DB2Table] | None = None,
) -> None:
    """
    Applies canonical relationships as DB2 foreign keys.

    Rules:
    - Owner record is parent table.
    - Member record is child table.
    - FK column reuses exact parent PK column name.
    - SET name is metadata only.
    """

    if table_lookup is None:
        table_lookup = build_table_lookup(
            model=model,
        )

    for relationship in getattr(schema, "relationships", []) or []:
        set_name = get_relationship_set_name(
            relationship=relationship,
        )

        owner_record = get_relationship_owner_record(
            relationship=relationship,
        )

        member_record = get_relationship_member_record(

```

```

        relationship=relationship,
    )

    if not owner_record or not member_record:
        continue

    owner_table = find_table_for_record(
        record_name=owner_record,
        table_lookup=table_lookup,
    )

    member_table = find_table_for_record(
        record_name=member_record,
        table_lookup=table_lookup,
    )

    if owner_table is None or member_table is None:
        continue

    owner_pk_columns = get_primary_key_columns(
        table=owner_table,
    )

    if not owner_pk_columns:
        owner_pk_columns = ensure_record_primary_key(
            table=owner_table,
            record_name=owner_record,
        )

    for owner_pk_column in owner_pk_columns:
        fk_column = ensure_fk_column_reusing_parent_name(
            child_table=member_table,
            parent_pk_column=owner_pk_column,
        )

        add_foreign_key_if_missing(
            child_table=member_table,
            child_column=fk_column.name,
            parent_table=owner_table,
            parent_column=owner_pk_column.name,
            set_name=set_name,
        )

def build_table_lookup(
    model: DB2Model,
) -> dict[str, DB2Table]:
    """
    Builds lookup keys for DB2 tables.

    Keeps compatibility with:
    - Actual table name
    - Normalized table name
    - Suffix-removed table name
    """

    lookup: dict[str, DB2Table] = {}

    for table in getattr(model, "tables", []) or []:
        table_name = getattr(table, "name", "") or ""

        normalized_table_name = normalize_name(
            value=table_name,
        )

        if normalized_table_name:
            lookup[normalized_table_name] = table

        suffix_removed = remove_record_suffix(
            name=normalized_table_name,
        )

        if suffix_removed:
            lookup[suffix_removed] = table

    return lookup

```

```

def find_table_for_record(
    record_name: str,
    table_lookup: dict[str, DB2Table],
) -> DB2Table | None:
    """
    Finds DB2 table for a canonical record name.

    Supports current generated table naming:
    - RECORD -> DZCORDTB style through normalize_table_name()
    """

    normalized_record_name = normalize_table_name(
        value=record_name,
    )

    if normalized_record_name in table_lookup:
        return table_lookup[normalized_record_name]

    suffix_removed = remove_record_suffix(
        name=normalized_record_name,
    )

    if suffix_removed in table_lookup:
        return table_lookup[suffix_removed]

    for table_key, table in table_lookup.items():
        if remove_record_suffix(table_key) == suffix_removed:
            return table

    normalized_plain_record = normalize_name(
        value=record_name,
    )

    if normalized_plain_record in table_lookup:
        return table_lookup[normalized_plain_record]

    plain_suffix_removed = remove_record_suffix(
        name=normalized_plain_record,
    )

    if plain_suffix_removed in table_lookup:
        return table_lookup[plain_suffix_removed]

    return None

def ensure_fk_column_reusing_parent_name(
    child_table: DB2Table,
    parent_pk_column: DB2Column,
) -> DB2Column:
    """
    Ensures child table has FK column using exact parent PK column name.
    """

    column_name = getattr(parent_pk_column, "name", "") or ""

    existing_column = find_column(
        table=child_table,
        column_name=column_name,
    )

    if existing_column is not None:
        existing_column.nullable = True
        existing_column.generated = True
        existing_column.primary_key = False
        existing_column.source_kind = "SET_FK"

        if not getattr(existing_column, "datatype", None):
            existing_column.datatype = (
                getattr(parent_pk_column, "datatype", "") or TECHNICAL_KEY_DATATYPE
            )

    return existing_column

```

```

new_column = DB2Column(
    name=column_name,
    datatype=getattr(parent_pk_column, "datatype", "") or TECHNICAL_KEY_DATATYPE,
    nullable=True,
    primary_key=False,
    generated=True,
    source_kind="SET_FK",
)

child_table.columns.append(
    new_column,
)

return new_column

def add_foreign_key_if_missing(
    child_table: DB2Table,
    child_column: str,
    parent_table: DB2Table,
    parent_column: str,
    set_name: str = "",
) -> None:
    """
    Adds FK metadata only if the same FK does not already exist.
    """

    if foreign_key_exists(
        child_table=child_table,
        child_column=child_column,
        parent_table=parent_table,
        parent_column=parent_column,
    ):
        return

    child_table.foreign_keys.append(
        DB2ForeignKey(
            column_name=child_column,
            reference_table=parent_table.name,
            reference_column=parent_column,
            set_name=normalize_name(
                value=set_name,
            ),
        )
    )

def foreign_key_exists(
    child_table: DB2Table,
    child_column: str,
    parent_table: DB2Table,
    parent_column: str,
) -> bool:
    normalized_child_column = normalize_name(
        value=child_column,
    )

    normalized_parent_table = normalize_name(
        value=parent_table.name,
    )

    normalized_parent_column = normalize_name(
        value=parent_column,
    )

    for foreign_key in getattr(child_table, "foreign_keys", []) or []:
        existing_child_column = normalize_name(
            value=getattr(foreign_key, "column_name", "") or "",
        )

        existing_parent_table = normalize_name(
            value=getattr(foreign_key, "reference_table", "") or "",
        )

        existing_parent_column = normalize_name(
            value=getattr(foreign_key, "reference_column", "") or "",

```

```

    )

    if (
        existing_child_column == normalized_child_column
        and existing_parent_table == normalized_parent_table
        and existing_parent_column == normalized_parent_column
    ):
        return True

    return False

def get_relationship_set_name(
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("set_name")
            or relationship.get("set")
            or relationship.get("name")
            or "",
        )

    return str(
        getattr(relationship, "set_name", None)
        or getattr(relationship, "set", None)
        or getattr(relationship, "name", None)
        or "",
    )

def get_relationship_owner_record(
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("owner_record")
            or relationship.get("parent_record")
            or relationship.get("owner")
            or relationship.get("parent")
            or "",
        )

    return str(
        getattr(relationship, "owner_record", None)
        or getattr(relationship, "parent_record", None)
        or getattr(relationship, "owner", None)
        or getattr(relationship, "parent", None)
        or "",
    )

def get_relationship_member_record(
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("member_record")
            or relationship.get("child_record")
            or relationship.get("member")
            or relationship.get("child")
            or "",
        )

    return str(
        getattr(relationship, "member_record", None)
        or getattr(relationship, "child_record", None)
        or getattr(relationship, "member", None)
        or getattr(relationship, "child", None)
        or "",
    )

```



```
=====
FILE: src\idms_modernizer\rules\db2_table_mapping_rules.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.domain.db2_models import (
    DB2Column,
    DB2Model,
    DB2Table,
)
from idms_modernizer.rules.db2_mapping_constants import TECHNICAL_KEY_DATATYPE
from idms_modernizer.rules.db2_name_mapping_rules import (
    generated_primary_key_name,
    normalize_name,
    remove_record_suffix,
)
```

```
def build_table_lookup(
    model: DB2Model,
) -> dict[str, DB2Table]:
    lookup: dict[str, DB2Table] = {}

    for table in getattr(model, "tables", []) or []:
        table_name = getattr(table, "name", "") or ""
        normalized_table_name = normalize_name(
            value=table_name,
        )

        if normalized_table_name:
            lookup[normalized_table_name] = table

        suffix_removed = remove_record_suffix(
            name=normalized_table_name,
        )

        if suffix_removed:
            lookup[suffix_removed] = table

    return lookup
```

```
def find_column(
    table: DB2Table,
    column_name: str,
) -> DB2Column | None:
    normalized_column_name = normalize_name(
        value=column_name,
    )

    if not normalized_column_name:
        return None

    for column in getattr(table, "columns", []) or []:
        current_name = normalize_name(
            value=getattr(column, "name", "") or "",
        )

        if current_name == normalized_column_name:
            return column

    return None
```

```
def get_primary_key_columns(
    table: DB2Table,
) -> list[DB2Column]:
    primary_key_names = list(
        getattr(table, "primary_keys", []) or []
    )

    if not primary_key_names and getattr(table, "primary_key", None):
        primary_key_names = [
            table.primary_key,
        ]
```

```

if not primary_key_names:
    primary_key_names = [
        getattr(column, "name", "") or ""
        for column in getattr(table, "columns", []) or []
        if getattr(column, "primary_key", False)
    ]

result: list[DB2Column] = []

for primary_key_name in primary_key_names:
    column = find_column(
        table=table,
        column_name=primary_key_name,
    )

    if column is not None:
        result.append(column)

return result

def ensure_record_primary_key(
    table: DB2Table,
    record_name: str,
) -> list[DB2Column]:
    primary_key_columns: list[DB2Column] = []

    for primary_key_name in list(getattr(table, "primary_keys", []) or []):
        column = find_column(
            table=table,
            column_name=primary_key_name,
        )

        if column is None:
            continue

        column.nullable = False
        column.primary_key = True

        if not getattr(column, "source_kind", ""):
            column.source_kind = "COBOL"

        primary_key_columns.append(column)

    if primary_key_columns:
        table.primary_keys = [
            column.name
            for column in primary_key_columns
        ]
        table.primary_key = table.primary_keys[0]

        return primary_key_columns

    generated_pk_name = generated_primary_key_name(
        record_name=record_name,
    )

    existing_generated = find_column(
        table=table,
        column_name=generated_pk_name,
    )

    if existing_generated is not None:
        existing_generated.nullable = False
        existing_generated.primary_key = True
        existing_generated.generated = True
        existing_generated.source_kind = "GENERATED PK"

        table.primary_keys = [
            existing_generated.name,
        ]
        table.primary_key = existing_generated.name

    return [
        existing_generated,
    ]

```

```

generated_column = DB2Column(
    name=generated_pk_name,
    datatype=TECHNICAL_KEY_DATATYPE,
    nullable=False,
    primary_key=True,
    generated=True,
    source_kind="GENERATED PK",
)

table.columns.insert(
    0,
    generated_column,
)

table.primary_keys = [
    generated_column.name,
]
table.primary_key = generated_column.name

return [
    generated_column,
]

```

```

=====
FILE: src\idms_modernizer\rules\excel_basic_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules.excel_row_template_rules import (
    empty_row,
)
from idms_modernizer.rules.excel_text_rules import (
    name_lookup_keys,
    name_parts,
    remove_record_suffix,
    safe_int,
    to_db2_name,
    to_string,
)
from idms_modernizer.rules.excel_field_identity_rules import (
    get_field_raw_text,
    get_redefines_target,
    is_filler_field,
    is_group_field,
    is_redefines_field,
)
from idms_modernizer.rules.excel_field_picture_rules import (
    get_field_picture,
    parse_length_from_pic_clause,
)
from idms_modernizer.rules.excel_field_position_rules import (
    get_field_end_position,
    get_field_length,
    get_field_start_position,
)
from idms_modernizer.rules.excel_field_remark_rules import (
    hopex_remark,
)
from idms_modernizer.rules.excel_cobol_zone_rules import (
    build_record_level_01_row,
    format_cobol_level_and_name,
    format_cobol_zone,
    has_level_01_row,
)

def get_field_basetype(
    field,
) -> str:
    return str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or getattr(field, "datatype", None)
        or "",
    )

```

)

```
__all__ = [  
    "empty_row",  
    "to_string",  
    "safe_int",  
    "to_db2_name",  
    "name_parts",  
    "remove_record_suffix",  
    "name_lookup_keys",  
    "get_field_raw_text",  
    "is_filler_field",  
    "is_group_field",  
    "is_redefines_field",  
    "get_redefines_target",  
    "get_field_picture",  
    "parse_length_from_pic_clause",  
    "get_field_start_position",  
    "get_field_length",  
    "get_field_end_position",  
    "get_field_basetype",  
    "hopex_remark",  
    "format_cobol_level_and_name",  
    "format_cobol_zone",  
    "has_level_01_row",  
    "build_record_level_01_row",  
]
```

```
=====
```

FILE: src\idms_modernizer\rules\excel_cobol_zone_rules.py
ENCODING: utf-8

```
=====
```

```
from idms_modernizer.services.name_normalizer import NameNormalizer  
from idms_modernizer.rules.excel_row_template_rules import (  
    empty_row,  
)  
from idms_modernizer.rules.excel_text_rules import (  
    remove_record_suffix,  
)
```

```
def format_cobol_level_and_name(  
    level,  
    name,  
) -> str:  
    level_text = ""  
  
    if level not in (  
        "",  
        None,  
    ):  
        try:  
            level_text = f"{int(level):02d}"  
        except Exception:  
            level_text = str(level or "").strip()  
  
    name_text = str(name or "").strip()  
  
    if level_text and name_text:  
        return f"{level_text} {name_text}"  
  
    if name_text:  
        return name_text  
  
    return level_text
```

```
def format_cobol_zone(  
    field,  
) -> str:  
    level = getattr(field, "level", None)  
    name = getattr(field, "name", "") or ""
```

```

        return format_cobol_level_and_name(
            level=level,
            name=name,
        )

def has_level_01_row(
    record,
    mapping_fields,
) -> bool:
    record_name = NameNormalizer.normalize(
        getattr(record, "name", "") or "",
    )

    for field in mapping_fields or []:
        level = getattr(field, "level", None)

        try:
            if int(level) != 1:
                continue
        except Exception:
            continue

        field_name = NameNormalizer.normalize(
            getattr(field, "name", "") or "",
        )

        if field_name == record_name:
            return True

        if remove_record_suffix(
            field_name,
        ) == remove_record_suffix(
            record_name,
        ):
            return True

    return False

def build_record_level_01_row(
    record,
) -> dict[str, str]:
    row = empty_row()
    record_name = getattr(record, "name", "") or ""

    row["Cobol Record IDMS"] = record_name
    row["Cobol Zone"] = format_cobol_level_and_name(
        level=1,
        name=record_name,
    )

    return row

```

```

=====
FILE: src\idms_modernizer\rules\excel_datatype_post_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.services.db2_datatype_mapper import (
    DB2DatatypeMapper,
)

def force_pic_based_datatypes(
    rows: list[dict[str, str]],
) -> list[dict[str, str]]:
    corrected_rows: list[dict[str, str]] = []

    for row in rows or []:
        pic_clause = str(row.get("IDMS PIC Clause", "") or "").strip()
        db2_record = str(row.get("New DB2 Record", "") or "").strip()
        db2_field = str(row.get("New DB2 Field name", "") or "").strip()
        current_type = str(row.get("New DB2 Data Type", "") or "").strip().upper()

```

```

        if current_type == "DATE":
            corrected_rows.append(
                row,
            )
            continue

        if pic_clause and db2_record and db2_field:
            row["New DB2 Data Type"] = DB2DatatypeMapper.map_picture(
                picture=pic_clause,
                fallback_length=None,
                fallback_scale=None,
            )

            corrected_rows.append(
                row,
            )

    return corrected_rows

```

```

=====
FILE: src\idms_modernizer\rules\excel_date_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.services.db2_datatype_mapper import DB2DatatypeMapper
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.rules.excel_basic_rules import (
    get_field_picture,
    is_group_field,
    name_parts,
    safe_int,
)
from idms_modernizer.rules.excel_mapping_constants import (
    DAY_PARTS,
    MONTH_PARTS,
    YEAR_PARTS,
)
from idms_modernizer.rules.excel_name_rules import (
    is_date_like_name,
    is_date_part_name,
)

```

```

def is_yyyymmdd_picture(
    pic_clause: str,
) -> bool:
    clean = DB2DatatypeMapper.clean_picture(
        pic_clause,
    )

    core = DB2DatatypeMapper.picture_core(
        clean,
    )

    return core in {
        "9(8)",
        "S9(8)",
        "99999999",
        "S99999999",
    }

```

```

def is_potential_outer_date(
    field,
) -> bool:
    field_name = getattr(field, "name", "") or ""

    if is_date_like_name(
        field_name,
    ):
        return True

    datatype = str(
        getattr(field, "datatype", None)
        or getattr(field, "data_type", None)
    )

```

```

        or getattr(field, "type", None)
        or "",
    ).strip().upper()

    basetype = str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or "",
    ).strip().upper()

    return datatype == "DATE" or basetype == "DATE"

def is_direct_date_field(
    field,
) -> bool:
    if is_group_field(
        field=field,
    ):
        return False

    datatype = str(
        getattr(field, "datatype", None)
        or getattr(field, "data_type", None)
        or getattr(field, "type", None)
        or "",
    ).strip().upper()

    basetype = str(
        getattr(field, "basetype", None)
        or getattr(field, "base_type", None)
        or "",
    ).strip().upper()

    if datatype == "DATE" or basetype == "DATE":
        return not is_date_part_name(
            field_name=getattr(field, "name", "") or "",
        )

    pic_clause = get_field_picture(
        field=field,
    )

    if is_date_like_name(
        getattr(field, "name", "") or "",
    ):
        return is_yyyymmdd_picture(
            pic_clause=pic_clause,
        )

    return False

def build_date_scope(
    fields,
) -> dict[int, bool]:
    scope: dict[int, bool] = {}
    active_date_levels: list[int] = []

    for field in fields or []:
        level = safe_int(
            getattr(field, "level", None),
            default=0,
        )

        active_date_levels = [
            current_level
            for current_level in active_date_levels
            if current_level < level
        ]

        if active_date_levels:
            scope[id(field)] = True

        if is_potential_outer_date(
            field=field,

```

```

    ):
        active_date_levels.append(
            level,
        )

    return scope

def group_has_ymd_children(
    field,
    fields,
    start_index: int,
) -> bool:
    parent_level = safe_int(
        getattr(field, "level", None),
        default=0,
    )

    found_year = False
    found_month = False
    found_day = False

    for child in list(fields or [])[start_index + 1:]:
        child_level = safe_int(
            getattr(child, "level", None),
            default=0,
        )

        if child_level <= parent_level:
            break

        child_name = getattr(child, "name", "") or ""

        parts = name_parts(
            child_name,
        )

        if any(part in YEAR_PARTS for part in parts):
            found_year = True

        if any(part in MONTH_PARTS for part in parts):
            found_month = True

        if any(part in DAY_PARTS for part in parts):
            found_day = True

    return found_year and found_month and found_day

def collect_date_group_info(
    fields,
) -> dict[str, dict]:
    info: dict[str, dict] = {}

    for index, field in enumerate(fields or []):
        if not is_group_field(
            field=field,
        ):
            continue

        if not is_potential_outer_date(
            field=field,
        ):
            continue

        if not group_has_ymd_children(
            field=field,
            fields=fields,
            start_index=index,
        ):
            continue

        field_name = NameNormalizer.normalize(
            getattr(field, "name", "") or "",
        )

```



```

        if field_name:
            info[field_name] = {
                "field": field,
            }

    return info

def is_outer_date_group(
    field,
    date_group_info: dict[str, dict],
) -> bool:
    if not is_group_field(
        field=field,
    ):
        return False

    field_name = NameNormalizer.normalize(
        getattr(field, "name", "") or "",
    )

    return field_name in date_group_info

def is_date_child(
    field,
    date_scope: dict[int, bool],
) -> bool:
    if id(field) not in date_scope:
        return False

    return is_date_part_name(
        field_name=getattr(field, "name", "") or "",
    )

```

```

=====
FILE: src\idms_modernizer\rules\excel_db2_mapping_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules.excel_key_rules import (
    infer_db2_datatype_from_field,
    should_emit_db2_field,
)
from idms_modernizer.rules.excel_name_rules import (
    convert_cobol_zone_to_db2_field_name,
    convert_date_field_to_db2_field_name,
    enforce_middle_name_max_length,
)
from idms_modernizer.rules.excel_db2_record_name_rules import (
    db2_record_name,
)

def clear_db2_mapping(
    row,
) -> None:
    row["New DB2 Record"] = ""
    row["New DB2 Field name"] = ""
    row["New DB2 Data Type"] = ""
    row["DB2 Key"] = ""

def apply_date_db2_mapping(
    row,
    record,
    field,
    table,
) -> None:
    row["New DB2 Record"] = db2_record_name(
        record=record,
        table=table,
    )
    row["New DB2 Field name"] = enforce_middle_name_max_length(
        db2_field_name=convert_date_field_to_db2_field_name(

```

```

        record=record,
        field=field,
    ),
)
row["New DB2 Data Type"] = "DATE"

def apply_standard_db2_mapping(
    row,
    record,
    field,
    table,
    db2_column,
) -> None:
    row["New DB2 Record"] = db2_record_name(
        record=record,
        table=table,
    )

    if db2_column is not None and getattr(db2_column, "name", None):
        field_name = getattr(db2_column, "name", "") or ""
    else:
        field_name = convert_cobol_zone_to_db2_field_name(
            record=record,
            field=field,
        )

    row["New DB2 Field name"] = enforce_middle_name_max_length(
        db2_field_name=field_name,
    )

    row["New DB2 Data Type"] = infer_db2_datatype_from_field(
        field=field,
    )

def apply_db2_mapping(
    row,
    record,
    field,
    table,
    db2_column,
    is_outer_date,
    is_date_child_value,
    is_direct_date,
    calc_status,
    is_group,
    is_redefines,
) -> None:
    if is_redefines:
        clear_db2_mapping(
            row=row,
        )
        return

    if is_outer_date:
        apply_date_db2_mapping(
            row=row,
            record=record,
            field=field,
            table=table,
        )
        return

    if is_date_child_value:
        clear_db2_mapping(
            row=row,
        )
        return

    if is_direct_date:
        apply_date_db2_mapping(
            row=row,
            record=record,
            field=field,
            table=table,

```

```

    )
    return

if calc_status == "ROOT":
    clear_db2_mapping(
        row=row,
    )
    return

if is_group:
    clear_db2_mapping(
        row=row,
    )
    return

if should_emit_db2_field(
    field=field,
):
    apply_standard_db2_mapping(
        row=row,
        record=record,
        field=field,
        table=table,
        db2_column=db2_column,
    )

```

```

=====
FILE: src\idms_modernizer\rules\excel_db2_record_name_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.db2_models import DB2Table
from idms_modernizer.rules.excel_basic_rules import (
    to_db2_name,
)

```

```

def db2_record_name(
    record,
    table: DB2Table | None,
) -> str:
    if table is not None and getattr(table, "name", None):
        return getattr(table, "name", "") or ""

    return to_db2_name(
        getattr(record, "name", "") or "",
    )

```

```

=====
FILE: src\idms_modernizer\rules\excel_extra_row_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.db2_models import DB2Table
from idms_modernizer.rules.excel_basic_rules import (
    empty_row,
    is_redefines_field,
    to_db2_name,
)
from idms_modernizer.rules.excel_fk_rules import (
    get_db2_datatype,
    get_relationship_member_record,
)
from idms_modernizer.rules.excel_mapping_constants import (
    DB2_COLUMN_CODE_PREFIX,
)

```

```

def build_generated_pk_rows(
    record,
    table: DB2Table | None,
) -> list[dict[str, str]]:
    if table is None:
        return []

```

```

rows: list[dict[str, str]] = []

for column in getattr(table, "columns", []) or []:
    if getattr(column, "source_kind", "") != "GENERATED PK":
        continue

    row = empty_row()
    row["Cobol Record IDMS"] = getattr(record, "name", "") or ""
    row["DB2 Key"] = "PK"
    row["New DB2 Record"] = getattr(table, "name", "") or ""
    row["New DB2 Field name"] = getattr(column, "name", "") or ""
    row["New DB2 Data Type"] = get_db2_datatype(
        db2_column=column,
    )

    rows.append(
        row,
    )

return rows

def build_audit_rows(
    record,
    table: DB2Table | None,
) -> list[dict[str, str]]:
    if table is None:
        return []

    rows: list[dict[str, str]] = []

    record_name = getattr(record, "name", "") or ""

    suffix = audit_field_suffix(
        record_name=record_name,
    )

    audit_fields = [
        (
            f"TS_CREATE_{suffix}",
            "TIMESTAMP",
        ),
    ]

    if not all_physical_fields_are_calc_keys(
        record=record,
    ):
        audit_fields.append(
            (
                f"TS_UPDATE_{suffix}",
                "TIMESTAMP",
            )
        )

    audit_fields.append(
        (
            f"ID_USERID_{suffix}",
            "CHAR(8)",
        )
    )

    for field_name, datatype in audit_fields:
        row = empty_row()
        row["Cobol Record IDMS"] = record_name
        row["New DB2 Record"] = getattr(table, "name", "") or ""
        row["New DB2 Field name"] = field_name
        row["New DB2 Data Type"] = datatype

        rows.append(
            row,
        )

    return rows

```

```

def build_sequence_number_rows(
    metadata,
    record,
    table: DB2Table | None,
) -> list[dict[str, str]]:
    if table is None:
        return []

    if not record_requires_sequence_number(
        metadata=metadata,
        record=record,
    ):
        return []

    record_name = getattr(record, "name", "") or ""

    suffix = audit_field_suffix(
        record_name=record_name,
    )

    row = empty_row()
    row["Cobol Record IDMS"] = record_name
    row["New DB2 Record"] = getattr(table, "name", "") or ""
    row["New DB2 Field name"] = f"NS_SEQUENCE_{suffix}"
    row["New DB2 Data Type"] = "DECIMAL(9,0)"
    row["Hopex Expression TypeRemark"] = "SEQUENCE NUMBER FOR DUPLICATE LAST"

    return [
        row,
    ]

def audit_field_suffix(
    record_name: str | None,
) -> str:
    normalized_record_name = to_db2_name(
        record_name,
    )

    compact_record_name = normalized_record_name.replace(
        "_",
        "",
    )

    if not compact_record_name:
        return DB2_COLUMN_CODE_PREFIX

    record_suffix = (
        compact_record_name[-4:]
        if len(compact_record_name) >= 4
        else compact_record_name
    )

    return f"{DB2_COLUMN_CODE_PREFIX}{record_suffix}"

def all_physical_fields_are_calc_keys(
    record,
) -> bool:
    from idms_modernizer.rules.excel_key_rules import (
        build_calc_scope,
        calc_status_for_field,
        should_emit_db2_field,
    )

    mapping_fields = (
        getattr(record, "mapping_fields", None)
        or getattr(record, "fields", None)
        or []
    )

    calc_scope = build_calc_scope(
        record=record,
        fields=mapping_fields,
    )

```

```

physical_fields = []

for field in mapping_fields:
    if is_redefines_field(
        field=field,
    ):
        continue

    if should_emit_db2_field(
        field=field,
    ):
        physical_fields.append(
            field,
        )

if not physical_fields:
    return False

for field in physical_fields:
    if calc_status_for_field(
        field=field,
        calc_scope=calc_scope,
    ) != "DESCENDANT":
        return False

return True

def get_relationship_order_mode(
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("order_mode")
            or relationship.get("order")
            or relationship.get("sort_order")
            or relationship.get("duplicates")
            or "",
        ).upper()

    return str(
        getattr(relationship, "order_mode", None)
        or getattr(relationship, "order", None)
        or getattr(relationship, "sort_order", None)
        or getattr(relationship, "duplicates", None)
        or "",
    ).upper()

def is_duplicate_last_relationship(
    relationship,
) -> bool:
    order_mode = get_relationship_order_mode(
        relationship=relationship,
    )

    return (
        "DL" in order_mode
        or "DUPLICATE LAST" in order_mode
        or "DUPLICATES LAST" in order_mode
    )

def record_requires_sequence_number(
    metadata,
    record,
) -> bool:
    record_name = to_db2_name(
        getattr(record, "name", "") or "",
    )

    for relationship in getattr(metadata, "relationships", []) or []:
        member_record = to_db2_name(
            get_relationship_member_record(
                relationship=relationship,
            )

```

```

    )

    if member_record != record_name:
        continue

    if is_duplicate_last_relationship(
        relationship=relationship,
    ):
        return True

    return False

=====
FILE: src\idms_modernizer\rules\excel_field_identity_rules.py
ENCODING: utf-8
=====

import re

from idms_modernizer.services.name_normalizer import NameNormalizer

def get_field_raw_text(
    field,
) -> str:
    values = [
        getattr(field, "name", None),
        getattr(field, "raw_line", None),
        getattr(field, "original_line", None),
        getattr(field, "rest", None),
    ]

    return " ".join(
        str(value or "")
        for value in values
        if value is not None
    ).strip()

def is_filler_field(
    field,
) -> bool:
    return NameNormalizer.normalize(
        getattr(field, "name", "") or "",
    ).startswith(
        "FILLER",
    )

def is_group_field(
    field,
) -> bool:
    return bool(
        getattr(field, "has_child", False)
        or getattr(field, "is_group", False)
    )

def is_redefines_field(
    field,
) -> bool:
    text = get_field_raw_text(
        field=field,
    ).upper()

    return (
        "REDEFINES" in text
        or "REDFINES" in text
    )

def get_redefines_target(
    field,
) -> str:
    explicit_target = (

```

```

        getattr(field, "redefines_target", None)
        or getattr(field, "redefines", None)
    )

    if explicit_target:
        return str(explicit_target or "").strip().upper()

    text = get_field_raw_text(
        field=field,
    ).upper()

    match = re.search(
        r"\b(?:REDEFINES|REDFINES)\s+([A-Z][A-Z0-9-]*)",
        text,
        re.IGNORECASE,
    )

    if not match:
        return ""

    return match.group(
        1,
    ).strip().upper()

```

```

=====
FILE: src\idms_modernizer\rules\excel_field_picture_rules.py
ENCODING: utf-8
=====

```

```

import re

from idms_modernizer.rules.excel_field_identity_rules import (
    is_group_field,
)

def get_field_picture(
    field,
) -> str:
    if is_group_field(
        field=field,
    ):
        return ""

    text = str(
        getattr(field, "picture", None)
        or getattr(field, "pic", None)
        or getattr(field, "pic_clause", None)
        or "",
    ).strip()

    if not text:
        return ""

    if text.upper().startswith("PIC "):
        return text

    if text.upper().startswith("PICTURE "):
        return "PIC " + text[8:].strip()

    return f"PIC {text}"

def parse_length_from_pic_clause(
    pic_clause: str | None,
):
    text = str(pic_clause or "").strip().upper()

    if not text:
        return None

    text = text.replace(
        "PICTURE",
        "PIC",
    )

```



```

text = text.replace(
    ".",
    "",
)
text = re.sub(
    r"\s+",
    " ",
    text,
).strip()

text = re.sub(
    r"\bCOMP-3\b",
    "",
    text,
)
text = re.sub(
    r"\bCOMP\b",
    "",
    text,
)
text = re.sub(
    r"\bDISPLAY\b",
    "",
    text,
)

text = text.strip()

if text.startswith("PIC "):
    text = text[4:].strip()

text = text.replace(
    " ",
    "",
)

char_match = re.fullmatch(
    r"X(?:$(\d+)?)?",
    text,
)

if char_match:
    length_text = char_match.group(1)

    if length_text:
        return int(length_text)

    return 1

decimal_match = re.fullmatch(
    r"S?9(?:$(\d+)?)?V9(?:$(\d+)?)?",
    text,
)

if decimal_match:
    integer_digits = decimal_match.group(1)
    decimal_digits = decimal_match.group(2)

    integer_length = int(integer_digits) if integer_digits else 1
    decimal_length = int(decimal_digits) if decimal_digits else 1

    return integer_length + decimal_length

numeric_match = re.fullmatch(
    r"S?9(?:$(\d+)?)?",
    text,
)

if numeric_match:
    length_text = numeric_match.group(1)

    if length_text:
        return int(length_text)

    return 1

```

```
return None
```

```
=====
FILE: src\idms_modernizer\rules\excel_field_position_rules.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.rules.excel_field_picture_rules import (
    get_field_picture,
    parse_length_from_pic_clause,
)
```

```
def get_field_start_position(
    field,
):
    return (
        getattr(field, "start_position", None)
        or getattr(field, "start", None)
    )
```

```
def get_field_length(
    field,
):
    length = (
        getattr(field, "length", None)
        or getattr(field, "storage_length", None)
        or getattr(field, "physical_length", None)
        or getattr(field, "byte_length", None)
        or getattr(field, "bytes", None)
    )

    if length not in (
        "",
        None,
    ):
        return length

    pic_clause = get_field_picture(
        field=field,
    )

    parsed_length = parse_length_from_pic_clause(
        pic_clause=pic_clause,
    )

    if parsed_length is not None:
        return parsed_length

    return ""
```

```
def get_field_end_position(
    field,
):
    start = get_field_start_position(
        field=field,
    )

    length = get_field_length(
        field=field,
    )

    try:
        if start is not None and length not in (
            "",
            None,
        ):
            return int(start) + int(length) - 1
    except Exception:
        pass

    end_position = (
        getattr(field, "end_position", None)
    )
```

```

        or getattr(field, "field_end_position", None)
        or getattr(field, "end", None)
    )

    if end_position is not None:
        return end_position

    return ""

```

```

=====
FILE: src\idms_modernizer\rules\excel_field_remark_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules.occurs_rules import (
    format_occurs_clause,
)
from idms_modernizer.rules.excel_field_identity_rules import (
    get_redefines_target,
    is_redefines_field,
)

```

```

def hopex_remark(
    field,
) -> str:
    remarks: list[str] = []

    if is_redefines_field(
        field=field,
    ):
        target = get_redefines_target(
            field=field,
        )

        if target:
            remarks.append(
                f"REDEFINES {target}",
            )
        else:
            remarks.append(
                "REDEFINES",
            )

    if getattr(field, "occurs", False):
        occurs_text = format_occurs_clause(
            occurs_min=getattr(field, "occurs_min", None),
            occurs_max=getattr(field, "occurs_max", None),
        )

        if occurs_text:
            remarks.append(
                occurs_text,
            )

    return "; ".join(
        remarks,
    )

```

```

=====
FILE: src\idms_modernizer\rules\excel_filler_row_builder.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules.excel_basic_rules import (
    empty_row,
    format_cobol_zone,
    get_field_end_position,
    get_field_length,
    get_field_picture,
    hopex_remark,
    to_string,
)

```

```

def build_filler_row(
    record,
    field,
) -> dict[str, str]:
    row = empty_row()

    row["Cobol Record IDMS"] = getattr(record, "name", "") or ""
    row["Cobol Zone"] = format_cobol_zone(
        field=field,
    )
    row["IDMS PIC Clause"] = get_field_picture(
        field=field,
    )
    row["Length of Field Bytes"] = to_string(
        get_field_length(
            field=field,
        )
    )
    row["Field end position"] = to_string(
        get_field_end_position(
            field=field,
        )
    )
    row["Hopex Expression TypeRemark"] = hopex_remark(
        field=field,
    )

    return row

```

```

=====
FILE: src\idms_modernizer\rules\excel_fk_accessors.py
ENCODING: utf-8
=====

```

```

def get_foreign_key_column_name(
    foreign_key,
) -> str:
    if isinstance(foreign_key, dict):
        return str(
            foreign_key.get("column_name")
            or foreign_key.get("column")
            or foreign_key.get("fk_column")
            or foreign_key.get("foreign_key_column")
            or "",
        )

    return str(
        getattr(foreign_key, "column_name", None)
        or getattr(foreign_key, "column", None)
        or getattr(foreign_key, "fk_column", None)
        or getattr(foreign_key, "foreign_key_column", None)
        or "",
    )

def get_foreign_key_reference_table(
    foreign_key,
) -> str:
    if isinstance(foreign_key, dict):
        return str(
            foreign_key.get("reference_table")
            or foreign_key.get("parent_table")
            or foreign_key.get("referenced_table")
            or "",
        )

    return str(
        getattr(foreign_key, "reference_table", None)
        or getattr(foreign_key, "parent_table", None)
        or getattr(foreign_key, "referenced_table", None)
        or "",
    )

```

```

def get_foreign_key_reference_column(
    foreign_key,
) -> str:
    if isinstance(foreign_key, dict):
        return str(
            foreign_key.get("reference_column")
            or foreign_key.get("parent_column")
            or foreign_key.get("referenced_column")
            or foreign_key.get("parent_key")
            or "",
        )

    return str(
        getattr(foreign_key, "reference_column", None)
        or getattr(foreign_key, "parent_column", None)
        or getattr(foreign_key, "referenced_column", None)
        or getattr(foreign_key, "parent_key", None)
        or "",
    )

```

```

def get_relationship_set_name(
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("set_name")
            or relationship.get("set")
            or relationship.get("name")
            or "",
        )

    return str(
        getattr(relationship, "set_name", None)
        or getattr(relationship, "set", None)
        or getattr(relationship, "name", None)
        or "",
    )

```

```

def get_relationship_owner_record(
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("owner_record")
            or relationship.get("parent_record")
            or relationship.get("owner")
            or relationship.get("parent")
            or "",
        )

    return str(
        getattr(relationship, "owner_record", None)
        or getattr(relationship, "parent_record", None)
        or getattr(relationship, "owner", None)
        or getattr(relationship, "parent", None)
        or "",
    )

```

```

def get_relationship_member_record(
    relationship,
) -> str:
    if isinstance(relationship, dict):
        return str(
            relationship.get("member_record")
            or relationship.get("child_record")
            or relationship.get("member")
            or relationship.get("child")
            or "",
        )

    return str(
        getattr(relationship, "member_record", None)
        or getattr(relationship, "child_record", None)

```

```

        or getattr(relationship, "member", None)
        or getattr(relationship, "child", None)
        or "",
    )

```

```

=====
FILE: src\idms_modernizer\rules\excel_fk_datatype_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.db2_models import (
    DB2Column,
)
from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.db2_datatype_mapper import DB2DatatypeMapper
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.rules.excel_basic_rules import (
    get_field_picture,
    name_lookup_keys,
    remove_record_suffix,
)
from idms_modernizer.rules.excel_fk_accessors import (
    get_foreign_key_reference_column,
    get_relationship_owner_record,
    get_relationship_set_name,
)
from idms_modernizer.rules.excel_name_rules import (
    convert_cobol_zone_to_db2_field_name,
    convert_date_field_to_db2_field_name,
)

```

```

def get_db2_datatype(
    db2_column: DB2Column | None,
) -> str:
    if db2_column is None:
        return ""

    return str(
        getattr(db2_column, "datatype", None)
        or getattr(db2_column, "data_type", None)
        or getattr(db2_column, "type", None)
        or "",
    ).strip().upper()

```

```

def resolve_foreign_key_reference_datatype(
    metadata: SchemaMetadata | None,
    foreign_key,
    fallback_column: DB2Column | None,
) -> str:
    set_name = str(
        getattr(foreign_key, "set_name", "") or "",
    )

    reference_column_name = get_foreign_key_reference_column(
        foreign_key=foreign_key,
    )

    owner_record_name = find_owner_record_name_for_set(
        metadata=metadata,
        set_name=set_name,
    )

    if owner_record_name and reference_column_name:
        datatype = resolve_owner_reference_datatype_by_column_name(
            metadata=metadata,
            owner_record_name=owner_record_name,
            owner_column_name=reference_column_name,
        )

        if datatype:
            return datatype

    return get_db2_datatype(

```

```

        db2_column=fallback_column,
    )

def find_owner_record_name_for_set(
    metadata: SchemaMetadata | None,
    set_name: str,
) -> str:
    normalized_set_name = NameNormalizer.normalize(
        set_name,
    )

    if not normalized_set_name:
        return ""

    for relationship in getattr(metadata, "relationships", []) or []:
        relationship_set_name = get_relationship_set_name(
            relationship=relationship,
        )

        if NameNormalizer.normalize(
            relationship_set_name,
        ) != normalized_set_name:
            continue

        return get_relationship_owner_record(
            relationship=relationship,
        )

    return ""

def resolve_owner_reference_datatype_by_column_name(
    metadata: SchemaMetadata | None,
    owner_record_name: str,
    owner_column_name: str,
) -> str:
    owner_record = find_metadata_record(
        metadata=metadata,
        record_name=owner_record_name,
    )

    if owner_record is None:
        return ""

    owner_fields = []
    owner_fields.extend(
        getattr(owner_record, "fields", []) or []
    )
    owner_fields.extend(
        getattr(owner_record, "mapping_fields", []) or []
    )

    for field in owner_fields:
        if field_matches_db2_column(
            record=owner_record,
            field=field,
            db2_column_name=owner_column_name,
        ):
            return infer_fk_datatype_from_owner_field(
                field=field,
            )

    return ""

def resolve_owner_reference_datatype(
    metadata: SchemaMetadata | None,
    owner_record_name: str,
    owner_pk_column: DB2Column,
) -> str:
    owner_pk_column_name = getattr(owner_pk_column, "name", "") or ""

    return resolve_owner_reference_datatype_by_column_name(
        metadata=metadata,
        owner_record_name=owner_record_name,
    )

```

```

        owner_column_name=owner_pk_column_name,
    )

def infer_fk_datatype_from_owner_field(
    field,
) -> str:
    datatype = str(
        getattr(field, "datatype", "") or "",
    ).upper()

    basetype = str(
        getattr(field, "basetype", "") or "",
    ).upper()

    field_name = NameNormalizer.normalize(
        getattr(field, "name", "") or "",
    )

    pic_clause = get_field_picture(
        field=field,
    )

    clean_picture = DB2DatatypeMapper.clean_picture(
        picture=pic_clause,
    )

    picture_core = DB2DatatypeMapper.picture_core(
        picture=clean_picture,
    )

    is_yyyymmdd_picture = picture_core in {
        "9(8)",
        "S9(8)",
        "99999999",
        "S99999999",
    }

    is_date_like_field = (
        datatype == "DATE"
        or basetype == "DATE"
        or field_name.startswith("DA_")
        or field_name.endswith("_DATE")
        or "_DATE_" in field_name
    )

    if is_date_like_field and is_yyyymmdd_picture:
        return "DATE"

    if datatype == "DATE" or basetype == "DATE":
        return "DATE"

    if pic_clause:
        return DB2DatatypeMapper.map_picture(
            picture=pic_clause,
            fallback_length=None,
            fallback_scale=getattr(field, "scale", None),
        )

    length = (
        getattr(field, "length", None)
        or getattr(field, "storage_length", None)
        or getattr(field, "physical_length", None)
        or getattr(field, "byte_length", None)
        or getattr(field, "bytes", None)
    )

    scale = getattr(
        field,
        "scale",
        None,
    )

    if datatype in {
        "TIMESTAMP",
        "DATETIME",
    }

```



```

    }:
        return "TIMESTAMP"

    if datatype in {
        "CHAR",
        "VARCHAR",
        "DISPLAY",
        "TEXT",
    }:
        actual_length = safe_int_for_fk(
            value=length,
            default=1,
        )

        return f"CHAR({actual_length})"

    if datatype in {
        "DECIMAL",
        "NUMERIC",
    }:
        actual_length = safe_int_for_fk(
            value=length,
            default=1,
        )

        actual_scale = safe_int_for_fk(
            value=scale,
            default=0,
        )

        return f"DECIMAL({actual_length},{actual_scale})"

    return ""

def find_metadata_record(
    metadata: SchemaMetadata | None,
    record_name: str,
):
    normalized_record_name = NameNormalizer.normalize(
        record_name,
    )

    for record in getattr(metadata, "records", []) or []:
        current_record_name = NameNormalizer.normalize(
            getattr(record, "name", "") or "",
        )

        if current_record_name == normalized_record_name:
            return record

        if remove_record_suffix(current_record_name) == remove_record_suffix(
            normalized_record_name,
        ):
            return record

    return None

def field_matches_db2_column(
    record,
    field,
    db2_column_name: str,
) -> bool:
    normalized_db2_column_name = NameNormalizer.normalize(
        db2_column_name,
    )

    if not normalized_db2_column_name:
        return False

    candidate_names = []

    field_name = getattr(field, "name", "") or ""

    candidate_names.append(

```

```

        field_name,
    )

    candidate_names.append(
        remove_record_suffix(
            field_name,
        )
    )

    try:
        candidate_names.append(
            convert_cobol_zone_to_db2_field_name(
                record=record,
                field=field,
            )
        )
    except Exception:
        pass

    try:
        candidate_names.append(
            convert_date_field_to_db2_field_name(
                record=record,
                field=field,
            )
        )
    except Exception:
        pass

    db2_column_keys = set(
        name_lookup_keys(
            normalized_db2_column_name,
        )
    )

    db2_column_keys.add(
        normalized_db2_column_name,
    )

    db2_column_keys.add(
        remove_record_suffix(
            normalized_db2_column_name,
        )
    )

    for candidate_name in candidate_names:
        normalized_candidate_name = NameNormalizer.normalize(
            candidate_name,
        )

        if not normalized_candidate_name:
            continue

        candidate_keys = set(
            name_lookup_keys(
                normalized_candidate_name,
            )
        )

        candidate_keys.add(
            normalized_candidate_name,
        )

        candidate_keys.add(
            remove_record_suffix(
                normalized_candidate_name,
            )
        )

        if candidate_keys & db2_column_keys:
            return True

    return False

```

```

def safe_int_for_fk(

```

```

        value,
        default: int = 0,
    ) -> int:
        try:
            if value is None:
                return default

            text = str(value).strip()

            if not text:
                return default

            return int(text)
        except Exception:
            return default

```

```

=====
FILE: src\idms_modernizer\rules\excel_fk_key_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.db2_models import (
    DB2Column,
    DB2Table,
)
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.rules.excel_fk_accessors import (
    get_foreign_key_column_name,
)

```

```

def fk_emit_key(
    cobol_record_name: str,
    db2_table_name: str,
    fk_column_name: str,
) -> tuple[str, str, str]:
    return (
        NameNormalizer.normalize(
            cobol_record_name,
        ),
        NameNormalizer.normalize(
            db2_table_name,
        ),
        NameNormalizer.normalize(
            fk_column_name,
        ),
    )

```

```

def is_primary_key_column(
    table: DB2Table,
    column: DB2Column,
) -> bool:
    if getattr(column, "primary_key", False):
        return True

    column_name = NameNormalizer.normalize(
        getattr(column, "name", "") or "",
    )

    primary_keys = list(
        getattr(table, "primary_keys", []) or []
    )

    if not primary_keys and getattr(table, "primary_key", None):
        primary_keys = [
            table.primary_key,
        ]

    normalized_primary_keys = {
        NameNormalizer.normalize(
            primary_key,
        )
        for primary_key in primary_keys
        if primary_key
    }

```

```

    }

    return column_name in normalized_primary_keys

def is_foreign_key_column(
    table: DB2Table,
    column: DB2Column,
) -> bool:
    column_name = NameNormalizer.normalize(
        getattr(column, "name", "") or "",
    )

    for foreign_key in getattr(table, "foreign_keys", []) or []:
        fk_column_name = NameNormalizer.normalize(
            get_foreign_key_column_name(
                foreign_key=foreign_key,
            )
        )

        if fk_column_name == column_name:
            return True

    source_kind = str(
        getattr(column, "source_kind", "") or "",
    ).upper()

    return source_kind in {
        "SET_FK",
        "FK",
        "FOREIGN KEY",
    }

def db2_key_label(
    table: DB2Table | None,
    column: DB2Column | None,
) -> str:
    if table is None or column is None:
        return ""

    labels: list[str] = []

    if is_primary_key_column(
        table=table,
        column=column,
    ):
        labels.append(
            "PK",
        )

    if is_foreign_key_column(
        table=table,
        column=column,
    ):
        labels.append(
            "FK",
        )

    return "/".join(
        labels,
    )

def find_primary_key_columns(
    table: DB2Table | None,
) -> list[DB2Column]:
    if table is None:
        return []

    primary_key_names = list(
        getattr(table, "primary_keys", []) or []
    )

    if not primary_key_names and getattr(table, "primary_key", None):
        primary_key_names = [

```

```

        table.primary_key,
    ]

columns = list(
    getattr(table, "columns", []) or []
)

if primary_key_names:
    normalized_primary_keys = {
        NameNormalizer.normalize(
            primary_key,
        )
        for primary_key in primary_key_names
        if primary_key
    }

    matched_columns = [
        column
        for column in columns
        if NameNormalizer.normalize(
            getattr(column, "name", "") or "",
        )
        in normalized_primary_keys
    ]

    if matched_columns:
        return matched_columns

return [
    column
    for column in columns
    if getattr(column, "primary_key", False)
]

```

```

=====
FILE: src\idms_modernizer\rules\excel_fk_relationship_lookup_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.rules.excel_fk_accessors import (
    get_relationship_member_record,
    get_relationship_owner_record,
    get_relationship_set_name,
)

def build_relationship_lookup(
    metadata: SchemaMetadata,
) -> dict[str, str]:
    lookup: dict[str, str] = {}

    for relationship in getattr(metadata, "relationships", []) or []:
        owner_record = get_relationship_owner_record(
            relationship=relationship,
        )

        member_record = get_relationship_member_record(
            relationship=relationship,
        )

        set_name = get_relationship_set_name(
            relationship=relationship,
        )

        for record_name in [
            owner_record,
            member_record,
        ]:
            normalized_record = NameNormalizer.normalize(
                record_name,
            )

            if not normalized_record:

```

```

        continue

    existing = lookup.get(
        normalized_record,
        "",
    )

    if existing:
        if set_name and set_name not in existing:
            lookup[normalized_record] = f"{existing}; {set_name}"
        else:
            lookup[normalized_record] = set_name

    return lookup

```

```

=====
FILE: src\idms_modernizer\rules\excel_fk_row_builder.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.db2_models import (
    DB2Table,
)
from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.rules.excel_basic_rules import (
    empty_row,
)
from idms_modernizer.rules.excel_fk_accessors import (
    get_foreign_key_column_name,
    get_relationship_member_record,
    get_relationship_owner_record,
    get_relationship_set_name,
)
from idms_modernizer.rules.excel_fk_datatype_rules import (
    get_db2_datatype,
    resolve_foreign_key_reference_datatype,
    resolve_owner_reference_datatype,
)
from idms_modernizer.rules.excel_fk_key_rules import (
    find_primary_key_columns,
    fk_emit_key,
)
from idms_modernizer.rules.excel_lookup_rules import (
    build_column_lookup,
    find_matching_column,
    find_primary_key_column,
    find_table_for_record,
)

def build_set_fk_rows_for_record(
    metadata: SchemaMetadata,
    record,
    table: DB2Table | None,
    table_lookup: dict[str, DB2Table],
) -> list[dict[str, str]]:
    if table is None:
        return []

    rows: list[dict[str, str]] = []
    emitted: dict[tuple[str, str, str], dict[str, str]] = {}

    rows.extend(
        build_set_fk_rows_from_metadata_relationships(
            metadata=metadata,
            record=record,
            member_table=table,
            table_lookup=table_lookup,
            emitted=emitted,
        )
    )

    rows.extend(
        build_set_fk_rows_from_db2_foreign_keys(

```

```

        metadata=metadata,
        record=record,
        table=table,
        emitted=emitted,
    )
)

return rows

def build_set_fk_rows_from_db2_foreign_keys(
    metadata: SchemaMetadata | None,
    record,
    table: DB2Table,
    emitted: dict[tuple[str, str, str], dict[str, str]],
) -> list[dict[str, str]]:
    rows: list[dict[str, str]] = []

    cobol_record_name = getattr(record, "name", "") or ""

    column_lookup = build_column_lookup(
        table=table,
    )

    for foreign_key in getattr(table, "foreign_keys", []) or []:
        fk_column_name = get_foreign_key_column_name(
            foreign_key=foreign_key,
        )

        if not fk_column_name:
            continue

        fk_column = find_matching_column(
            normalized_column_name=fk_column_name,
            column_lookup=column_lookup,
        )

        if fk_column is None:
            continue

        set_name = getattr(foreign_key, "set_name", "") or ""

        fk_datatype = resolve_foreign_key_reference_datatype(
            metadata=metadata,
            foreign_key=foreign_key,
            fallback_column=fk_column,
        )

        if not fk_datatype:
            fk_datatype = get_db2_datatype(
                db2_column=fk_column,
            )

        add_or_merge_fk_row(
            rows=rows,
            emitted=emitted,
            cobol_record_name=cobol_record_name,
            db2_table_name=getattr(table, "name", "") or "",
            fk_column_name=getattr(fk_column, "name", "") or "",
            fk_datatype=fk_datatype,
            set_name=set_name,
        )

    return rows

def build_set_fk_rows_from_metadata_relationships(
    metadata: SchemaMetadata,
    record,
    member_table: DB2Table,
    table_lookup: dict[str, DB2Table],
    emitted: dict[tuple[str, str, str], dict[str, str]],
) -> list[dict[str, str]]:
    rows: list[dict[str, str]] = []

    cobol_record_name = getattr(record, "name", "") or ""

```

```

normalized_current_record = NameNormalizer.normalize(
    cobol_record_name,
)

normalized_current_table = NameNormalizer.normalize(
    getattr(member_table, "name", "") or "",
)

for relationship in getattr(metadata, "relationships", []) or []:
    set_name = get_relationship_set_name(
        relationship=relationship,
    )

    owner_record_name = get_relationship_owner_record(
        relationship=relationship,
    )

    member_record_name = get_relationship_member_record(
        relationship=relationship,
    )

    if not set_name:
        continue

    normalized_member_record = NameNormalizer.normalize(
        member_record_name,
    )

    if normalized_member_record not in {
        normalized_current_record,
        normalized_current_table,
    }:
        continue

    owner_table = find_table_for_record(
        record_name=owner_record_name,
        table_lookup=table_lookup,
    )

    owner_pk_columns = find_primary_key_columns(
        table=owner_table,
    )

    if not owner_pk_columns:
        owner_pk_column = find_primary_key_column(
            table=owner_table,
        )

        if owner_pk_column is not None:
            owner_pk_columns = [
                owner_pk_column,
            ]

    for owner_pk_column in owner_pk_columns:
        fk_column_name = getattr(owner_pk_column, "name", "") or ""

        if not fk_column_name:
            continue

        fk_datatype = resolve_owner_reference_datatype(
            metadata=metadata,
            owner_record_name=owner_record_name,
            owner_pk_column=owner_pk_column,
        )

        if not fk_datatype:
            fk_datatype = get_db2_datatype(
                db2_column=owner_pk_column,
            )

        add_or_merge_fk_row(
            rows=rows,
            emitted=emitted,
            cobol_record_name=cobol_record_name,
            db2_table_name=getattr(member_table, "name", "") or "",

```



```

        fk_column_name=fk_column_name,
        fk_datatype=fk_datatype,
        set_name=set_name,
    )

    return rows

def add_or_merge_fk_row(
    rows: list[dict[str, str]],
    emitted: dict[tuple[str, str, str], dict[str, str]],
    cobol_record_name: str,
    db2_table_name: str,
    fk_column_name: str,
    fk_datatype: str,
    set_name: str,
) -> None:
    key = fk_emit_key(
        cobol_record_name=cobol_record_name,
        db2_table_name=db2_table_name,
        fk_column_name=fk_column_name,
    )

    existing_row = emitted.get(
        key,
    )

    if existing_row is not None:
        existing_relation = str(
            existing_row.get("Relation", "") or "",
        ).strip()

        if set_name and set_name not in existing_relation:
            if existing_relation:
                existing_row["Relation"] = f"{existing_relation}; {set_name}"
            else:
                existing_row["Relation"] = set_name

        existing_datatype = str(
            existing_row.get("New DB2 Data Type", "") or "",
        ).strip()

        if not existing_datatype and fk_datatype:
            existing_row["New DB2 Data Type"] = fk_datatype

        return

    row = build_fk_row(
        cobol_record_name=cobol_record_name,
        db2_table_name=db2_table_name,
        fk_column_name=fk_column_name,
        fk_datatype=fk_datatype,
        set_name=set_name,
    )

    emitted[key] = row

    rows.append(
        row,
    )

def build_fk_row(
    cobol_record_name: str,
    db2_table_name: str,
    fk_column_name: str,
    fk_datatype: str,
    set_name: str,
) -> dict[str, str]:
    row = empty_row()

    row["Cobol Record IDMS"] = cobol_record_name
    row["IDMS Key"] = "SET"
    row["DB2 Key"] = "FK"
    row["New DB2 Record"] = db2_table_name
    row["New DB2 Field name"] = fk_column_name

```

```

row["New DB2 Data Type"] = fk_datatype
row["Relation"] = set_name

return row

```

```

=====
FILE: src\idms_modernizer\rules\excel_fk_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules.excel_fk_accessors import (
    get_foreign_key_column_name,
    get_foreign_key_reference_column,
    get_foreign_key_reference_table,
    get_relationship_member_record,
    get_relationship_owner_record,
    get_relationship_set_name,
)
from idms_modernizer.rules.excel_fk_datatype_rules import (
    field_matches_db2_column,
    find_metadata_record,
    find_owner_record_name_for_set,
    get_db2_datatype,
    infer_fk_datatype_from_owner_field,
    resolve_foreign_key_reference_datatype,
    resolve_owner_reference_datatype,
    resolve_owner_reference_datatype_by_column_name,
    safe_int_for_fk,
)
from idms_modernizer.rules.excel_fk_key_rules import (
    db2_key_label,
    find_primary_key_columns,
    fk_emit_key,
    is_foreign_key_column,
    is_primary_key_column,
)
from idms_modernizer.rules.excel_fk_relationship_lookup_rules import (
    build_relationship_lookup,
)
from idms_modernizer.rules.excel_fk_row_builder import (
    build_set_fk_rows_for_record,
    build_set_fk_rows_from_db2_foreign_keys,
    build_set_fk_rows_from_metadata_relationships,
)

__all__ = [
    "get_foreign_key_column_name",
    "get_foreign_key_reference_column",
    "get_foreign_key_reference_table",
    "get_relationship_member_record",
    "get_relationship_owner_record",
    "get_relationship_set_name",
    "get_db2_datatype",
    "is_primary_key_column",
    "is_foreign_key_column",
    "db2_key_label",
    "build_relationship_lookup",
    "fk_emit_key",
    "build_set_fk_rows_for_record",
    "build_set_fk_rows_from_db2_foreign_keys",
    "build_set_fk_rows_from_metadata_relationships",
    "find_primary_key_columns",
    "resolve_foreign_key_reference_datatype",
    "find_owner_record_name_for_set",
    "resolve_owner_reference_datatype_by_column_name",
    "resolve_owner_reference_datatype",
    "infer_fk_datatype_from_owner_field",
    "find_metadata_record",
    "field_matches_db2_column",
    "safe_int_for_fk",
]

```

```
=====
FILE: src\idms_modernizer\rules\excel_key_rules.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.services.db2_datatype_mapper import DB2DatatypeMapper
from idms_modernizer.rules.excel_basic_rules import (
    get_field_length,
    get_field_picture,
    is_filler_field,
    is_group_field,
    remove_record_suffix,
)
from idms_modernizer.rules.excel_name_rules import (
    is_date_part_name,
)

def build_calc_scope(
    record,
    fields,
) -> dict[str, str]:
    scope: dict[str, str] = {}

    primary_key = getattr(record, "primary_key", None)

    if not primary_key:
        return scope

    normalized_primary_key = NameNormalizer.normalize(
        primary_key,
    )

    suffix_removed_primary_key = remove_record_suffix(
        normalized_primary_key,
    )

    mapping_fields = list(fields or [])

    key_field = None
    key_index = None

    for index, field in enumerate(mapping_fields):
        field_name = NameNormalizer.normalize(
            getattr(field, "name", "") or "",
        )

        if not field_name:
            continue

        if field_name == normalized_primary_key:
            key_field = field
            key_index = index
            break

        if remove_record_suffix(field_name) == suffix_removed_primary_key:
            key_field = field
            key_index = index
            break

    if key_field is None or key_index is None:
        if normalized_primary_key:
            scope[normalized_primary_key] = "DESCENDANT"

    return scope

    key_name = NameNormalizer.normalize(
        getattr(key_field, "name", "") or "",
    )

    if not key_name:
        return scope

    key_is_group = is_group_field(
        field=key_field,
```

```

)

key_level = getattr(key_field, "level", None)

if not key_is_group:
    scope[key_name] = "DESCENDANT"
    return scope

scope[key_name] = "ROOT"

if key_level is None:
    return scope

try:
    key_level_int = int(key_level)
except Exception:
    return scope

for child_field in mapping_fields[key_index + 1:]:
    child_name = NameNormalizer.normalize(
        getattr(child_field, "name", "") or "",
    )

    child_level = getattr(child_field, "level", None)

    if child_level is None:
        continue

    try:
        child_level_int = int(child_level)
    except Exception:
        continue

    if child_level_int <= key_level_int:
        break

    if child_name:
        scope[child_name] = "DESCENDANT"

return scope

def calc_status_for_field(
    field,
    calc_scope: dict[str, str],
) -> str:
    field_name = NameNormalizer.normalize(
        getattr(field, "name", "") or "",
    )

    if not field_name:
        return ""

    if field_name in calc_scope:
        return calc_scope[field_name]

    suffix_removed = remove_record_suffix(
        field_name,
    )

    for key, value in calc_scope.items():
        if remove_record_suffix(key) == suffix_removed:
            return value

    return ""

def idms_key_label(
    calc_status: str,
    is_date: bool,
    is_group: bool,
    db2_key: str,
) -> str:
    if is_date:
        return ""

```

```

if is_group:
    return ""

labels: list[str] = []

if calc_status == "DESCENDANT":
    labels.append(
        "CALC",
    )

db2_key_parts = [
    part.strip().upper()
    for part in str(db2_key or "").split("/")
    if part.strip()
]

if "FK" in db2_key_parts:
    labels.append(
        "SET",
    )

output: list[str] = []

for label in labels:
    if label not in output:
        output.append(
            label,
        )

return "; ".join(output)

def add_pk_to_db2_key(
    db2_key: str,
) -> str:
    parts = [
        part.strip().upper()
        for part in str(db2_key or "").split("/")
        if part.strip()
    ]

    if "PK" not in parts:
        parts.insert(
            0,
            "PK",
        )

    ordered_parts: list[str] = []

    if "PK" in parts:
        ordered_parts.append(
            "PK",
        )

    if "FK" in parts:
        ordered_parts.append(
            "FK",
        )

    return "/".join(ordered_parts)

def should_emit_db2_field(
    field,
) -> bool:
    if is_filler_field(
        field=field,
    ):
        return False

    if is_group_field(
        field=field,
    ):
        return False

    if is_date_part_name(

```

```

        field_name=getattr(field, "name", "") or "",
    ):
        return False

    picture = get_field_picture(
        field=field,
    )

    if picture:
        return True

    datatype = str(
        getattr(field, "datatype", "") or "",
    ).upper()

    return datatype in {
        "CHAR",
        "VARCHAR",
        "DECIMAL",
        "NUMERIC",
        "DATE",
    }

def infer_db2_datatype_from_field(
    field,
) -> str:
    pic_clause = get_field_picture(
        field=field,
    )

    if pic_clause:
        return DB2DatatypeMapper.map_picture(
            picture=pic_clause,
            fallback_length=None,
            fallback_scale=getattr(field, "scale", None),
        )

    datatype = str(
        getattr(field, "datatype", "") or "",
    ).upper()

    if datatype == "DATE":
        return "DATE"

    if datatype in {
        "TIMESTAMP",
        "DATETIME",
    }:
        return "TIMESTAMP"

    length = get_field_length(
        field=field,
    ) or 1

    if datatype in {
        "CHAR",
        "VARCHAR",
        "DISPLAY",
        "TEXT",
    }:
        return f"CHAR({int(length)})"

    if datatype in {
        "DECIMAL",
        "NUMERIC",
    }:
        scale = getattr(field, "scale", None)

        if scale is not None and int(scale) > 0:
            return f"DECIMAL({int(length)},{int(scale)})"

        return f"DECIMAL({int(length)})"

    return ""

```

```
=====
FILE: src\idms_modernizer\rules\excel_lookup_rules.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.domain.db2_models import (
    DB2Column,
    DB2Model,
    DB2Table,
)
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.rules.excel_basic_rules import (
    name_lookup_keys,
    remove_record_suffix,
    to_db2_name,
)
from idms_modernizer.rules.excel_name_rules import (
    cobol_field_base_name,
    convert_cobol_zone_to_db2_field_name,
    convert_date_field_to_db2_field_name,
    date_field_base_name,
)
```

```
def build_table_lookup(
    db2_model: DB2Model,
) -> dict[str, DB2Table]:
    lookup: dict[str, DB2Table] = {}

    if db2_model is None:
        return lookup

    for table in getattr(db2_model, "tables", []) or []:
        table_name = getattr(table, "name", "") or ""

        for key in name_lookup_keys(
            table_name,
        ):
            lookup[key] = table

    return lookup
```

```
def build_column_lookup(
    table: DB2Table | None,
) -> dict[str, DB2Column]:
    lookup: dict[str, DB2Column] = {}

    if table is None:
        return lookup

    for column in getattr(table, "columns", []) or []:
        column_name = getattr(column, "name", "") or ""

        for key in name_lookup_keys(
            column_name,
        ):
            lookup[key] = column

    return lookup
```

```
def find_table_for_record(
    record_name: str,
    table_lookup: dict[str, DB2Table],
) -> DB2Table | None:
    if not table_lookup:
        return None

    for key in name_lookup_keys(
        record_name,
    ):
        if key in table_lookup:
            return table_lookup[key]
```

```

normalized_record_name = to_db2_name(
    record_name,
)

compact_record_name = normalized_record_name.replace(
    "_",
    "",
)

if compact_record_name:
    generated_table_name = f"DZ{compact_record_name[-4:]}TB"

    for key in name_lookup_keys(
        generated_table_name,
    ):
        if key in table_lookup:
            return table_lookup[key]

suffix_removed_record_name = remove_record_suffix(
    normalized_record_name,
)

for table_key, table in table_lookup.items():
    if remove_record_suffix(table_key) == suffix_removed_record_name:
        return table

return None

def find_column_for_field(
    record,
    field,
    column_lookup: dict[str, DB2Column],
) -> DB2Column | None:
    field_name = getattr(field, "name", "") or ""

    if not field_name:
        return None

    possible_names = [
        field_name,
        remove_record_suffix(
            field_name,
        ),
        cobol_field_base_name(
            field_name=field_name,
        ),
        date_field_base_name(
            field_name=field_name,
        ),
        convert_cobol_zone_to_db2_field_name(
            record=record,
            field=field,
        ),
        convert_date_field_to_db2_field_name(
            record=record,
            field=field,
        ),
    ]

    for name in possible_names:
        column = find_matching_column(
            normalized_column_name=name,
            column_lookup=column_lookup,
        )

        if column is not None:
            return column

    return None

def find_matching_column(
    normalized_column_name: str,
    column_lookup: dict[str, DB2Column],
) -> DB2Column | None:

```



```

    if not normalized_column_name:
        return None

    if not column_lookup:
        return None

    for key in name_lookup_keys(
        normalized_column_name,
    ):
        if key in column_lookup:
            return column_lookup[key]

    suffix_removed = remove_record_suffix(
        normalized_column_name,
    )

    for key in name_lookup_keys(
        suffix_removed,
    ):
        if key in column_lookup:
            return column_lookup[key]

    target_suffix_removed = remove_record_suffix(
        to_db2_name(
            normalized_column_name,
        )
    )

    for current_name, column in column_lookup.items():
        if remove_record_suffix(current_name) == target_suffix_removed:
            return column

    return None

def find_primary_key_column(
    table: DB2Table | None,
) -> DB2Column | None:
    if table is None:
        return None

    primary_key_names = list(
        getattr(table, "primary_keys", []) or []
    )

    if not primary_key_names and getattr(table, "primary_key", None):
        primary_key_names = [
            table.primary_key,
        ]

    normalized_primary_keys = {
        NameNormalizer.normalize(primary_key)
        for primary_key in primary_key_names
        if primary_key
    }

    for column in getattr(table, "columns", []) or []:
        column_name = NameNormalizer.normalize(
            getattr(column, "name", "") or "",
        )

        if column_name in normalized_primary_keys:
            return column

        if getattr(column, "primary_key", False):
            return column

    columns = list(
        getattr(table, "columns", []) or []
    )

    if columns:
        return columns[0]

    return None

```

```
=====
FILE: src\idms_modernizer\rules\excel_mapping_builder.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.domain.db2_models import (
    DB2Model,
    DB2Table,
)
from idms_modernizer.domain.schema_models import SchemaMetadata

from idms_modernizer.rules.excel_basic_rules import (
    build_record_level_01_row,
    has_level_01_row,
)
from idms_modernizer.rules.excel_fk_rules import (
    build_relationship_lookup,
    build_set_fk_rows_for_record,
)
from idms_modernizer.rules.excel_lookup_rules import (
    build_table_lookup,
    find_table_for_record,
)
from idms_modernizer.rules.excel_row_rules import (
    build_record_mapping_rows,
    force_pic_based_datatypes,
)
from idms_modernizer.rules.restart_record_rules import (
    is_restart_control_record,
)
from idms_modernizer.rules.excel_restart_row_rules import (
    build_restart_mapping_rows,
)

def build(
    metadata: SchemaMetadata,
    db2_model: DB2Model,
) -> list[dict[str, str]]:
    print(
        "USING ExcelSheetMappingService.build VERSION MODULAR-EXCEL-BUILDER-RESTART-2026-09-01"
    )

    rows: list[dict[str, str]] = []

    table_lookup = build_table_lookup(
        db2_model=db2_model,
    )

    relationship_lookup = build_relationship_lookup(
        metadata=metadata,
    )

    restart_rows_added = False

    for record in getattr(metadata, "records", []) or []:
        record_name = getattr(record, "name", "") or ""

        if is_restart_control_record(
            record_name=record_name,
        ):
            if not restart_rows_added:
                rows.extend(
                    build_restart_mapping_rows(),
                )
                restart_rows_added = True

            continue

        mapping_fields = (
            getattr(record, "mapping_fields", None)
            or getattr(record, "fields", None)
            or []
        )
```

```

if not has_level_01_row(
    record=record,
    mapping_fields=mapping_fields,
):
    rows.append(
        build_record_level_01_row(
            record=record,
        )
    )

table = find_table_for_record(
    record_name=record_name,
    table_lookup=table_lookup,
)

rows.extend(
    build_record_mapping_rows(
        record=record,
        table=table,
        relationship_lookup=relationship_lookup,
    )
)

rows.extend(
    build_generated_pk_rows_proxy(
        record=record,
        table=table,
    )
)

rows.extend(
    build_sequence_number_rows_proxy(
        metadata=metadata,
        record=record,
        table=table,
    )
)

rows.extend(
    build_audit_rows_proxy(
        record=record,
        table=table,
    )
)

rows.extend(
    build_set_fk_rows_for_record(
        metadata=metadata,
        record=record,
        table=table,
        table_lookup=table_lookup,
    )
)

rows = force_pic_based_datatypes(
    rows=rows,
)

return rows

```

```

def build_generated_pk_rows_proxy(
    record,
    table: DB2Table | None,
) -> list[dict[str, str]]:
    from idms_modernizer.rules.excel_extra_row_rules import (
        build_generated_pk_rows,
    )

    return build_generated_pk_rows(
        record=record,
        table=table,
    )

```

```

def build_sequence_number_rows_proxy(

```

```

        metadata,
        record,
        table: DB2Table | None,
) -> list[dict[str, str]]:
    from idms_modernizer.rules.excel_extra_row_rules import (
        build_sequence_number_rows,
    )

    return build_sequence_number_rows(
        metadata=metadata,
        record=record,
        table=table,
    )

def build_audit_rows_proxy(
    record,
    table: DB2Table | None,
) -> list[dict[str, str]]:
    from idms_modernizer.rules.excel_extra_row_rules import (
        build_audit_rows,
    )

    return build_audit_rows(
        record=record,
        table=table,
    )

```

```

=====
FILE: src\idms_modernizer\rules\excel_mapping_constants.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules.db2_mapping_constants import (
    DB2_COLUMN_CODE_PREFIX,
    DB2_TABLE_CORE_LEN,
    DB2_TABLE_PREFIX,
    DB2_TABLE_SUFFIX,
    DATE_TOKENS,
    DAY_PARTS,
    GENERIC_TRAILING_TOKENS,
    MONTH_PARTS,
    PROJECT_ABBREVIATIONS,
    YEAR_PARTS,
)

```

```

COLUMNS = [
    "Cobol Record IDMS",
    "Cobol Zone",
    "IDMS Key",
    "IDMS PIC Clause",
    "Length of Field Bytes",
    "Field end position",
    "DB2 Key",
    "New DB2 Record",
    "New DB2 Field name",
    "New DB2 Data Type",
    "Hopex Expression TypeRemark",
    "Relation",
    "Reference Field Name (CopyBook) ",
    "Reference Field PIC Clause",
    "Cross Application DB2 Field Name",
    "Cross Appln DB2 Data Type",
    "Basetype",
]

```

```
DATE_CHILD_TOKENS = YEAR_PARTS | MONTH_PARTS | DAY_PARTS
```

```

DATE_TRAILING_TOKENS = {
    "SIC",
    "FC",
    "GDIFC",
    "GDIFR",
    "GDIF",
}

```

```

    "GDI",
}

```

```

=====
FILE: src\idms_modernizer\rules\excel_mapping_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules.date_rules import (
    is_date_field,
    is_date_part_name,
)
from idms_modernizer.rules.field_rules import is_group_field
from idms_modernizer.rules.naming_rules import normalize_name
from idms_modernizer.utils.safe_cast import safe_int

```

```

def build_date_scope(fields) -> dict[int, bool]:
    scope: dict[int, bool] = {}
    active_date_levels: list[int] = []

    for field in fields or []:
        level = safe_int(getattr(field, "level", None), default=0)

        active_date_levels = [
            current_level
            for current_level in active_date_levels
            if current_level < level
        ]

        if active_date_levels:
            scope[id(field)] = True

        if is_potential_outer_date(field):
            active_date_levels.append(level)

    return scope

```

```

def collect_date_group_info(fields) -> dict[str, dict]:
    info: dict[str, dict] = {}

    for index, field in enumerate(fields or []):
        if not is_group_field(field):
            continue

        if not is_potential_outer_date(field):
            continue

        if not group_has_ymd_children(
            field=field,
            fields=fields,
            start_index=index,
        ):
            continue

        field_name = normalize_name(getattr(field, "name", "") or "")

        if field_name:
            info[field_name] = {
                "field": field,
            }

    return info

```

```

def group_has_ymd_children(field, fields, start_index: int) -> bool:
    parent_level = safe_int(getattr(field, "level", None), default=0)

    found_year = False
    found_month = False
    found_day = False

    for child in fields[start_index + 1:] or []:
        child_level = safe_int(getattr(child, "level", None), default=0)

```

```

        if child_level <= parent_level:
            break

        child_name = normalize_name(getattr(child, "name", "") or "")

        if not child_name:
            continue

        parsed = is_date_part_name(child_name)

        if not parsed:
            continue

        if child_name.endswith("_YEAR") or child_name.endswith("_YR"):
            found_year = True

        if child_name.endswith("_MONTH") or child_name.endswith("_MON") or child_name.endswith("_MO"):
            found_month = True

        if child_name.endswith("_DAY") or child_name.endswith("_DD") or child_name.endswith("_DY"):
            found_day = True

    return found_year and found_month and found_day

def is_potential_outer_date(field) -> bool:
    if field is None:
        return False

    if is_date_field(field):
        return True

    field_name = normalize_name(getattr(field, "name", "") or "")

    return (
        field_name.endswith("_DATE")
        or field_name == "DATE"
        or "_DATE_" in field_name
    )

def should_include_mapping_field(field) -> bool:
    if field is None:
        return False

    if not getattr(field, "name", None):
        return False

    return True

=====
FILE: src\idms_modernizer\rules\excel_name_rules.py
ENCODING: utf-8
=====

import re

from idms_modernizer.rules.excel_basic_rules import (
    name_parts,
    remove_record_suffix,
    to_db2_name,
)
from idms_modernizer.rules.excel_mapping_constants import (
    DATE_TOKENS,
    DAY_PARTS,
    DB2_COLUMN_CODE_PREFIX,
    DB2_TABLE_CORE_LEN,
    DB2_TABLE_PREFIX,
    DB2_TABLE_SUFFIX,
    GENERIC_TRAILING_TOKENS,
    MONTH_PARTS,
    PROJECT_ABBREVIATIONS,
    YEAR_PARTS,
)

```

```

def record_code(
    record_name: str | None,
) -> str:
    normalized = to_db2_name(
        value=record_name or "",
    )

    compact = re.sub(
        r"^[^A-Z0-9]",
        "",
        normalized,
    )

    if not compact:
        return ""

    if len(compact) <= 4:
        return compact

    return compact[-4:]

def normalize_table_name(
    value: str | None,
) -> str:
    clean = to_db2_name(
        value=value or "",
    )

    clean = clean.replace(
        "_",
        "",
    )

    if not clean:
        return ""

    core = (
        clean[-DB2_TABLE_CORE_LEN:]
        if len(clean) >= DB2_TABLE_CORE_LEN
        else clean
    )

    return f"{DB2_TABLE_PREFIX}{core}{DB2_TABLE_SUFFIX}"

def remove_generic_trailing_tokens(
    parts: list[str],
) -> list[str]:
    output = list(parts or [])

    while output and output[-1] in GENERIC_TRAILING_TOKENS:
        output.pop()

    return output

def apply_project_abbreviations(
    parts: list[str],
) -> list[str]:
    return [
        PROJECT_ABBREVIATIONS.get(part, part)
        for part in parts or []
    ]

def apply_date_project_abbreviations(
    parts: list[str],
) -> list[str]:
    output: list[str] = []

    for part in parts or []:
        if part == "EVPRCP":
            output.append(

```

```

        "ERCP",
    )
    continue

    if part == "GDIFAR":
        output.append(
            "GD",
        )
        continue

    if part in {
        "GDIFC",
        "GDIFR",
    }:
        continue

    output.append(
        PROJECT_ABBREVIATIONS.get(part, part)
    )

return output

def enforce_middle_name_max_length(
    db2_field_name: str,
    max_length: int = 7,
) -> str:
    text = str(db2_field_name or "").strip()

    marker = f"_{DB2_COLUMN_CODE_PREFIX}"

    if marker not in text:
        return text

    base, suffix = text.split(
        marker,
        1,
    )

    base = base.rstrip(
        "_",
    )

    if "_" not in base:
        trimmed_base = base[:max_length]

        return f"{trimmed_base}{marker}{suffix}"

    prefix, middle = base.split(
        "_",
        1,
    )

    if len(middle) <= max_length:
        return text

    trimmed_middle = middle[:max_length]

    return f"{prefix}_{trimmed_middle}{marker}{suffix}"

def cobol_field_base_name(
    field_name: str,
) -> str:
    normalized = to_db2_name(
        value=field_name,
    )

    normalized = remove_record_suffix(
        name=normalized,
    )

    parts = [
        part
        for part in normalized.split("_")
        if part
    ]

```



```

]

if not parts:
    return normalized

parts = remove_generic_trailing_tokens(
    parts=parts,
)

parts = apply_project_abbreviations(
    parts=parts,
)

if len(parts) == 1:
    return parts[0]

first = parts[0]
rest = "".join(
    parts[1:],
)

if rest:
    return f"{first}_{rest}"

return first

def date_field_base_name(
    field_name: str,
) -> str:
    normalized = to_db2_name(
        value=field_name,
    )

    normalized = remove_record_suffix(
        name=normalized,
    )

    parts = [
        part
        for part in normalized.split("_")
        if part
    ]

    if not parts:
        return normalized

    if parts[0] == "DA":
        main_parts = parts[1:]
    else:
        main_parts = parts

    if not main_parts:
        return "DA"

    if main_parts[0] in {
        "UB",
        "UE",
    }:
        return f"DA_{main_parts[0]}DATE"

    main_parts = apply_date_project_abbreviations(
        parts=main_parts,
    )

    main_parts = remove_generic_trailing_tokens(
        parts=main_parts,
    )

    if not main_parts:
        return "DA"

    if len(main_parts) == 1:
        main = main_parts[0]
    else:
        main = "".join(

```

```

        main_parts,
    )

    return f"DA_{main}"

def convert_cobol_zone_to_db2_field_name(
    record,
    field,
) -> str:
    field_name = getattr(field, "name", "") or ""

    base = cobol_field_base_name(
        field_name=field_name,
    )

    code = record_code(
        record_name=getattr(record, "name", "") or "",
    )

    if code:
        return enforce_middle_name_max_length(
            db2_field_name=f"{base}_{DB2_COLUMN_CODE_PREFIX}{code}",
        )

    return base

def convert_date_field_to_db2_field_name(
    record,
    field,
) -> str:
    field_name = getattr(field, "name", "") or ""

    base = date_field_base_name(
        field_name=field_name,
    )

    code = record_code(
        record_name=getattr(record, "name", "") or "",
    )

    if code:
        return enforce_middle_name_max_length(
            db2_field_name=f"{base}_{DB2_COLUMN_CODE_PREFIX}{code}",
        )

    return base

def is_date_like_name(
    value: str,
) -> bool:
    parts = name_parts(
        value=value or "",
    )

    if any(part in DATE_TOKENS for part in parts):
        return True

    if parts and parts[0] == "DA":
        return True

    compact = "".join(
        parts,
    )

    return any(token in compact for token in DATE_TOKENS)

def is_date_part_name(
    field_name: str,
) -> bool:
    parts = name_parts(
        value=field_name or "",
    )

```

```

for part in parts:
    if part in YEAR_PARTS:
        return True

    if part in MONTH_PARTS:
        return True

    if part in DAY_PARTS:
        return True

return False

```

```

=====
FILE: src\idms_modernizer\rules\excel_occurs_rules.py
ENCODING: utf-8
=====

```

```

from copy import deepcopy

```

```

from idms_modernizer.rules.excel_basic_rules import (
    is_filler_field,
    is_group_field,
    is_redefines_field,
    safe_int,
)

```

```

def build_occurs_scope(
    fields,
) -> dict[int, dict]:
    """
    Builds OCCURS scope for all descendants of an OCCURS group.

```

Example:

```

    03 KY-VMBFGDC OCCURS 2 TIMES
    04 NR-CIO-GDIFC
    04 NR-ID-GDIFC

```

The child fields are marked as being inside OCCURS.

```

    """
    scope: dict[int, dict] = {}
    active_occurs_groups: list[dict] = []

    for field in fields or []:
        level = safe_int(
            getattr(field, "level", None),
            default=0,
        )

        active_occurs_groups = [
            item
            for item in active_occurs_groups
            if item["level"] < level
        ]

        if active_occurs_groups:
            current_group = active_occurs_groups[-1]

            scope[id(field)] = {
                "occurs": True,
                "occurs_min": current_group.get("occurs_min"),
                "occurs_max": current_group.get("occurs_max"),
                "parent": current_group.get("field"),
                "parent_level": current_group.get("level"),
            }

        if getattr(field, "occurs", False):
            occurs_max = safe_int(
                getattr(field, "occurs_max", None),
                default=0,
            )

```

```

        occurs_min = safe_int(
            getattr(field, "occurs_min", None),
            default=occurs_max,
        )

        if occurs_max > 0:
            active_occurs_groups.append(
                {
                    "level": level,
                    "occurs": True,
                    "occurs_min": occurs_min,
                    "occurs_max": occurs_max,
                    "field": field,
                }
            )

    return scope

def occurs_info_for_field(
    field,
    occurs_scope: dict[int, dict],
) -> dict:
    if id(field) not in occurs_scope:
        return {
            "occurs": False,
            "occurs_min": None,
            "occurs_max": None,
        }

    return occurs_scope[id(field)]

def should_expand_occurs_row(
    row: dict[str, str],
    field,
    occurs_info: dict,
    is_group: bool,
    is_date_child: bool,
    is_redefines: bool = False,
) -> bool:
    """
    Decides whether an Excel mapping row should be expanded for OCCURS.

    Rules:
    - Do not expand the OCCURS group row itself.
    - Do not expand group rows.
    - Do not expand filler rows.
    - Do not expand redefines rows.
    - Do not expand date child rows.
    - Expand only rows that have a DB2 field name.
    """

    if not occurs_info.get("occurs", False):
        return False

    if is_group:
        return False

    if is_date_child:
        return False

    if is_filler_field(field):
        return False

    if is_redefines:
        return False

    if is_redefines_field(field):
        return False

    db2_field_name = str(
        row.get("New DB2 Field name", "") or ""
    ).strip()

    if not db2_field_name:

```

```

        return False

    occurs_max = safe_int(
        occurs_info.get("occurs_max"),
        default=0,
    )

    return occurs_max > 1

def expand_occurs_row(
    row: dict[str, str],
    occurs_info: dict,
) -> list[dict[str, str]]:
    """
    Expands one mapped row into occurrence-numbered DB2 rows.

    Naming rule:
    - Keep total business-code length before _479 within 7 characters.
    - Occurrence number is part of that 7-character segment.
    - Therefore, if base code is already 7 chars, remove last char before adding index.

    Example:
    - CT_RKTGDSV_479JFGD + 1 -> CT_RKTGDS1_479JFGD
    - CT_RKTGDSV_479JFGD + 2 -> CT_RKTGDS2_479JFGD

    For shorter codes:
    - NR_CIO_479JGDC + 1 -> NR_CIO1_479JGDC
    """

    occurs_max = safe_int(
        occurs_info.get("occurs_max"),
        default=0,
    )

    if occurs_max <= 1:
        return [
            row,
        ]

    expanded_rows: list[dict[str, str]] = []

    for occurrence_index in range(1, occurs_max + 1):
        new_row = deepcopy(row)

        new_row["New DB2 Field name"] = add_occurrence_number_to_db2_field(
            field_name=str(
                new_row.get("New DB2 Field name", "") or ""
            ),
            occurrence_index=occurrence_index,
        )

        existing_remark = str(
            new_row.get("Hopex Expression TypeRemark", "") or ""
        ).strip()

        occurs_remark = f"OCCURS INSTANCE {occurrence_index}"

        if existing_remark:
            new_row["Hopex Expression TypeRemark"] = (
                f"{existing_remark}; {occurs_remark}"
            )
        else:
            new_row["Hopex Expression TypeRemark"] = occurs_remark

        expanded_rows.append(new_row)

    return expanded_rows

def add_occurrence_number_to_db2_field(
    field_name: str,
    occurrence_index: int,
) -> str:
    text = str(field_name or "").strip()

```

```

if not text:
    return text

marker = "_479"

if marker not in text:
    return f"{text}{occurrence_index}"

base, suffix = text.split(marker, 1)

base = base.rstrip("_")

if "_" not in base:
    adjusted_base = trim_last_segment_for_occurs(
        base=base,
        occurrence_index=occurrence_index,
    )

    return f"{adjusted_base}_{marker.strip('_')}{suffix}"

prefix, business_code = base.split("_", 1)

adjusted_business_code = trim_last_segment_for_occurs(
    base=business_code,
    occurrence_index=occurrence_index,
)

return f"{prefix}_{adjusted_business_code}_{marker.strip('_')}{suffix}"

def trim_last_segment_for_occurs(
    base: str,
    occurrence_index: int,
) -> str:
    """
    Ensures occurrence suffix keeps segment length <= 7.

    Example:
    - RKTGDSV + 1 -> RKTGDS1
    - CIO + 1 -> CIO1
    """

    text = str(base or "").strip()

    occurrence_text = str(occurrence_index)

    max_length = 7

    allowed_base_length = max_length - len(occurrence_text)

    if allowed_base_length < 1:
        allowed_base_length = 1

    trimmed = text[:allowed_base_length]

    return f"{trimmed}{occurrence_text}"

=====
FILE: src\idms_modernizer\rules\excel_record_mapping_row_builder.py
ENCODING: utf-8
=====

from idms_modernizer.domain.db2_models import DB2Table
from idms_modernizer.rules.excel_basic_rules import (
    is_group_field,
    is_redefines_field,
)
from idms_modernizer.rules.excel_date_rules import (
    is_date_child,
)
from idms_modernizer.rules.excel_occurs_rules import (
    expand_occurs_row,
    occurs_info_for_field,
    should_expand_occurs_row,
)

```

```

from idms_modernizer.rules.excel_row_context_rules import (
    build_excel_row_mapping_context,
)
from idms_modernizer.rules.excel_single_mapping_row_builder import (
    build_single_mapping_row,
)

def build_record_mapping_rows(
    record,
    table: DB2Table | None,
    relationship_lookup: dict[str, str],
) -> list[dict[str, str]]:
    rows: list[dict[str, str]] = []

    context = build_excel_row_mapping_context(
        record=record,
        table=table,
    )

    for field in context.mapping_fields:
        row = build_single_mapping_row(
            record=record,
            field=field,
            table=table,
            column_lookup=context.column_lookup,
            date_scope=context.date_scope,
            date_group_info=context.date_group_info,
            calc_scope=context.calc_scope,
            redefines_scope=context.redefines_scope,
            relationship_lookup=relationship_lookup,
        )

        is_group = is_group_field(
            field=field,
        )

        is_redefines = (
            is_redefines_field(
                field=field,
            )
            or id(field) in context.redefines_scope
        )

        is_date_child_value = is_date_child(
            field=field,
            date_scope=context.date_scope,
        )

        current_occurs_info = occurs_info_for_field(
            field=field,
            occurs_scope=context.occurs_scope,
        )

        if should_expand_occurs_row(
            row=row,
            field=field,
            occurs_info=current_occurs_info,
            is_group=is_group,
            is_date_child=is_date_child_value,
            is_redefines=is_redefines,
        ):
            rows.extend(
                expand_occurs_row(
                    row=row,
                    occurs_info=current_occurs_info,
                )
            )
        else:
            rows.append(
                row,
            )

    return rows

```

```
=====
FILE: src\idms_modernizer\rules\excel_redefines_scope_rules.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.rules.excel_basic_rules import (
    is_redefines_field,
)
```

```
def build_redefines_scope(
    fields,
) -> dict[int, bool]:
    scope: dict[int, bool] = {}
    active_redefines_levels: list[int] = []

    for field in fields or []:
        level = getattr(field, "level", None)

        try:
            current_level = int(level)
        except Exception:
            continue

        active_redefines_levels = [
            active_level
            for active_level in active_redefines_levels
            if active_level < current_level
        ]

        if active_redefines_levels:
            scope[id(field)] = True

        if is_redefines_field(
            field=field,
        ):
            active_redefines_levels.append(
                current_level,
            )

            scope[id(field)] = True

    return scope
```

```
=====
FILE: src\idms_modernizer\rules\excel_restart_row_rules.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.rules.excel_basic_rules import empty_row
from idms_modernizer.rules.db2_mapping_constants import (
    RESTART_CONTROL_COBOL_ZONE,
    RESTART_DB2_VIEW_NAME,
    RESTART_DCLGEN_COLUMNS,
    RESTART_MAPPING_REMARK,
    RESTART_TECHNICAL_SOURCE_NAME,
)
```

```
def build_restart_mapping_rows() -> list[dict[str, str]]:
    rows: list[dict[str, str]] = []

    for column in RESTART_DCLGEN_COLUMNS:
        row = empty_row()

        row["Cobol Record IDMS"] = RESTART_TECHNICAL_SOURCE_NAME
        row["Cobol Zone"] = RESTART_CONTROL_COBOL_ZONE
        row["Length of Field Bytes"] = column.get("length", "")
        row["DB2 Key"] = column.get("db2_key", "")
        row["New DB2 Record"] = RESTART_DB2_VIEW_NAME
        row["New DB2 Field name"] = column.get("column_name", "")
        row["New DB2 Data Type"] = column.get("data_type", "")
        row["Hopex Expression TypeRemark"] = RESTART_MAPPING_REMARK
        row["Relation"] = column.get("relation", "")
        row["Basetype"] = column.get("basetype", "")
```



```

        rows.append(
            row,
        )

    return rows

```

```

=====
FILE: src\idms_modernizer\rules\excel_row_context_rules.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass

from idms_modernizer.domain.db2_models import DB2Table
from idms_modernizer.rules.excel_date_rules import (
    build_date_scope,
    collect_date_group_info,
)
from idms_modernizer.rules.excel_key_rules import (
    build_calc_scope,
)
from idms_modernizer.rules.excel_lookup_rules import (
    build_column_lookup,
)
from idms_modernizer.rules.excel_occurs_rules import (
    build_occurs_scope,
)
from idms_modernizer.rules.excel_redefines_scope_rules import (
    build_redefines_scope,
)

```

```

@dataclass
class ExcelRowMappingContext:
    mapping_fields: list
    column_lookup: dict
    date_scope: dict
    date_group_info: dict
    calc_scope: dict
    redefines_scope: dict
    occurs_scope: dict

```

```

def get_mapping_fields(
    record,
) -> list:
    return (
        getattr(record, "mapping_fields", None)
        or getattr(record, "fields", None)
        or []
    )

```

```

def build_excel_row_mapping_context(
    record,
    table: DB2Table | None,
) -> ExcelRowMappingContext:
    mapping_fields = get_mapping_fields(
        record=record,
    )

```

```

    column_lookup = (
        build_column_lookup(
            table=table,
        )
        if table is not None
        else {}
    )

```

```

    date_scope = build_date_scope(
        fields=mapping_fields,
    )

```

```

    date_group_info = collect_date_group_info(

```

```

        fields=mapping_fields,
    )

    calc_scope = build_calc_scope(
        record=record,
        fields=mapping_fields,
    )

    redefines_scope = build_redefines_scope(
        fields=mapping_fields,
    )

    occurs_scope = build_occurs_scope(
        fields=mapping_fields,
    )

    return ExcelRowMappingContext(
        mapping_fields=mapping_fields,
        column_lookup=column_lookup,
        date_scope=date_scope,
        date_group_info=date_group_info,
        calc_scope=calc_scope,
        redefines_scope=redefines_scope,
        occurs_scope=occurs_scope,
    )

```

```

=====
FILE: src\idms_modernizer\rules\excel_row_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules.excel_record_mapping_row_builder import (
    build_record_mapping_rows,
)
from idms_modernizer.rules.excel_redefines_scope_rules import (
    build_redefines_scope,
)
from idms_modernizer.rules.excel_single_mapping_row_builder import (
    build_single_mapping_row,
)
from idms_modernizer.rules.excel_db2_mapping_rules import (
    apply_db2_mapping,
)
from idms_modernizer.rules.excel_db2_record_name_rules import (
    db2_record_name,
)
from idms_modernizer.rules.excel_filler_row_builder import (
    build_filler_row,
)
from idms_modernizer.rules.excel_datatype_post_rules import (
    force_pic_based_datatypes,
)

```

```

__all__ = [
    "build_record_mapping_rows",
    "build_redefines_scope",
    "build_single_mapping_row",
    "apply_db2_mapping",
    "db2_record_name",
    "build_filler_row",
    "force_pic_based_datatypes",
]

```

```

=====
FILE: src\idms_modernizer\rules\excel_row_template_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules.excel_mapping_constants import COLUMNS

```

```

def empty_row() -> dict[str, str]:
    return {
        column: ""
    }

```

```

        for column in COLUMNS
    }

=====
FILE: src\idms_modernizer\rules\excel_single_mapping_row_builder.py
ENCODING: utf-8
=====

from idms_modernizer.domain.db2_models import DB2Table
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.rules.excel_basic_rules import (
    empty_row,
    format_cobol_zone,
    get_field_basetype,
    get_field_end_position,
    get_field_length,
    get_field_picture,
    hopex_remark,
    is_filler_field,
    is_group_field,
    is_redefines_field,
    to_string,
)
from idms_modernizer.rules.excel_date_rules import (
    is_date_child,
    is_direct_date_field,
    is_outer_date_group,
)
from idms_modernizer.rules.excel_fk_rules import (
    db2_key_label,
)
from idms_modernizer.rules.excel_key_rules import (
    add_pk_to_db2_key,
    calc_status_for_field,
    idms_key_label,
)
from idms_modernizer.rules.excel_lookup_rules import (
    find_column_for_field,
)
from idms_modernizer.rules.excel_db2_mapping_rules import (
    apply_db2_mapping,
)
from idms_modernizer.rules.excel_filler_row_builder import (
    build_filler_row,
)

def build_single_mapping_row(
    record,
    field,
    table: DB2Table | None,
    column_lookup,
    date_scope,
    date_group_info,
    calc_scope,
    redefines_scope,
    relationship_lookup,
) -> dict[str, str]:
    if is_filler_field(
        field=field,
    ):
        return build_filler_row(
            record=record,
            field=field,
        )

    is_group = is_group_field(
        field=field,
    )

    is_redefines = (
        is_redefines_field(
            field=field,
        )
        or id(field) in redefines_scope

```

```

)

is_outer_date = is_outer_date_group(
    field=field,
    date_group_info=date_group_info,
)

is_date_child_value = is_date_child(
    field=field,
    date_scope=date_scope,
)

is_direct_date = is_direct_date_field(
    field=field,
)

is_date = (
    is_outer_date
    or is_date_child_value
    or is_direct_date
)

calc_status = calc_status_for_field(
    field=field,
    calc_scope=calc_scope,
)

db2_column = None

if not is_redefines:
    db2_column = find_column_for_field(
        record=record,
        field=field,
        column_lookup=column_lookup,
    )

db2_key = db2_key_label(
    table=table,
    column=db2_column,
)

is_mapped_date = (
    is_outer_date
    or is_direct_date
)

is_date_key = (
    calc_status == "DESCENDANT"
    and is_mapped_date
    and not is_date_child_value
    and not is_redefines
)

if calc_status == "ROOT":
    db2_key = ""

if is_group and not is_outer_date:
    db2_key = ""

if (
    calc_status == "DESCENDANT"
    and not is_date
    and not is_group
    and not is_redefines
):
    db2_key = add_pk_to_db2_key(
        db2_key=db2_key,
    )

if is_date_child_value:
    db2_key = ""

if is_date and not is_date_key:
    db2_key = ""

if is_date_key:

```

```

        db2_key = add_pk_to_db2_key(
            db2_key=db2_key,
        )

    if is_redefines:
        db2_key = ""

    row = empty_row()

    row["Cobol Record IDMS"] = getattr(record, "name", "") or ""
    row["Cobol Zone"] = format_cobol_zone(
        field=field,
    )

    row["IDMS Key"] = idms_key_label(
        calc_status=calc_status,
        is_date=is_date,
        is_group=is_group,
        db2_key=db2_key,
    )

    if is_date_key:
        row["IDMS Key"] = "CALC"

    if is_redefines:
        row["IDMS Key"] = ""

    row["IDMS PIC Clause"] = get_field_picture(
        field=field,
    )

    row["Length of Field Bytes"] = to_string(
        get_field_length(
            field=field,
        )
    )

    row["Field end position"] = to_string(
        get_field_end_position(
            field=field,
        )
    )

    row["DB2 Key"] = db2_key

    apply_db2_mapping(
        row=row,
        record=record,
        field=field,
        table=table,
        db2_column=db2_column,
        is_outer_date=is_outer_date,
        is_date_child_value=is_date_child_value,
        is_direct_date=is_direct_date,
        calc_status=calc_status,
        is_group=is_group,
        is_redefines=is_redefines,
    )

    row["Hopex Expression TypeRemark"] = hopex_remark(
        field=field,
    )

    row["Relation"] = relationship_lookup.get(
        NameNormalizer.normalize(
            getattr(record, "name", "") or "",
        ),
        "",
    )

    row["Basetype"] = get_field_basetype(
        field=field,
    )

    return row

```

```
=====
FILE: src\idms_modernizer\rules\excel_text_rules.py
ENCODING: utf-8
=====
```

```
import re

from idms_modernizer.services.name_normalizer import NameNormalizer

def to_string(
    value,
) -> str:
    if value is None:
        return ""

    return str(value)

def safe_int(
    value,
    default: int = 0,
) -> int:
    try:
        if value is None:
            return default

        text = str(value).strip()

        if not text:
            return default

        return int(text)
    except Exception:
        return default

def to_db2_name(
    value: str | None,
) -> str:
    normalized = str(value or "").upper().strip()
    normalized = normalized.replace("\u00a0", " ")
    normalized = normalized.replace("\t", " ")
    normalized = re.sub(
        r"^[A-Z0-9]+",
        "_",
        normalized,
    )
    normalized = re.sub(
        r"_{1,}",
        "_",
        normalized,
    )
    normalized = normalized.strip("_")

    return normalized

def name_parts(
    value: str | None,
) -> list[str]:
    normalized = to_db2_name(
        value=value or "",
    )

    return [
        part
        for part in normalized.split("_")
        if part
    ]

def remove_record_suffix(
    name: str | None = None,
    value: str | None = None,
) -> str:
```

```

actual_value = name

if actual_value is None:
    actual_value = value

text = str(actual_value or "").strip().upper()
text = re.sub(
    r"[_\-\s]{0-9}{4}$",
    "",
    text,
)
text = re.sub(
    r"[0-9]{4}$",
    "",
    text,
)
text = re.sub(
    r"[_\-\s]+$",
    "",
    text,
)

return text

def name_lookup_keys(
    value: str | None,
) -> list[str]:
    keys: list[str] = []

    candidates = [
        value,
        NameNormalizer.normalize(value or ""),
        to_db2_name(value or ""),
        remove_record_suffix(value or ""),
        remove_record_suffix(
            NameNormalizer.normalize(
                value or "",
            )
        ),
        remove_record_suffix(
            to_db2_name(
                value or "",
            )
        ),
    ]

    for candidate in candidates:
        candidate_text = str(candidate or "").strip()

        if not candidate_text:
            continue

        normalized_name = NameNormalizer.normalize(
            candidate_text,
        )

        db2_name = to_db2_name(
            candidate_text,
        )

        for key in [
            candidate_text,
            normalized_name,
            db2_name,
        ]:
            key = str(key or "").strip()

            if key and key not in keys:
                keys.append(
                    key,
                )

    return keys

```

```
=====
FILE: src\idms_modernizer\rules\field_rules.py
ENCODING: utf-8
=====
```

```
import re

FIELD_NAME_PATTERN = re.compile(
    r"^\s*(?P<level>0[1-9]|[1-4][0-9]|88)\s+"
    r"(?P<name>[A-Z][A-Z0-9-]*|FILLER)\b"
    r"(?P<rest>.*)$",
    re.IGNORECASE,
)

FIELD_LINE_PATTERN = re.compile(
    r"^\s*(0[1-9]|[1-4][0-9]|88)\s+"
    r"([A-Z][A-Z0-9-]*|FILLER)\b",
    re.IGNORECASE,
)

EXPLICIT_REDEFINES_PATTERN = re.compile(
    r"\b(?:REDEFINES|REDFINES)\s+(?P<target>[A-Z][A-Z0-9-]*)",
    re.IGNORECASE,
)

USAGE_WORDS = {
    "DISPLAY",
    "COMP",
    "COMP-3",
}

NON_REDEFINES_WORDS = {
    "",
    "PIC",
    "PICTURE",
    "VALUE",
    "VALUES",
    "OCCURS",
    "DEPENDING",
    "ON",
    "SET",
    "CONTROL",
    "ITEM",
    "FOR",
    "CALC",
    "ASC",
    "DUP",
    "FIRST",
    "LAST",
    "NOT",
    "ALLOWED",
}

def clean_field_line(
    line: str | None,
) -> str:
    value = str(line or "").strip()

    if not value:
        return ""

    value = value.replace(
        "\u00a0",
        " ",
    )

    value = value.replace(
        "\t",
        " ",
    )

    value = re.sub(
        r"\s+",
        " ",
    )
```



```

        value,
    )

    return value.strip()

def is_field_line(
    line: str | None,
) -> bool:
    text = clean_field_line(
        line,
    )

    if not text:
        return False

    return FIELD_LINE_PATTERN.search(
        text,
    ) is not None

def parse_field_line(
    line: str | None,
) -> dict | None:
    text = clean_field_line(
        line,
    )

    if not text:
        return None

    match = FIELD_NAME_PATTERN.search(
        text,
    )

    if not match:
        return None

    return {
        "level": safe_int(
            match.group("level"),
        ),
        "name": match.group("name").strip().upper(),
        "rest": match.group("rest").strip(),
        "raw_line": text,
    }

def is_filler_name(
    name: str | None,
) -> bool:
    return str(name or "").strip().upper().startswith(
        "FILLER",
    )

def is_redefines_text(
    text: str | None,
) -> bool:
    value = clean_field_line(
        text,
    ).upper()

    if not value:
        return False

    if EXPLICIT_REDEFINES_PATTERN.search(
        value,
    ):
        return True

    return infer_redefines_target_from_listing_line(
        value,
    ) != ""

```

```

def extract_redefines_target(
    text: str | None,
) -> str:
    value = clean_field_line(
        text,
    ).upper()

    explicit_match = EXPLICIT_REDEFINES_PATTERN.search(
        value,
    )

    if explicit_match:
        return explicit_match.group(
            "target",
        ).strip().upper()

    return infer_redefines_target_from_listing_line(
        value,
    )

def infer_redefines_target_from_listing_line(
    text: str | None,
) -> str:
    """
    Detects IDMS fixed-column REDEFINES format.

    In the IDMS listing, the data row may not contain the literal word
    REDEFINES. The target appears as the token between data item and usage.

    Example:
        05 CT-RK-TGOOD      CT-RK-TGDSV      DISPLAY      9(4)      1      4

    Meaning:
        05 CT-RK-TGOOD REDEFINES CT-RK-TGDSV
    """

    value = clean_field_line(
        text,
    ).upper()

    match = FIELD_NAME_PATTERN.match(
        value,
    )

    if not match:
        return ""

    field_name = match.group(
        "name",
    ).strip().upper()

    rest = match.group(
        "rest",
    ).strip()

    if not rest:
        return ""

    tokens = rest.split()

    if not tokens:
        return ""

    first_token = tokens[0].strip().upper().rstrip(".")

    if first_token in USAGE_WORDS:
        return ""

    if first_token in NON_REDEFINES_WORDS:
        return ""

    if not re.fullmatch(
        r"[A-Z][A-Z0-9-]*",
        first_token,
    ):

```

```

        return ""

    if first_token == field_name:
        return ""

    if len(tokens) >= 2:
        second_token = tokens[1].strip().upper().rstrip(".")

        if (
            second_token not in USAGE_WORDS
            and second_token not in NON_REDEFINES_WORDS
        ):
            return ""

    return first_token

def is_group_field(
    field,
) -> bool:
    if getattr(field, "is_group", False):
        return True

    if getattr(field, "has_child", False):
        return True

    return False

def has_physical_definition(
    field,
) -> bool:
    if getattr(field, "picture", None):
        return True

    if getattr(field, "datatype", None):
        return True

    if getattr(field, "length", None):
        return True

    return False

def safe_int(
    value,
) -> int | None:
    try:
        if value is None:
            return None

        text = str(value).strip()

        if not text:
            return None

        return int(text)
    except Exception:
        return None

=====
FILE: src\idms_modernizer\rules\naming_rules.py
ENCODING: utf-8
=====

import re

from idms_modernizer.rules.db2_mapping_constants import (
    DB2_COLUMN_CODE_PREFIX,
)

def normalize_name(
    name: str | None,
) -> str:

```

```

if not name:
    return ""

normalized = str(name).strip().upper()
normalized = normalized.replace("-", "_")
normalized = re.sub(
    r"^[A-Z0-9_]",
    "_",
    normalized,
)
normalized = re.sub(
    r"_{1,}",
    "_",
    normalized,
)

return normalized.strip("_")

def split_tokens(
    value: str | None,
) -> list[str]:
    normalized = normalize_name(
        value,
    )

    return [
        token.upper()
        for token in re.split(
            r"[\s_-]+",
            normalized,
        )
        if token
    ]

def remove_record_suffix(
    name: str | None,
) -> str:
    value = str(name or "").strip().upper()
    value = re.sub(
        r"[_\-\s]+[0-9]{4}$",
        "",
        value,
    )
    value = re.sub(
        r"[0-9]{4}$",
        "",
        value,
    )
    value = re.sub(
        r"[_\-\s]+$",
        "",
        value,
    )

    return normalize_name(
        value,
    )

def remove_four_digit_suffix(
    field_name: str | None,
) -> str:
    normalized = normalize_name(
        field_name,
    )

    return re.sub(
        r"_[0-9]{4}$",
        "",
        normalized,
    )

def record_code(

```

```

        record_name: str | None,
    ) -> str:
        normalized = normalize_name(
            record_name,
        )

        compact = re.sub(
            r"^[^A-Z0-9]",
            "",
            normalized,
        )

        if not compact:
            return ""

        if len(compact) <= 4:
            return compact

        return compact[-4:]

def build_generated_record_id(
    record_name: str | None,
) -> str:
    from idms_modernizer.rules.db2_record_name_rules import (
        generated_primary_key_name,
    )

    return generated_primary_key_name(
        record_name=record_name or "",
    )

```

```

=====
FILE: src\idms_modernizer\rules\occurs_rules.py
ENCODING: utf-8
=====

```

```
import re
```

```

OCCURS_PATTERN = re.compile(
    r"\bOCCURS\s+"
    r"(?P<min>[0-9]+)"
    r"(?:\s+TO\s+(?P<max>[0-9]+))?"
    r"(?:\s+TIMES)?\b",
    re.IGNORECASE,
)

```

```

def parse_occurs(
    text: str | None,
) -> dict:
    match = OCCURS_PATTERN.search(
        text or "",
    )

    if not match:
        return {
            "occurs": False,
            "occurs_min": None,
            "occurs_max": None,
        }

    occurs_min = safe_int(
        match.group("min"),
    )

    occurs_max_raw = match.group(
        "max",
    )

    occurs_max = (
        safe_int(
            occurs_max_raw,
        )
    )

```

```

        if occurs_max_raw is not None
        else occurs_min
    )

    return {
        "occurs": True,
        "occurs_min": occurs_min,
        "occurs_max": occurs_max,
    }

def has_occurs(
    text: str | None,
) -> bool:
    return OCCURS_PATTERN.search(
        text or "",
    ) is not None

def format_occurs_clause(
    occurs_min,
    occurs_max,
) -> str:
    min_value = safe_int(
        occurs_min,
    )

    max_value = safe_int(
        occurs_max,
    )

    if min_value is None and max_value is None:
        return ""

    if min_value is not None and max_value is not None:
        if min_value == max_value:
            return f"OCCURS {max_value} TIMES"

        return f"OCCURS {min_value} TO {max_value}"

    if max_value is not None:
        return f"OCCURS {max_value} TIMES"

    return f"OCCURS {min_value} TIMES"

def safe_int(
    value,
) -> int | None:
    try:
        if value is None:
            return None

        text = str(value).strip()

        if not text:
            return None

        return int(text)
    except Exception:
        return None

=====
FILE: src\idms_modernizer\rules\picture_rules.py
ENCODING: utf-8
=====

import re

START_LENGTH_PATTERN = re.compile(
    r"\b(?:P<start>[0-9]+\s+(?:P<length>[0-9]+\s*$",
    re.IGNORECASE,
)

```

```

COMPACT_PIC_PATTERNS = [
    re.compile(r"S?9[()][0-9]+[()]]V9[()][0-9]+[()]]", re.IGNORECASE),
    re.compile(r"S?9[()][0-9]+[()]]V[90]+", re.IGNORECASE),
    re.compile(r"S?[90]+V9[()][0-9]+[()]]", re.IGNORECASE),
    re.compile(r"S?[90]+V[90]+", re.IGNORECASE),
    re.compile(r"S?9[()][0-9]+[()]]", re.IGNORECASE),
    re.compile(r"S?[90]+", re.IGNORECASE),
    re.compile(r"X[()][0-9]+[()]]", re.IGNORECASE),
    re.compile(r"X+", re.IGNORECASE),
]

```

```

def clean_picture(value: str | None) -> str:
    text = str(value or "").strip()
    text = text.replace("\u00a0", " ")
    text = text.replace("\t", " ")
    text = text.replace(".", "")
    text = re.sub(r"\s+", " ", text)

    return text.strip()

```

```

def picture_core(picture: str | None) -> str:
    text = str(picture or "").upper()
    text = text.replace("PICTURE", "")
    text = text.replace("PIC", "")
    text = text.replace("COMP-3", "")
    text = re.sub(r"\bCOMP\b", "", text)
    text = text.replace("DISPLAY", "")
    text = text.replace("USAGE", "")
    text = text.replace("IS", "")
    text = text.replace(".", "")
    text = text.replace("\u00a0", " ")
    text = text.replace("\t", " ")
    text = re.sub(r"\s+", "", text)

    return text.strip()

```

```

def picture_score(core: str | None) -> int:
    text = str(core or "").upper()

    score = 0

    if "V" in text:
        score += 1000

    if "(" in text and ")" in text:
        score += 100

    if text.startswith("S"):
        score += 10

    if text.startswith("X"):
        score += 5

    score += len(text)

    return score

```

```

def extract_picture_core(
    value: str | None,
    patterns=None,
) -> str | None:
    compact = str(value or "").upper()
    compact = re.sub(r"\s+", "", compact)
    compact = compact.replace("PICTURE", "")
    compact = compact.replace("PIC", "")
    compact = compact.replace(".", "")

    selected_patterns = patterns or COMPACT_PIC_PATTERNS
    candidates: list[tuple[int, int, str]] = []

    for pattern in selected_patterns:
        for match in pattern.finditer(compact):

```

```

        raw = match.group(0)
        core = picture_core(raw)

        if not core:
            continue

        candidates.append(
            (
                picture_score(core),
                match.start(),
                core,
            )
        )

    if not candidates:
        return None

    candidates.sort(
        key=lambda item: (item[0], item[1]),
    )

    return candidates[-1][2]

def extract_parenthesized_int(value: str | None) -> int | None:
    match = re.search(r"$(\d+)$", str(value or ""))

    if not match:
        return None

    try:
        return int(match.group(1))
    except Exception:
        return None

def picture_is_character(picture: str | None) -> bool:
    text = picture_core(picture)

    if text.startswith("X(") and text.endswith(")"):
        return extract_parenthesized_int(text) is not None

    return bool(text) and set(text) == {"X"}

def character_length(
    picture: str | None,
    fallback_length: int | None = None,
) -> int | None:
    text = picture_core(picture)

    if text.startswith("X(") and text.endswith(")"):
        parsed = extract_parenthesized_int(text)

        if parsed is not None:
            return parsed

    if re.fullmatch(r"X+", text):
        return len(text)

    return fallback_length

def picture_is_decimal(picture: str | None) -> bool:
    text = picture_core(picture)

    if text.startswith("S"):
        text = text[1:]

    return "V" in text and "9" in text

def picture_is_numeric(picture: str | None) -> bool:
    text = picture_core(picture)

    if text.startswith("S"):

```



```

        text = text[1:]

    return "9" in text and "V" not in text

def count_numeric_digits(value: str | None) -> int:
    text = str(value or "").upper()

    if text.startswith("S"):
        text = text[1:]

    total = 0
    index = 0

    while index < len(text):
        current = text[index]

        if current not in {"9", "0"}:
            index += 1
            continue

        if index + 1 < len(text) and text[index + 1] == "(":
            close_index = text.find(")", index + 2)

            if close_index != -1:
                number_text = text[index + 2:close_index]

                if number_text.isdigit():
                    total += int(number_text)
                    index = close_index + 1
                    continue

            total += 1
            index += 1

    return total

def numeric_precision(picture: str | None) -> int:
    return count_numeric_digits(
        value=picture_core(picture),
    )

def decimal_precision_scale(picture: str | None) -> tuple[int, int]:
    text = picture_core(picture)

    if text.startswith("S"):
        text = text[1:]

    if "V" not in text:
        return count_numeric_digits(text), 0

    before_v, after_v = text.split("V", 1)

    integer_digits = count_numeric_digits(before_v)
    decimal_digits = count_numeric_digits(after_v)

    return integer_digits + decimal_digits, decimal_digits

def normalize_x_part(
    part: str | None,
    storage_length: int | None = None,
) -> str:
    text = str(part or "").upper()

    if text.startswith("X(") and text.endswith(")"):
        return text

    if text and set(text) == {"X"}:
        if len(text) == 1 and storage_length and int(storage_length) > 1:
            return f"X({int(storage_length)})"

        if len(text) == 1:
            return "X"

```

```

        return f"X({len(text)})"

    return text

def normalize_9_part(
    part: str | None,
    storage_length: int | None = None,
) -> str:
    text = str(part or "").upper()
    signed = False

    if text.startswith("S"):
        signed = True
        text = text[1:]

    if text.startswith("9(") and text.endswith(")"):
        return f"{'S' if signed else ''}{text}"

    count = count_numeric_digits(text)

    if count <= 0:
        return f"{'S' if signed else ''}{text}"

    if count == 1 and storage_length and int(storage_length) > 1:
        return f"{'S' if signed else ''}9({int(storage_length)})"

    if count == 1:
        return f"{'S' if signed else ''}9"

    return f"{'S' if signed else ''}9({count})"

def normalize_picture_preserving_v(
    picture: str | None,
    storage_length: int | None = None,
) -> str | None:
    core = picture_core(picture)

    if not core:
        return None

    if "V" in core:
        before_v, after_v = core.split("V", 1)

        before_v_normalized = normalize_9_part(
            part=before_v,
            storage_length=None,
        )

        after_v_normalized = normalize_9_part(
            part=after_v,
            storage_length=None,
        )

        return f"{before_v_normalized}V{after_v_normalized}"

    if core.startswith("X"):
        return normalize_x_part(
            part=core,
            storage_length=storage_length,
        )

    return normalize_9_part(
        part=core,
        storage_length=storage_length,
    )

def render_cobol_pic_clause(
    picture: str | None,
    usage: str | None,
    storage_length: int | None,
) -> str | None:
    if not picture:

```

```

        return None

    normalized_picture = normalize_picture_preserving_v(
        picture=picture,
        storage_length=storage_length,
    )

    if not normalized_picture:
        return None

    usage_upper = str(usage or "DISPLAY").upper()

    if usage_upper in {"COMP", "COMP-3"}:
        if usage_upper not in normalized_picture.upper():
            normalized_picture = f"{normalized_picture} {usage_upper}"

    if normalized_picture.upper().startswith("PIC "):
        return normalized_picture

    return f"PIC {normalized_picture}"

def derive_type_from_picture(
    picture: str | None,
    usage: str | None,
    storage_length: int | None,
    fallback_datatype: str | None = None,
) -> tuple[str | None, int | None, int | None]:
    if not picture:
        return fallback_datatype, storage_length, None

    core = picture_core(picture)

    if not core:
        return fallback_datatype, storage_length, None

    if picture_is_character(core):
        length = character_length(
            picture=core,
            fallback_length=storage_length,
        )

        return "CHAR", length, None

    if picture_is_decimal(core):
        precision, scale = decimal_precision_scale(core)

        return "DECIMAL", precision, scale

    if picture_is_numeric(core):
        precision = numeric_precision(core)

        return "DECIMAL", precision, 0

    return fallback_datatype, storage_length, None

=====
FILE: src\idms_modernizer\rules\primary_key_rules.py
ENCODING: utf-8
=====

from idms_modernizer.rules.date_rules import (
    is_date_field,
    is_date_part_name,
)
from idms_modernizer.rules.field_rules import (
    is_filler_name,
    is_group_field,
    has_physical_definition,
)
from idms_modernizer.rules.naming_rules import normalize_name

def is_physical_primary_key_candidate(field) -> bool:
    if field is None:

```

```

        return False

    field_name = getattr(field, "name", None)

    if not field_name:
        return False

    if is_filler_name(field_name):
        return False

    if is_date_field(field):
        return False

    if is_date_part_name(field_name):
        return False

    if is_group_field(field) and not has_physical_definition(field):
        return False

    return True

def find_field_for_primary_key(record, primary_key: str | None):
    if not record or not primary_key:
        return None

    normalized_primary_key = normalize_name(primary_key)

    fields = []
    fields.extend(getattr(record, "fields", []) or [])
    fields.extend(getattr(record, "mapping_fields", []) or [])

    for field in fields:
        normalized_field_name = normalize_name(
            getattr(field, "name", "") or ""
        )

        if normalized_field_name == normalized_primary_key:
            return field

    return None

def collect_primary_key_fields(record, primary_key: str | None) -> list:
    field = find_field_for_primary_key(
        record=record,
        primary_key=primary_key,
    )

    if field is None:
        return []

    if is_physical_primary_key_candidate(field):
        return [field]

    return []

def normalize_primary_key_name(primary_key: str | None) -> str | None:
    normalized = normalize_name(primary_key)

    if not normalized:
        return None

    return normalized

```

```

=====
FILE: src\idms_modernizer\rules\record_rules.py
ENCODING: utf-8
=====

```

```
import re
```

```
RECORD_PATTERN = re.compile(
```

```

        r"\bRECORD\S*NAME\b.*?([A-Z][A-Z0-9-]+)",
        re.IGNORECASE,
    )

ALT_RECORD_PATTERN = re.compile(
    r"\bRECORD\s+NAME\b.*?([A-Z][A-Z0-9-]+)",
    re.IGNORECASE,
)

def is_record_line(line: str | None) -> bool:
    text = clean_record_line(line)

    if not text:
        return False

    return parse_record_name(text) is not None

def parse_record_name(line: str | None) -> str | None:
    text = clean_record_line(line)

    if not text:
        return None

    match = RECORD_PATTERN.search(text)

    if match:
        return match.group(1).strip().upper()

    match = ALT_RECORD_PATTERN.search(text)

    if match:
        return match.group(1).strip().upper()

    return None

def clean_record_line(line: str | None) -> str:
    if line is None:
        return ""

    value = getattr(line, "text", None)

    if value is None:
        value = str(line)

    value = str(value)
    value = value.replace("\u00a0", " ")
    value = value.replace("\t", " ")
    value = re.sub(r"\s+", " ", value)

    return value.strip()

def normalize_record_name(name: str | None) -> str:
    value = str(name or "").strip().upper()
    value = value.replace("-", "_")
    value = re.sub(r"^[A-Z0-9_]", "_", value)
    value = re.sub(r"_+", "_", value)

    return value.strip("_")

=====
FILE: src\idms_modernizer\rules\relationship_rules.py
ENCODING: utf-8
=====

from idms_modernizer.rules.naming_rules import normalize_name

def normalize_relationship(rel) -> dict | None:
    set_name = getattr(rel, "set_name", None)
    owner_record = getattr(rel, "owner_record", None)
    member_record = getattr(rel, "member_record", None)

```

```

if not set_name or not owner_record or not member_record:
    return None

normalized_owner = normalize_name(owner_record)
normalized_member = normalize_name(member_record)

if not normalized_owner or not normalized_member:
    return None

if normalized_owner == normalized_member:
    return None

return {
    "set_name": normalize_name(set_name),
    "owner_record": normalized_owner,
    "member_record": normalized_member,
    "cardinality": getattr(rel, "cardinality", "1:N") or "1:N",
}

def relationship_key(rel) -> tuple | None:
    normalized = normalize_relationship(rel)

    if not normalized:
        return None

    return (
        normalized["owner_record"],
        normalized["member_record"],
        normalized["set_name"],
    )

def validate_relationships(metadata) -> list[str]:
    errors: list[str] = []

    record_names = {
        normalize_name(getattr(record, "name", "") or "")
        for record in getattr(metadata, "records", []) or []
    }

    for rel in getattr(metadata, "relationships", []) or []:
        owner_record = normalize_name(getattr(rel, "owner_record", "") or "")
        member_record = normalize_name(getattr(rel, "member_record", "") or "")

        if owner_record and owner_record not in record_names:
            errors.append(
                f"Missing Owner Record: {getattr(rel, 'owner_record', '')}"
            )

        if member_record and member_record not in record_names:
            errors.append(
                f"Missing Member Record: {getattr(rel, 'member_record', '')}"
            )

    return errors

def build_record_set_lookup(metadata) -> dict[str, str]:
    lookup: dict[str, str] = {}

    for rel in getattr(metadata, "relationships", []) or []:
        set_name = normalize_name(getattr(rel, "set_name", "") or "")
        owner_record = normalize_name(getattr(rel, "owner_record", "") or "")
        member_record = normalize_name(getattr(rel, "member_record", "") or "")

        for record_name in [owner_record, member_record]:
            if not record_name:
                continue

            existing = lookup.get(record_name, "")

            if existing:
                if set_name and set_name not in existing:
                    lookup[record_name] = f"{existing} ; {set_name}"

```

```

        else:
            lookup[record_name] = set_name

    return lookup

```

```

=====
FILE: src\idms_modernizer\rules\restart_record_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules.db2_identifier_rules import to_db2_name
from idms_modernizer.rules.db2_mapping_constants import (
    RESTART_DB2_SCHEMA_NAME,
    RESTART_DB2_VIEW_NAME,
    RESTART_IDMS_RECORD_NAMES,
)

```

```

def restart_record_candidates() -> set[str]:
    candidates: set[str] = set()

    for record_name in RESTART_IDMS_RECORD_NAMES:
        normalized_record_name = to_db2_name(
            record_name,
        )

        if not normalized_record_name:
            continue

        candidates.add(
            normalized_record_name,
        )

        candidates.add(
            to_db2_name(
                f"{record_name}-{RESTART_DB2_SCHEMA_NAME}",
            )
        )

        candidates.add(
            to_db2_name(
                f"{record_name} - {RESTART_DB2_SCHEMA_NAME}",
            )
        )

        candidates.add(
            to_db2_name(
                f"{RESTART_DB2_SCHEMA_NAME}{record_name}",
            )
        )

        candidates.add(
            to_db2_name(
                RESTART_DB2_VIEW_NAME,
            )
        )

    return candidates

```

```

def is_restart_control_record(
    record_name: str | None,
) -> bool:
    normalized_record_name = to_db2_name(
        record_name,
    )

    if not normalized_record_name:
        return False

    return normalized_record_name in restart_record_candidates()

```

```
=====
FILE: src\idms_modernizer\rules\schema_enrichment_rules.py
ENCODING: utf-8
=====
```

```
import re

from idms_modernizer.rules.field_rules import FIELD_NAME_PATTERN
from idms_modernizer.rules.usage_rules import USAGE_PATTERN
from idms_modernizer.rules.picture_rules import START_LENGTH_PATTERN
from idms_modernizer.services.name_normalizer import NameNormalizer
```

```
DATE_NAME_TOKENS = {
    "DA",
    "DATE",
    "DT",
    "DTE",
    "YYMMDD",
    "YYYYMMDD",
    "YMD",
}
```

```
def normalize_schema_lines(lines: list[str]) -> list[str]:
    result: list[str] = []

    for line in lines or []:
        if line is None:
            continue

        value = getattr(line, "text", None)

        if value is None:
            value = str(line)

        value = str(value)
        value = value.replace("\t", " ")
        value = value.replace("\u00a0", " ")
        value = value.replace("PICTURE .", "PICTURE")
        value = value.replace("PIC .", "PIC")
        value = re.sub(r"<[^>]+>", " ", value)
        value = re.sub(r"\s+", " ", value).strip()

        if value:
            result.append(value)

    return result
```

```
def build_field_blocks(
    normalized_lines: list[str],
) -> dict[str, str]:
    starts: list[tuple[int, str]] = []

    for index, line in enumerate(normalized_lines or []):
        match = FIELD_NAME_PATTERN.match(line)

        if not match:
            continue

        field_name = match.group("name") or ""

        starts.append(
            (
                index,
                field_name.upper(),
            )
        )

    blocks: dict[str, str] = {}

    for index, item in enumerate(starts):
        start_index, field_name = item

        if index + 1 < len(starts):
```



```

        end_index = starts[index + 1][0]
    else:
        end_index = len(normalized_lines)

    block = " ".join(
        normalized_lines[start_index:end_index],
    )

    normalized_field_name = NameNormalizer.normalize(
        field_name,
    )

    if normalized_field_name:
        blocks[normalized_field_name] = block

    suffix_removed = remove_record_suffix(
        value=normalized_field_name,
    )

    if suffix_removed:
        blocks.setdefault(
            suffix_removed,
            block,
        )

    return blocks

def find_block_for_field(
    field_name: str,
    field_blocks: dict[str, str],
) -> str:
    normalized_name = NameNormalizer.normalize(
        field_name or "",
    )

    if not normalized_name:
        return ""

    if normalized_name in field_blocks:
        return field_blocks[normalized_name]

    suffix_removed = remove_record_suffix(
        value=normalized_name,
    )

    if suffix_removed in field_blocks:
        return field_blocks[suffix_removed]

    for block_name, block in field_blocks.items():
        if remove_record_suffix(block_name) == suffix_removed:
            return block

    return ""

def parse_schema_block(block: str) -> dict:
    text = str(block or "").strip()

    usage = extract_usage(
        block=text,
    )

    start_position, storage_length = extract_start_and_length(
        block=text,
    )

    body_without_positions = remove_trailing_start_and_length(
        value=text,
    )

    rest = extract_rest_after_level_and_name(
        block=body_without_positions,
    )

    picture_area = remove_usage_words(

```

```

        value=rest,
    )

    picture_area = remove_schema_noise(
        value=picture_area,
    )

    return {
        "usage": usage,
        "start_position": start_position,
        "storage_length": storage_length,
        "picture_area": picture_area,
    }

def extract_rest_after_level_and_name(block: str) -> str:
    match = FIELD_NAME_PATTERN.match(
        block or ""
    )

    if not match:
        return str(block or "")

    return str(match.group("rest") or "").strip()

def extract_usage(block: str) -> str:
    upper = str(block or "").upper()

    if "COMP-3" in upper:
        return "COMP-3"

    if re.search(r"\bCOMP\b", upper):
        return "COMP"

    usage_match = USAGE_PATTERN.search(
        block or ""
    )

    if usage_match:
        return usage_match.group("usage").upper()

    return "DISPLAY"

def extract_start_and_length(
    block: str,
) -> tuple[int | None, int | None]:
    match = START_LENGTH_PATTERN.search(
        block or ""
    )

    if not match:
        return None, None

    return int(match.group("start")), int(match.group("length"))

def remove_trailing_start_and_length(value: str) -> str:
    return re.sub(
        r"\b[0-9]+\s+[0-9]+\s*$",
        "",
        str(value or ""),
    ).strip()

def remove_usage_words(value: str) -> str:
    text = str(value or "")

    text = re.sub(r"\bUSAGE\s+IS\s+COMP-3\b", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bUSAGE\s+IS\s+COMP\b", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bUSAGE\s+IS\s+DISPLAY\b", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bCOMP-3\b", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bCOMP\b", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bDISPLAY\b", " ", text, flags=re.IGNORECASE)

```

```

return re.sub(r"\s+", " ", text).strip()

def remove_schema_noise(value: str) -> str:
    text = str(value or "")

    text = re.sub(r"\bOCCURS\b.*$", " ", text, flags=re.IGNORECASE)
    text = re.sub(r"\bREDEFINES\b.*$", " ", text, flags=re.IGNORECASE)
    text = text.replace(".", " ")
    text = text.replace("\u00a0", " ")
    text = text.replace("\t", " ")

    return re.sub(r"\s+", " ", text).strip()

def remove_record_suffix(value: str | None) -> str:
    text = str(value or "").strip().upper()
    text = re.sub(r"[\s_-]+\d{4}$", "", text)
    text = re.sub(r"\d{4}$", "", text)
    text = re.sub(r"[\s_-]+$", "", text)

    return text

def calculate_end_position(
    start_position: int | None,
    storage_length: int | None,
    fallback=None,
) -> int | None:
    try:
        if start_position is not None and storage_length is not None:
            return int(start_position) + int(storage_length) - 1
    except Exception:
        pass

    try:
        if fallback is not None:
            return int(fallback)
    except Exception:
        pass

    return None

def basetype_for_datatype(datatype: str | None) -> str | None:
    datatype_upper = str(datatype or "").upper()

    if datatype_upper == "DATE":
        return "DATE"

    if datatype_upper in {"DECIMAL", "NUMERIC", "INTEGER"}:
        return "NUMERIC"

    if datatype_upper in {"CHAR", "VARCHAR"}:
        return "CHAR"

    return datatype

def is_date_field(
    field,
    picture: str | None,
    storage_length: int | None,
    datatype: str | None,
) -> bool:
    datatype_upper = str(datatype or "").upper()

    if datatype_upper == "DATE":
        return True

    field_name = NameNormalizer.normalize(
        getattr(field, "name", "") or "",
    )

    parts = [
        part

```

```

        for part in field_name.replace("-", " ").replace("_", " ").split()
        if part
    ]

    if "DATE" in parts or "DA" in parts:
        if storage_length == 8:
            return True

    compact_name = field_name.replace("_", "")

    if compact_name.startswith("DA") and storage_length == 8:
        return True

    if field_name.endswith("_DATE") and storage_length == 8:
        return True

    return False

def field_to_dict(field) -> dict:
    if hasattr(field, "model_dump"):
        return field.model_dump()

    if hasattr(field, "__dict__"):
        return dict(field.__dict__)

    return {}

=====
FILE: src\idms_modernizer\rules\schema_field_enrichment_rules.py
ENCODING: utf-8
=====

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.rules import picture_rules
from idms_modernizer.rules import schema_enrichment_rules
from idms_modernizer.rules.schema_field_factory_rules import (
    copy_field_with_defaults,
    make_field,
)

def enrich_field(
    field: DataField,
    block: str,
    debug_label: str = "",
    debug_verbose: bool = False,
    compact_pic_patterns=None,
) -> DataField:
    field_name = getattr(field, "name", "") or ""

    if not block:
        return copy_field_with_defaults(
            field=field,
        )

    parsed_line = parse_schema_block(
        block=block,
        field_name=field_name,
        debug_label=debug_label,
        debug_verbose=debug_verbose,
        compact_pic_patterns=compact_pic_patterns,
    )

    usage = parsed_line["usage"]
    start_position = parsed_line["start_position"]
    storage_length = parsed_line["storage_length"]
    picture_core = parsed_line["picture_core"]

    is_group = bool(
        getattr(field, "has_child", False)
        or getattr(field, "is_group", False)
    )

    if is_group:

```

```

    return make_field(
        source=field,
        datatype=getattr(field, "datatype", None),
        length=storage_length or getattr(field, "length", None),
        scale=getattr(field, "scale", None),
        picture=None,
        start_position=start_position or getattr(field, "start_position", None),
        end_position=schema_enrichment_rules.calculate_end_position(
            start_position=start_position,
            storage_length=storage_length,
            fallback=getattr(field, "end_position", None),
        ),
        basetype=getattr(field, "basetype", None),
    )

rendered_picture = picture_rules.render_cobol_pic_clause(
    picture=picture_core,
    usage=usage,
    storage_length=storage_length,
)

datatype, logical_length, scale = picture_rules.derive_type_from_picture(
    picture=rendered_picture,
    usage=usage,
    storage_length=storage_length,
    fallback_datatype=getattr(field, "datatype", None),
)

if schema_enrichment_rules.is_date_field(
    field=field,
    picture=rendered_picture,
    storage_length=storage_length,
    datatype=datatype,
):
    datatype = "DATE"
    scale = None

end_position = schema_enrichment_rules.calculate_end_position(
    start_position=start_position,
    storage_length=storage_length,
    fallback=getattr(field, "end_position", None),
)

actual_length = storage_length

if actual_length is None:
    actual_length = logical_length

if debug_verbose:
    print(
        "SCHEMA_ENRICH_RESULT",
        f"debug_label={debug_label}",
        f"field={repr(field_name)}",
        f"block={repr(block)}",
        f"usage={repr(usage)}",
        f"picture_core={repr(picture_core)}",
        f"rendered_picture={repr(rendered_picture)}",
        f"datatype={repr(datatype)}",
        f"logical_length={logical_length}",
        f"storage_length={storage_length}",
        f"actual_length={actual_length}",
        f"scale={scale}",
        f"start={start_position}",
        f"end={end_position}",
    )

return make_field(
    source=field,
    datatype=datatype,
    length=actual_length,
    scale=scale,
    picture=rendered_picture,
    start_position=start_position,
    end_position=end_position,
    basetype=schema_enrichment_rules.basetype_for_datatype(
        datatype=datatype,

```

```

    ),
)

def parse_schema_block(
    block: str,
    field_name: str,
    debug_label: str = "",
    debug_verbose: bool = False,
    compact_pic_patterns=None,
) -> dict:
    parsed = schema_enrichment_rules.parse_schema_block(
        block=block,
    )

    selected_picture_core = extract_picture_core(
        value=parsed["picture_area"],
        field_name=field_name,
        debug_label=debug_label,
        debug_verbose=debug_verbose,
        compact_pic_patterns=compact_pic_patterns,
    )

    if debug_verbose:
        print(
            "SCHEMA_PARSE_DEBUG",
            f"debug_label={debug_label}",
            f"field={repr(field_name)}",
            f"block={repr(block)}",
            f"usage={repr(parsed['usage'])}",
            f"start={repr(parsed['start_position'])}",
            f"length={repr(parsed['storage_length'])}",
            f"picture_area={repr(parsed['picture_area'])}",
            f"picture_core={repr(selected_picture_core)}",
        )

    return {
        "usage": parsed["usage"],
        "start_position": parsed["start_position"],
        "storage_length": parsed["storage_length"],
        "picture_core": selected_picture_core,
    }

def extract_picture_core(
    value: str,
    field_name: str = "",
    debug_label: str = "",
    debug_verbose: bool = False,
    compact_pic_patterns=None,
) -> str | None:
    selected = picture_rules.extract_picture_core(
        value=value,
        patterns=compact_pic_patterns,
    )

    if debug_verbose:
        print(
            "PIC_EXTRACT_DEBUG_RESULT",
            f"debug_label={debug_label}",
            f"field={repr(field_name)}",
            f"value={repr(value)}",
            f"selected={repr(selected)}",
        )

    return selected

```

```

=====
FILE: src\idms_modernizer\rules\schema_field_factory_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.rules import schema_enrichment_rules

```

```

def make_field(
    source: DataField,
    datatype: str | None,
    length: int | None,
    scale: int | None,
    picture: str | None,
    start_position: int | None,
    end_position: int | None,
    basetype: str | None,
) -> DataField:
    data = schema_enrichment_rules.field_to_dict(
        field=source,
    )

    data.update(
        {
            "datatype": datatype,
            "length": length,
            "scale": scale,
            "picture": picture,
            "start_position": start_position,
            "end_position": end_position,
            "basetype": basetype,
        }
    )

    return DataField(**data)

def copy_field_with_defaults(
    field: DataField,
) -> DataField:
    return make_field(
        source=field,
        datatype=getattr(field, "datatype", None),
        length=getattr(field, "length", None),
        scale=getattr(field, "scale", None),
        picture=getattr(field, "picture", None),
        start_position=getattr(field, "start_position", None),
        end_position=getattr(field, "end_position", None),
        basetype=getattr(field, "basetype", None),
    )

```

```

=====
FILE: src\idms_modernizer\rules\schema_group_position_rules.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.rules.schema_field_factory_rules import make_field

```

```

def enrich_group_positions_from_children(
    fields: list[DataField],
) -> list[DataField]:
    output = list(fields or [])

    for index, field in enumerate(output):
        is_group = bool(
            getattr(field, "has_child", False)
            or getattr(field, "is_group", False)
        )

        if not is_group:
            continue

        try:
            parent_level = int(getattr(field, "level", None))
        except Exception:
            continue

        child_starts: list[int] = []
        child_ends: list[int] = []

```

```

for child in output[index + 1:]:
    try:
        current_level = int(getattr(child, "level", None))
    except Exception:
        continue

    if current_level <= parent_level:
        break

    start = getattr(child, "start_position", None)
    end = getattr(child, "end_position", None)

    if start is not None:
        try:
            child_starts.append(int(start))
        except Exception:
            pass

    if end is not None:
        try:
            child_ends.append(int(end))
        except Exception:
            pass

    if not child_starts and not child_ends:
        continue

    group_start = (
        min(child_starts)
        if child_starts
        else getattr(field, "start_position", None)
    )

    group_end = (
        max(child_ends)
        if child_ends
        else getattr(field, "end_position", None)
    )

    group_length = getattr(field, "length", None)

    try:
        if group_start is not None and group_end is not None:
            group_length = int(group_end) - int(group_start) + 1
    except Exception:
        pass

    output[index] = make_field(
        source=field,
        datatype=getattr(field, "datatype", None),
        length=group_length,
        scale=getattr(field, "scale", None),
        picture=None,
        start_position=group_start,
        end_position=group_end,
        basetype=getattr(field, "basetype", None),
    )

return output

```

```

=====
FILE: src\idms_modernizer\rules\usage_rules.py
ENCODING: utf-8
=====

```

```
import re
```

```

USAGE_PATTERN = re.compile(
    r"\b(?:USAGE\s+IS\s+)?(?:P<usage>DISPLAY|COMP-3|COMP)\b",
    re.IGNORECASE,
)

```

```

PIC_OR_USAGE_PATTERN = re.compile(
    r"\b(PIC|PICTURE|DISPLAY|COMP-3|COMP)\b",
)

```



```

    re.IGNORECASE,
)

def extract_usage(text: str | None) -> str | None:
    match = USAGE_PATTERN.search(text or "")

    if not match:
        return None

    return normalize_usage(match.group("usage"))

def normalize_usage(value: str | None) -> str:
    text = str(value or "").strip().upper()

    if text in {"COMP3", "COMP-3"}:
        return "COMP-3"

    if text == "COMP":
        return "COMP"

    if text == "DISPLAY":
        return "DISPLAY"

    return text

def has_pic_or_usage(text: str | None) -> bool:
    return PIC_OR_USAGE_PATTERN.search(text or "") is not None

```

```

=====
FILE: src\idms_modernizer\services\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_modernizer\services\business_datatype_inference.py
ENCODING: utf-8
=====

```

```

class BusinessDatatypeInference:
    """
    Generic datatype inference.

    This class intentionally does not inspect field names.

    Reason:
    Field-name based rules such as PHONE, ZIP, SSN, CODE, ACCOUNT,
    POLICY, or LICENSE are business/domain assumptions and should not be
    hardcoded.

    Datatype should be decided upstream from:
    - IDMS PIC clause
    - USAGE
    - length
    - scale
    """

    @staticmethod
    def infer(
        field_name: str,
        current_type: str,
    ) -> str:
        if current_type:
            return current_type.upper()

        return "VARCHAR"

```

```
=====
FILE: src\idms_modernizer\services\canonical_reconciler.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.domain.canonical_models import (
    CanonicalSchema,
    CanonicalField
)
from idms_modernizer.domain.schema_models import SchemaMetadata
from idms_modernizer.services.name_normalizer import NameNormalizer
```

```
class CanonicalReconciler:
    """
    Reconciles canonical schema with IDMS schema metadata.

    IDMS schema is authoritative for:
    - datatype
    - length
    - scale
    - picture

    Canonical schema is authoritative for:
    - normalized record names
    - primary keys
    - relationships
    """

    def reconcile(
        self,
        canonical_schema: CanonicalSchema,
        metadata: SchemaMetadata
    ) -> CanonicalSchema:

        idms_field_lookup = self._build_idms_field_lookup(
            metadata
        )

        for record in canonical_schema.records:
            normalized_record_name = NameNormalizer.normalize(
                record.name
            )

            reconciled_fields: list[CanonicalField] = []

            for field in record.fields:
                normalized_field_name = NameNormalizer.normalize(
                    field.name
                )

                idms_field = idms_field_lookup.get(
                    (
                        normalized_record_name,
                        normalized_field_name
                    )
                )

                if idms_field is None:
                    idms_field = idms_field_lookup.get(
                        (
                            normalized_record_name,
                            self._remove_record_suffix(
                                normalized_field_name
                            )
                        )
                    )

                if idms_field is None:
                    reconciled_fields.append(field)
                    continue

                reconciled_fields.append(
                    CanonicalField(
                        name=field.name,
                        datatype=idms_field.datatype,
```

```

        length=idms_field.length,
        scale=idms_field.scale,
        occurs=field.occurs,
        occurs_max=field.occurs_max
    )
)

record.fields = reconciled_fields

return canonical_schema

def _build_idms_field_lookup(
    self,
    metadata: SchemaMetadata
) -> dict[tuple[str, str], object]:

    lookup: dict[tuple[str, str], object] = {}

    for record in metadata.records:
        record_name = NameNormalizer.normalize(
            record.name
        )

        for field in record.fields:
            field_name = NameNormalizer.normalize(
                field.name
            )

            lookup[
                (
                    record_name,
                    field_name
                )
            ] = field

            lookup[
                (
                    record_name,
                    self._remove_record_suffix(field_name)
                )
            ] = field

    return lookup

def _remove_record_suffix(
    self,
    field_name: str
) -> str:

    parts = field_name.split("_")

    if (
        len(parts) > 1
        and parts[-1].isdigit()
        and len(parts[-1]) == 4
    ):
        return "_".join(parts[:-1])

    return field_name

```

```

=====
FILE: src\idms_modernizer\services\canonical_schema_builder.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.canonical_models import (
    CanonicalSchema,
    CanonicalRecord,
    CanonicalField,
)
from idms_modernizer.services.name_normalizer import NameNormalizer
from idms_modernizer.services.date_field_consolidator import DateFieldConsolidator

from idms_modernizer.rules.canonical_field_rules import (
    adjust_occurs_field,

```

```

        build_date_name_set,
        build_group_name_set,
        is_date_part_name,
        is_non_physical_group_field,
    )

from idms_modernizer.rules.canonical_primary_key_rules import (
    apply_primary_key_rules,
)

from idms_modernizer.rules.canonical_relationship_rules import (
    add_sets_and_relationships,
)

from idms_modernizer.rules.restart_record_rules import (
    is_restart_control_record,
)

class CanonicalSchemaBuilder:
    """
    Builds canonical schema from IDMS metadata.

    Generic behavior only:
    - No hardcoded record names.
    - No hardcoded column names.
    - No hardcoded SET names.

    Rules:
    - Physical DB2 columns come from record.fields, after date consolidation.
    - Group detection uses mapping_fields level hierarchy.
    - CALC outer group is never a physical DB2 PK column.
    - CALC intermediate groups are not physical DB2 columns.
    - CALC physical non-date leaf children become actual composite PK columns.
    - Date group/date parts are never physical PK.
    - FILLER is never physical PK.
    """

    def build(
        self,
        metadata,
    ) -> CanonicalSchema:
        schema = CanonicalSchema()

        for record in getattr(metadata, "records", []) or []:
            record_name = getattr(record, "name", "") or ""

            if is_restart_control_record(record_name):
                continue

            canonical_record = self.build_record(
                record=record,
            )

            schema.records.append(
                canonical_record,
            )

        add_sets_and_relationships(
            schema=schema,
            metadata=metadata,
        )

        return schema

    def build_record(
        self,
        record,
    ) -> CanonicalRecord:
        record_name = NameNormalizer.normalize(
            getattr(record, "name", "") or "",
        )

        primary_key = (
            NameNormalizer.normalize(
                getattr(record, "primary_key", "") or "",
            )

```

```

        if getattr(record, "primary_key", None)
        else None
    )

    canonical_record = CanonicalRecord(
        name=record_name,
        primary_key=primary_key,
        primary_keys=[],
    )

    mapping_fields = (
        getattr(record, "mapping_fields", None)
        or getattr(record, "fields", None)
        or []
    )

    group_names = build_group_name_set(
        fields=mapping_fields,
    )

    date_names = build_date_name_set(
        fields=mapping_fields,
    )

    added_fields: set[str] = set()

    self.add_physical_fields(
        canonical_record=canonical_record,
        record=record,
        group_names=group_names,
        date_names=date_names,
        added_fields=added_fields,
    )

    apply_primary_key_rules(
        canonical_record=canonical_record,
        record=record,
        added_fields=added_fields,
        group_names=group_names,
        date_names=date_names,
    )

    return canonical_record

def add_physical_fields(
    self,
    canonical_record: CanonicalRecord,
    record,
    group_names: set[str],
    date_names: set[str],
    added_fields: set[str],
) -> None:
    physical_fields = getattr(record, "fields", []) or []

    normalized_fields = DateFieldConsolidator.consolidate(
        physical_fields,
    )

    for field in normalized_fields:
        field_name = NameNormalizer.normalize(
            getattr(field, "name", "") or "",
        )

        if not field_name:
            continue

        if is_date_part_name(field_name):
            continue

        if field_name in added_fields:
            continue

        if field_name in group_names and field_name not in date_names:
            continue

        if is_non_physical_group_field(

```

```

        field=field,
    ):
        continue

    adjusted_field = adjust_occurs_field(
        field=field,
    )

    canonical_record.fields.append(
        CanonicalField(
            name=field_name,
            datatype=getattr(adjusted_field, "datatype", None),
            length=getattr(adjusted_field, "length", None),
            scale=getattr(adjusted_field, "scale", None),
            occurs=getattr(adjusted_field, "occurs", False),
            occurs_max=getattr(adjusted_field, "occurs_max", None),
        )
    )

    added_fields.add(
        field_name,
    )

```

```

=====
FILE: src\idms_modernizer\services\date_field_consolidator.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.schema_models import DataField
from idms_modernizer.services.name_normalizer import NameNormalizer

```

```

from idms_modernizer.rules import date_pattern_rules
from idms_modernizer.rules import date_group_rules

```

```

class DateFieldConsolidator:

```

```

    """

```

```

    Consolidates legacy date child fields into one DATE field.

```

```

    Supports:

```

```

    - YEAR / MONTH / DAY
    - YR / MO / DY
    - Y / M / D
    - YY / MM / DD
    - YYYY / MM / DD
    - DY / DM / DD
    """

```

```

    YEAR_PARTS = date_pattern_rules.YEAR_PARTS
    MONTH_PARTS = date_pattern_rules.MONTH_PARTS
    DAY_PARTS = date_pattern_rules.DAY_PARTS

```

```

    @staticmethod

```

```

    def consolidate(
        fields: list[DataField],
    ) -> list[DataField]:
        result: list[DataField] = []
        consumed: set[str] = set()
        added_dates: set[str] = set()

```

```

        date_groups = date_group_rules.collect_date_groups(
            fields=fields,
        )

```

```

        for date_name, parts in date_groups.items():
            if not date_group_rules.should_build_date_field(
                date_name=date_name,
                parts=parts,
                added_dates=added_dates,
            ):
                continue

```

```

        date_field = date_group_rules.build_date_field(
            date_name=date_name,
            parts=parts,

```

```

    )

    result.append(
        date_field,
    )

    normalized_date_name = NameNormalizer.normalize(
        date_name,
    )

    added_dates.add(
        normalized_date_name,
    )

    for part_field in parts.values():
        consumed.add(
            NameNormalizer.normalize(
                part_field.name,
            )
        )

    for field in fields or []:
        if not field or not field.name:
            continue

        normalized_name = NameNormalizer.normalize(
            field.name,
        )

        if normalized_name in consumed:
            continue

        if normalized_name in added_dates:
            continue

        result.append(
            field,
        )

    return result

@staticmethod
def collect_date_groups(
    fields: list[DataField],
) -> dict[str, dict[str, DataField]]:
    return date_group_rules.collect_date_groups(
        fields=fields,
    )

@staticmethod
def parse_date_part(
    field_name: str,
) -> dict[str, str] | None:
    return date_pattern_rules.parse_date_part(
        field_name=field_name,
    )

@staticmethod
def date_part_type(
    token: str,
    tokens: list[str],
) -> str | None:
    return date_pattern_rules.date_part_type(
        token=token,
        tokens=tokens,
    )

@staticmethod
def has_complete_date_group(
    parts: dict[str, DataField],
) -> bool:
    return date_pattern_rules.has_complete_date_group(
        parts=parts,
    )

@staticmethod

```

```

def parent_level(
    field: DataField | None,
) -> int | None:
    return date_group_rules.parent_level(
        field=field,
    )

@staticmethod
def min_start_position(
    parts: dict[str, DataField],
) -> int | None:
    return date_group_rules.min_start_position(
        parts=parts,
    )

@staticmethod
def max_end_position(
    parts: dict[str, DataField],
) -> int | None:
    return date_group_rules.max_end_position(
        parts=parts,
    )

@staticmethod
def split_tokens(
    value: str,
) -> list[str]:
    return date_pattern_rules.split_tokens(
        value=value,
    )

```

```

=====
FILE: src\idms_modernizer\services\db2_datatype_mapper.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.rules import db2_picture_rules
from idms_modernizer.rules import db2_datatype_rules

```

```

class DB2DatatypeMapper:
    """
    Generic DB2 datatype mapper for Schema Listing conversion.

    Rules:
    - Sheet Mapping and DB2 model must use the same datatype logic.
    - Numeric COBOL PIC always maps to DECIMAL.
    - Do not emit SMALLINT, INTEGER, or BIGINT from Schema Listing fields.
    - Character PIC maps to CHAR by default.
    - VARCHAR is used only for fallback text fields without a COBOL PIC.
    - DATE and TIMESTAMP are preserved.
    """

    DEFAULT_CHAR_LENGTH = db2_picture_rules.DEFAULT_CHAR_LENGTH
    DEFAULT_DECIMAL_PRECISION = db2_picture_rules.DEFAULT_DECIMAL_PRECISION
    DEFAULT_DECIMAL_SCALE = db2_picture_rules.DEFAULT_DECIMAL_SCALE
    DEBUG = False

    @staticmethod
    def map(
        field,
    ) -> str:
        datatype = (
            getattr(field, "datatype", None)
            or getattr(field, "data_type", None)
            or getattr(field, "type", None)
            or ""
        )

        datatype = str(datatype or "").strip().upper()

        picture = (
            getattr(field, "picture", None)
            or getattr(field, "pic", None)
            or getattr(field, "pic_clause", None)

```



```

        or ""
    )

    picture = str(picture or "").strip()

    length = db2_datatype_rules.first_present(
        getattr(field, "length", None),
        getattr(field, "precision", None),
        getattr(field, "logical_length", None),
        getattr(field, "storage_length", None),
        getattr(field, "physical_length", None),
        getattr(field, "byte_length", None),
        getattr(field, "bytes", None),
    )

    scale = getattr(field, "scale", None)

    if datatype == "DATE":
        return "DATE"

    if datatype in {"TIMESTAMP", "DATETIME"}:
        return "TIMESTAMP"

    if datatype == "TIME":
        return "TIME"

    if picture:
        return db2_datatype_rules.map_picture(
            picture=picture,
            fallback_length=length,
            fallback_scale=scale,
            fallback_datatype=datatype,
        )

    return db2_datatype_rules.map_without_picture(
        fallback_datatype=datatype,
        fallback_length=length,
        fallback_scale=scale,
    )

    )

    @staticmethod
    def map_datatype(
        field,
    ) -> str:
        return DB2DatatypeMapper.map(
            field=field,
        )

    @staticmethod
    def map_picture(
        picture: str,
        fallback_length=None,
        fallback_scale=None,
        fallback_datatype: str = "",
    ) -> str:
        return db2_datatype_rules.map_picture(
            picture=picture,
            fallback_length=fallback_length,
            fallback_scale=fallback_scale,
            fallback_datatype=fallback_datatype,
        )

    @staticmethod
    def map_without_picture(
        fallback_datatype: str,
        fallback_length=None,
        fallback_scale=None,
    ) -> str:
        return db2_datatype_rules.map_without_picture(
            fallback_datatype=fallback_datatype,
            fallback_length=fallback_length,
            fallback_scale=fallback_scale,
        )

    @staticmethod
    def clean_picture(

```

```

        picture: str,
    ) -> str:
        return db2_picture_rules.clean_picture(
            picture=picture,
        )

    @staticmethod
    def picture_core(
        picture: str | None,
    ) -> str:
        return db2_picture_rules.picture_core(
            picture=picture,
        )

    @staticmethod
    def picture_is_char(
        picture: str,
    ) -> bool:
        return db2_picture_rules.picture_is_char(
            picture=picture,
        )

    @staticmethod
    def char_length(
        picture: str,
        fallback_length=None,
    ) -> int:
        return db2_picture_rules.char_length(
            picture=picture,
            fallback_length=fallback_length,
        )

    @staticmethod
    def picture_is_numeric(
        picture: str,
    ) -> bool:
        return db2_picture_rules.picture_is_numeric(
            picture=picture,
        )

    @staticmethod
    def picture_is_decimal(
        picture: str,
    ) -> bool:
        return db2_picture_rules.picture_is_decimal(
            picture=picture,
        )

    @staticmethod
    def numeric_precision(
        picture: str,
        fallback_length=None,
    ) -> int:
        return db2_picture_rules.numeric_precision(
            picture=picture,
            fallback_length=fallback_length,
        )

    @staticmethod
    def decimal_precision_scale(
        picture: str,
    ) -> tuple[int, int]:
        return db2_picture_rules.decimal_precision_scale(
            picture=picture,
        )

    @staticmethod
    def count_numeric_digits(
        value: str,
    ) -> int:
        return db2_picture_rules.count_numeric_digits(
            value=value,
        )

    @staticmethod
    def format_decimal(

```

```

        precision: int,
        scale: int,
        force_scale: bool = False,
    ) -> str:
        return db2_datatype_rules.format_decimal(
            precision=precision,
            scale=scale,
            force_scale=force_scale,
        )

    @staticmethod
    def extract_parenthesized_int(
        value: str,
    ) -> int | None:
        return db2_picture_rules.extract_parenthesized_int(
            value=value,
        )

    @staticmethod
    def safe_int(
        value,
        default: int,
    ) -> int:
        return db2_picture_rules.safe_int(
            value=value,
            default=default,
        )

    @staticmethod
    def first_present(
        *values,
    ):
        return db2_datatype_rules.first_present(
            *values,
        )

```

```

=====
FILE: src\idms_modernizer\services\db2_mapping_service.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import DB2Model
from idms_modernizer.rules import db2_mapping_constants
from idms_modernizer.rules import db2_model_mapping_rules
from idms_modernizer.rules import db2_name_mapping_rules
from idms_modernizer.rules import db2_relationship_mapping_rules
from idms_modernizer.rules import db2_table_mapping_rules

```

```

print("LOADED DB2MappingService VERSION MODULAR-FACADE-2026-08-31")

```

```

class DB2MappingService:
    """
    Builds DB2 model from canonical schema.

    This service is intentionally thin.

    Main responsibilities:
    - Delegate DB2 model creation to rule modules.
    - Preserve backward-compatible method names used elsewhere.
    - Keep naming, table, PK, and FK logic in separate small files.
    """

    DB2_TABLE_PREFIX = db2_mapping_constants.DB2_TABLE_PREFIX
    DB2_TABLE_SUFFIX = db2_mapping_constants.DB2_TABLE_SUFFIX
    DB2_TABLE_CORE_LEN = db2_mapping_constants.DB2_TABLE_CORE_LEN

    TECHNICAL_KEY_DATATYPE = db2_mapping_constants.TECHNICAL_KEY_DATATYPE

    GENERIC_TRAILING_TOKENS = db2_mapping_constants.GENERIC_TRAILING_TOKENS
    PROJECT_ABBREVIATIONS = db2_mapping_constants.PROJECT_ABBREVIATIONS

    DATE_TOKENS = db2_mapping_constants.DATE_TOKENS

```

```

YEAR_PARTS = db2_mapping_constants.YEAR_PARTS
MONTH_PARTS = db2_mapping_constants.MONTH_PARTS
DAY_PARTS = db2_mapping_constants.DAY_PARTS

def build_db2_model(
    self,
    schema: CanonicalSchema,
) -> DB2Model:
    print(
        "USING DB2MappingService.build_db2_model VERSION MODULAR-FACADE-2026-08-31"
    )

    return db2_model_mapping_rules.build_db2_model(
        schema=schema,
    )

def build_table_for_record(
    self,
    record,
):
    return db2_model_mapping_rules.build_table_for_record(
        record=record,
    )

def get_record_primary_keys(
    self,
    record,
) -> list[str]:
    return db2_model_mapping_rules.get_record_primary_keys(
        record=record,
    )

def apply_relationship_foreign_keys(
    self,
    schema: CanonicalSchema,
    model: DB2Model,
) -> None:
    table_lookup = self.build_table_lookup(
        model=model,
    )

    db2_relationship_mapping_rules.apply_relationship_foreign_keys(
        schema=schema,
        model=model,
        table_lookup=table_lookup,
    )

def ensure_fk_column_reusing_parent_name(
    self,
    child_table,
    parent_pk_column,
):
    return db2_relationship_mapping_rules.ensure_fk_column_reusing_parent_name(
        child_table=child_table,
        parent_pk_column=parent_pk_column,
    )

def add_foreign_key_if_missing(
    self,
    child_table,
    child_column: str,
    parent_table,
    parent_column: str,
    set_name: str = "",
) -> None:
    db2_relationship_mapping_rules.add_foreign_key_if_missing(
        child_table=child_table,
        child_column=child_column,
        parent_table=parent_table,
        parent_column=parent_column,
        set_name=set_name,
    )

def foreign_key_exists(
    self,
    child_table,

```

```

        child_column: str,
        parent_table,
        parent_column: str,
    ) -> bool:
        return db2_relationship_mapping_rules.foreign_key_exists(
            child_table=child_table,
            child_column=child_column,
            parent_table=parent_table,
            parent_column=parent_column,
        )

    def ensure_record_primary_key(
        self,
        table,
        record_name: str,
    ):
        return db2_table_mapping_rules.ensure_record_primary_key(
            table=table,
            record_name=record_name,
        )

    def generated_primary_key_name(
        self,
        record_name: str,
    ) -> str:
        return db2_name_mapping_rules.generated_primary_key_name(
            record_name=record_name,
        )

    def get_primary_key_columns(
        self,
        table,
    ):
        return db2_table_mapping_rules.get_primary_key_columns(
            table=table,
        )

    def find_column(
        self,
        table,
        column_name: str,
    ):
        return db2_table_mapping_rules.find_column(
            table=table,
            column_name=column_name,
        )

    def build_table_lookup(
        self,
        model,
    ):
        return db2_table_mapping_rules.build_table_lookup(
            model=model,
        )

    def find_table_for_record(
        self,
        record_name: str,
        table_lookup,
    ):
        return db2_relationship_mapping_rules.find_table_for_record(
            record_name=record_name,
            table_lookup=table_lookup,
        )

    def normalize_table_name(
        self,
        value: str | None,
    ) -> str:
        return db2_name_mapping_rules.normalize_table_name(
            value=value,
        )

    def normalize_column_name(
        self,
        value: str | None,
    )

```

```

        record_name: str | None,
        field=None,
    ) -> str:
        return db2_name_mapping_rules.normalize_column_name(
            value=value,
            record_name=record_name,
            field=field,
        )

    def normal_business_column_name(
        self,
        field_name: str,
        record_name: str | None,
    ) -> str:
        return db2_name_mapping_rules.normal_business_column_name(
            field_name=field_name,
            record_name=record_name,
        )

    def date_column_name(
        self,
        field_name: str,
        record_name: str | None,
    ) -> str:
        return db2_name_mapping_rules.date_column_name(
            field_name=field_name,
            record_name=record_name,
        )

    def business_field_base_name(
        self,
        field_name: str,
    ) -> str:
        return db2_name_mapping_rules.business_field_base_name(
            field_name=field_name,
        )

    def date_field_base_name(
        self,
        field_name: str,
    ) -> str:
        return db2_name_mapping_rules.date_field_base_name(
            field_name=field_name,
        )

    def apply_date_project_abbreviations(
        self,
        parts: list[str],
    ) -> list[str]:
        return db2_name_mapping_rules.apply_date_project_abbreviations(
            parts=parts,
        )

    def remove_generic_trailing_tokens(
        self,
        parts: list[str],
    ) -> list[str]:
        return db2_name_mapping_rules.remove_generic_trailing_tokens(
            parts=parts,
        )

    def record_code(
        self,
        record_name: str | None,
    ) -> str:
        return db2_name_mapping_rules.record_code(
            record_name=record_name,
        )

    def is_date_field(
        self,
        field,
    ) -> bool:
        return db2_name_mapping_rules.is_date_field(
            field=field,
        )

```

```

def is_yyyymmdd_picture(
    self,
    picture: str,
) -> bool:
    return db2_name_mapping_rules.is_yyyymmdd_picture(
        picture=picture,
    )

def is_date_like_name(
    self,
    value: str,
) -> bool:
    return db2_name_mapping_rules.is_date_like_name(
        value=value,
    )

def is_date_part_name(
    self,
    value: str,
) -> bool:
    return db2_name_mapping_rules.is_date_part_name(
        value=value,
    )

def get_relationship_set_name(
    self,
    relationship,
) -> str:
    return db2_relationship_mapping_rules.get_relationship_set_name(
        relationship=relationship,
    )

def get_relationship_owner_record(
    self,
    relationship,
) -> str:
    return db2_relationship_mapping_rules.get_relationship_owner_record(
        relationship=relationship,
    )

def get_relationship_member_record(
    self,
    relationship,
) -> str:
    return db2_relationship_mapping_rules.get_relationship_member_record(
        relationship=relationship,
    )

def normalize_name(
    self,
    value: str | None,
) -> str:
    return db2_name_mapping_rules.normalize_name(
        value=value,
    )

def to_db2_name(
    self,
    value: str | None,
) -> str:
    return db2_name_mapping_rules.to_db2_name(
        value=value,
    )

def name_parts(
    self,
    value: str | None,
) -> list[str]:
    return db2_name_mapping_rules.name_parts(
        value=value,
    )

def remove_record_suffix(
    self,
    name: str | None,

```

```

) -> str:
    return db2_name_mapping_rules.remove_record_suffix(
        name=name,
    )

```

```

=====
FILE: src\idms_modernizer\services\db2_model_builder.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.canonical_models import CanonicalSchema
from idms_modernizer.domain.db2_models import DB2Model
from idms_modernizer.services.db2_mapping_service import DB2MappingService

```

```

class DB2ModelBuilder:

    def __init__(self):
        self.mapping_service = DB2MappingService()

    def build(
        self,
        schema: CanonicalSchema
    ) -> DB2Model:

        return self.mapping_service.build_db2_model(
            schema
        )

```

```

=====
FILE: src\idms_modernizer\services\db2_model_service.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.db2_models import (
    DB2Column,
    DB2Model,
    DB2Table
)

```

```

class DB2ModelService:

    def build(
        self,
        metadata
    ) -> DB2Model:

        tables = []

        for record in metadata.records:

            columns = []

            for field in record.fields:

                columns.append(
                    DB2Column(
                        name=field.name,
                        datatype="VARCHAR(255)"
                    )
                )

            tables.append(
                DB2Table(
                    name=record.name,
                    columns=columns,
                    foreign_keys=[]
                )
            )

        return DB2Model(
            tables=tables
        )

```



```
=====
FILE: src\idms_modernizer\services\db2_name_normalizer.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.services.name_normalizer import NameNormalizer
```

```
class DB2NameNormalizer:
```

```
    REMOVE_RECORD_SUFFIX = False
```

```
    @staticmethod
```

```
    def normalize_record_name(
        name: str | None
    ) -> str:
```

```
        return NameNormalizer.normalize(name)
```

```
    @staticmethod
```

```
    def normalize_column_name(
        name: str | None
    ) -> str:
```

```
        normalized = NameNormalizer.normalize(name)
```

```
        if not DB2NameNormalizer.REMOVE_RECORD_SUFFIX:
            return normalized
```

```
        parts = normalized.split("_")
```

```
        if (
            len(parts) > 1
            and parts[-1].isdigit()
            and len(parts[-1]) == 4
        ):
            return "_".join(parts[:-1])
```

```
        return normalized
```

```
=====
FILE: src\idms_modernizer\services\document_segmenter.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.domain.section_models import RecordSection
from idms_modernizer.rules.record_rules import (
    RECORD_PATTERN,
    parse_record_name,
)
```

```
class DocumentSegmenter:
```

```
    """
```

```
    Splits extracted schema document pages into record sections.
```

```
    Important:
```

- Uses permissive RECORD...NAME matching.
- Preserves old logic that accepted RECORDNAME / RECORD-NAME / RECORD NAME.
- Keeps every line under the active record until the next record starts.

```
    """
```

```
    RECORD_PATTERN = RECORD_PATTERN
```

```
    def segment(
        self,
        document,
```

```
    ) -> list[RecordSection]:
        sections: list[RecordSection] = []
        current_section: RecordSection | None = None
```

```
        for page in getattr(document, "pages", []) or []:
            for line in getattr(page, "lines", []) or []:
```

```

        text = getattr(line, "text", None)

        if text is None:
            text = str(line)

        text = str(text or "").strip()

        if not text:
            continue

        record_name = parse_record_name(text)

        if record_name:
            if current_section:
                sections.append(
                    current_section,
                )

            current_section = RecordSection(
                record_name=record_name,
                lines=[],
            )

            current_section.lines.append(
                text,
            )

            continue

        if current_section:
            current_section.lines.append(
                text,
            )

    if current_section:
        sections.append(
            current_section,
        )

    return sections

```

```

=====
FILE: src\idms_modernizer\services\excel_sheet_mapping_service.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.domain.db2_models import (
    DB2Model,
    DB2Table,
)
from idms_modernizer.domain.schema_models import SchemaMetadata

from idms_modernizer.rules import excel_basic_rules
from idms_modernizer.rules import excel_date_rules
from idms_modernizer.rules import excel_fk_rules
from idms_modernizer.rules import excel_key_rules
from idms_modernizer.rules import excel_lookup_rules
from idms_modernizer.rules import excel_mapping_builder
from idms_modernizer.rules import excel_name_rules
from idms_modernizer.rules import excel_row_rules

from idms_modernizer.rules.excel_mapping_constants import (
    COLUMNS,
    DATE_CHILD_TOKENS,
    DATE_TOKENS,
    DAY_PARTS,
    DATE_TRAILING_TOKENS,
    GENERIC_TRAILING_TOKENS,
    MONTH_PARTS,
    PROJECT_ABBREVIATIONS,
    YEAR_PARTS,
)

print("LOADED ExcelSheetMappingService VERSION MODULAR-FACADE-2026-08-31")

```

```

class ExcelSheetMappingService:
    """
    Builds Excel Sheet Mapping rows.

    This service is intentionally thin.

    Detailed logic is split across:
    - excel_mapping_constants.py
    - excel_basic_rules.py
    - excel_name_rules.py
    - excel_date_rules.py
    - excel_lookup_rules.py
    - excel_key_rules.py
    - excel_fk_rules.py
    - excel_row_rules.py
    - excel_extra_row_rules.py
    - excel_mapping_builder.py
    """

    COLUMNS = COLUMNS

    YEAR_PARTS = YEAR_PARTS
    MONTH_PARTS = MONTH_PARTS
    DAY_PARTS = DAY_PARTS

    DATE_TOKENS = DATE_TOKENS
    DATE_CHILD_TOKENS = DATE_CHILD_TOKENS
    DATE_TRAILING_TOKENS = DATE_TRAILING_TOKENS

    PROJECT_ABBREVIATIONS = PROJECT_ABBREVIATIONS
    GENERIC_TRAILING_TOKENS = GENERIC_TRAILING_TOKENS

    def build(
        self,
        metadata: SchemaMetadata,
        db2_model: DB2Model,
    ) -> list[dict[str, str]]:
        return excel_mapping_builder.build(
            metadata=metadata,
            db2_model=db2_model,
        )

    def build_record_mapping_rows(
        self,
        record,
        table: DB2Table | None,
        relationship_lookup: dict[str, str],
    ) -> list[dict[str, str]]:
        return excel_row_rules.build_record_mapping_rows(
            record=record,
            table=table,
            relationship_lookup=relationship_lookup,
        )

    def __getattr__(
        self,
        name: str,
    ):
        """
        Backward-compatible fallback for older calls.
        """

        modules = [
            excel_basic_rules,
            excel_name_rules,
            excel_date_rules,
            excel_lookup_rules,
            excel_key_rules,
            excel_fk_rules,
            excel_row_rules,
            excel_mapping_builder,
        ]

        for module in modules:

```

```

        if hasattr(module, name):
            return getattr(module, name)

    raise AttributeError(
        f"{self.__class__.__name__} object has no attribute {name!r}"
    )

```

```

=====
FILE: src\idms_modernizer\services\metadata_builder.py
ENCODING: utf-8
=====

```

```
import re
```

```

from idms_modernizer.domain.document_models import (
    DocumentModel
)

```

```

from idms_modernizer.domain.schema_models import (
    DataField,
    Record,
    SchemaMetadata,
)

```

```
class MetadataBuilder:
```

```

    RECORD_PATTERN = re.compile(
        r"RECORD\s+NAME.*?([A-Z][A-Z0-9-]+)",
        re.IGNORECASE
    )

```

```

    FIELD_PATTERN = re.compile(
        r"^\d{2}\s+([A-Z][A-Z0-9-]+)",
        re.IGNORECASE
    )

```

```

    def build(
        self,
        document: DocumentModel
    ) -> SchemaMetadata:

```

```
        metadata = SchemaMetadata()
```

```
        current_record = None
```

```
        seen_records = set()
```

```
        for page in document.pages:
```

```
            for line in page.lines:
```

```
                text = line.text.strip()
```

```
                if not text:
                    continue
```

```

                #
                # RECORD DETECTION
                #

```

```

                record_match = (
                    self.RECORD_PATTERN.search(
                        text
                    )
                )

```

```
                if record_match:
```

```

                    record_name = (
                        record_match.group(1)
                        .strip()
                        .upper()
                    )

```

```

        if (
            record_name
            not in seen_records
        ):

            current_record = Record(
                name=record_name
            )

            metadata.records.append(
                current_record
            )

            seen_records.add(
                record_name
            )

        continue

#
# FIELD DETECTION
#

if current_record:

    field_match = (
        self.FIELD_PATTERN.match(
            text
        )
    )

    if field_match:

        field_name = (
            field_match.group(1)
            .strip()
            .upper()
        )

        current_record.fields.append(
            DataField(
                name=field_name
            )
        )

#
# DEBUG OUTPUT
#

print(
    "\n===== RECORDS ====="
)

for record in metadata.records:

    print(
        f"{record.name} -> "
        f"{len(record.fields)} fields"
    )

print(
    "=====\n"
)

return metadata

```

```

=====
FILE: src\idms_modernizer\services\metadata_service.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.parsers.text_extractor import TextExtractor
from idms_modernizer.services.document_segmenter import DocumentSegmenter
from idms_modernizer.extractors.owner_member_extractor import OwnerMemberExtractor
from idms_modernizer.extractors.field_extractor import FieldExtractor

```

```

from idms_modernizer.extractors.cobol_zone_extractor import CobolZoneExtractor
from idms_modernizer.services.schema_picture_enricher import SchemaPictureEnricher
from idms_modernizer.services.relationship_resolver import RelationshipResolver
from idms_modernizer.domain.schema_models import SchemaMetadata, Record
from idms_modernizer.extractors.primary_key_extractor import PrimaryKeyExtractor

from idms_modernizer.rules.field_rules import FIELD_LINE_PATTERN
from idms_modernizer.rules.usage_rules import PIC_OR_USAGE_PATTERN

print("LOADED MetadataService VERSION MODULAR-RULES-ZONE-FIX-2026-08-31")

class MetadataService:
    """
    Builds SchemaMetadata from the schema PDF.

    Critical rule:
    - record.fields is used for physical DB2 / DDL / canonical schema.
    - record.mapping_fields is used for Excel Sheet Mapping.
    - BOTH must be enriched with:
      - PIC clause
      - datatype
      - length
      - scale
      - start_position
      - end_position
      - basetype

    Generic behavior:
    - No hardcoded record names.
    - No hardcoded field names.
    - No hardcoded business/application field names.
    - Debug logging is pattern-based, not field-name based.
    """

    DEBUG = True

    FIELD_LINE_PATTERN = FIELD_LINE_PATTERN
    PIC_OR_USAGE_PATTERN = PIC_OR_USAGE_PATTERN

    def __init__(self):
        self.extractor = TextExtractor()
        self.segmenter = DocumentSegmenter()

        self.membership_extractor = OwnerMemberExtractor()
        self.owner_member_extractor = self.membership_extractor

        self.field_extractor = FieldExtractor()

        self.zone_extractor = CobolZoneExtractor()
        self.cobol_zone_extractor = self.zone_extractor

        self.picture_enricher = SchemaPictureEnricher()
        self.relationship_resolver = RelationshipResolver()
        self.primary_key_extractor = PrimaryKeyExtractor()

    def build_metadata(
        self,
        pdf_path: str,
    ) -> SchemaMetadata:
        print(
            "USING MetadataService.build_metadata VERSION MODULAR-RULES-ZONE-FIX-2026-08-31"
        )

        document = self.extractor.extract_document(
            pdf_path,
        )

        sections = self.segmenter.segment(
            document,
        )

        if self.DEBUG:
            print(
                "METADATA_DOCUMENT_DEBUG",

```

```

        f"sections_count={len(sections or [])}",
    )

metadata = SchemaMetadata()

for section in sections or []:
    record = Record(
        name=section.record_name,
    )

    record.cobol_zone = self.zone_extractor.extract(
        section.lines,
    )

    physical_fields = self.field_extractor.extract(
        section.lines,
    )

    mapping_fields = self.field_extractor.extract_all(
        section.lines,
        include_filler=True,
    )

    if self.DEBUG:
        print("=" * 100)
        print(f"METADATA DEBUG RECORD: {record.name}")
        print("=" * 100)
        print(f"RAW SECTION LINE COUNT: {len(section.lines or [])}")
        print(
            f"PHYSICAL FIELD COUNT BEFORE ENRICH: "
            f"{len(physical_fields or [])}"
        )
        print(
            f"MAPPING FIELD COUNT BEFORE ENRICH: "
            f"{len(mapping_fields or [])}"
        )

        self.print_generic_raw_schema_lines(
            record_name=record.name,
            lines=section.lines,
        )

    record.fields = self.picture_enricher.enrich(
        fields=physical_fields,
        lines=section.lines,
        debug_label=f"{record.name} PHYSICAL",
    )

    record.mapping_fields = self.picture_enricher.enrich(
        fields=mapping_fields,
        lines=section.lines,
        debug_label=f"{record.name} MAPPING",
    )

    record.set_memberships = self.membership_extractor.extract(
        section.record_name,
        section.lines,
    )

    record.primary_key = self.primary_key_extractor.extract(
        section.lines,
    )

    if self.DEBUG:
        print("=" * 100)
        print(f"FIELDS EXTRACTED FOR {record.name} - PHYSICAL")
        print("=" * 100)

        for field in record.fields or []:
            self.print_field_debug(
                label="PHYSICAL_FIELD_DEBUG",
                field=field,
            )

        print("=" * 100)
        print(f"FIELDS EXTRACTED FOR {record.name} - MAPPING")

```

```

        print("=" * 100)

        for field in record.mapping_fields or []:
            self.print_field_debug(
                label="MAPPING_FIELD_DEBUG",
                field=field,
            )

        print(
            f"MetadataService: {record.name} "
            f"PK={record.primary_key} "
            f"ZONE={record.cobol_zone} "
            f"PHYSICAL_FIELDS={len(record.fields or [])} "
            f"MAPPING_FIELDS={len(record.mapping_fields or [])}"
        )

        metadata.records.append(
            record,
        )

    metadata.relationships = self.relationship_resolver.resolve(
        metadata,
    )

    if self.DEBUG:
        print(
            f"Relationships Found: {len(metadata.relationships)}"
        )

    return metadata

def print_generic_raw_schema_lines(
    self,
    record_name: str,
    lines,
) -> None:
    """
    Generic raw-line debug.

    Prints:
    - COBOL/IDMS data item lines such as 02 FIELD-NAME, 04 FIELD-NAME.
    - Lines containing PIC / PICTURE / DISPLAY / COMP / COMP-3.
    """

    print("=" * 100)
    print(f"RAW GENERIC SCHEMA LINES FOR {record_name}")
    print("=" * 100)

    for index, line in enumerate(lines or []):
        value = getattr(line, "text", None)

        if value is None:
            value = str(line)

        text = str(value or "").strip()

        if not text:
            continue

        is_field_line = self.FIELD_LINE_PATTERN.search(text) is not None
        is_pic_or_usage_line = (
            self.PIC_OR_USAGE_PATTERN.search(text) is not None
        )

        if is_field_line or is_pic_or_usage_line:
            print(
                "METADATA_RAW_SCHEMA_LINE",
                f"record={record_name}",
                f"index={index}",
                f"line={repr(text)}",
            )

def print_field_debug(
    self,
    label: str,
    field,

```



```

) -> None:
    print(
        label,
        "name=", getattr(field, "name", None),
        "level=", getattr(field, "level", None),
        "datatype=", getattr(field, "datatype", None),
        "length=", getattr(field, "length", None),
        "scale=", getattr(field, "scale", None),
        "picture=", getattr(field, "picture", None),
        "start=", getattr(field, "start_position", None),
        "end=", getattr(field, "end_position", None),
        "basetype=", getattr(field, "basetype", None),
        "group=", getattr(field, "is_group", None),
        "has_child=", getattr(field, "has_child", None),
    )

```

```

=====
FILE: src\idms_modernizer\services\name_normalizer.py
ENCODING: utf-8
=====

```

```
import re
```

```
class NameNormalizer:
```

```

    @staticmethod
    def normalize(
        name: str | None
    ) -> str:

        if not name:
            return ""

        normalized = name.strip().upper()
        normalized = normalized.replace("-", "_")
        normalized = re.sub(
            r"^[A-Z0-9_]",
            "_",
            normalized
        )
        normalized = re.sub(
            r"_+",
            "_",
            normalized
        )

        return normalized.strip("_")

```

```

=====
FILE: src\idms_modernizer\services\relationship_resolver.py
ENCODING: utf-8
=====

```

```
from collections import defaultdict
```

```

from idms_modernizer.domain.relationship_models import (
    Relationship,
)

```

```
class RelationshipResolver:
```

```

    """
    Resolves IDMS SET owner/member relationships from record set memberships.

    Generic behavior only:
    - No hardcoded SET names.
    - No hardcoded record names.
    - OWNER and MEMBER are read from schema listing memberships.
    """

```

```

    def resolve(
        self,
        metadata,

```

```

) -> list[Relationship]:
    owner_map: dict[str, list[str]] = defaultdict(list)
    member_map: dict[str, list[str]] = defaultdict(list)

    for record in getattr(metadata, "records", []) or []:
        record_name = getattr(record, "name", "") or ""

        if not record_name:
            continue

        for membership in getattr(record, "set_memberships", []) or []:
            set_name = (
                getattr(membership, "set_name", None)
                or getattr(membership, "name", None)
                or ""
            )

            role = (
                getattr(membership, "role", None)
                or getattr(membership, "relation_type", None)
                or getattr(membership, "type", None)
                or ""
            )

            set_name = str(set_name or "").strip().upper()
            role = str(role or "").strip().upper()

            if not set_name:
                continue

            if not role:
                continue

            if set_name == "CALC":
                continue

            if role == "OWNER":
                if record_name not in owner_map[set_name]:
                    owner_map[set_name].append(record_name)

            elif role == "MEMBER":
                if record_name not in member_map[set_name]:
                    member_map[set_name].append(record_name)

    relationships: list[Relationship] = []

    all_sets = set(owner_map.keys()) | set(member_map.keys())

    for set_name in sorted(all_sets):
        owners = owner_map.get(set_name, [])
        members = member_map.get(set_name, [])

        if not owners:
            continue

        if not members:
            continue

        for owner in owners:
            for member in members:
                if not owner or not member:
                    continue

                if owner == member:
                    continue

                relationships.append(
                    Relationship(
                        set_name=set_name,
                        owner_record=owner,
                        member_record=member,
                        cardinality="1:N",
                    )
                )

    metadata.relationships = relationships

```

```
return relationships
```

```
=====
FILE: src\idms_modernizer\services\schema_picture_enricher.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.domain.schema_models import DataField

from idms_modernizer.rules import picture_rules
from idms_modernizer.rules import schema_enrichment_rules
from idms_modernizer.rules import schema_field_enrichment_rules
from idms_modernizer.rules import schema_group_position_rules
from idms_modernizer.rules.schema_field_factory_rules import (
    copy_field_with_defaults,
    make_field,
)
from idms_modernizer.rules.field_rules import FIELD_NAME_PATTERN
from idms_modernizer.rules.usage_rules import USAGE_PATTERN
from idms_modernizer.rules.picture_rules import (
    START_LENGTH_PATTERN,
    COMPACT_PIC_PATTERNS,
)

print("LOADED SchemaPictureEnricher VERSION MODULAR-SMALL-FILES-2026-08-31")
```

```
class SchemaPictureEnricher:
    """
    Enriches DataField objects using schema listing lines.

    Critical rules:
    - IDMS PIC Clause must come from schema PICTURE, not byte length.
    - PIC S9(13)V9(2) must remain PIC S9(13)V9(2).
    - PIC S9(13)V9(2) COMP-3 must remain PIC S9(13)V9(2) COMP-3.
    - Storage length remains byte length.
    - Numeric logical precision comes from PIC.

    This service is intentionally thin.
    The detailed parsing/enrichment logic lives in rules modules.
    """

    DEBUG = True
    DEBUG_VERBOSE = False

    FIELD_START_PATTERN = FIELD_NAME_PATTERN
    USAGE_PATTERN = USAGE_PATTERN
    START_LENGTH_PATTERN = START_LENGTH_PATTERN
    COMPACT_PIC_PATTERNS = COMPACT_PIC_PATTERNS

    def enrich(
        self,
        fields: list[DataField],
        lines: list[str],
        debug_label: str = "",
    ) -> list[DataField]:
        if self.DEBUG:
            print(
                "USING SchemaPictureEnricher.enrich VERSION MODULAR-SMALL-FILES-2026-08-31",
                f"debug_label={debug_label}",
            )

        normalized_lines = self.normalize_lines(
            lines=lines,
        )

        if self.DEBUG:
            print(
                "SCHEMA_ENRICH_DEBUG_LINES",
                f"debug_label={debug_label}",
                f"line_count={len(normalized_lines)}",
            )
```

```

field_blocks = self.build_field_blocks(
    normalized_lines=normalized_lines,
    debug_label=debug_label,
)

enriched_fields: list[DataField] = []

for field in fields or []:
    field_name = getattr(field, "name", "") or ""

    block = self.find_block_for_field(
        field_name=field_name,
        field_blocks=field_blocks,
    )

    if not block:
        if self.DEBUG:
            print(
                "SCHEMA_ENRICH_NO_BLOCK",
                f"debug_label={debug_label}",
                f"field={repr(field_name)}",
            )

            enriched_fields.append(
                self.copy_field_with_defaults(
                    field=field,
                )
            )
            continue

        enriched_fields.append(
            self.enrich_field(
                field=field,
                block=block,
                debug_label=debug_label,
            )
        )

    return self.enrich_group_positions_from_children(
        fields=enriched_fields,
        debug_label=debug_label,
    )

def normalize_lines(
    self,
    lines: list[str],
) -> list[str]:
    return schema_enrichment_rules.normalize_schema_lines(
        lines=lines,
    )

def build_field_blocks(
    self,
    normalized_lines: list[str],
    debug_label: str = "",
) -> dict[str, str]:
    blocks = schema_enrichment_rules.build_field_blocks(
        normalized_lines=normalized_lines,
    )

    if self.DEBUG:
        print(
            "SCHEMA_BLOCK_STARTS_DEBUG",
            f"debug_label={debug_label}",
            f"starts_count={len(blocks)}",
        )

    if self.DEBUG_VERBOSE:
        for field_name, block in blocks.items():
            print(
                "SCHEMA_BLOCK_DEBUG",
                f"debug_label={debug_label}",
                f"field={repr(field_name)}",
                f"block={repr(block)}",
            )

```

```

        return blocks

def find_block_for_field(
    self,
    field_name: str,
    field_blocks: dict[str, str],
) -> str:
    return schema_enrichment_rules.find_block_for_field(
        field_name=field_name,
        field_blocks=field_blocks,
    )

def enrich_field(
    self,
    field: DataField,
    block: str,
    debug_label: str = "",
) -> DataField:
    return schema_field_enrichment_rules.enrich_field(
        field=field,
        block=block,
        debug_label=debug_label,
        debug_verbose=self.DEBUG_VERBOSE,
        compact_pic_patterns=self.COMPACT_PIC_PATTERNS,
    )

def parse_schema_block(
    self,
    block: str,
    field_name: str,
    debug_label: str = "",
) -> dict:
    return schema_field_enrichment_rules.parse_schema_block(
        block=block,
        field_name=field_name,
        debug_label=debug_label,
        debug_verbose=self.DEBUG_VERBOSE,
        compact_pic_patterns=self.COMPACT_PIC_PATTERNS,
    )

def extract_picture_core(
    self,
    value: str,
    field_name: str = "",
    debug_label: str = "",
) -> str | None:
    return schema_field_enrichment_rules.extract_picture_core(
        value=value,
        field_name=field_name,
        debug_label=debug_label,
        debug_verbose=self.DEBUG_VERBOSE,
        compact_pic_patterns=self.COMPACT_PIC_PATTERNS,
    )

def render_cobol_pic_clause(
    self,
    picture: str | None,
    usage: str,
    storage_length: int | None,
) -> str | None:
    return picture_rules.render_cobol_pic_clause(
        picture=picture,
        usage=usage,
        storage_length=storage_length,
    )

def derive_type_from_picture(
    self,
    picture: str | None,
    usage: str,
    storage_length: int | None,
    fallback_datatype: str | None = None,
) -> tuple[str | None, int | None, int | None]:
    return picture_rules.derive_type_from_picture(
        picture=picture,
        usage=usage,

```

```

        storage_length=storage_length,
        fallback_datatype=fallback_datatype,
    )

def calculate_end_position(
    self,
    start_position: int | None,
    storage_length: int | None,
    fallback=None,
) -> int | None:
    return schema_enrichment_rules.calculate_end_position(
        start_position=start_position,
        storage_length=storage_length,
        fallback=fallback,
    )

def is_date_field(
    self,
    field: DataField,
    picture: str | None,
    storage_length: int | None,
    datatype: str | None,
) -> bool:
    return schema_enrichment_rules.is_date_field(
        field=field,
        picture=picture,
        storage_length=storage_length,
        datatype=datatype,
    )

def basetype_for_datatype(
    self,
    datatype: str | None,
) -> str | None:
    return schema_enrichment_rules.basetype_for_datatype(
        datatype=datatype,
    )

def enrich_group_positions_from_children(
    self,
    fields: list[DataField],
    debug_label: str = "",
) -> list[DataField]:
    return schema_group_position_rules.enrich_group_positions_from_children(
        fields=fields,
    )

def copy_field_with_defaults(
    self,
    field: DataField,
) -> DataField:
    return copy_field_with_defaults(
        field=field,
    )

def make_field(
    self,
    source: DataField,
    datatype: str | None,
    length: int | None,
    scale: int | None,
    picture: str | None,
    start_position: int | None,
    end_position: int | None,
    basetype: str | None,
) -> DataField:
    return make_field(
        source=source,
        datatype=datatype,
        length=length,
        scale=scale,
        picture=picture,
        start_position=start_position,
        end_position=end_position,
        basetype=basetype,
    )

```

```

def remove_record_suffix(
    self,
    value: str,
) -> str:
    return schema_enrichment_rules.remove_record_suffix(
        value=value,
    )

def extract_usage(
    self,
    block: str,
) -> str:
    return schema_enrichment_rules.extract_usage(
        block=block,
    )

def extract_start_and_length(
    self,
    block: str,
) -> tuple[int | None, int | None]:
    return schema_enrichment_rules.extract_start_and_length(
        block=block,
    )

def remove_trailing_start_and_length(
    self,
    value: str,
) -> str:
    return schema_enrichment_rules.remove_trailing_start_and_length(
        value=value,
    )

def remove_usage_words(
    self,
    value: str,
) -> str:
    return schema_enrichment_rules.remove_usage_words(
        value=value,
    )

def remove_schema_noise(
    self,
    value: str,
) -> str:
    return schema_enrichment_rules.remove_schema_noise(
        value=value,
    )

def picture_core(
    self,
    picture: str | None,
) -> str:
    return picture_rules.picture_core(
        picture=picture,
    )

def picture_is_character(
    self,
    picture: str,
) -> bool:
    return picture_rules.picture_is_character(
        picture=picture,
    )

def character_length(
    self,
    picture: str,
) -> int | None:
    return picture_rules.character_length(
        picture=picture,
    )

def picture_is_numeric(
    self,
    picture: str,

```

```

) -> bool:
    return picture_rules.picture_is_numeric(
        picture=picture,
    )

def picture_is_decimal(
    self,
    picture: str,
) -> bool:
    return picture_rules.picture_is_decimal(
        picture=picture,
    )

def decimal_precision_scale(
    self,
    picture: str,
) -> tuple[int, int]:
    return picture_rules.decimal_precision_scale(
        picture=picture,
    )

def numeric_precision(
    self,
    picture: str,
) -> int:
    return picture_rules.numeric_precision(
        picture=picture,
    )

def count_9_digits(
    self,
    value: str,
) -> int:
    return picture_rules.count_numeric_digits(
        value=value,
    )

def extract_parenthesized_int(
    self,
    value: str,
) -> int | None:
    return picture_rules.extract_parenthesized_int(
        value=value,
    )

```

```

=====
FILE: src\idms_modernizer\services\validation_service.py
ENCODING: utf-8
=====

```

```

class ValidationService:

    def validate_metadata(
        self,
        metadata
    ) -> list[str]:

        errors = []

        record_names = {
            record.name
            for record in metadata.records
        }

        for rel in metadata.relationships:

            if rel.owner_record not in record_names:

                errors.append(
                    f"Missing Owner Record: {rel.owner_record}"
                )

            if rel.member_record not in record_names:

                errors.append(

```



```

        f"Missing Member Record: {rel.member_record}"
    )

    return errors

```

```

=====
FILE: src\idms_modernizer\tests\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_modernizer\tests\test_date_field_consolidator.py
ENCODING: utf-8
=====

```

```

from idms_modernizer.services.date_field_consolidator import (
    DateFieldConsolidator
)

```

```

from idms_modernizer.domain.schema_models import (
    DataField
)

```

```

def test Consolidate_year_month_day():
    fields = [
        DataField(name="START_YEAR"),
        DataField(name="START_MONTH"),
        DataField(name="START_DAY")
    ]

    out = DateFieldConsolidator.Consolidate(fields)

    assert len(out) == 1
    assert out[0].name == "START_DATE"
    assert out[0].datatype == "DATE"

```

```

def test Consolidate_variants():
    fields = [
        DataField(name="FINISH_YR"),
        DataField(name="FINISH_MO"),
        DataField(name="FINISH_DY")
    ]

    out = DateFieldConsolidator.Consolidate(fields)

    assert len(out) == 1
    assert out[0].name == "FINISH_DATE"
    assert out[0].datatype == "DATE"

```

```

def test Preserve_existing_full_date():
    fields = [
        DataField(name="BIRTH_DATE", datatype="DATE"),
        DataField(name="BIRTH_YEAR"),
        DataField(name="BIRTH_MONTH")
    ]

    out = DateFieldConsolidator.Consolidate(fields)

    # Should keep the existing BIRTH_DATE and not create duplicate
    names = [f.name for f in out]
    assert "BIRTH_DATE" in names
    assert all(n.startswith("BIRTH_") for n in names)

```

```

def test Keep_year_only():
    fields = [
        DataField(name="CLAIM_YEAR")
    ]

```

```

out = DateFieldConsolidator.consolidate(fields)

assert len(out) == 1
assert out[0].name == "CLAIM_YEAR"

=====
FILE: src\idms_modernizer\tests\test_diagram_parser.py
ENCODING: utf-8
=====

from idms_modernizer.parsers.pdf_diagram_parser import (
    PdfDiagramParser
)

def test_diagram_parser():

    parser = (
        PdfDiagramParser()
    )

    relationships = (
        parser.parse(
            """
            DEPARTMENT
            EMPLOYEE
            """
        )
    )

    assert (
        len(relationships)
        == 1
    )

    assert (
        relationships[0].owner
        == "DEPARTMENT"
    )

=====
FILE: src\idms_modernizer\tests\test_field_extractor.py
ENCODING: utf-8
=====

import re
import sys
from dataclasses import dataclass, asdict
from typing import List, Optional, Set

# ----- PDF READER ----- #
def read_pdf_lines(pdf_path: str) -> List[str]:
    """
    Extract text lines from a PDF file using pdfminer.six.
    Returns a list of lines.
    """
    try:
        from pdfminer.high_level import extract_text
    except ImportError as e:
        raise RuntimeError(
            "pdfminer.six is required. Install it with:\n\n"
            "    pip install pdfminer.six\n"
        ) from e

    text = extract_text(pdf_path) or ""
    # Split into lines while preserving basic line structure
    lines = text.splitlines()
    # Trim trailing spaces for consistency
    return [ln.rstrip("\r\n") for ln in lines]

# ----- DATA TYPES ----- #
@dataclass(frozen=True)

```

```

class DataField:
    name: str
    datatype: str
    length: Optional[int]
    scale: Optional[int]
    picture: str

# ----- EXTRACTOR ----- #
class FieldExtractor:
    """
    Robust parser for IDMS-like listing lines.

    - Extracts PICTURE regardless of position and with/without PIC/PICTURE keyword
    - Recognizes DISPLAY, COMP, COMP-3 in any order relative to PICTURE
    - Skips FILLER items by default (configurable)
    - Removes duplicate names
    - Maps:
      * X -> CHAR(1)
      * X(n) -> VARCHAR(n)
      * DISPLAY + 9/S9[/ (n)] -> INTEGER(length = digits)
      * COMP/COMP-3 + 9/S9[/ (n)] -> DECIMAL(precision = digits)
      * S9(n)V9(m)/9(n)V9(m) -> DECIMAL(precision = n+m, scale = m)
    """

    def __init__(
        self,
        require_explicit_length: bool = False,
        include_filler: bool = False,
        auto_number_filler: bool = True,
    ):
        self.require_explicit_length = require_explicit_length
        self.include_filler = include_filler
        self.auto_number_filler = auto_number_filler

    # Start: <level> <name>
    _R_START = re.compile(r"^\s*(\d{2})\s+([A-Z0-9\-\+])\b", re.IGNORECASE)

    # USAGE anywhere (allow "USAGE IS ...")
    _R_USAGE = re.compile(r"\b(?:USAGE\s+IS\s+)?(DISPLAY|COMP-3|COMP)\b", re.IGNORECASE)

    # PICTURE anywhere; optional PIC/PICTURE keyword; tolerate spaces; no trailing \b
    _R_PIC_ANYWHERE = re.compile(
        r"""
        (?:\b(?:PIC|PICTURE)\s*)?
        (
            S\s*9\s*$\s*\d+\s*$\s*V\s*9\s*$\s*\d+\s*$ | # S9(n)V9(m) / 9(n)V9(m)
            S\s*9\s*$\s*\d+\s*$ | # S9(n)
            9\s*$\s*\d+\s*$ | # 9(n)
            S\s*9 | # S9
            9 | # 9
            X\s*$\s*\d+\s*$ | # X(n)
            X | # X
        )
        """,
        re.IGNORECASE | re.VERBOSE,
    )

    # Helpers (operate on normalized picture like X(30), 9(7), S9(7)V9(2))
    _R_CHAR = re.compile(r"^\s*X(\d+)\s*$", re.IGNORECASE) # X(n)
    _R_INT = re.compile(r"^\s*9(?:$|(\d+)\s*)?\s*$", re.IGNORECASE) # 9 / 9(n) / S9 / S9(n)
    _R_DEC = re.compile(r"^\s*9$|(\d+)\s*V9$|(\d+)\s*$", re.IGNORECASE) # S9(n)V9(m) / 9(n)V9(m)

    def extract(self, lines: List[str]) -> List[DataField]:
        seen: Set[str] = set()
        out: List[DataField] = []
        filler_idx = 0

        for raw in lines:
            line = raw.strip()
            if not line:
                continue

            m_start = self._R_START.match(line)
            if not m_start:
                continue

```

```

_, name = m_start.groups()
name_up = name.upper()

# Handle FILLER items
if name_up.startswith("FILLER"):
    if not self.include_filler:
        continue
    if self.auto_number_filler:
        filler_idx += 1
        name_up = f"FILLER_{filler_idx}"

if name_up in seen:
    continue

# USAGE default to DISPLAY if absent
m_usage = self._R_USAGE.search(line)
usage = m_usage.group(1).upper() if m_usage else "DISPLAY"

# PICTURE capture
m_pic = self._R_PIC_ANYWHERE.search(line)
if not m_pic:
    continue

# Normalize: remove spaces inside the picture and uppercase
pic = re.sub(r"\s+", "", m_pic.group(1).upper())

# Optional filter: require explicit length (skip X, 9, S9)
if self.require_explicit_length and pic in {"X", "9", "S9"}:
    continue

dtype: str = "VARCHAR"
length: Optional[int] = None
scale: Optional[int] = None

# Character
if pic == "X":
    dtype, length = "CHAR", 1
elif self._R_CHAR.fullmatch(pic):
    length = int(self._R_CHAR.fullmatch(pic).group(1))
    dtype = "VARCHAR"

# Decimal with V
elif self._R_DEC.fullmatch(pic):
    int_len, frac_len = map(int, self._R_DEC.fullmatch(pic).groups())
    length = int_len + frac_len
    scale = frac_len
    dtype = "DECIMAL"

# Integer-like 9/S9 with optional (n)
elif self._R_INT.fullmatch(pic):
    digits_str = self._R_INT.fullmatch(pic).group(1)
    digits = int(digits_str) if digits_str else 1
    length = digits
    dtype = "INTEGER" if usage == "DISPLAY" else "DECIMAL"
else:
    # Unrecognized picture; skip
    continue

seen.add(name_up)
out.append(
    DataField(
        name=name_up,
        datatype=dtype,
        length=length,
        scale=scale,
        picture=pic,
    )
)
)

return out

```

```

# ----- DEMO (PDF PATH) ----- #
if __name__ == "__main__":
    # Option A: Provide the PDF file path here (edit this string)

```

```
pdf_path = r"C:\VSCode\idms-db2-modernizer-master\idms-db2-modernizer-master\src\idms_modernizer\data\IDMS_Schema_Listing.pdf"
```

```
# Option B: Or pass the path as first CLI argument
if len(sys.argv) > 1:
    pdf_path = sys.argv[1]

# Read lines from PDF
lines = read_pdf_lines(pdf_path)

# Configure extractor as needed
extractor = FieldExtractor(
    require_explicit_length=False, # set True to skip X / 9 / S9 without (n)
    include_filler=False,         # set True to include FILLER items
    auto_number_filler=True
)

fields = extractor.extract(lines)
for f in fields:
    print(asdict(f))
```

```
=====
FILE: src\idms_modernizer\tests\test_schema_parser.py
ENCODING: utf-8
=====
```

```
from idms_modernizer.parsers.pdf_schema_parser import (
    PdfSchemaParser
)
```

```
def test_schema_parser():
```

```
    sample_text = """
```

```
    RECORD NAME..... EMPLOYEE
```

```
    EMP-ID-0415
```

```
    EMP-FIRST-NAME-0415
```

```
    EMP-LAST-NAME-0415
```

```
    RECORD NAME..... DEPARTMENT
```

```
    DEPT-ID-0416
```

```
    DEPT-NAME-0416
```

```
    """
```

```
    parser = (
        PdfSchemaParser()
    )
```

```
    metadata = (
        parser.parse(
            sample_text
        )
    )
```

```
    assert (
        len(metadata.records)
        == 2
    )
```

```
    assert (
        metadata.records[0].name
        == "EMPLOYEE"
    )
```

```
    assert (
        len(
            metadata.records[0].fields
        )
    )
```

```
    == 3
)
```

```
=====
FILE: src\idms_modernizer\ui\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_modernizer\ui\schema_listing_only_app.py
ENCODING: utf-8
=====
```

```
from pathlib import Path
import sys
from tempfile import NamedTemporaryFile

CURRENT_FILE = Path(__file__).resolve()
SRC_ROOT = CURRENT_FILE.parents[2]

if str(SRC_ROOT) not in sys.path:
    sys.path.insert(
        0,
        str(SRC_ROOT),
    )

import streamlit as st

from idms_modernizer.core.config import settings
from idms_modernizer.core.utils import ensure_directory
from idms_modernizer.services.metadata_service import MetadataService
from idms_modernizer.services.canonical_schema_builder import (
    CanonicalSchemaBuilder,
)
from idms_modernizer.services.canonical_reconciler import (
    CanonicalReconciler,
)
from idms_modernizer.services.db2_model_builder import DB2ModelBuilder
from idms_modernizer.generators.ddl_generator import DDLGenerator
from idms_modernizer.services.excel_sheet_mapping_service import (
    ExcelSheetMappingService,
)

SUPPORTED_SCHEMA_EXTENSIONS = [
    "pdf",
    "txt",
    "text",
    "log",
    "cbl",
    "cpy",
]

def initialize_directories() -> None:
    ensure_directory(
        settings.output_dir,
    )

def initialize_session_state() -> None:
    defaults = {
        "schema_only_generated": False,
        "schema_only_error": "",
        "schema_only_schema_path": "",
        "schema_only_metadata": None,
        "schema_only_canonical_schema": None,
        "schema_only_db2_model": None,
        "schema_only_ddl_text": "",
        "schema_only_excel_mapping_rows": [],
    }
```

```

for key, value in defaults.items():
    if key not in st.session_state:
        st.session_state[key] = value

def reset_schema_only_outputs() -> None:
    st.session_state.schema_only_generated = False
    st.session_state.schema_only_error = ""
    st.session_state.schema_only_schema_path = ""
    st.session_state.schema_only_metadata = None
    st.session_state.schema_only_canonical_schema = None
    st.session_state.schema_only_db2_model = None
    st.session_state.schema_only_ddl_text = ""
    st.session_state.schema_only_excel_mapping_rows = []

def save_uploaded_schema_file(
    uploaded_file,
) -> str:
    suffix = Path(
        uploaded_file.name,
    ).suffix

    if not suffix:
        suffix = ".txt"

    with NamedTemporaryFile(
        delete=False,
        suffix=suffix,
    ) as temp_file:
        temp_file.write(
            uploaded_file.getbuffer(),
        )

        return temp_file.name

def generate_schema_outputs(
    schema_file,
) -> None:
    reset_schema_only_outputs()

    try:
        schema_path = save_uploaded_schema_file(
            uploaded_file=schema_file,
        )

        metadata_service = MetadataService()

        metadata = metadata_service.build_metadata(
            pdf_path=schema_path,
        )

        canonical_builder = CanonicalSchemaBuilder()

        canonical_schema = canonical_builder.build(
            metadata=metadata,
        )

        canonical_reconciler = CanonicalReconciler()

        canonical_schema = canonical_reconciler.reconcile(
            canonical_schema=canonical_schema,
            metadata=metadata,
        )

        db2_builder = DB2ModelBuilder()

        db2_model = db2_builder.build(
            canonical_schema,
        )

        ddl_generator = DDLGenerator()

        ddl_text = ddl_generator.generate(
            db2_model,

```

```

    )

    excel_mapping_service = ExcelSheetMappingService()

    excel_mapping_rows = excel_mapping_service.build(
        metadata=metadata,
        db2_model=db2_model,
    )

    st.session_state.schema_only_generated = True
    st.session_state.schema_only_error = ""
    st.session_state.schema_only_schema_path = schema_path
    st.session_state.schema_only_metadata = metadata
    st.session_state.schema_only_canonical_schema = canonical_schema
    st.session_state.schema_only_db2_model = db2_model
    st.session_state.schema_only_ddl_text = ddl_text
    st.session_state.schema_only_excel_mapping_rows = excel_mapping_rows

except Exception as error:
    st.session_state.schema_only_generated = False
    st.session_state.schema_only_error = str(
        error,
    )
)

def build_mapping_row_count_by_table(
    rows,
    include_total: bool = True,
) -> list[dict[str, int | str]]:
    output_rows: list[dict[str, int | str]] = []

    if not rows:
        return output_rows

    counts: dict[str, int] = {}

    for row in rows:
        table_name = (
            row.get("New DB2 Record")
            or row.get("Cobol Record IDMS")
            or ""
        )

        table_name = str(
            table_name,
        ).strip()

        if not table_name:
            table_name = "UNMAPPED"

        counts[table_name] = counts.get(
            table_name,
            0,
        ) + 1

    total_count = 0

    for table_name in sorted(counts):
        count = counts[table_name]
        total_count += count

        output_rows.append(
            {
                "Mapping Row Count": count,
                "Table Name": table_name,
            }
        )

    if include_total:
        output_rows.append(
            {
                "Mapping Row Count": total_count,
                "Table Name": "TOTAL",
            }
        )
)

```



```

return output_rows

def build_db2_physical_column_count_rows(
    db2_model,
    include_total: bool = True,
) -> list[dict[str, int | str]]:
    table_rows: list[dict[str, int | str]] = []

    if db2_model is None:
        return table_rows

    total_count = 0

    for table in getattr(
        db2_model,
        "tables",
        [],
    ) or []:
        table_name = getattr(
            table,
            "name",
            "",
        )

        columns = getattr(
            table,
            "columns",
            [],
        ) or []

        count = len(
            columns,
        )

        table_rows.append(
            {
                "Column Field Count": count,
                "Table Name": table_name,
            }
        )

        total_count += count

    if include_total:
        table_rows.append(
            {
                "Column Field Count": total_count,
                "Table Name": "TOTAL",
            }
        )

    return table_rows

def render_main_tab() -> None:
    st.markdown(
        "## Schema Listing Only"
    )

    st.info(
        "Upload a Schema Listing file to generate Metadata Overview, "
        "DB2 DDL, and Excel Sheet Mapping. Supported formats: PDF and TXT."
    )

    schema_file = st.file_uploader(
        "Upload Schema Listing",
        type=SUPPORTED_SCHEMA_EXTENSIONS,
        key="schema_only_schema_listing_file",
    )

    generate_clicked = st.button(
        "Generate Schema Outputs",
        type="primary",
        use_container_width=True,
    )

```

```

if generate_clicked:
    if schema_file is None:
        st.error(
            "Please upload a Schema Listing PDF or TXT file."
        )
        return

    with st.spinner(
        "Processing schema listing and generating outputs ..."
    ):
        generate_schema_outputs(
            schema_file=schema_file,
        )

    if st.session_state.schema_only_generated:
        st.success(
            "Schema outputs generated successfully."
        )
    else:
        st.error(
            "Schema output generation failed."
        )

        if st.session_state.schema_only_error:
            st.error(
                st.session_state.schema_only_error,
            )

if st.session_state.schema_only_generated:
    st.markdown(
        "### Output Status"
    )

    metadata = st.session_state.schema_only_metadata
    db2_model = st.session_state.schema_only_db2_model
    mapping_rows = st.session_state.schema_only_excel_mapping_rows

    col1, col2, col3, col4 = st.columns(
        4,
    )

    with col1:
        st.metric(
            "Records",
            len(
                metadata.records,
            )
            if metadata is not None
            else 0,
        )

    with col2:
        st.metric(
            "Relationships",
            len(
                metadata.relationships,
            )
            if metadata is not None
            else 0,
        )

    with col3:
        st.metric(
            "DB2 Tables",
            len(
                db2_model.tables,
            )
            if db2_model is not None
            else 0,
        )

    with col4:
        st.metric(
            "Sheet Mapping Rows",
            len(

```

```

        mapping_rows,
    )
    if mapping_rows is not None
    else 0,
)

def render_metadata_overview_tab() -> None:
    if not st.session_state.schema_only_generated:
        st.info(
            "Generate outputs from the Main tab first."
        )
        return

    st.markdown(
        "## Metadata Overview"
    )

    metadata = st.session_state.schema_only_metadata
    db2_model = st.session_state.schema_only_db2_model

    if metadata is None:
        st.info(
            "No metadata generated."
        )
        return

    st.markdown(
        "### DB2 Physical Column Counts"
    )

    physical_column_count_rows = build_db2_physical_column_count_rows(
        db2_model=db2_model,
        include_total=True,
    )

    if physical_column_count_rows:
        st.dataframe(
            physical_column_count_rows,
            use_container_width=True,
            hide_index=True,
        )
    else:
        st.info(
            "No DB2 physical column counts available."
        )

    st.caption(
        "This count is based on DB2 physical columns. It can be lower than "
        "the mapping row count when COBOL YEAR, MONTH, and DAY fields map "
        "to one DB2 DATE column."
    )

    st.divider()

    st.markdown(
        "### Records and Fields"
    )

    record_names = [
        record.name
        for record in metadata.records
    ]

    if not record_names:
        st.info(
            "No records found."
        )
        return

    selected_record = st.selectbox(
        "Select a record",
        options=record_names,
    )

    selected_record_obj = None

```

```

for record in metadata.records:
    if record.name == selected_record:
        selected_record_obj = record
        break

if selected_record_obj is not None:
    st.write(
        f"Fields found: `{len(selected_record_obj.fields)}`"
    )

    field_rows = []

    for field in selected_record_obj.fields:
        field_rows.append(
            {
                "Field": field.name,
                "Level": field.level,
                "Datatype": field.datatype,
                "Length": field.length,
                "Scale": field.scale,
                "Picture": field.picture,
                "Start Position": field.start_position,
                "End Position": field.end_position,
                "Basetype": field.basetype,
            }
        )

    st.dataframe(
        field_rows,
        use_container_width=True,
        hide_index=True,
    )

with st.expander(
    "View all records and fields as JSON",
    expanded=False,
):
    st.json(
        [
            {
                "record": record.name,
                "cobol_zone": record.cobol_zone,
                "primary_key": record.primary_key,
                "fields": [
                    field.model_dump()
                    for field in record.fields
                ],
                "set_memberships": [
                    {
                        "set_name": getattr(
                            membership,
                            "set_name",
                            "",
                        ),
                        "role": getattr(
                            membership,
                            "role",
                            "",
                        ),
                        "relation_type": getattr(
                            membership,
                            "relation_type",
                            "",
                        ),
                        "owner_record": getattr(
                            membership,
                            "owner_record",
                            "",
                        ),
                        "member_record": getattr(
                            membership,
                            "member_record",
                            "",
                        ),
                    }
                ]
            }
        ]
    )

```

```

        for membership in record.set_memberships
    ],
    }
    for record in metadata.records
]
)

def render_ddl_tab() -> None:
    if not st.session_state.schema_only_generated:
        st.info(
            "Generate outputs from the Main tab first."
        )
        return

    st.markdown(
        "## DB2 DDL"
    )

    ddl_text = st.session_state.schema_only_ddl_text

    if not ddl_text:
        st.info(
            "No DDL generated."
        )
        return

    st.code(
        ddl_text,
        language="sql",
    )

def render_excel_sheet_mapping_tab() -> None:
    if not st.session_state.schema_only_generated:
        st.info(
            "Generate outputs from the Main tab first."
        )
        return

    st.markdown(
        "## Excel Sheet Mapping"
    )

    rows = st.session_state.schema_only_excel_mapping_rows

    if not rows:
        st.info(
            "No mapping rows generated."
        )
        return

    st.markdown(
        "### Sheet Map Abstract"
    )

    abstract_rows = build_mapping_row_count_by_table(
        rows=rows,
        include_total=True,
    )

    if abstract_rows:
        st.dataframe(
            abstract_rows,
            use_container_width=True,
            hide_index=True,
        )
    else:
        st.info(
            "No table abstract rows available."
        )

    st.caption(
        "The TOTAL row matches the number of detailed mapping rows below."
    )

```

```

st.markdown(
    "### Sheet Mapping Details"
)

st.dataframe(
    rows,
    use_container_width=True,
    hide_index=True,
)

def main() -> None:
    st.set_page_config(
        page_title="Schema Listing Mapping",
        layout="wide",
    )

    initialize_directories()
    initialize_session_state()

    st.title(
        "Schema Listing Mapping"
    )

    st.caption(
        "Upload Schema Listing PDF or TXT and generate DB2 DDL plus "
        "Excel-style mapping."
    )

    tabs = st.tabs(
        [
            "Main",
            "Metadata Overview",
            "DB2 DDL",
            "Excel Sheet Mapping",
        ]
    )

    with tabs[0]:
        render_main_tab()

    with tabs[1]:
        render_metadata_overview_tab()

    with tabs[2]:
        render_ddl_tab()

    with tabs[3]:
        render_excel_sheet_mapping_tab()

if __name__ == "__main__":
    main()

```

```

=====
FILE: src\idms_modernizer\utils\__init__.py
ENCODING: utf-8
=====

```

```

"""
Utility helpers for IDMS DB2 modernization.

Keep this package focused on generic helpers only.
"""

```

```

=====
FILE: src\idms_modernizer\utils\safe_cast.py
ENCODING: utf-8
=====

```

```

def safe_int(value, default: int | None = None) -> int | None:
    try:
        if value is None:
            return default

```

```

        text = str(value).strip()

        if not text:
            return default

        return int(text)
    except Exception:
        return default

def first_present(*values):
    for value in values:
        if value is None:
            continue

        if isinstance(value, str) and not value.strip():
            continue

        return value

    return None

def stringify(value) -> str:
    if value is None:
        return ""

    return str(value)

def safe_upper(value) -> str:
    return stringify(value).strip().upper()

=====
FILE: src\idms_modernizer\utils\text_utils.py
ENCODING: utf-8
=====

import re

def normalize_text(text: str | None) -> str:
    return " ".join(str(text or "").split())

def clean_line(line: str | None) -> str:
    if not line:
        return ""

    cleaned = str(line).strip()
    cleaned = re.sub(r"<[^>]+>", "", cleaned)
    cleaned = cleaned.replace("&nbsp;", " ")
    cleaned = cleaned.replace("\t", " ")
    cleaned = cleaned.replace("|", " ")
    cleaned = cleaned.replace("\u00a0", " ")
    cleaned = re.sub(r"\s+", " ", cleaned)

    return cleaned.strip()

def clean_right(line: str | None) -> str:
    if line is None:
        return ""

    cleaned = str(line).rstrip()
    cleaned = cleaned.replace("\t", " ")
    cleaned = cleaned.replace("\u00a0", " ")
    cleaned = re.sub(r"[ ]+", " ", cleaned)

    return cleaned

def is_blank(text: str | None) -> bool:
    return not str(text or "").strip()

```

```
=====
FILE: src\tools\audit_hardcoding.py
ENCODING: utf-8
=====
```

```
import re
from pathlib import Path
```

```
SEARCH_TERMS = [
    "0400",
    "0405",
    "0410",
    "0415",
    "0420",
    "0425",
    "0430",
    "0435",
    "0440",
    "0445",
    "0450",
    "0455",
    "0460",
    "DEPARTMENT",
    "EMPLOYEE",
    "EMPOSITION",
    "JOB",
    "OFFICE",
    "COVERAGE",
    "EXPERTISE",
    "NON-HOSP",
    "SKILL",
    "STRUCTURE",
    "DEPT-EMPLOYEE",
    "EMP-EMPOSITION",
    "JOB-EMPOSITION",
    "OFFICE-EMPLOYEE",
    "NEW-DEPT-STORE-DEPARTMENT",
    "OLD-DEPT-ERASE-DEPARTMENT",
    "OLD-LASTNAME-MODIFY-EMPLOYEE",
]
```

```
SKIP_DIRS = {
    ".git",
    ".venv",
    "__pycache__",
    "data",
    "output",
    "temp",
    "logs",
    "tests",
}
```

```
SKIP_FILES = {
    "audit_hardcoding.py",
    "installed.txt",
}
```

```
INCLUDE_SUFFIXES = {
    ".py",
}
```

```
FALSE_POSITIVE_SUBSTRINGS = {
    "STRUCTUREDBLOCKS",
    "CLASS STRUCTUREDBLOCKS",
    "FROM IDMS_DB2_CONVERTER.TRANSFORMERS.STRUCTURED_BLOCKS",
    "IDMS_DB2_CONVERTER.TRANSFORMERS.STRUCTURED_BLOCKS",
}
```



```

def should_skip(
    path: Path,
) -> bool:
    if path.name in SKIP_FILES:
        return True

    parts = set(
        part.lower()
        for part in path.parts
    )

    if parts.intersection(SKIP_DIRS):
        return True

    if path.suffix.lower() not in INCLUDE_SUFFIXES:
        return True

    return False

```

```

def strip_docstrings(
    text: str,
) -> str:
    text = re.sub(
        r'(""".*?""")',
        "",
        text,
        flags=re.DOTALL,
    )

    text = re.sub(
        r'('''.*?''')',
        "",
        text,
        flags=re.DOTALL,
    )

    return text

```

```

def is_likely_comment_line(
    line: str,
) -> bool:
    stripped = line.strip()

    if not stripped:
        return True

    return stripped.startswith(
        (
            "#",
            "*",
            "_",
        )
    )

```

```

def contains_false_positive(
    line: str,
) -> bool:
    upper_line = line.upper()

    for value in FALSE_POSITIVE_SUBSTRINGS:
        if value in upper_line:
            return True

    return False

```

```

def term_pattern(
    term: str,
) -> re.Pattern:
    escaped = re.escape(
        term,
    )

```

```

    if re.fullmatch(
        r"\d+",
        term,
    ):
        return re.compile(
            rf"(?<!\d){escaped}(?!\\d)",
            re.IGNORECASE,
        )

    return re.compile(
        rf"(?<![A-Z0-9_-]){escaped}(?![A-Z0-9_-])",
        re.IGNORECASE,
    )

def main() -> None:
    root = Path.cwd()

    findings: list[str] = []

    compiled_terms = [
        (
            term,
            term_pattern(term),
        )
        for term in SEARCH_TERMS
    ]

    for path in root.rglob("*"):
        if not path.is_file():
            continue

        if should_skip(path):
            continue

        try:
            text = path.read_text(
                encoding="utf-8",
                errors="ignore",
            )
        except Exception:
            continue

        text_without_docstrings = strip_docstrings(
            text,
        )

        for line_number, line in enumerate(
            text_without_docstrings.splitlines(),
            start=1,
        ):
            if is_likely_comment_line(
                line,
            ):
                continue

            if contains_false_positive(
                line,
            ):
                continue

            for term, pattern in compiled_terms:
                if pattern.search(line):
                    findings.append(
                        f"{path}:{line_number}: {line.strip()}"
                    )

    if not findings:
        print(
            "No hardcoded demo terms found in scanned Python logic."
        )
        return

    print(
        "Potential hardcoded demo terms found in Python logic:"
    )

```

```

print()

for item in findings:
    print(
        item,
    )

if __name__ == "__main__":
    main()

=====
FILE: src\tools\validate_field_mapping_output.py
ENCODING: utf-8
=====

import csv
import re
import sys
from collections import defaultdict
from pathlib import Path

REQUIRED_COLUMNS = [
    "Cobol Record IDMS",
    "Cobol Zone",
    "IDMS Key",
    "IDMS PIC Clause",
    "Length of Field Bytes",
    "Field end position",
    "DB2 Key",
    "New DB2 Record",
    "New DB2 Field name",
    "New DB2 Data Type",
    "Hopex Expression TypeRemark",
    "Relation",
    "Basetype",
]

def normalize(value: str | None) -> str:
    text = str(value or "").strip().upper()
    text = text.replace("-", "_")
    text = re.sub(r"^[A-Z0-9_]+", "_", text)
    text = re.sub(r"_"+, "_", text)
    return text.strip("_")

def parse_level(cobol_zone: str) -> int | None:
    text = str(cobol_zone or "").strip()
    match = re.match(r"^\d{2}\s+", text)
    if not match:
        return None
    return int(match.group(1))

def parse_name(cobol_zone: str) -> str:
    text = str(cobol_zone or "").strip()
    match = re.match(r"^\d{2}\s+(.+)$", text)
    if not match:
        return text
    return match.group(1).strip()

def is_filler(row: dict) -> bool:
    return normalize(parse_name(row.get("Cobol Zone", ""))).startswith("FILLER")

def is_redefines(row: dict) -> bool:
    text = " ".join(
        [
            str(row.get("Cobol Zone", "") or ""),
            str(row.get("Hopex Expression TypeRemark", "") or ""),
        ]
    )

```

```

).upper()
return "REDEFINES" in text or "REDFINES" in text

def has_db2_mapping(row: dict) -> bool:
    return bool(
        str(row.get("New DB2 Record", "") or "").strip()
        or str(row.get("New DB2 Field name", "") or "").strip()
        or str(row.get("New DB2 Data Type", "") or "").strip()
        or str(row.get("DB2 Key", "") or "").strip()
    )

def parse_pic_length(pic: str) -> int | None:
    text = str(pic or "").strip().upper()

    if not text:
        return None

    text = text.replace("PICTURE", "PIC")
    text = text.replace(".", "")
    text = re.sub(r"\bCOMP-3\b", "", text)
    text = re.sub(r"\bCOMP\b", "", text)
    text = re.sub(r"\bDISPLAY\b", "", text)
    text = re.sub(r"\s+", " ", text).strip()

    if text.startswith("PIC "):
        text = text[4:].strip()

    text = text.replace(" ", "")

    match = re.fullmatch(r"X(?:$(\d+)?)?", text)
    if match:
        return int(match.group(1)) if match.group(1) else 1

    match = re.fullmatch(r"S?9(?:$(\d+)?)?V9(?:$(\d+)?)?", text)
    if match:
        left = int(match.group(1)) if match.group(1) else 1
        right = int(match.group(2)) if match.group(2) else 1
        return left + right

    match = re.fullmatch(r"S?9(?:$(\d+)?)?", text)
    if match:
        return int(match.group(1)) if match.group(1) else 1

    return None

def validate_required_columns(rows: list[dict]) -> list[str]:
    if not rows:
        return ["CSV has no rows."]

    missing = [
        column
        for column in REQUIRED_COLUMNS
        if column not in rows[0]
    ]

    if missing:
        return [
            f"Missing required column: {column}"
            for column in missing
        ]

    return []

def validate_redefines(rows: list[dict]) -> list[str]:
    issues = []
    active_redefines_levels: list[int] = []

    for index, row in enumerate(rows, start=2):
        level = parse_level(row.get("Cobol Zone", ""))

        if level is None:
            active_redefines_levels = []

```

```

        continue

    active_redefines_levels = [
        active_level
        for active_level in active_redefines_levels
        if active_level < level
    ]

    current_is_redefines = is_redefines(row)

    if active_redefines_levels or current_is_redefines:
        if has_db2_mapping(row):
            issues.append(
                f"Line {index}: REDEFINES row/child has DB2 mapping: {row.get('Cobol Zone')}"
            )

        if current_is_redefines:
            active_redefines_levels.append(level)

    return issues

def validate_occurs(rows: list[dict]) -> list[str]:
    issues = []

    for index, row in enumerate(rows, start=2):
        remark = str(row.get("Hopex Expression TypeRemark", "") or "").upper()
        field_name = str(row.get("New DB2 Field name", "") or "")

        if "OCCURS INSTANCE" in remark and not field_name:
            issues.append(
                f"Line {index}: OCCURS INSTANCE row has no DB2 field name."
            )

        if "OCCURS INSTANCE" in remark and "_479" in field_name:
            base = field_name.split("_479", 1)[0]
            if "_" in base:
                middle = base.split("_", 1)[1]
                if len(middle) > 7:
                    issues.append(
                        f"Line {index}: OCCURS middle name exceeds 7 chars: {field_name}"
                    )

    return issues

def validate_date_keys(rows: list[dict]) -> list[str]:
    issues = []

    for index, row in enumerate(rows, start=2):
        datatype = str(row.get("New DB2 Data Type", "") or "").upper()
        idms_key = str(row.get("IDMS Key", "") or "").upper()
        db2_key = str(row.get("DB2 Key", "") or "").upper()

        if datatype == "DATE" and "CALC" in idms_key and "PK" not in db2_key:
            issues.append(
                f"Line {index}: DATE CALC field missing DB2 PK: {row.get('Cobol Zone')}"
            )

    return issues

def validate_generated_names(rows: list[dict]) -> list[str]:
    issues = []

    for index, row in enumerate(rows, start=2):
        field_name = str(row.get("New DB2 Field name", "") or "").strip()

        if field_name.startswith("ID_RECORD_"):
            if not re.match(r"^ID_RECORD_479[A-Z0-9]+$ ", field_name):
                issues.append(
                    f"Line {index}: generated PK should be ID_RECORD_479xxxx: {field_name}"
                )

        if field_name.startswith(("TS_CREATE_", "TS_UPDATE_", "ID_USERID_", "NS_SEQUENCE_")):
            if "_479" not in field_name:

```

```

        issues.append(
            f"Line {index}: audit/sequence field missing 479 prefix: {field_name}"
        )

        if re.search(r"_479\s+", field_name):
            issues.append(
                f"Line {index}: audit/sequence field has space after 479: {field_name}"
            )

    return issues

def validate_middle_name_length(rows: list[dict]) -> list[str]:
    issues = []

    for index, row in enumerate(rows, start=2):
        field_name = str(row.get("New DB2 Field name", "") or "").strip()

        if not field_name or "_479" not in field_name:
            continue

        if field_name.startswith(("ID_RECORD_", "TS_CREATE_", "TS_UPDATE_", "ID_USERID_", "NS_SEQUENCE_")):
            continue

        base = field_name.split("_479", 1)[0].rstrip("_")

        if "_" not in base:
            continue

        _prefix, middle = base.split("_", 1)

        if len(middle) > 7:
            issues.append(
                f"Line {index}: middle DB2 field name exceeds 7 chars: {field_name}"
            )

    return issues

def validate_fk_duplicates_and_datatypes(rows: list[dict]) -> list[str]:
    issues = []
    fk_rows_by_key = defaultdict(list)
    pk_datatype_by_column = {}

    for row in rows:
        db2_key = str(row.get("DB2 Key", "") or "").upper()
        field_name = str(row.get("New DB2 Field name", "") or "").strip()
        datatype = str(row.get("New DB2 Data Type", "") or "").strip()

        if not field_name:
            continue

        if "PK" in db2_key:
            pk_datatype_by_column[normalize(field_name)] = datatype

    for index, row in enumerate(rows, start=2):
        if str(row.get("DB2 Key", "") or "").upper() != "FK":
            continue

        key = (
            normalize(row.get("Cobol Record IDMS", "")),
            normalize(row.get("New DB2 Record", "")),
            normalize(row.get("New DB2 Field name", "")),
        )

        fk_rows_by_key[key].append((index, row))

    for key, grouped_rows in fk_rows_by_key.items():
        if len(grouped_rows) > 1:
            lines = ", ".join(str(line) for line, _row in grouped_rows)
            issues.append(
                f"Lines {lines}: duplicate FK rows for record/table/column {key}"
            )

        for line, row in grouped_rows:
            field_name = normalize(row.get("New DB2 Field name", ""))

```

```

        fk_datatype = str(row.get("New DB2 Data Type", "") or "").strip()

        pk_datatype = pk_datatype_by_column.get(field_name)

        if pk_datatype and fk_datatype and pk_datatype != fk_datatype:
            issues.append(
                f"Line {line}: FK datatype differs from PK datatype for {field_name}: FK={fk_datatype}, PK={pk_data
type}"
            )

    return issues

def validate_filler(rows: list[dict]) -> list[str]:
    issues = []

    for index, row in enumerate(rows, start=2):
        if not is_filler(row):
            continue

        pic = str(row.get("IDMS PIC Clause", "") or "").strip()
        length = str(row.get("Length of Field Bytes", "") or "").strip()

        expected_length = parse_pic_length(pic)

        if expected_length is not None and length != str(expected_length):
            issues.append(
                f"Line {index}: FILLER length mismatch. Expected {expected_length}, got {length or '<blank>'}"
            )

    return issues

def validate_decimal_zero_scale(rows: list[dict]) -> list[str]:
    issues = []

    for index, row in enumerate(rows, start=2):
        pic = str(row.get("IDMS PIC Clause", "") or "").upper()
        datatype = str(row.get("New DB2 Data Type", "") or "").upper()

        if "PIC" not in pic:
            continue

        is_numeric_no_v = "9" in pic and "V" not in pic and "X" not in pic

        if is_numeric_no_v and datatype.startswith("DECIMAL("):
            if "," not in datatype:
                issues.append(
                    f"Line {index}: numeric PIC should emit explicit scale zero: {datatype}"
                )

    return issues

def validate_csv(path: Path) -> list[str]:
    with path.open("r", encoding="utf-8-sig", newline="") as file:
        reader = csv.DictReader(file)
        rows = list(reader)

    issues = []

    issues.extend(
        validate_required_columns(
            rows=rows,
        )
    )

    if issues:
        return issues

    issues.extend(
        validate_redefines(
            rows=rows,
        )
    )
    issues.extend(

```

```

        validate_occurs(
            rows=rows,
        )
    )
    issues.extend(
        validate_date_keys(
            rows=rows,
        )
    )
    issues.extend(
        validate_generated_names(
            rows=rows,
        )
    )
    issues.extend(
        validate_middle_name_length(
            rows=rows,
        )
    )
    issues.extend(
        validate_fk_duplicates_and_datatypes(
            rows=rows,
        )
    )
    issues.extend(
        validate_filler(
            rows=rows,
        )
    )
    issues.extend(
        validate_decimal_zero_scale(
            rows=rows,
        )
    )
    )

    return issues

def main() -> int:
    if len(sys.argv) < 2:
        print("Usage:")
        print("python tools/validate_field_mapping_output.py path\\to\\field_mapping.csv")
        return 2

    path = Path(sys.argv[1])

    if not path.exists():
        print(f"File not found: {path}")
        return 2

    issues = validate_csv(
        path=path,
    )

    if not issues:
        print("PASS: no field mapping validation issues found.")
        return 0

    print(f"FAIL: {len(issues)} issue(s) found.")
    print()

    for issue in issues:
        print(f"- {issue}")

    return 1

if __name__ == "__main__":
    raise SystemExit(
        main()
    )

```



```
=====
FILE: test
ENCODING: utf-8
=====
```

```
digraph G { A -> B }
```

```
=====
EXPORT SUMMARY
=====
```

```
Project folder: idms-db2-Database-Sheet-Mapping
Text/code files processed: 131
```